

Patterns und Frameworks

für die Entwicklung paralleler und verteilter Anwendungen in Java

Inhaltsverzeichnis

- Einleitung
 - Ziele und Struktur des Moduls
 - Objektorientierung und UML-Klassendiagramm
- Entwurfsmuster
 - Einführung in Entwurfsmuster
 - Erzeugungsmuster
 - Singleton
 - Fabrikmethode
 - Dependency Injection
 - Strukturmuster
 - Kompositum
 - Adapter
 - Fassade
 - Proxy
 - Verhaltensmuster
 - Beobachter
 - Strategie
- Architektur
 - Modularisierung und Architektur
 - Schichtenarchitektur
 - Model-View-Controller
 - MVC und MVVM in JavaFX
- Verteilte Programmierung
 - Kommunikation und Skalierbarkeit im verteilten System
 - Remote Method Invocation (RMI)
 - SOAP-Webservices
 - REST-Webservices
 - Object-Relational Mapping (ORM)
 - REST und Reactive Streams in Spring
 - WebSockets
 - Sockets
- Nebenläufige Programmierung
 - Threads in Java
 - Synchronisation von Threads
 - Futures und parallele Streams
- UI-Frameworks
 - Desktop-Anwendungen mit JavaFX
 - Web-Anwendungen mit jQuery und Angular

Ziele und Struktur des Moduls

Kurzbildung des Moduls

In dem vorliegenden Modul **Patterns und Frameworks** werden die erworbenen Kompetenzen aus vorherigen Modulen wie Grundlagen der Programmierung I+II, Softwaretechnik und Datenbanken zusammengeführt. Die Prüfungsleistung ist ein Softwareprojekt, dass die Studierenden i.d.R. in einem kleinen Team bearbeiten. Die Studierenden lernen bewährte Entwurfs- und Architekturmuster sowie aktuelle Frameworks aus dem Java-Umfeld kennen. Diese sollen im Projekt selbstständig eingesetzt werden. Im Mittelpunkt des Projekts stehen die Entwurfs- und die Implementierungsphase. Die Studierenden lernen während des Projekts verschiedene Ansätze zur Parallelverarbeitung und zur Kommunikation in verteilten Java-Anwendungen anzuwenden.

Vorausgesetzte Kenntnisse und Fähigkeiten

- Objektorientiertes Programmieren in Java (bekannt aus Grundlagen der Programmierung I+II)
- Modellierung in UML (bekannt aus Softwaretechnik)
- Konfigurationsmanagement in Softwareprojekten, insb. Versionsverwaltung mit Git und Buildmanagement mit Maven (bekannt aus Softwaretechnik)
- Grundlagen relationaler Datenbanksysteme und SQL (bekannt aus Datenbanken)

Lernergebnisse des Moduls

- **Softwareprojekt in Java:** Die Studierenden können ein Softwareprojekt mit vorgegebenen Anforderungen in der Programmiersprache Java selbstständig implementieren.
- **Anwendung von Entwurfsmustern:** Die Studierenden können die vorgestellten Entwurfsmuster in der Anwendungsentwicklung erkennen und selbst bewusst einsetzen.
- **Auswahl von Architekturmustern und Frameworks:** Die Studierenden können eine adäquate Softwarearchitektur unter Berücksichtigung der vorgestellten Architekturmuster und Frameworks entwerfen. Sie können die vorgestellten Frameworks in einem Projekt gezielt einsetzen.
- **Verteilte Kommunikation:** Die Studierenden können synchrone und asynchrone Kommunikation jeweils mittels verschiedener Ansätze (RMI, Webservices, Sockets, WebSockets) in einer verteilten Java-Anwendung umsetzen und diesbezüglich Vor- und Nachteile erörtern.
- **Parallelverarbeitung:** Die Studierenden können komplexe Verarbeitungsprozesse in Java aufteilen, effizient parallelisieren und synchronisieren. Sie können diesbezüglich Vor- und Nachteile unterschiedlicher Ansätze erörtern.

Literatur

Auf die folgenden Einträge des Literaturverzeichnisses wird an den entsprechenden Stellen im Modul über die hier definierten Kürzel verwiesen.

- [Bal11] Helmut Balzert: Lehrbuch der Softwaretechnik - Entwurf, Implementierung, Installation und Betrieb, Spektrum, 3. Aufl., 2011.
- [BC87] Kent Beck, Ward Cunningham: Using Pattern Languages for Object-Oriented Programs, in Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), 1987.
- [Ben05] Mordechai Ben-Ari: Principles of Concurrent and Distributed Programming, 2. Aufl., Addison-Wesley, 2005.
- [Blo17] Joshua Bloch: Effective Java, Addison-Wesley, 3. Aufl., 2017.
- [Fow02] Martin Fowler: Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.
- [Fow04] Martin Fowler: Inversion of Control Containers and the Dependency Injection pattern, <https://martinfowler.com/articles/injection.html>, 2004.
- [FRS+15] Eric Freeman, Elisabeth Robson, Kathy Sierra, Bert Bates: Entwurfsmuster von Kopf bis Fuß, O'Reilly, 2. Aufl. 2015.
- [Gei15] Matthias Geirhos: Entwurfsmuster - Das umfassende Handbuch, Rheinwerk Computing, 2015.
- [GHJ+10] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: Entwurfsmuster - Elemente wiederverwendbarer objektorientierter Software, Addison-Wesley, 6. Aufl., 2010.
- [GJM03] Carlo Ghezzi, Mehdi Jazayeri, Dino Mandrioli: Fundamentals of Software Engineering, Prentice Hall, 2nd ed., 2003.
- [Ind16] Michael Inden: Der Java-Profi - Persistenzlösungen und REST-Services, dpunkt, 2016.
- [Ind17] Michael Inden: Der Weg zum Java-Profi - Konzepte und Techniken für die professionelle Java-Entwicklung, dpunkt, 4. Aufl., 2017.
- [KB18] Veikko Krypczyk, Olena Bochkor: Handbuch für Software-Entwickler, Rheinwerk Computing, 2018.
- [Kle17] Martin Kleppmann: Designing Data-Intensive Applications - The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, O'Reilly, 2017.
- [KSH17] Christoph Kecher, Alexander Salvanos, Ralf Hoffmann-Elbern: UML 2.5 - Das umfassende Handbuch, Rheinwerk Computing, 6. Aufl., 2017.
- [LRS18] Bernhard Lahres, Gregor Rayman, Stefan Strich: Objektorientierte Programmierung - Das umfassende Handbuch. Prinzipien guter Objektorientierung auf den Punkt erklärt, Rheinwerk Computing, 4. Aufl., 2018.
- [LW07] Kung-Kiu Lau, Zheng Wang: Software Component Models, IEEE Transactions on Software Engineering 33(10), 2007.
- [Mar09] Robert Martin: Clean Code - Refactoring, Patterns, Testen und Techniken für sauberen Code, mitp, 2009.
- [Mar18] Robert Martin: Clean Architecture - Gute Softwarearchitekturen - Das Praxis-Handbuch für professionelles Softwaredesign - Regeln und Paradigmen für effiziente Softwarestrukturierung, mitp, 2018.
- [OS13] Bernd Oestereich, Axel Scheithauer: Analyse und Design mit der UML 2.5 - Objektorientierte Softwareentwicklung, De Gruyter Oldenbourg, 11. Aufl., 2013.
- [RQ12] Chris Rupp, Stefan Queins: UML 2 glasklar - Praxiswissen für die UML-Modellierung, Hanser, 4. Auflage, 2012.
- [SGM02] Clemens Szyperski, Dominik Gruntz, Stephan Murer: Component Software - Beyond Object-Oriented Programming, Addison Wesley, 2nd ed., 2002.
- [Spi19] Kai Spichale: API-Design: Praxishandbuch für Java- und Webservice-Entwickler, dpunkt, 2. Aufl., 2019.
- [SW01] Connie Smith, Lloyd Williams: Performance Solutions - A Practical Guide to Creating Responsive, Scalable Software, Addison-Wesley, 2001.
- [TvS16] Andrew Tanenbaum, Maarten van Steen: Distributed Systems - Principles and Paradigms, CreateSpace, 2nd ed., 2016.

Objektorientierung und UML-Klassendiagramm

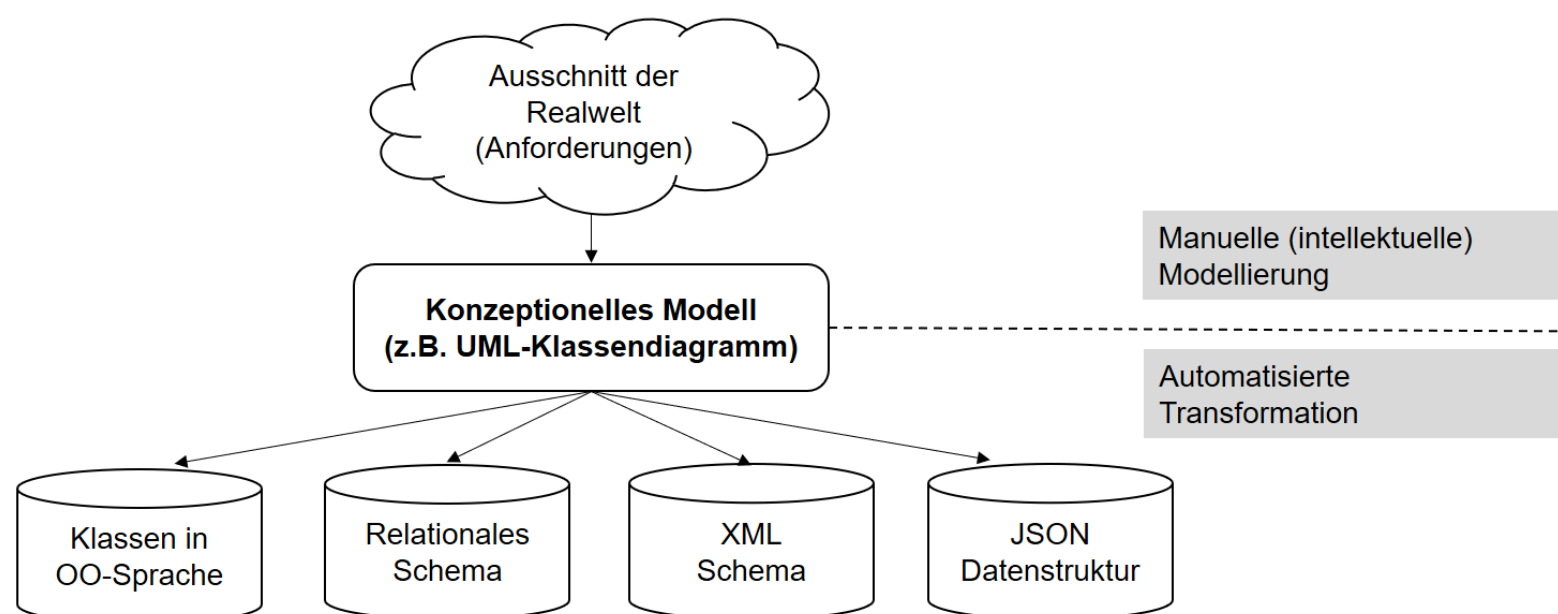
Dieses Kapitel behandelt die Modellierung von objektorientierten Datenstrukturen. Es soll der Zusammenhang zwischen einem Modell, das hier grundsätzlich in der Notation eines UML-Klassendiagramms dargestellt wird, und dem zugehörigen Code in Form von Klassen in der Programmiersprache Java verdeutlicht werden. Dieses Kapitel ist eine zusammenfassende Wiederholung von Inhalten, die bereits aus den vorausgehenden Modulen Softwaretechnik und Grundlagen der Programmierung bekannt sein sollten. Dieses Kapitel kann also gerne übersprungen werden, falls diese Grundlagen bereits gefestigt sind. Der sichere Umgang mit UML-Klassendiagrammen ist Voraussetzung für das Verständnis der Entwurfsmuster im folgenden Kapitel.

Konzeptionelle Modellierung

Bei der konzeptionellen Modellierung geht es darum, Datenstrukturen (und Operationen auf diesen) für ein konkretes Anwendungsszenario zu entwerfen. Das Modell bildet einen Ausschnitt der Wirklichkeit ab, wobei nur für die Anwendung relevante Aspekte der Wirklichkeit ins Modell aufgenommen werden. Aufgrund der Komplexität der Wirklichkeit ist verkürzende Abstraktion erforderlich. Welche Details der Wirklichkeit versteckt werden, hängt vom pragmatischen Zweck der Anwendung ab. Die Ziele jeder Modellbildung sind also Abbildung, Verkürzung und Pragmatismus.

Im Rahmen einer Anforderungsanalyse wird ermittelt, welche Daten für eine Anwendung relevant sind und erfasst werden müssen. Das Datenmodell speichert den Zustand einer Anwendung über ihren gesamten Lebenszyklus. In der Entwurfsphase wird üblicherweise mit Testdaten gearbeitet. Ab der Inbetriebnahme der Anwendung werden auf dem Produktivsystem dann die Echt Daten der Anwender erfasst. Da Anwendungen häufig über viele Jahre in Betrieb sind, wird es im Laufe der Zeit auch Änderungen am Datenmodell geben. In Abhängigkeit von der Stabilität der Anforderungen spielt also die Anpassungsmöglichkeit des Datenmodells eine mehr oder weniger wichtige Rolle. Da zur Laufzeit ständig Daten gelesen, geändert und hinzugefügt werden, ist es in Abhängigkeit der erwarteten Frequenz und Parallelität der Lese- und Schreibzugriffe auch erforderlich sich mit der Effizienz dieser Zugriffe zu beschäftigen.

Ein konzeptionelles Datenmodell ist noch nicht an eine konkrete Technologie zur Implementierung der Anwendung gebunden. Es ist eben ein rein konzeptioneller Entwurf, dessen Entstehung eine Tätigkeit intellektueller Modellbildung ist. In der Softwaretechnik hat sich hier die Unified Modelling Language (UML) als Standard etabliert. Wortwörtlich übersetzt ist die UML also eine vereinheitlichte Modellierungssprache. Aus Modellen in dieser Sprache ist es wiederum möglich, automatisiert Datenmodelle für konkrete Technologien abzuleiten. Das können z.B. die Datenstrukturen in Form von Klassen für eine objektorientierte Programmiersprache sein, wenn die Anwendung z.B. in Java, C# oder ähnlichen Sprachen programmiert werden soll. Es kann auch ein relationales Schema aus einem konzeptionellen Modell abgeleitet werden, wenn die Daten in einem relationalen Datenbanksystem wie MariaDB, PostgreSQL o.ä. gespeichert werden sollen. Es können natürlich auch Datenstrukturen für strukturierte Datenaustauschformate wie XML oder JSON abgeleitet werden. Wichtig ist, dass das konzeptionelle Modell möglichst nicht an eine konkrete Technologie gekoppelt ist. Auch bei einem Wechsel der Technologie, z.B. auf eine nicht-objektorientierte Programmiersprache oder eine nicht-relationale Datenbank können die neuen Datenstrukturen über einen automatisierten Transformationsschritt aus dem Modell generiert werden.



Konzeptionelles Modell als Schnittstelle zwischen Anforderungen und technischer Implementierung

Unified Modeling Language (UML)

Die UML ist eine weit verbreitete Sprache zur Modellierung von Softwaresystemen. In den präskriptiven Phasen der Software-Entwicklung unterstützt die UML die Spezifikation und die Konstruktion einer Anwendung, d.h. während die Anwendung noch nicht existiert, kann die UML helfen, die Anforderungen an die Anwendung in eine Softwarearchitektur zu überführen und vorzuschreiben, wie die Anwendung zu implementieren ist. In den deskriptiven Phasen der Software-Entwicklung dient die UML der Visualisierung und der Dokumentation, d.h. nachdem die Anwendung implementiert worden ist, helfen die grafischen Diagramme der UML die Architektur der Anwendung zu veranschaulichen und zu dokumentieren. Da die Notation von UML-Diagrammen unter Software-Entwicklern allgemein bekannt ist, eignen sich die Diagramme insbesondere als Diskussionsgrundlage für den Entwurf in

Teambesprechungen. Häufig werden UML-Diagramme als handschriftliche Skizzen gemeinsam an Whiteboards erstellt und diskutiert. Dabei geht es nicht darum, dass diese Skizzen formal korrekt sein müssen, sie dienen eher der Entwicklung und Zusammenführung von Ideen. Dabei ist eine grundlegende, allgemein bekannte Notation hilfreich.

Das Konsortium, das die UML seit 1998 pflegt, nennt sich Object Management Group (OMG). Viele namhafte Softwarehersteller wie IBM, Oracle, SAP oder Microsoft sind Mitglied der OMG und engagieren sich herstellerunabhängige Standards zu entwickeln. Seit 2015 ist UML 2.5 die [aktuelle Version der UML-Spezifikation](#). Die Diagramme der UML teilen sich mittlerweile ein gemeinsames Metamodell. Historisch gesehen gab es aber verschiedene, in den 90er-Jahren unabhängig voneinander entworfene Notationsideen zur Modellierung spezifischer Aspekte von Softwaresystemen. Diese wurden später in der UML zusammengeführt und nicht ohne Kompromisse mit einem gemeinsamen Metamodell versehen. Heute besteht der Anspruch, dass mehrere UML-Diagramme, die im Rahmen eines Projekts für eine Anwendung erstellt werden, jeweils als abstrahierende Sicht aus unterschiedlichen Perspektiven auf ein gemeinsames und widerspruchsfreies Modell der Anwendung interpretiert werden können.

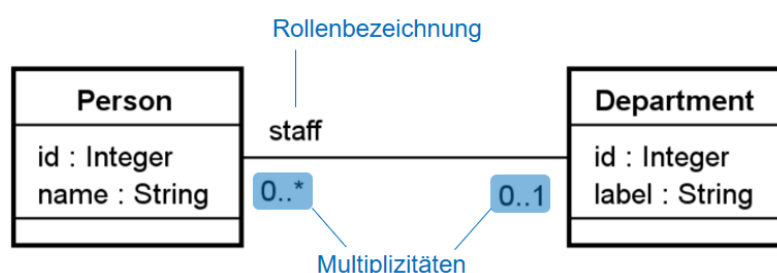
Es gibt mehrere gute deutschsprachige Lehrbücher zum Thema UML: [\[KSH17\]](#), [\[OS13\]](#), [\[RQ12\]](#). Außerdem existiert eine Vielzahl von Software-Werkzeugen, mit denen UML-Diagramme erstellt werden können. Diese Werkzeuge unterstützen meist auch *Forward Engineering* und *Reverse Engineering*. Forward Engineering heißt dabei, dass ausgehend von dem erstellten Modell, Datenstrukturen und Operationen in Form von Code in der Syntax der geläufigen Programmiersprachen generiert werden kann. Reverse Engineering heißt das ausgehend von vorliegendem Code ein Modell zurückgewonnen werden kann. Die UML-Werkzeuge am Markt unterscheiden sich in ihrem Funktionsumfang, ihrer Konformität zur UML-Spezifikation und ihren Preis- und Lizenzmodellen. Das IT-Beratungsunternehmen oose hat in 2018 in einer Veröffentlichung [verschiedene UML-Werkzeuge einander gegenübergestellt](#). Natürlich werden diese Werkzeuge kontinuierlich weiterentwickelt. Interessant bleiben aber die Kriterien, nach denen die Werkzeuge miteinander verglichen werden. Dem Vergleich kann entnommen werden, welche der 14 UML-Diagramme von den jeweiligen Werkzeugen unterstützt werden. Zusätzlich werden weitere Kriterien wie unterstützte Programmiersprache für Forward Engineering und Reverse Engineering, Simulationsunterstützung, Integration in die populären Entwicklungsumgebungen, XMI-Import und -Export, unterstützte Betriebssysteme und Lizenzkosten dargestellt.

Klassen und Objekte

In der objektorientierten Modellierung und Programmierung sind Klassen eine Abstraktion für gleichartige Objekte der abzubildenden Realwelt. Klassen erlauben mittels ihrer Attribute eigene komplexe Datentypen zu definieren. Die Attribute können als mehrere sachlogisch zusammengehörige Variablen aufgefasst werden, die eine gemeinsame Datenstruktur bilden. Zusätzlich können an einer Klasse Operationen definiert werden, die mögliches Verhalten der Objekte einer Klasse zur Laufzeit beschreiben und die in Java als Methoden zu implementieren sind.

Die Objekte einer Klasse sind ihre Instanzen, d.h. konkreten Ausprägungen zur Laufzeit. Jedes Objekt, das einmal erzeugt worden ist, hat seine eigene Identität. Der Zustand eines Objekts, der sich durch die Ausprägungen seiner Attribute ergibt, kann sich zur Laufzeit häufig verändern. Auch wenn 2 Objekte zur Laufzeit zufälligerweise den gleichen Zustand haben, sind es nicht dieselben Objekte. Für jedes Objekt kann zur Laufzeit einer Anwendung festgestellt werden, von welchem konkreten Typ es ist, d.h. zu welcher Klasse es gehört.

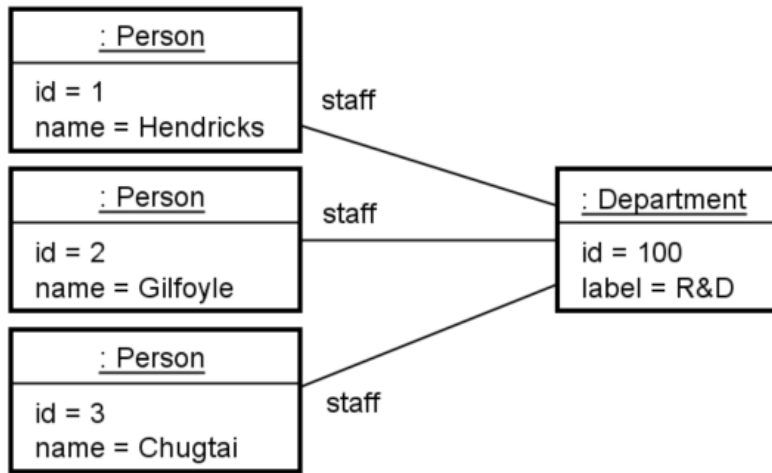
Wir erklären die Notation des UML-Klassendiagramms folgend anhand eines Beispiels. In diesem Beispiel sollen Datenstrukturen für die Mitarbeiter eines Unternehmens modelliert werden. Die Mitarbeiter werden Abteilungen zugeordnet und arbeiten in Projekten. In der Programmierung hat es sich durchgesetzt alle Klassen, Methoden, Variablen, etc. in englischer Sprache zu bezeichnen. Daher werden wir auch in diesem Modul für alle Modell- und Code-Beispiele englische Bezeichner wählen.



2 Klassen, die durch eine Assoziation verbunden sind

Zunächst wird eine Klasse als ein Rechteck mit 3 Abschnitten dargestellt. Im oberen Abschnitt steht der Name der Klasse. Im mittleren Abschnitt folgen alle Attribute der Klasse, die von einem Basisdatentyp sind. Die Typen der Attribute können angegeben oder versteckt werden. Im unteren Abschnitt der Klasse werden später die Operationen stehen. In der obigen Abbildung ist durch die Multiplizitäten an der Assoziationskante zwischen den Klassen modelliert, dass jede Person einer oder keiner Abteilung (0..1) zugeordnet ist, wobei in einer Abteilung mehrere Personen (0..*) zusammengefasst werden. Der Default-Wert für Multiplizitäten ist 1 und muss nicht explizit angegeben werden.

In der folgenden Abbildung sehen wir ein Objektdiagramm, das einen beliebigen Zeitpunkt zur Laufzeit der Anwendung abbildet. Im dargestellten Zustand existieren 4 Objekte: ein Objekt vom Typ Abteilung mit 3 zugeordneten Objekten vom Typ Person, die jeweils unterschiedliche Ausprägungen in ihren Attributen aufweisen.



Objektdiagramm, das den Zustand von 4 Objekten zur Laufzeit darstellt

Assoziationen und Navigierbarkeit

Die beiden Klassen des obigen Modells entsprechen folgendem Code in Java.

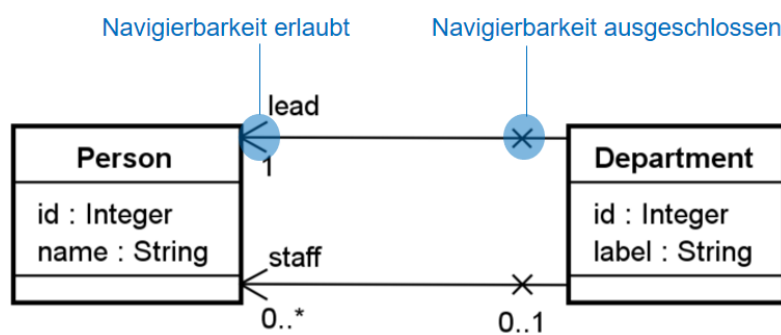
```

class Person {
    Integer id;
    String name;
    Department department;
}

class Department {
    Integer id;
    String label;
    Person[] staff;
}
  
```

Die Klasse **Person** enthält ein Attribut vom komplexen Typ **Department**, das im Modell nicht weiter bezeichnet worden ist und daher wie sein Typ **department** heißt. Die Klasse **Department** enthält ein Array-Attribut vom komplexen Typ **Person**, für das im Modell die Bezeichnung **staff** spezifiziert worden ist. Assoziationen sind also die Grundlage, um eigene komplexe Datentypen zu konstruieren. Die Navigierbarkeit der Assoziation ist im Modell bisher unspezifiziert. Im Code ist einfach angenommen worden, dass die Navigierbarkeit in beide Richtungen möglich sein soll, d.h. das eine bidirektionale Assoziation vorliegt.

Das folgende Modell erweitert das vorherige um eine zweite Assoziation zwischen den Klassen. Nun sind Rollenbezeichnungen an den Assoziationsenden zwingend erforderlich, um deren Semantik unterscheiden zu können. Außerdem ist die Navigierbarkeit nun durch das Pfeil-Symbol explizit erlaubt bzw. durch das Kreuz-Symbol explizit ausgeschlossen. Es liegen nur noch unidirektionale Assoziationen vor, d.h. von der Klasse **Person** aus kann die zugeordnete Abteilung nicht mehr ermittelt werden.



2 Klassen, die durch 2 unterschiedliche Assoziationen verbunden sind

Dieses Modell entspricht folgendem Code in Java.

```

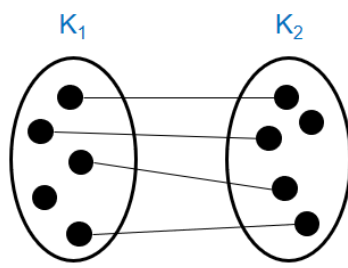
class Person {
    Integer id;
    String name;
}

class Department {
    Integer id;
    String label;
    Person lead;
    Person[] staff;
}
  
```

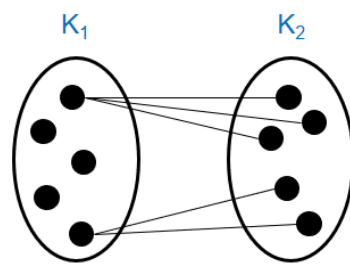
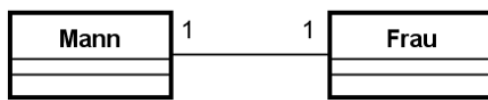
Grundsätzlich lassen sich folgende Arten von Assoziationen hinsichtlich ihrer Kardinalität unterscheiden:

- **1:1-Beziehung:** Jedem Objekt vom Typ K_1 ist max. ein Objekt vom Typ K_2 zugeordnet und umgekehrt. Beispiel: "Mann ist verheiratet mit Frau."

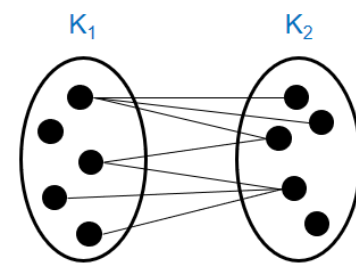
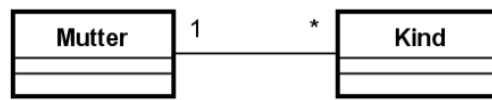
- **1:N-Beziehung:** Jedem Objekt vom Typ K_1 sind beliebig viele Objekte vom Typ K_2 zugeordnet, aber zu jedem Objekt vom Typ K_2 gibt es max. ein Objekt vom Typ K_1 . Beispiel: "Mutter hat Kinder."
- **N:N-Beziehung:** Jedem Objekt vom Typ K_1 sind beliebig viele Objekte vom Typ K_2 zugeordnet und umgekehrt Beispiel: "Bestellung beinhaltet Produkt."



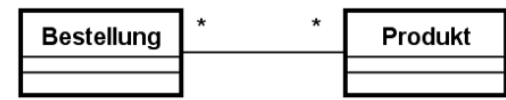
1:1-Beziehung



1:N-Beziehung



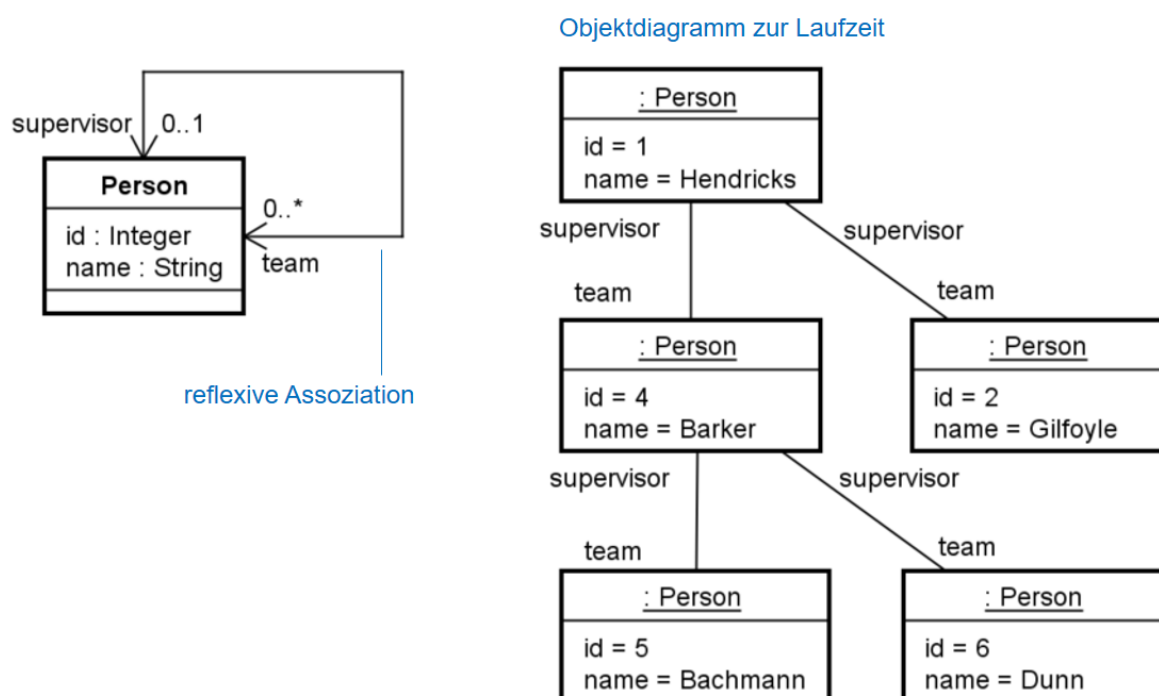
N:N-Beziehung



Kardinalitäten von Assoziationen

Reflexive Assoziationen

Eine reflexive Assoziation tritt auf, wenn mehrere Objekte derselben Klasse zueinander in Beziehung stehen. In dem folgenden Beispiel hat jede Person eine oder keine andere Person als direkten Vorgesetzten. Eine Person kann mehrere andere Personen als untergebenes Team zugeordnet haben. Nur die Personen ganz oben in der Hierarchie haben keinen Vorgesetzten. Im zugehörigen Objektdiagramm sehen wir eine beispielhafte Hierarchie mit mehreren Ebenen. Die Hierarchie kann über beliebig viele Ebenen tief verschachtelt werden und ist auch in der Breite nicht begrenzt.



Reflexive Assoziation

Dieses Modell entspricht folgendem Code in Java.

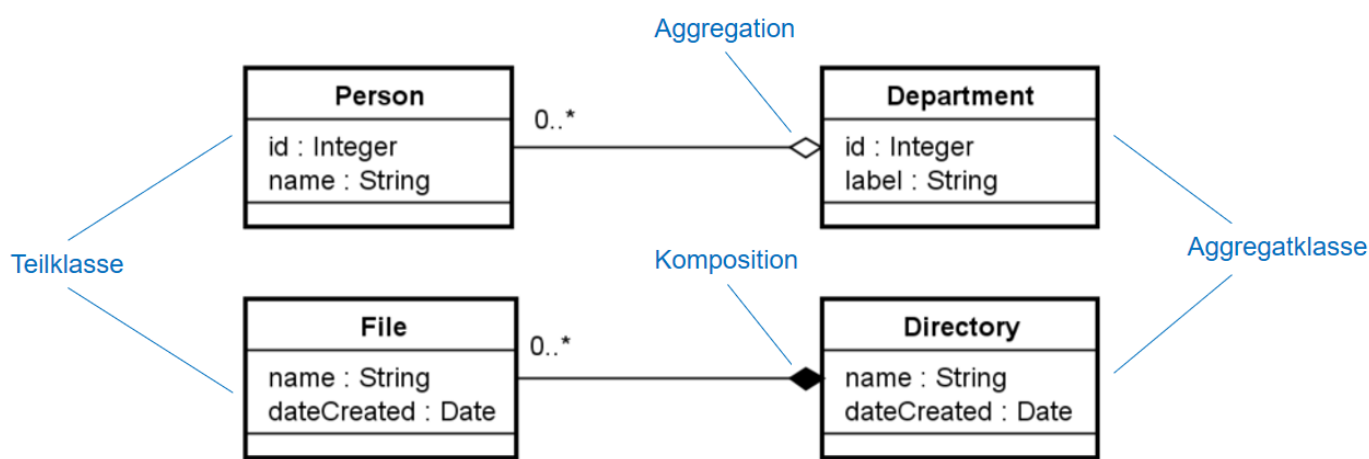
```
class Person {
    Integer id;
    String name;
    Person supervisor;
    Person[] team;
}
```

Aggregation und Komposition

Soll ein Assoziationsende hervorgehoben werden, um auszudrücken, dass es sich um eine Teil-Ganzes-Beziehung handelt, bietet die UML dafür die Aggregation und die Komposition als spezielle Assoziationen an. Eine nicht ausgefüllte Raute auf der Seite des Aggregats kennzeichnet eine Aggregation, eine ausgefüllte Raute stellt eine Komposition dar. Die beteiligten Objekte einer Teil-Ganzes-Beziehung bilden immer einen azyklischen Graphen.

- **Aggregation:** Die Semantik einer schwachen Teil-Ganzes-Beziehung besteht nur darin, dass das Aggregat sachlogisch irgendwelche Aufgaben als Stellvertreter für seine Teile übernehmen kann, z.B. eine Abteilung für alle ihre Mitarbeiter. Grundsätzlich ist die Abgrenzung zwischen Assoziation und Aggregation schwierig.
- **Komposition:** Die starke Teil-Ganzes-Beziehung ist die Komposition. Bei der Komposition kann ein Teil zu einem bestimmten Zeitpunkt nur zu einem Aggregat gehören. Das Teil wird mit dem Aggregat zusammen kopiert oder gelöscht. Es ist in seiner Existenz vom Aggregat abhängig. Für das hier dargestellte Beispiel von Dateien als Teile eines Verzeichnisses im Dateisystem

erscheint das einleuchtend. Wenn wir z.B ein Verzeichnis im Dateisystem löschen, entfernen wir auch alle Dateien in diesem Verzeichnis. Auch wenn wir das Verzeichnis an eine andere Stelle kopieren, vervielfältigen wir gleichermaßen alle Dateien im Verzeichnis. Eine Datei kann auch nur in einem Verzeichnis gleichzeitig sein.

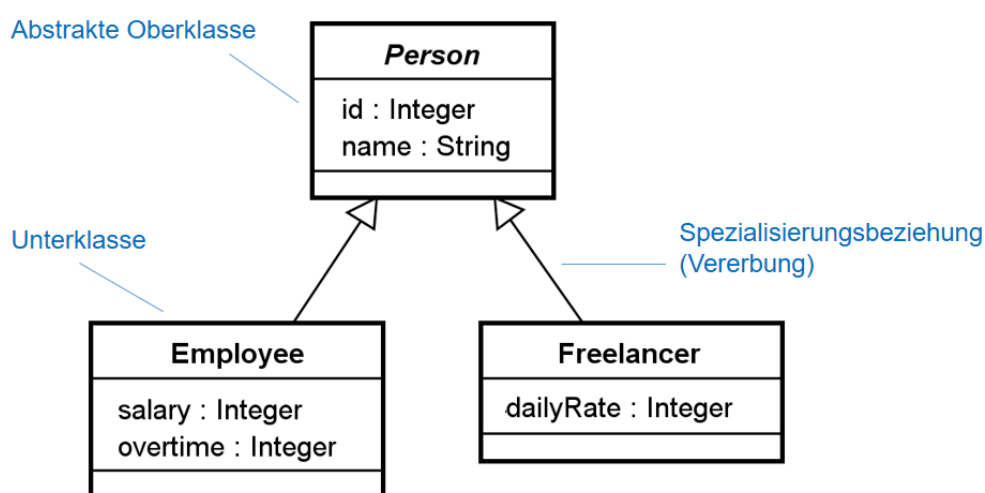


Aggregation und Komposition in der UML

Vererbung und Polymorphie

Klassen können andere Klassen erweitern. Spezialisierte Unterklassen (auch Subklassen genannt) können Attribute und Methoden von allgemeineren Oberklassen (auch Superklassen genannt) erben. Die Vererbung ist das Grundkonzept zur Wiederverwendung von Klassen. Eine erbende/abgeleitete Klasse besitzt alle Attribute und Methoden der Klasse, von der sie erbt/abgeleitet ist. Der neuen Klasse können weitere spezielle Attribute und Methoden hinzugefügt werden, aber sie bleibt stets kompatibel mit der Oberklasse, d.h. sie erfüllt deren Schnittstelle. Ein Objekt der Unterklasse kann überall dort verwendet werden, wo ein Objekt der Oberklasse erwartet wird, z.B. als Input-Argument einer Methode.

In der UML wird die Vererbung durch Blockpfeile notiert, die ausgehend von der Unterklasse zur Oberklasse gerichtet sind. Im folgenden Beispiel gibt es 2 spezielle Typen von Personen, nämlich **Employee** und **Freelancer**. Diese beiden Unterklassen teilen sich die allgemeinen Attribute der Oberklasse **Person** und ergänzen jeweils spezifische Attribute. Gleichermaßen können auch Methoden (und Konstruktoren) aus der Oberklasse übernommen und neue Methoden (und Konstruktoren) in der Unterklasse hinzugefügt werden.



Vererbung in der UML

Dieses Modell entspricht folgendem Code in Java.

```

abstract class Person {
    Integer id;
    String name;
}

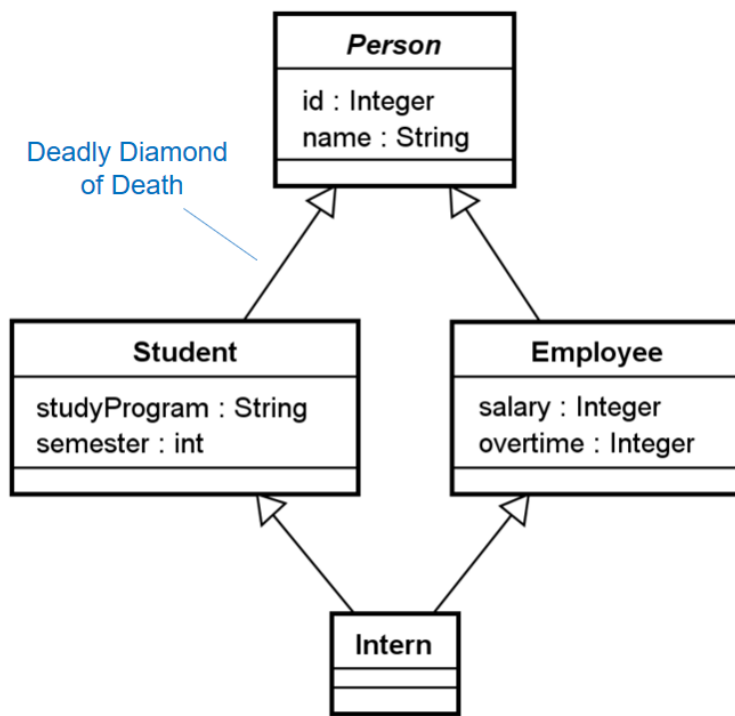
class Employee extends Person {
    Integer salary;
    Integer overtime;
}

class Freelancer extends Person {
    Integer dailyRate;
}

```

Im obigen Beispiel ist die Klasse **Person** abstrakt. In der UML wird der Name von abstrakten Klassen *kursiv* geschrieben. Von einer abstrakten Klasse können zur Laufzeit keine Objekte erzeugt werden, d.h. es ist kein Konstruktor-Aufruf `new Person()` möglich. Abstrakte Klassen sind nur eine Grundlage für konkrete Unterklassen, die von ihr abgeleitet werden. Sie stellen i.d.R. allgemeine Oberbegriffe dar, wie z.B. **Person**. Wenn eine Klasse abstrakte Methoden enthält, d.h. Methoden für die nur ihre Signatur ohne Implementierung vorliegt, muss sie abstrakt sein. Von Klassen mit abstrakten Methoden kann es zur Laufzeit keine Objekte geben, da diese Methoden auf den Objekten nicht ausgeführt werden könnten.

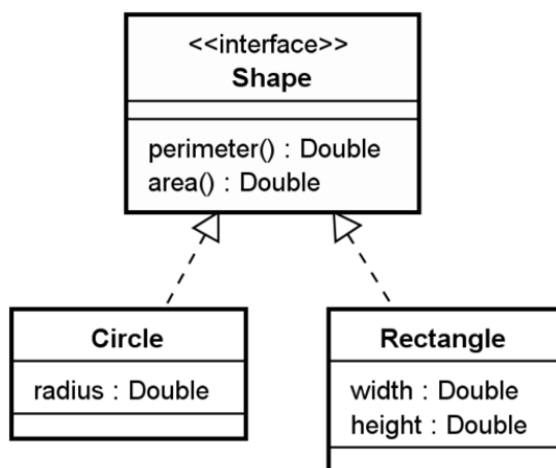
Eine Unterklasse kann in der UML auch mehrere Oberklassen erweitern, was Mehrfachvererbung genannt wird. Mehrfachvererbung ist nur in wenigen Programmiersprachen zulässig (z.B. C++, Python) und in vielen anderen nicht (z.B. Java, C#). In der Programmierung werden Schnittstellen (*Interfaces*) gegenüber Mehrfachvererbung i.d.R. bevorzugt. Im folgenden Modell ist Mehrfachvererbung am Beispiel der Praktikanten-Klasse **Intern** illustriert. Probleme entstehen, wenn an die Unterklasse über verschiedene Vererbungspfade jeweils ein Attribut mit gleicher Bezeichnung vererbt wird oder wenn eine Unterklasse über mehrere Vererbungspfade von derselben Oberklasse erbt. Letzteres wird bildlich als *Deadly Diamond of Death* bezeichnet.



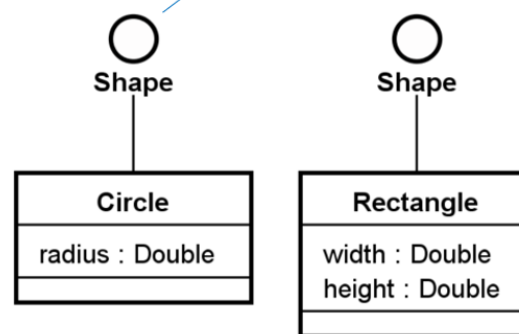
Mehrfachvererbung in der UML

Ein Interface spezifiziert i.d.R. nur Methoden-Signaturen, die dann eine gemeinsame Schnittstelle für alle Klassen bilden, die das Interface implementieren. Damit entspricht ein Interface einer abstrakten Klasse mit ausschließlich abstrakten Methoden. Seit Java 8 können in Interfaces aber auch Default-Implementierungen für Methoden angegeben werden. Da ein Interface keine Attribute und keine Assoziationen besitzt, ist es zustandslos. Das folgende Beispiel zeigt die beiden Klassen **Rectangle** und **Circle**, die jeweils das Interface **Shape** implementieren und damit dessen Methoden überschreiben müssen. Wenn die Methoden des Interface allgemein bekannt sind und eine Klasse ggf. sehr viele Interfaces implementiert, bietet sich die sogenannte *Lollipop-Notation* als Kurzschreibweise an, um die Übersichtlichkeit in einem größeren Modell zu erhalten.

Implementierung eines Interface



Interface-Implementierung in Lollipop-Notation



Interfaces in der UML

Dieses Modell entspricht folgendem Code in Java. Der Code enthält zusätzlich unterschiedliche Implementierungen der Methoden-Signaturen im Interface.

```

interface Shape {
    Double perimeter();
    Double area();
}

class Circle implements Shape {
    Double radius;

    @Override
    Double perimeter() { return 2 * Math.PI * radius; }
    @Override
    Double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    Double width, height;

    @Override
    Double perimeter() { return 2 * (width + height); }
    @Override
    Double area() { return width * height; }
}

```

Eine Default-Implementierung für die beiden Methoden im Interface **Shape** könnte wie folgt aussehen. In diesem Fall müssten die Klassen, die das Interface implementieren, die Methoden nicht notwendigerweise überschreiben (aber sinnvoll wäre es doch).

```

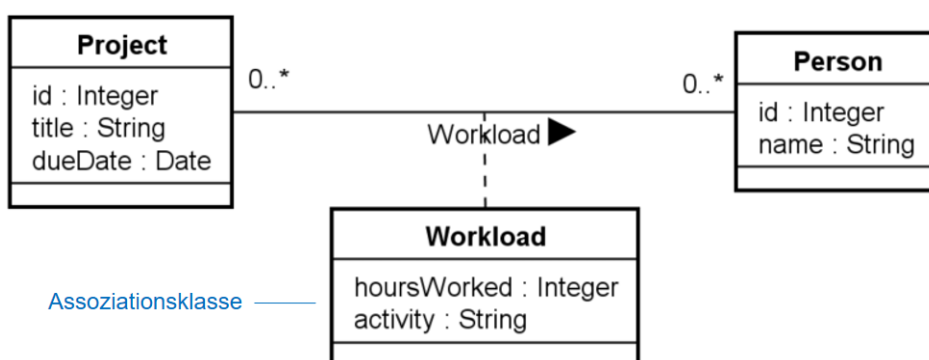
interface Shape {
    default Double perimeter() { return null; }
    default Double area() { return null; }
}

```

In Unterklassen kann die Implementierung von Methoden einer Oberklasse oder eines Interface also überschrieben werden, was in Java durch die Annotation *Override* kenntlich gemacht wird. Die Aufrufe derselben Methode auf einer Menge von Objekten, die eine gemeinsame Oberklasse erweitern oder ein gemeinsames Interface implementieren, kann also zur Laufzeit zur Ausführung unterschiedlicher Implementierungen der Methode führen. Dies bezeichnen wir in der Objektorientierung als Polymorphie (= Vielgestaltigkeit).

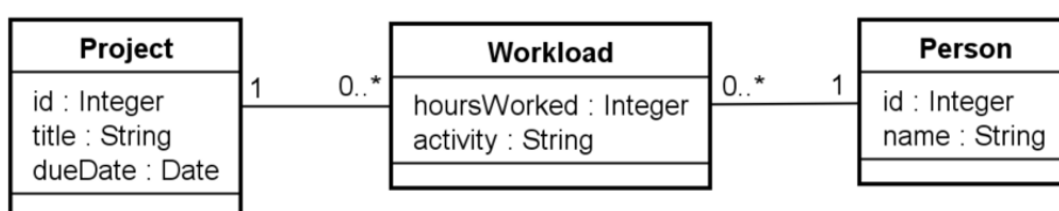
Assoziationsklassen und mehrstellige Beziehungen

Eine spezielle Art von Klassen sind Assoziationsklassen, die eine Assoziation um Attribute anreichern. Ein Objekt der Assoziationsklasse kann ohne ein zugehöriges Objekt an beiden Assoziationsenden nicht existieren. Im folgenden Beispiel arbeitet eine Person eine bestimmte Anzahl an Stunden (**hoursWorked**) mit einer bestimmten Tätigkeit (**activity**) für ein Projekt. Insgesamt können viele Personen für viele Projekte arbeiten.



Assoziationsklasse in der UML

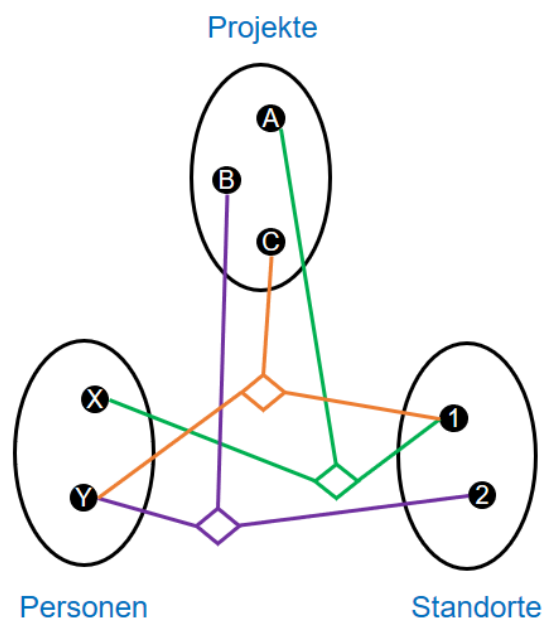
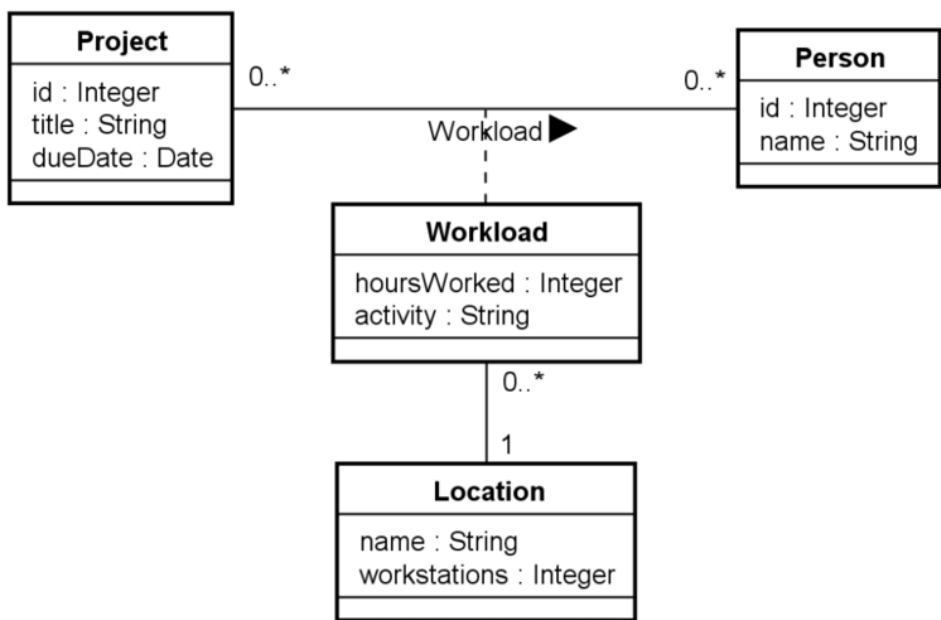
Derselbe Zusammenhang kann alternativ auch ohne Assoziationsklasse wie folgt modelliert werden.



Modellierung einer N:N-Beziehung mit Attributen ohne Assoziationsklasse

Eine Assoziationsklasse kann ihrerseits in Beziehung zu weiteren Klassen stehen. In folgendem Beispiel wird die Information ergänzt, von welchem Standort (**Location**) aus eine Person für ein Projekt arbeitet. Dadurch entsteht eine ternäre Beziehung zwischen den Klassen **Person**, **Project** und **Location**. In der unten skizzierten Mengendarstellung arbeitet **Person X von Standort 1 an Projekt A**,

Person Y von Standort 2 an Projekt B und Person Y von Standort 1 an Projekt C. Beliebige weitere Kombinationen sind auch möglich. Grundsätzlich können mehrstellige Beziehungen auf unterschiedliche Weise (auch ohne Verwendung einer Assoziationsklasse) in der UML modelliert werden.



Modellierung einer mehrstelligen Beziehung

Einführung in Entwurfsmuster

Als Ursprung objektorientierter Muster kann der Artikel [Using Pattern Languages for Object-Oriented Programs](#) von Beck und Cunningham auf der OOPSLA-Konferenz 1987 angesehen werden. Für eine allgemeine Verbreitung der grundlegenden Entwurfsmuster sorgten dann Gamma, Helm, Vlissides und Johnson, auch bekannt als *Gang of Four (GoF)*, mit ihrem Buch [Design Patterns - Elements of Reusable Object-Oriented Software](#), das 1994 in seiner 1. Auflage erschien und bis heute als Grundlagenwerk für Entwurfsmuster gilt. In diesem Buch werden insgesamt 23 Muster katalogisiert und jedes nach der gleichen Schablone beschrieben.

Die GoF-Schablone für Entwurfsmuster enthält folgende Aspekte:

- Zunächst werden Zweck, Motivation und Anwendbarkeit erläutert.
- Es folgt eine Darstellung der Struktur als UML-Klassendiagramm (im Original-Buch in mittlerweile veralteter Notation) und der einzelnen beteiligten Klassen als Akteure.
- Zu jedem Muster werden Vor- und Nachteile diskutiert und mind. eine mögliche Implementierung als Code in C++ und Smalltalk angegeben.

Eine umfängliche Darstellung der GoF-Entwurfsmuster inkl. Java-Beispielen findet sich im unterhaltsam gestalteten Lehrbuch [Entwurfsmuster von Kopf bis Fuß](#). Ein weiteres gelungenes Lehrbuch, das nicht auf die GoF-Entwurfsmuster beschränkt ist, findet sich in [\[Gei15\]](#).

Was sind Entwurfsmuster?

Entwurfsmuster stellen bewährte, verallgemeinerte Lösungsansätze für häufig wiederkehrende Probleme im objektorientierten Entwurf dar. Ausgerichtet sind die Entwurfsmuster auf eine flexible Erweiterbarkeit und Anpassungsfähigkeit des Codes bei zukünftig notwendigen Änderungen. Die Namen der Entwurfsmuster haben sich als Begriffe im alltäglichen Sprachgebrauch unter Softwareentwicklern etabliert und gehören als Fachvokabular damit zur Grundausbildung in der Softwaretechnik.

Die Verwendung von Entwurfsmustern ist als *Best Practice* in der professionellen Softwareentwicklung weit verbreitet und lässt sich mittels [statischer Codeanalyse](#) im Code vieler Softwareprojekte nachweisen. Geläufige Gegenbegriffe sind [Anti-Patterns](#) und [Code Smells](#), die ebenfalls durch Codeanalyse entdeckt werden können und i.d.R. zu vermeiden sind. Eine gute Zusammenfassung, wie lesbarer und gut strukturierter Code zu gestalten ist, liefert Robert Martin in seinem Klassiker [Clean Code](#).

Ein Entwurfsmuster bietet eine Lösung für ein konkretes (kleines) Teilproblem im Feinentwurf einer Software-Komponente. Entwurfsmuster lassen sich damit von allgemeinen Architekturmustern und konkreten Frameworks abgrenzen, die zur Strukturierung einer ganzen Anwendung dienen.

GoF-Entwurfsmuster

Die folgende Tabelle zeigt alle 23 Entwurfsmuster der [GoF](#), gegliedert in die Kategorien Erzeugungsmuster, Strukturmuster und Verhaltensmuster. Einige wichtige dieser Muster sind mittels Links hervorgehoben und werden in jeweils einem eigenen Kapitel vorgestellt.

Erzeugungsmuster	Strukturmuster	Verhaltensmuster
• Singleton (Einzelstück) • Factory Method (Fabrikmethode) • Abstract Factory (abstrakte Fabrik) • <i>Builder</i> (Erbauer) • <i>Prototype</i> (Prototyp)	• Composite (Kompositum) • Adapter (Adapter) • Facade (Fassade) • Proxy (Stellvertreter) • <i>Decorator</i> (Dekorierer) • <i>Bridge</i> (Brücke) • <i>Flyweight</i> (Fliegengewicht)	• Observer (Beobachter) • Strategy (Strategie) • <i>State</i> (Zustand) • <i>Command</i> (Kommando) • <i>Memento</i> (Memento) • <i>Visitor</i> (Besucher) • <i>Iterator</i> (Iterator) • <i>Interpreter</i> (Interpreter) • <i>Template Method</i> (Schablonenmethode) • <i>Mediator</i> (Vermittler) • <i>Chain of Responsibility</i> (Zuständigkeitskette)

Weitere Entwurfsmuster

Neben den GoF-Entwurfsmustern haben sich weitere Muster – wie z.B. [Dependency Injection](#) – im Sprachgebrauch der Softwareentwickler etabliert. Eine zusätzliche maßgebliche und frühe Quelle ist die Katalogisierung ([siehe hier](#)), die Martin Fowler in seinem Buch [Patterns of Enterprise Application Architecture](#) vorgenommen hat. In diesem Buch werden folgende Muster für den Entwurf von Anwendungen, die ihre Daten in relationalen Datenbanksystemen speichern, genannt. Auf einige dieser Muster wird in den jeweils verlinkten Kapiteln dieses Moduls eingegangen.

- **Domain Logic Patterns:** Transaction Script, Domain Model, Table Module, [Service Layer](#)
- **Data Source Architectural Patterns:** Table Data Gateway, Row Data Gateway, Active Record, Data Mapper

- **Object-Relational Behavioral Patterns:** Unit of Work, Identity Map, [Lazy Load](#)
- **Object-Relational Structural Patterns:** [Identity Field](#), [Foreign Key Mapping](#), [Association Table Mapping](#), Dependent Mapping, Embedded Value, Serialized LOB, [Single Table Inheritance](#), [Class Table Inheritance](#), [Concrete Table Inheritance](#), Inheritance Mappers
- **Object-Relational Metadata Mapping Patterns:** Metadata Mapping, Query Object, [Repository](#)
- **Web Presentation Patterns:** [Model View Controller](#), Page Controller, Front Controller, [Template View](#), Transform View, Two-Step View, Application Controller
- **Distribution Patterns:** Remote Facade, [Data Transfer Object](#)
- **Offline Concurrency Patterns:** [Optimistic Offline Lock](#), [Pessimistic Offline Lock](#), Coarse Grained Lock, Implicit Lock
- **Session State Patterns:** [Client Session State](#), Server Session State, Database Session State
- **Base Patterns:** Gateway, Mapper, Layer Supertype, Separated Interface, [Registry](#), Value Object, Money, Special Case, Plugin, [Service Stub](#), Record Set

Singleton

Beginnen wir mit einem der einfachsten Erzeugungsmuster: dem Einzelstück (engl. *Singleton*). Das Singleton-Muster ist eng mit der Frage verknüpft, wie zur Laufzeit einer objektorientierten Anwendung sichergestellt werden kann, dass max. 1 Objekt einer Klasse erzeugt werden kann.

Das Singleton-Muster soll über folgendes Beispiel motiviert werden: Stellen wir uns vor, dass wir in einer Java-Anwendung eine Verbindung zu einer Datenbank aufbauen wollen. Dazu wird die folgende Klasse `DatabaseConnection` entworfen, die eine Methode zum Verbindungsaufbau und weitere Methoden zum Abfragen und Verändern der Daten enthält. Diese fachlichen Methoden werden in den Code-Listings weiter unten ausgeblendet.

```
1 public class DatabaseConnection {
2     public boolean connect(String host, String user, String pass, String db) { /* ... */ }
3     public ResultSet query(String sqlStatement) { /* ...*/ }
4     /* ... */
5 }
```

Es ist ausreichend, wenn diese Datenbankverbindung beim Start der Anwendung nur einmal aufgebaut wird, und es demzufolge nur ein Objekt dieser Klasse gibt. Dieses Objekt entspricht einer globalen Variable. Es soll nicht möglich sein, dass aus Methoden in beliebigen anderen Klassen in unserem Projekt oder durch Dritte, die unsere Klasse weiterverwenden, `DatabaseConnection`-Instanzen erzeugt werden können. Um genau das zu verhindern, wird die Sichtbarkeit des Konstruktors der Klasse `DatabaseConnection` auf `private` gesetzt. Damit können nur noch aus der Klasse selbst Instanzen erzeugt werden. In der Klasse selbst wird ein statisches Attribut vom Typ `DatabaseConnection` angelegt, auf das über eine ebenfalls statische und öffentliche Methode zugegriffen werden kann. In folgendem Code wird das Attribut `instance` bereits beim Laden der Klasse instantiiert.

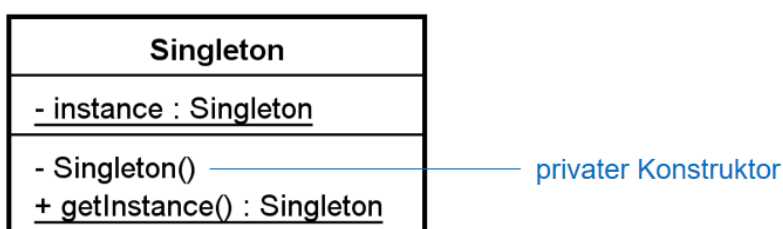
```
1 public class DatabaseConnection {
2     // Singleton object
3     private static DatabaseConnection instance = new DatabaseConnection();
4     // Private constructor
5     private DatabaseConnection() {}
6     // Static access to singleton object
7     public static DatabaseConnection getInstance() { return instance; }
8     /* ... */
9 }
```

Soll die Erzeugung des Singleton-Objekts nicht beim Laden der Klasse, sondern erst beim ersten Zugriff erfolgen (*Lazy Creation*), muss die Zugriffsmethode durch das Java-Schlüsselwort `synchronized` als kritischer Abschnitt geschützt werden, sodass nur ein paralleler *Thread* gleichzeitig diese Methode durchlaufen kann. Threads und kritische Abschnitte werden erst später im Kapitel [Synchronisation von Threads](#) behandelt.

Wenn das Beispiel der Datenbankverbindung verallgemeinert wird (d.h. die Klasse heißt nun `Singleton` anstatt `DatabaseConnection`), erhalten wir das geläufige Erzeugungsmuster Singleton, das zusichert, dass zur Laufzeit max. 1 Objekt einer Klasse instantiiert werden kann.

```
1 public class Singleton {
2     // Singleton object
3     private static Singleton instance;
4     // Private constructor
5     private Singleton() {}
6     // Static access to singleton object with lazy creation
7     public static synchronized Singleton getInstance() {
8         if (instance == null) { instance = new Singleton(); }
9         return instance;
10    }
11    /* ... */
12 }
```

Das folgende Modell entspricht dem obigen Java-Code. Unterstreichungen von Attributen oder Methoden in einem UML-Klassendiagramm bedeuten, dass diese statisch sind.



Entwurfsmuster Singleton

Das Muster realisiert eine einfache Zugriffskontrolle auf das Singleton-Objekt. Durch Unterklassen kann eine allgemeine Singleton-Klasse spezialisiert werden. Das Singleton-Muster sollte aber nicht als Ersatz für sämtliche globale Variablen genutzt werden. Wenn dies exzessiv erfolgt, würde zwar eine große Anzahl von Klassen entstehen, die objektorientierte Idee aber trotzdem untergraben werden.

Fabrikmethode

Die Fabrikmethode (engl. *Factory Method*) ist ein Erzeugungsmuster, das beschreibt, wie ein Objekt durch den Aufruf einer Methode anstatt eines Konstruktors erzeugt wird. Diese Methode ist Teil einer sogenannten Fabrik-Klasse, die für die Erzeugung von Objekten zuständig ist. Es ist irreführend, dass die Fabrikmethode im allgemeinen Sprachgebrauch von Software-Entwicklern sowohl eine beliebige statische Methode zur Objekterzeugung beschreibt als auch eines der ursprünglichen GoF-Entwurfsmuster.

Statische Fabrikmethode

Wir stellen hier zunächst die statische Fabrikmethode vor, die von Joshua Bloch in seinem bekannten Java-Lehrbuch "Effective Java" [\[Blo17\]](#) wie folgt definiert:

"A class can provide a public static factory method, which is simply a static method that returns an instance of the class." (Joshua Bloch)

Das folgende Code-Beispiel illustriert die statische Fabrikmethode anhand der Klasse `LoggerFactory`, die hier zwei unterschiedliche Logger (nämlich einen `ConsoleLogger` und einen `FileLogger`) erzeugen kann (Zeilen 5-12). Über ein Argument wird beispielhaft entschieden, welcher konkrete Logger erzeugt wird. Der `ConsoleLogger` ist hier der Default, falls die Methode `createLogger` ohne Argument aufgerufen wird (Zeilen 14-16). Dieser Default könnte auch aus einer Konfigurationsdatei eingelesen oder per *Dependency Injection* verknüpft werden.

LoggerFactory

```
1 public class LoggerFactory {
2
3     public enum LogType {CONSOLE, FILE};
4
5     public static MyLogger createLogger(LogType type) {
6         if (type.equals(LogType.CONSOLE)) {
7             return new ConsoleLogger();
8         } else if (type.equals(LogType.FILE)) {
9             return new FileLogger();
10        }
11        return null;
12    }
13
14    public static MyLogger createLogger() {
15        return createLogger(LogType.CONSOLE); // default logger
16    }
17 }
```

MyLogger

```
1 interface MyLogger {
2     void log(String message);
3     default void close() {}
4 }
```

ConsoleLogger

```
1 public class ConsoleLogger implements MyLogger {
2
3     @Override
4     public void log(String message) {
5         System.out.println(message);
6     }
7 }
```

FileLogger

```
1 public class FileLogger implements MyLogger {
2
3     BufferedWriter writer;
4
5     @Override
6     public void log(String message) {
7         try {
8             if (writer == null) {
9                 writer = new BufferedWriter(new FileWriter("log.txt", true));
10            }
11            writer.append(message + "\n");
12        } catch (IOException e) {}
13    }
14
15    @Override
16    public void close() {
17        try {
18            writer.close();
19        } catch (IOException e) {}
20    }
21 }
```

Client

```
1 class Client {
2
3     public static void main(String[] args) {
4         MyLogger a = new ConsoleLogger(); // constructor instantiation >> causes dependency to ConsoleLogger
5         a.log("Hello World!");
6
7         MyLogger b = LoggerFactory.createLogger(); // static factory method instantiation
8         b.log("Hello World!");
9     }
10 }
```

Das Interface `MyLogger` schreibt die gemeinsame Schnittstelle für alle konkreten Logger vor, die diese individuell implementieren. Der Client ist – im Gegensatz zur einfachen Instantiierung eines Loggers per Konstruktor – bei Verwendung der Fabrikmethode nicht mehr von einer konkreten Logger-Klasse abhängig, sondern kann ausschließlich auf dem Interface `MyLogger` arbeiten.

Die Stärke dieses Musters liegt in seiner Einfachheit. Im Vergleich zu dem GoF-Entwurfsmuster Fabrikmethode (s. unten) muss die Fabrik-Klasse nicht ihrerseits instanziiert werden, um Objekte zu erzeugen, und für die erzeugende Methode muss kein Interface definiert werden. Eine statische Fabrikmethode setzt natürlich voraus, dass die Programmiersprache Klassenmethoden ($\triangleq$ Schlüsselwort `static` in Java) kennt.

Die statische Fabrikmethode kann auch Gebrauch von der Java Reflection API machen, was in folgendem Code-Beispiel dargestellt ist (Zeile 4).

```
1 public class ReflectionLoggerFactory {
2
3     public static MyLogger createLogger(Class<? extends MyLogger> loggerClass) {
4         return loggerClass.getConstructor().newInstance(); // create instance by default constructor reflection
5     }
6
7     public static MyLogger createLogger() {
8         return createLogger(ConsoleLogger.class); // default logger
9     }
10 }
```

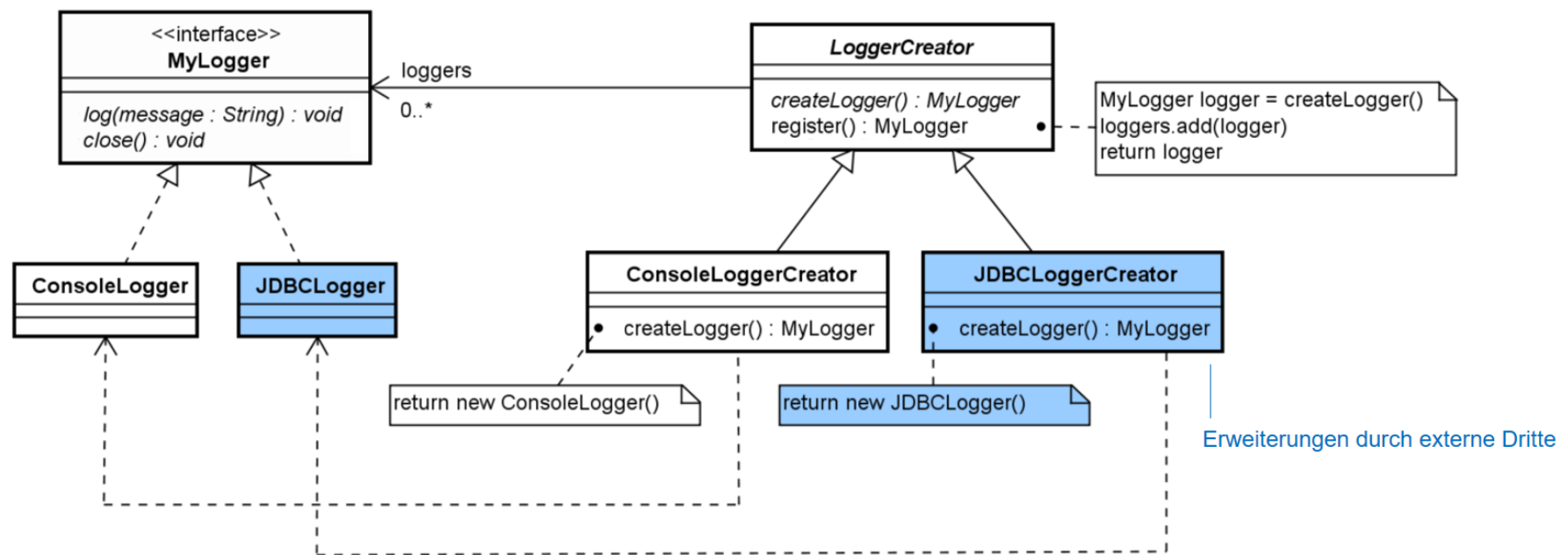
Auch die wirkliche Java Logging API setzt übrigens auf eine statische Fabrikmethode `getLogger` in der Klasse `java.util.logging.Logger`, um ein Logger-Objekt zu erzeugen:

```
1 import java.util.logging.Logger;
2 // ...
3 Logger logger = Logger.getLogger(getClass().getName());
4 logger.log(Level.INFO, "Hello World!");
```

Entwurfsmuster Fabrikmethode

Nun möchten wir die bisherige statische Fabrikmethode zu dem GoF-Entwurfsmuster Fabrikmethode erweitern. Das Ziel des Musters ist es, dass neue Klassen zu einer bestehenden Schnittstelle oder Vererbungshierarchie durch Dritte hinzugefügt werden können, und Objekte dieser neuen Klassen über eine zugehörige Fabrikmethode erzeugt werden können. Wir können uns z.B.

vorstellen, dass ein Dritter einen weiteren Logger für das obige Interface `MyLogger` implementieren möchte, z.B. einen `JDBCLogger` oder einen `RedisLogger`, um das Log in einer Datenbank abzulegen. Der Dritte hat aber keinen Zugriff auf unseren Code, insbesondere nicht auf die Klasse `LoggerFactory`. Demzufolge muss die `LoggerFactory` ihrerseits von außen erweiterbar sein. Eine Lösung könnte wie folgt aussehen, wobei die bisherige Klasse `LoggerFactory` in `LoggerCreator` umbenannt wird:



Anwendungsbeispiel für das Fabrikmethode-Muster

LoggerCreator

```

1  abstract class LoggerCreator {
2
3      Set<MyLogger> loggers;
4
5      abstract MyLogger createLogger();
6
7      MyLogger register() {
8          MyLogger logger = createLogger();
9          loggers.add(logger);
10         return logger;
11     }
12 }

```

ConsoleLoggerCreator

```

1  class ConsoleLoggerCreator extends LoggerCreator {
2
3      @Override
4      MyLogger createLogger() { return new ConsoleLogger(); }
5  }

```

Client

```

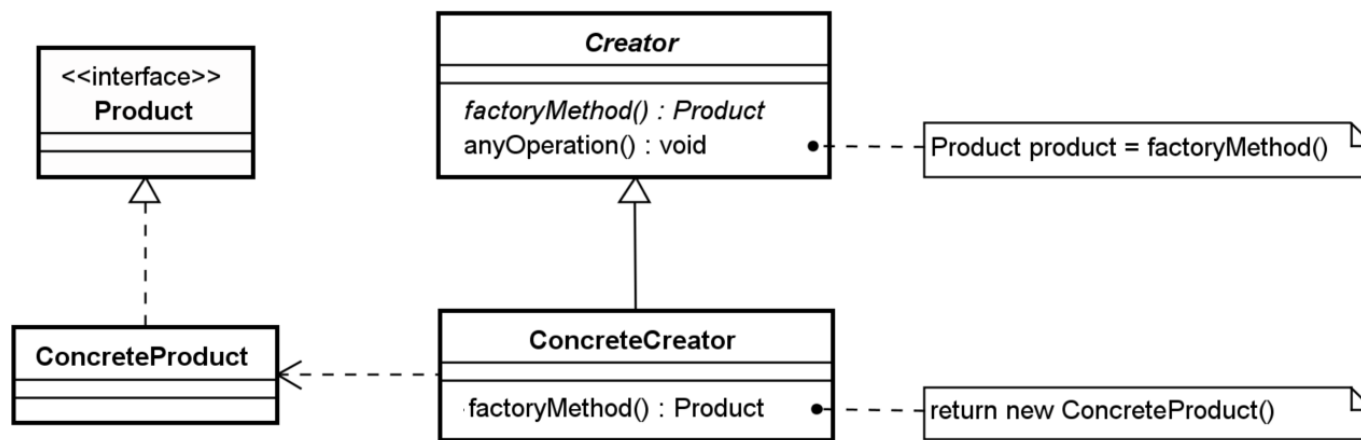
1  class Client {
2
3      public static void main(String[] args) {
4          LoggerCreator creator = new ConsoleLoggerCreator();
5
6          MyLogger a = creator.register();
7          a.log("Hello World!");
8
9          MyLogger b = creator.register();
10         b.log("Hello World!");
11
12         System.out.println("Created " + creator.loggers.size() + " loggers until now.");
13     }
14 }

```

Die abstrakte Klasse `LoggerCreator` definiert eine abstrakte Methode `createLogger`. Eine statische Methode zur Erzeugung der bekannten konkreten Logger gibt es hingegen nicht mehr. Es wird angenommen, dass zukünftig verschiedene konkrete Logger (wie z.B. der `JDBCLogger`) entstehen, die durch Dritte realisiert werden. Der Fokus liegt auf der Erweiterbarkeit. Die Oberklasse zur Objekterzeugung `LoggerCreator` kann auch Methoden beinhalten, die Verhalten implementieren, das für alle Logger gleich ist – hier beispielhaft die Methode `register`. Nur die konkreten Erzeuger (z.B. `JDBCLoggerCreator`) sind von einer zugehörigen Logger-Implementierung (z.B. `JDBCLogger`) abhängig.

Das folgende UML-Klassendiagramm verallgemeinert das Logger-Beispiel, sodass das allgemeingültige Muster der GoF-Fabrikmethode entsteht, das durch Erich Gamma et al. [\[GHJ+10\]](#) wie folgt definiert ist:

"Define an interface for creating an object, but let subclasses decide which class to instantiate. The factory method lets a class defer instantiation it uses to subclasses." (Erich Gamma et al.)



Entwurfsmuster Fabrikmethode

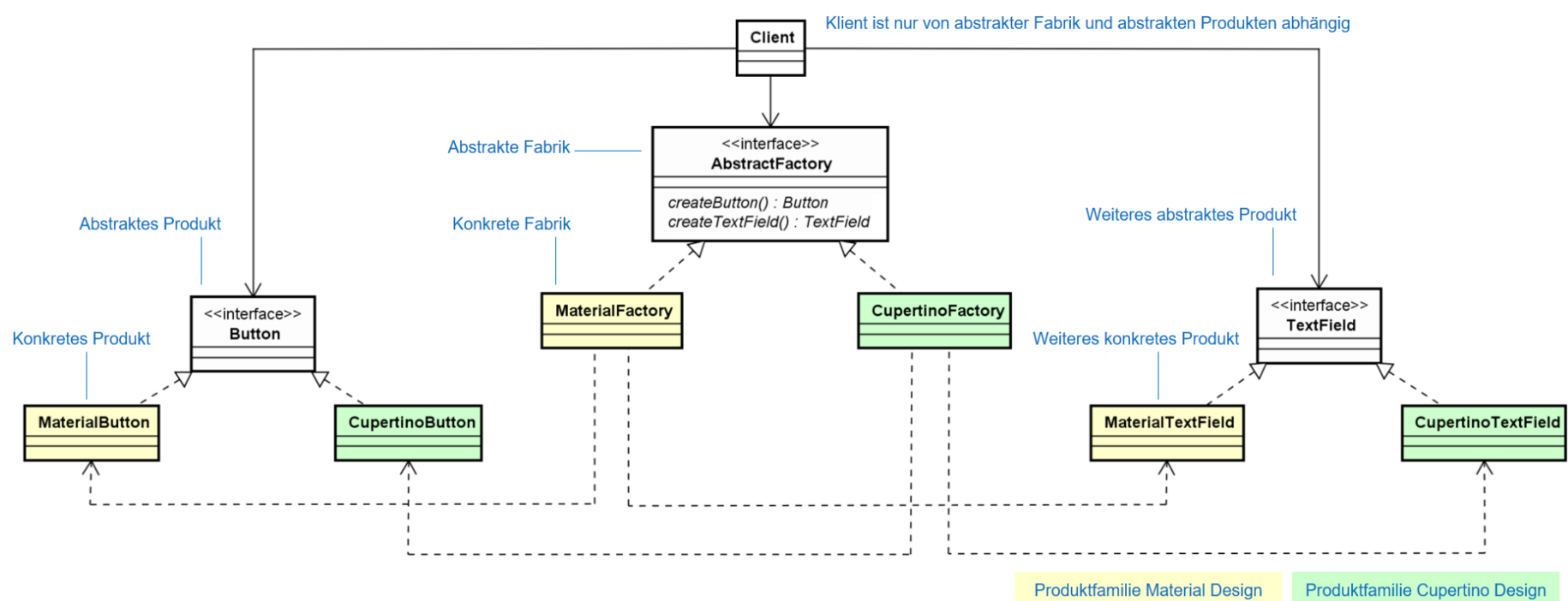
In diesem Muster gibt es eine allgemeine Oberklasse (oder Schnittstelle) zur Erstellung von ähnlichen Objekten (**Creator**). Den konkreten Erzeugern wird eine Fabrik-Methode (**factoryMethod**) zur Objekterzeugung vorgeschrieben. Die Oberklasse hat keine Abhängigkeit zu einem konkreten Erzeuger oder konkreten Produkt. Dieses Muster ist insbesondere dann sinnvoll, wenn die Oberklasse eine weitere Methode enthält, in der sie Verhalten implementiert, das unabhängig vom konkreten Typ eines Produkts für jedes Produkt gleichermaßen gilt. Im obigen UML-Klassendiagramm ist dies die Methode **anyOperation**. Aus objektorientierter Sicht gehört dieses gemeinsame Verhalten eigentlich eher zur Schnittstelle **Product** als zur Fabrik-Klasse. Es kann aber in der Praxis sein, dass die Fabrik-Klasse zusätzlich fachliches Verhalten für die Produkte implementiert, da letztere auf einfache Modellklassen reduziert sind, die über ihre Attribute nur Zustand erfassen, aber keine fachlichen Methoden enthalten sollen.

Der Nachteil des Entwurfsmusters Fabrikmethode ist, dass zwei parallele Spezialisierungshierarchien gepflegt werden müssen. Das ist der Preis für die Erweiterbarkeit durch Dritte. Im Gegensatz dazu ist eine statische Fabrikmethode immer dann als einfacherer Ansatz zu bevorzugen, wenn alle Produktklassen vorab bekannt sind oder die Erweiterbarkeit durch Dritte nicht wichtig ist.

Wenn die zu erzeugenden Produkte sich in den notwendigen Argumenten beim Konstruktoraufbau unterscheiden und sich daher zur Instantiierung nicht auf eine gemeinsame Schnittstelle einigen können, kann das Entwurfsmuster [Erbauer \(engl. Builder\)](#) herangezogen werden, ggf. in Kombination mit der Fabrikmethode.

Abstrakte Fabrik

Wenn die Fabrik-Klasse nicht nur ein Produkt, sondern gleich eine ganze Familie von zusammengehörigen Produkten erzeugen soll, wird die Fabrikmethode schnell zum Entwurfsmuster [Abstrakte Fabrik \(engl. Abstract Factory\)](#) ausgebaut. Ein beliebtes Beispiel ist hier im Kontext von UI-Frameworks die Erzeugung von UI-Elementen (wie Buttons, Textfelder, usw.), die jeweils einem bestimmten Design-Stil folgen sollen (wie Cupertino oder Material Design). Hierbei entsteht für jedes Produkt einer Produktfamilie eine separate Spezialisierungshierarchie. Die konkreten Produkte einer Familie werden alle aus einer gemeinsamen Fabrik-Klasse erzeugt, die dadurch einen konsistenten Design-Stil gewährleistet. Der Client arbeitet ausschließlich auf abstrakten Produkten, die er über eine konkrete Fabrik-Klasse erzeugt. Die Idee der Abstrakten Fabrik wird durch das folgende UML-Klassendiagramm und das zugehörige Code-Beispiel verdeutlicht.



Anwendungsbeispiel für das Entwurfsmuster Abstrakte Fabrik

Abstrakte Fabrik/Produkte

```
1 interface AbstractFactory {
2     Button createButton();
3     TextField createTextField();
4 }
5
6 interface Button {} // abstract product
7 interface TextField {} // another abstract product
```

Konkrete Fabrik/Produkte

```
1 class MaterialFactory implements AbstractFactory {
2
3     @Override
4     public Button createButton() { return new MaterialButton(); }
5
6     @Override
7     public TextField createTextField() { return new MaterialTextField(); }
8 }
9
10 class MaterialButton implements Button {} // concrete product
11 class MaterialTextField implements TextField {} // another concrete product
```

Client

```
1 AbstractFactory factory = new MaterialFactory();
2 Button button = factory.createButton();
3 TextField textField = factory.createTextField();
```

Die gezeigten Code-Beispiele zum Entwurfsmuster Fabrikmethode finden sich im Verzeichnis `/patterns/factory-method` des Modul-Repository.

Dependency Injection

Dependency Injection lässt sich am besten durch "Einbringen von Abhängigkeiten" übersetzen und ist in der Software-Entwicklung ein bekannter Begriff, bei dem es wie bei der Fabrikmethode um die Auslagerung von Konstruktoraufrufen zur Objekterzeugung geht. Die Ziel der Dependency Injection ist es, die Abhängigkeiten zwischen Klassen, die durch `import`-Anweisungen entstehen, zu reduzieren, und die Klassen dadurch zu entkoppeln. [Lose Kopplung](#) von Klassen (oder Komponenten im Allgemeinen) hat den Vorteil, dass sich Änderungen einfacher durchführen lassen, da diese sich nur lokal auswirken.

Dependency Injection ist kein GoF-Muster. Den Begriff hat Martin Fowler in seinem Artikel "[Inversion of Control Containers and the Dependency Injection pattern](#)" geprägt [\[Fow04\]](#). Er suchte nach einem Begriff für die Entkopplung bei der Objekterzeugung, der sich von dem allgemeinen Begriff *Inversion of Control (IoC)* (s. Kapitel [Spring](#)) abgrenzen lässt. Dependency Injection folgt dem [Prinzip der eindeutigen Verantwortlichkeit \(engl. Single-Responsibility-Prinzip\)](#) [\[Mar18\]](#), das nach Robert Martin besagt, dass jede Klasse in der objekt-orientierten Programmierung nur eine wesentliche Aufgabe erfüllen soll und entsprechend dieser Aufgabe weiterentwickelt wird:

"There should never be more than one reason for a class to change." (Robert Martin)

Demzufolge ist die Situation im folgenden Code-Beispiel zu vermeiden: Die Klasse `MyClass` verwendet intensiv das Interface `MyLogger`, ist aber aufgrund eines direkten Konstruktoraufrufs auch von der konkreten Klasse `ConsoleLogger` abhängig, die dieses Interface implementiert. Wenn nun die konkrete Klasse ausgetauscht werden soll, muss die Klasse `MyClass` geändert werden, obwohl sich an ihrer eigentlichen Funktionalität nichts geändert hat.

```
1 import MyLogger;
2 import ConsoleLogger; // this dependency should be avoided
3
4 class MyClass {
5     MyLogger logger = new ConsoleLogger();
6
7     // ...
8 }
```

Dependency Injection hat zum Ziel diese Abhängigkeit aufzulösen. Dies erfolgt im einfachsten Fall über einen Konstruktor oder über eine Setter-Methode.

```
1 import MyLogger; // no import of concrete implementation required
2
3 class MyClass {
4     MyLogger logger;
5
6     MyClass(MyLogger logger) { this.logger = logger; } // constructor dependency injection
7
8     void setLogger(MyLogger logger) { this.logger = logger; } // setter dependency injection
9
10    // ...
11 }
```

Nun stellt sich die Frage, wer das Objekt erzeugt und von außen der Klasse `MyClass` injiziert. Allgemein ist anzustreben, die Konfiguration darüber, welche konkrete Implementierungsklasse je Interface genutzt werden soll, an einer zentralen Stelle zusammenzuführen, z.B. in einer Konfigurationsklasse oder in einer Konfigurationsdatei (z.B. XML, YAML). Eine derartige Konfiguration wird i.d.R. von einem Framework in seiner Funktion als *IoC-Container* vorgesehen und verwaltet. Die Ansätze je Framework können sich dabei leicht unterscheiden. Im Folgenden werden die Dependency Injection-Ansätze von [Spring](#) und [Google Guice](#) vorgestellt. Beide nutzen die Java Reflection API, um zu ermitteln, welche konkrete Klasse an ein Interface gebunden werden kann. Guice spricht hier von *Binding*, während der entsprechende Begriff in Spring *Wiring* heißt.

Spring

Damit das Spring Framework die Objekte einer Klasse in Konstruktoren, in Methoden oder direkt in Attribute injizieren kann, sind diese dem Framework als sogenannte [Beans](#) bekanntzumachen. Spring definiert Beans wie folgt, wobei das Framework selbst die Funktion als IoC-Container übernimmt:

"In Spring, the objects that form the backbone of your application and that are managed by the Spring IoC container are called beans. A bean is an object that is instantiated, assembled, and otherwise managed by a Spring IoC container. Otherwise, a bean is simply one of many objects in your application. Beans, and the dependencies among them, are reflected in the configuration metadata used by a container." (Spring Docs)

Alle Klassen, die mit der Annotation `@Component` (aus dem Package `org.springframework.stereotype`) oder einer ihrer Spezialisierungen wie `@Service`, `@Repository` oder `@Controller` versehen sind, werden von Spring automatisch als Beans erkannt und registriert. Die registrierten Beans können zur Laufzeit über den Anwendungskontext erreicht werden. In dem folgenden Code-

Beispiel soll ein Objekt der Klasse `CapsLockConsoleLogger` als Bean an anderen Stellen eingebunden werden können. Daher ist die Klasse als `@Component` annotiert (Zeile 1).

```
1 | @Component
2 | class CapsLockConsoleLogger implements MyLogger {
3 |
4 |     @Override
5 |     public void log(String message) { System.out.println(message.toUpperCase()); }
6 | }
```

In der folgenden Klasse `SpringClient` wird diese Bean in das Attribut `MyLogger logger` injiziert (Zeilen 4-5), d.h. genau an dieser Stelle findet die Dependency Injection statt. Obwohl kein Konstruktoraufwurf zu sehen ist, kann der Logger später verwendet werden (Zeile 13). Die Annotation `@Autowired` sorgt dafür, dass das Framework per Reflection nach einer passenden Bean für das Interface `MyLogger` sucht und diese Bean an das Attribut bindet. Wichtig ist, dass genau eine passende Bean gefunden wird – und nicht keine oder mehrere.

Die Klasse `SpringClient` ist hier eine einfache [Spring Boot-Anwendung](#), die als solche annotiert ist (Zeile 1) und wie üblich gestartet wird (Zeile 8). Durch den Aufruf von `SpringApplication.run()` wird ein `ApplicationContext`-Objekt erzeugt, das den Spring IoC-Container repräsentiert und über das als zentraler Anwendungskontext alle Beans verwaltet werden und erreichbar sind.

```
1 | @SpringBootApplication
2 | class SpringClient { // this example is a Spring Boot application
3 |
4 |     @Autowired
5 |     MyLogger logger;
6 |
7 |     public static void main(String[] args) {
8 |         ApplicationContext ctx = SpringApplication.run(SpringClient.class, args);
9 |     }
10 |
11 |     @Bean
12 |     CommandLineRunner run(ApplicationContext ctx) {
13 |         return args -> logger.log("Hello World!");
14 |     }
15 | }
```

Wenn mehrere Beans dieselbe Schnittstelle erfüllen, muss über einen qualifizierenden Namen bestimmt werden, welche Bean das Framework injizieren soll. Dazu kann eine Konfigurationsklasse angelegt werden – wie z.B. die folgende Klasse `MyConfiguration`, in der die 3 Beans definiert sind (2 vom Typ `MyLogger`, 1 vom Typ `String`).

MyConfiguration

```
1 | @Configuration
2 | class MyConfiguration {
3 |
4 |     @Bean
5 |     MyLogger loggerA() { return new CapsLockConsoleLogger(); }
6 |
7 |     @Bean
8 |     MyLogger loggerB() { return new TimestampConsoleLogger(dateFormat()); }
9 |
10 |     @Bean
11 |     String dateFormat() { return "yyyy-MM-dd HH:mm:ss"; }
12 | }
```

TimestampConsoleLogger

```
1 | @Component
2 | class TimestampConsoleLogger implements MyLogger {
3 |
4 |     SimpleDateFormat dateFormat;
5 |
6 |     TimestampConsoleLogger(String pattern) { dateFormat = new SimpleDateFormat(pattern); }
7 |
8 |     @Override
9 |     public void log(String message) { System.out.println(dateFormat.format(new Date()) + "\t" + message); }
10 | }
```

Die Beans können über eine `@Qualifier`-Annotation und ihren Methodennamen wie folgt injiziert werden.

```
1 | @Autowired @Qualifier("loggerA")
2 | MyLogger logger;
```

Alternativ kann eine Bean über ihren Methodennamen beim Anwendungskontext abgerufen werden.

```
1 | logger = ctx.getBean("loggerA", MyLogger.class); // request bean from application context
2 | logger.log("Hello World!");
```

Anstatt in der Konfigurationsklasse `MyConfiguration` können die Beans auch in einer entsprechenden XML-Konfigurationsdatei definiert werden, welche dann als Anwendungskontext zu laden oder diesem hinzuzufügen ist.

beans.xml

```
1 | <beans xmlns="http://www.springframework.org/schema/beans" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2 |       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-
3 | beans.xsd">
4 |
5 |     <bean id="loggerA" class="CapsLockConsoleLogger"></bean>
6 |
7 |     <bean id="loggerB" class="TimestampConsoleLogger">
8 |         <constructor-arg ref="dateFormat"></constructor-arg>
9 |     </bean>
10 |
11 |     <bean id="dateFormat" class="java.lang.String">
12 |         <constructor-arg value="yyyy-MM-dd HH:mm:ss"></constructor-arg>
13 |     </bean>
14 | </beans>
```

SpringXMLConfigClient

```
1 | class SpringXMLConfigClient {
2 |
3 |     public static void main(String[] args) {
4 |         ApplicationContext ctx = new ClassPathXmlApplicationContext("beans.xml");
5 |         MyLogger logger = ctx.getBean("loggerA", MyLogger.class);
6 |         logger.log("Hello World!");
7 |     }
8 | }
```

Google Guice

Google Guice bietet eine bewährte Alternative für Dependency Injection, falls in einem Projekt bewusst ohne Spring gearbeitet werden soll. Als Guice in 2008 durch Google veröffentlicht wurde, war es das erste Framework, das Dependency Injection in Java mittels Annotationen möglich machte. In Guice wird die Konfigurationsklasse, die Interfaces an konkrete Implementierungsklassen bindet als Modul bezeichnet.

"Guice uses bindings to map types to their implementations. A module is a collection of bindings specified using fluent, English-like method calls." (Guice Docs)

Der folgende Beispiel-Code zeigt das Modul `MyModule`, das das Interface `MyLogger` an seine Implementierung `CapsLockConsoleLogger` bindet.

```
1 | class MyModule extends AbstractModule { // a Guice configuration module
2 |
3 |     @Override
4 |     protected void configure() {
5 |         bind(MyLogger.class).to(CapsLockConsoleLogger.class);
6 |         // more bindings follow here ...
7 |     }
8 | }
```

Diese Konfigurationsklasse wird in dem folgenden Code-Beispiel verwendet, um ein `Injector`-Objekt zu erzeugen (Zeile 9). Den Objekten, die später über die Methode `getInstance` dieses `Injector`-Objekts erzeugt werden (Zeile 12), können ihre Abhängigkeiten über die Annotation `@Inject` injiziert werden – wie z.B. bei dem Attribut `MyLogger logger` (Zeilen 3-4).


```

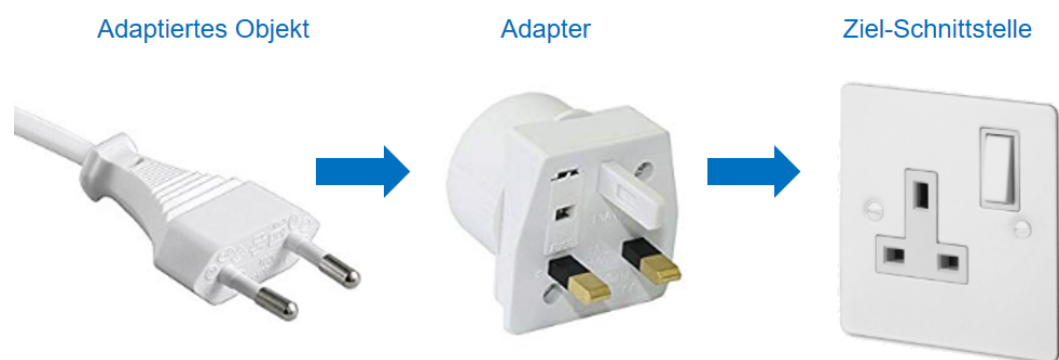
1  class GuiceClient {
2
3      @Inject
4      MyLogger logger;
5
6      public static void main(String[] args){
7
8          // create an injector based on module configuration
9          Injector injector = Guice.createInjector(new MyModule());
10
11         // create objects using the injector
12         GuiceClient client = injector.getInstance(GuiceClient.class);
13         client.logger.log("Hello World!");
14     }
15 }

```

Die gezeigten Code-Beispiele bezüglich Dependency Injection finden sich im Verzeichnis `/patterns/dependency-injection` des Modul-Repository.

Adapter

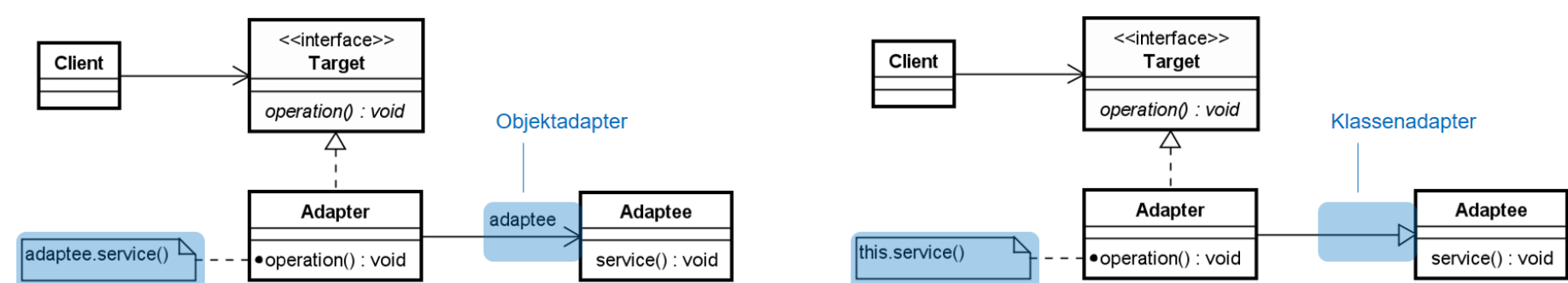
Ein Adapter hilft zwei zunächst inkompatible Schnittstellen miteinander zu verbinden. Die Analogie dieses Entwurfsmusters zu einem Stromnetzadapter für Auslandsreisen ist naheliegend. Der Stromnetzadapter ermöglicht es, eine Steckdose mit einem Stecker für ein elektrisches Gerät zu verbinden, auch wenn der Stecker nicht direkt passt, weil er eine andere Schnittstelle aufweist.



Analogie zum Adapter-Muster

In der Software-Entwicklung wird ein Adapter (engl. auch *Wrapper*) eingesetzt, wenn eine existierende Klasse verwendet werden soll, deren Interface nicht dem vorgesehenen Interface entspricht. Die existierende Klasse soll oder kann nicht verändert werden, insbesondere wenn sie Teil einer externen Bibliothek ist, die von Dritten entwickelt wird. Die zu adaptierende Klasse aus der Bibliothek ist häufig von Interesse, da sie komplexes Verhalten implementiert, das wiederverwendet werden soll. Das Prinzip der Wiederverwendung ist grundsätzlich effizient, wenn externe Bibliotheken in ein Projekt eingebunden werden, von denen bekannt ist, dass sie umfänglich getestet worden sind und dass sie sich in anderen Projekten bereits praktisch bewährt haben. Im Java-Umfeld werden Bibliotheken i.d.R. aus einem [zentralen Repository](#) über Build-Management-Werkzeuge wie [Maven](#) oder [Gradle](#) automatisiert in ein Projekt eingebunden.

Das Adapter-Muster gibt es in zwei Varianten: Objektadapter und Klassenadapter. In beiden Varianten wird die Ziel-Schnittstelle **Target** von der Adapter-Klasse **Adapter** implementiert. Weiterhin ruft der Adapter in der von **Target** vorgeschriebenen Methode **operation** die Methode **service** der adaptierten Klasse **Adaptee** auf. Es können natürlich auch mehrere Methoden adaptiert werden. Beim Objektadapter besteht eine Assoziation zwischen Adapter und adaptierter Klasse, während beim Klassenadapter der Adapter die adaptierte Klasse erweitert. Der Klassenadapter ist in Java nicht zu realisieren, falls **Target** kein Interface, sondern eine zustandsbehaftete Klasse ist, da in diesem Fall Mehrfachvererbung erforderlich wäre. Die folgenden UML-Klassendiagramme visualisieren Objekt- und Klassenadapter im Vergleich zueinander.



Entwurfsmuster Adapter (in Varianten Objekt- und Klassenadapter)

In dem folgenden Beispiel zur Motivation eines Adapters möchten wir die Fläche und den Umfang für unterschiedliche 2-dimensionale Formen bestimmen. Dazu haben wir das Interface **Shape** spezifiziert und bereits Implementierungen für einfache Formen wie Kreis, Rechteck und Dreieck entwickelt.

Shape

```
1 interface Shape {  
2     Double perimeter();  
3     Double area();  
4 }
```

Circle

```
1 class Circle implements Shape {
2     Point2D center;
3     Double radius;
4
5     Circle(Point2D center, Double radius) {
6         this.center = center;
7         this.radius = radius;
8     }
9
10    @Override
11    public Double area() { return Math.PI * radius * radius; }
12
13    @Override
14    public Double perimeter() { return 2 * Math.PI * radius; }
15 }
```

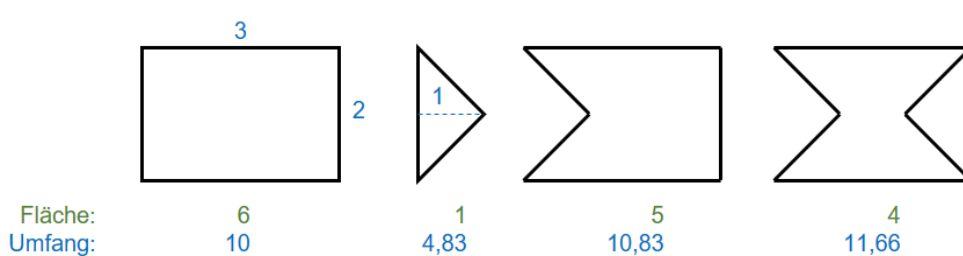
Rectangle

```
1 class Rectangle implements Shape {
2     Point2D topLeft;
3     Double width, height;
4
5     Rectangle(Point2D topLeft, Double width, Double height) {
6         this.topLeft = topLeft;
7         this.width = width;
8         this.height = height;
9     }
10
11    @Override
12    public Double perimeter() { return 2 * (width + height); }
13
14    @Override
15    public Double area() { return width * height; }
16 }
```

Triangle

```
1 class Triangle implements Shape {
2     Point2D p1, p2, p3; // points
3     Double e1, e2, e3; // edges
4
5     Triangle(Point2D p1, Point2D p2, Point2D p3) {
6         this.p1 = p1; this.p2 = p2; this.p3 = p3;
7         e1 = p1.distance(p2); e2 = p2.distance(p3); e3 = p3.distance(p1);
8     }
9
10    @Override
11    public Double perimeter() { return e1 + e2 + e3; }
12
13    @Override
14    public Double area() {
15        Double s = perimeter() / 2;
16        return Math.sqrt(s * (s - e1) * (s - e2) * (s - e3));
17    }
18 }
```

Das Interface **Shape** nutzen wir in unserer Anwendung (**Client**), um über verschiedene Formen zu iterieren und diese über ihre gemeinsame Schnittstelle zu verarbeiten. Es soll noch eine **Shape**-Implementierung für beliebige Polygone hinzugefügt werden. Ein Polygon besteht aus einer Reihe von Punkten, die über Kanten miteinander verbunden werden. Der **Client** soll zur Veranschaulichung die Fläche und den Umfang der folgenden 4 Formen berechnen.



Polygone zur Motivation eines Adapters

Die Bestimmung der Fläche eines Polygons ist nicht trivial. Aus diesem Grund möchten wir die Bibliothek [Apache Commons Math](#) einbinden, die eine API für sehr viele mathematische Grundlagen bereitstellt. Wir interessieren uns für die Methode **getSize** der Klasse [PolygonsSet](#), über die die Fläche für ein Polygon berechnet werden kann. Das Code-Beispiel zeigt, wie das Polygon über einen

Objektadapter (`PolygonObjectAdapter`) oder alternativ über einen Klassenadapter (`PolygonClassAdapter`) realisiert werden kann. In der Klasse `Client` ist zu erkennen, dass auch die neuen Adapter-Klassen das Interface `Shape` erfüllen.

PolygonObjectAdapter

```
1 import org.apache.commons.math3.geometry.euclidean.twod.PolygonsSet;
2 import org.apache.commons.math3.geometry.euclidean.twod.Vector2D;
3
4 class PolygonObjectAdapter implements Shape {
5     PolygonsSet adaptee;
6
7     PolygonObjectAdapter(Point2D... points) {
8         Vector2D[] vectors = Stream.of(points).map(p -> new Vector2D(p.getX(), p.getY())).toArray(size -> new Vector2D[size]);
9         adaptee = new PolygonsSet(.1, vectors);
10    }
11
12    @Override
13    public Double perimeter() { return adaptee.getBoundarySize(); }
14
15    @Override
16    public Double area() { return adaptee.getSize(); }
17 }
```

PolygonClassAdapter

```
1 import org.apache.commons.math3.geometry.euclidean.twod.PolygonsSet;
2 import org.apache.commons.math3.geometry.euclidean.twod.Vector2D;
3
4 class PolygonClassAdapter extends PolygonsSet implements Shape {
5
6     PolygonClassAdapter(Point2D... points) {
7         super(.1, Stream.of(points).map(p -> new Vector2D(p.getX(), p.getY())).toArray(size -> new Vector2D[size]));
8     }
9
10    @Override
11    public Double perimeter() { return this.getBoundarySize(); }
12
13    @Override
14    public Double area() { return this.getSize(); }
15 }
```

Client

```
1 class Client {
2
3     public static void main(String[] args) {
4         Shape s1 = new Rectangle(new Point2D(0, 0), 3., 2.);
5         Shape s2 = new Triangle(new Point2D(0, 0), new Point2D(0, 2), new Point2D(1, 1));
6         Shape s3 = new PolygonObjectAdapter(new Point2D(0, 0), new Point2D(3, 0), new Point2D(3, 2), new Point2D(0, 2), new
7 Point2D(1, 1));
8         Shape s4 = new PolygonClassAdapter(new Point2D(0, 0), new Point2D(3, 0), new Point2D(2, 1), new Point2D(3, 2), new
9 Point2D(0, 2), new Point2D(1, 1));
10
11         List.of(s1, s2, s3, s4).forEach(s -> {
12             System.out.printf("%s: perimeter = %.2f, area = %.2f\n", s.getClass().getSimpleName(), s.perimeter(), s.area());
13         });
14     }
15 }
```

Die gezeigten Code-Beispiele zum Entwurfsmuster Adapter finden sich im Verzeichnis `/patterns/adapter` des Modul-Repository.

Kompositum

Das Kompositum (engl. *Composite*) ist ein Strukturmuster, das angewendet wird, um Objekte, die sich ein gemeinsames Interface teilen, zu einer hierarchischer Baumstruktur zusammenzufügen. Die grundlegende Idee des Kompositum-Musters ist es, sowohl primitive Objekte als auch zusammengesetzte Objekte, die aus mehreren primitiven Objekten bestehen, in einem Interface zu repräsentieren. Dadurch können einzelne Objekte und ihre Kompositionen einheitlich behandelt werden.

Es soll also folgende Frage durch das Muster beantwortet werden: Wie können in einer objektorientierten Sprache einzelne Objekte als Teile (= Komponenten) geschickt zu einem Ganzen (= Kompositum) zusammengesetzt werden?

Die Herleitung des Musters soll wieder über ein Beispiel erfolgen. Es liegt die folgende Implementierung vor, um die 3 grundlegenden Formen **Line**, **Circle** und **Text** grafisch darzustellen. Aus diesen primitiven Formen sollen komplexe, zusammengesetzte Formen erzeugt werden können.

Line

```
1 | class Line {
2 |     GraphicsContext gc;
3 |     Point2D a, b;
4 |     void draw() { gc.strokeLine(a.getX(), a.getY(), b.getX(), b.getY()); }
5 | }
```

Circle

```
1 | class Circle {
2 |     GraphicsContext gc;
3 |     Point2D a;
4 |     Double radius;
5 |     void draw() { gc.strokeOval(a.getX(), a.getY(), radius, radius); }
6 | }
```

Text

```
1 | class Text {
2 |     GraphicsContext gc;
3 |     Point2D a;
4 |     String value;
5 |     void draw() { gc.fillText(value, a.getX(), a.getY()); }
6 | }
```

Es bietet sich an, ein gemeinsames Interface für die Formen zu spezifizieren, das die Methode **draw** für jede Form vorschreibt. Da sich alle Formen auch ein gemeinsames Attribut vom Typ **GraphicsContext** teilen und ein Interface zustandslos ist, wird eine abstrakte Klasse **Shape** eingeführt.

Shape

```
1 | abstract class Shape {
2 |     GraphicsContext gc;
3 |     abstract void draw();
4 | }
```

Line

```
1 | class Line extends Shape {
2 |     Point2D a, b;
3 |
4 |     @Override
5 |     void draw() { gc.strokeLine(a.getX(), a.getY(), b.getX(), b.getY()); }
6 | }
```

Circle

```
1 | class Circle extends Shape {
2 |     Point2D a;
3 |     Double radius;
4 |
5 |     @Override
6 |     void draw() { gc.strokeOval(a.getX(), a.getY(), radius, radius); }
7 | }
```

Text

```

1 class Text extends Shape {
2     Point2D a;
3     String value;
4
5     @Override
6     void draw() { gc.strokeText(value, a.getX(), a.getY()); }
7 }

```

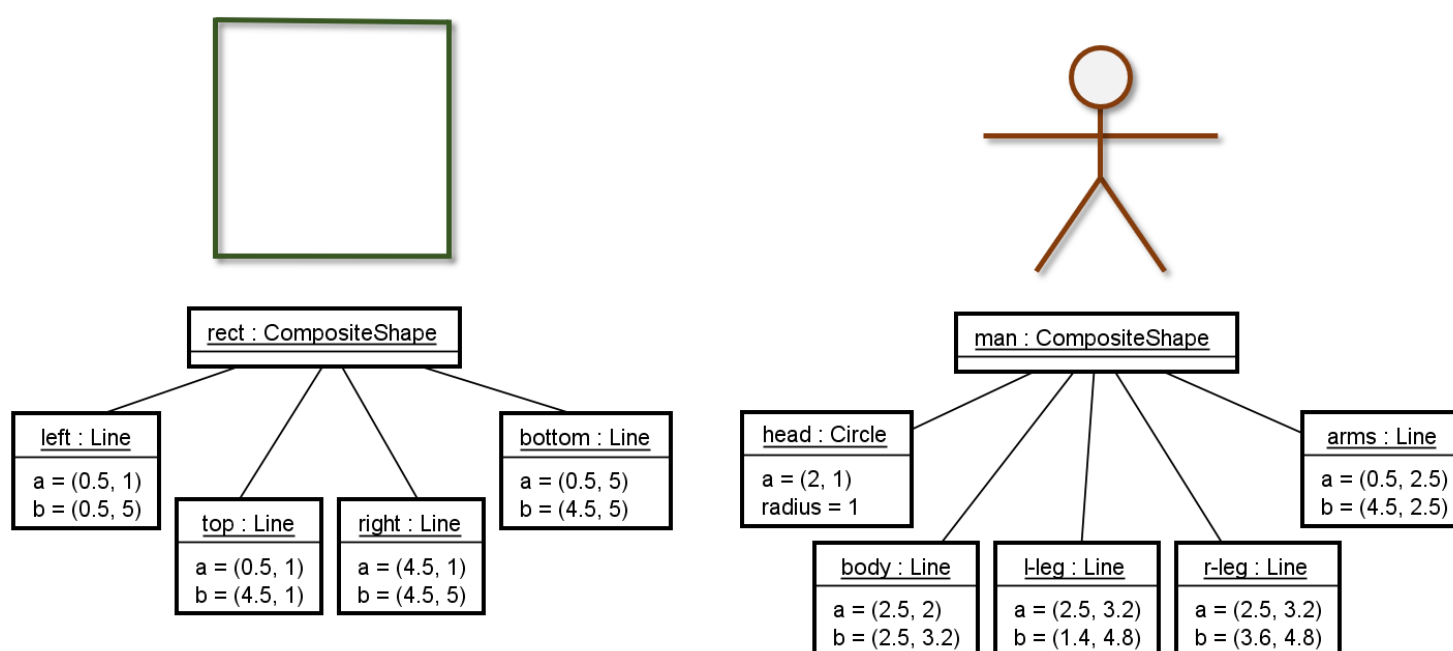
Das eigentliche Kompositum ist eine zusammengesetzte Form, die ihrerseits aus den anderen bisher zur Verfügung stehenden Formen besteht.

```

1 class CompositeShape extends Shape {
2     Set<Shape> components = new HashSet<>();
3
4     @Override
5     void draw() { components.forEach(c -> c.draw()); }
6 }

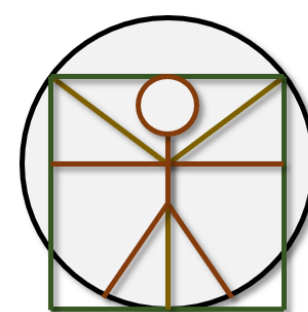
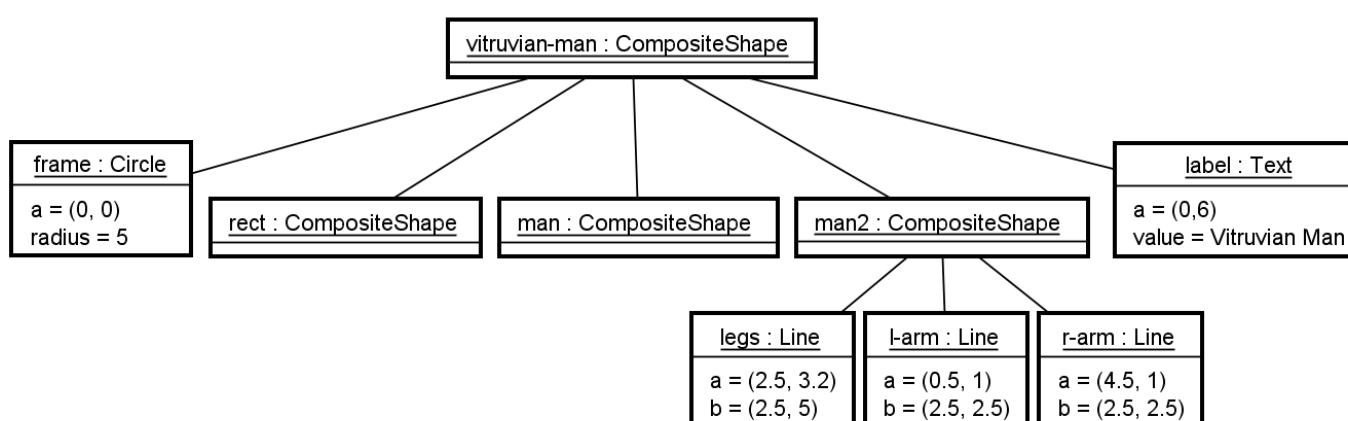
```

Beispiele für zusammengesetzte Formen sind das folgend dargestellte Rechteck, das aus 4 Linien besteht, und das Strichmännchen, das aus einem Kreis und 4 Linien besteht.



Zusammengesetzte Formen

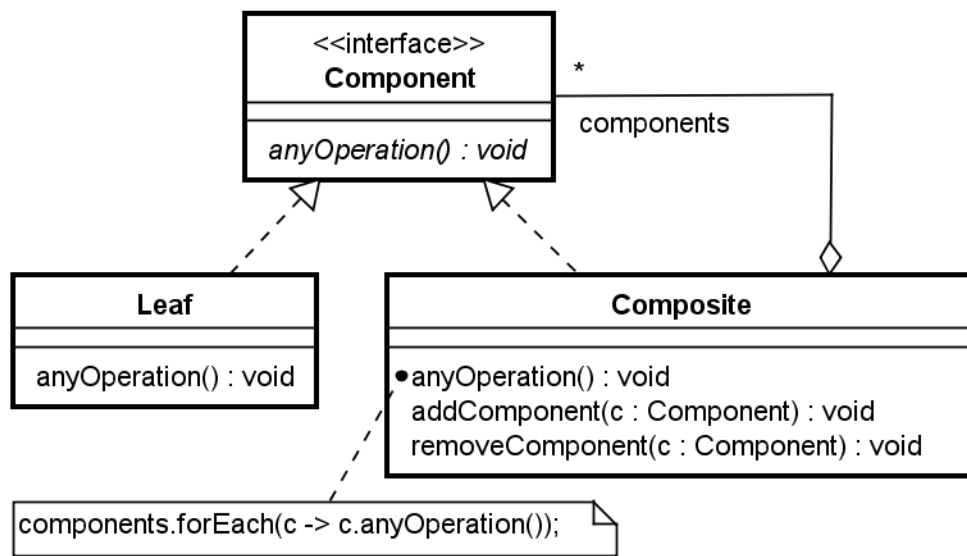
Dadurch, dass das Kompositum als Teile nicht nur primitive Formen, sondern auch andere zusammengesetzte Formen aufnehmen kann, entsteht eine Teil-Ganzes-Hierarchie, die in ihrer Tiefe grundsätzlich nicht beschränkt ist. Die Beziehungen der beteiligten Objekte stellen eine Baumstruktur dar, da zyklische Beziehungen i.d.R. nicht erwünscht sind und daher beim Einfügen von Teilen zu einem Ganzen verhindert werden sollten. Da es sich um eine Baumstruktur handelt, nennen wir die Basisformen auch Blätter (engl. *Leaf*). Das folgende Objektdiagramm verdeutlicht eine derartige verschachtelte Baumstruktur, die die obigen Formen weiterverwendet. Die resultierende komplexe Form ist daneben dargestellt. Auf Basis weniger Basisformen können so beliebig komplexe Kompositionen zusammengestellt werden.



Vitruvian Man

Zusammengesetzte Form "Vitruvian Man"

Das Entwurfsmuster Kompositum ist nun eine Verallgemeinerung des obigen Beispiels für die 2-dimensionalen Formen, d.h. die zu implementierende Methode heißt allgemein **anyOperation** anstatt **draw**, das gemeinsame Interface heißt **Component** anstatt **Shape**, usw. Der Klasse Kompositum werden noch Methoden hinzugefügt, um eine Komponente aufzunehmen bzw. zu entfernen. Das folgende UML-Klassendiagramm des Kompositum-Musters entspricht dem darunter stehenden Code.



Entwurfsmuster Kompositum

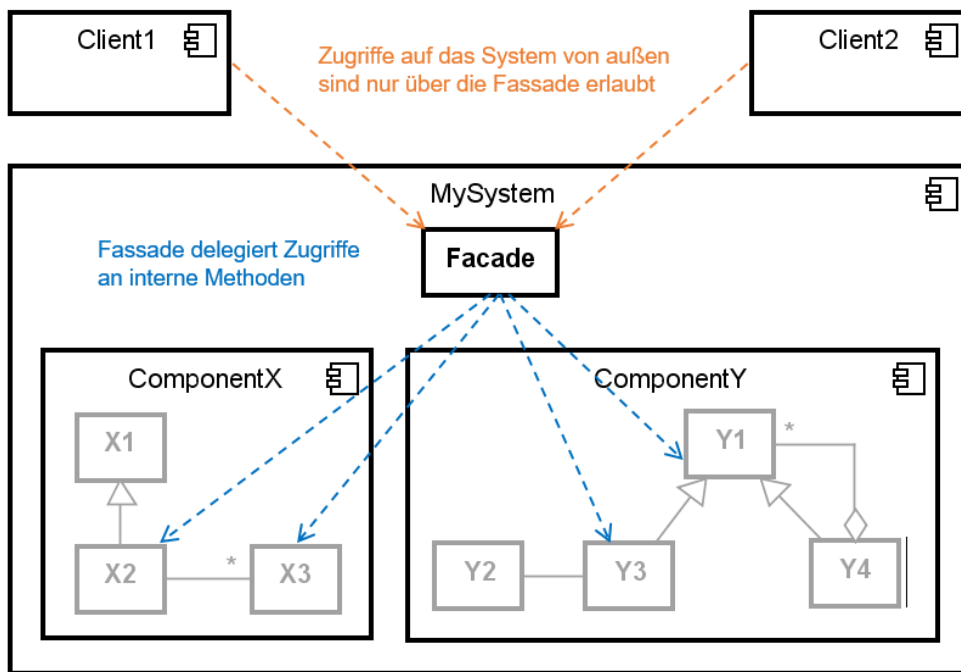
```

1  interface Component {
2      void anyOperation();
3  }
4
5  class Leaf implements Component {
6      @Override
7      void anyOperation() { /* ... */ }
8  }
9
10 class Composite implements Component {
11     Set<Component> components = new HashSet<>();
12
13     @Override
14     void anyOperation() { components.forEach(c -> c.anyOperation()); }
15
16     void addComponent(Component c) { components.add(c); }
17     void removeComponent(Component c) { components.remove(c); }
18 }

```

Fassade

Wie der Adapter ist auch die Fassade (engl. *Facade*) einer Hüllenklasse. Während der Adapter hilft Methoden aus externen Komponenten aufzurufen und diese in das eigene System zu integrieren, ist es das Ziel der Fassade das eigene System nach außen so zu gestalten, dass es einfach durch externe Komponenten (= *Clients*) aufzurufen ist. Die Fassade gestaltet also die Schnittstelle über die Zugriffe von außen erfolgen. Es sollen i.d.R. nicht alle Methoden mit ihrer internen Signatur nach außen zur Verfügung gestellt werden. Die folgende Abbildung skizziert die Idee einer Fassade in einem UML-Komponentendiagramm.



Entwurfsmuster Fassade

Eine Fassade entkoppelt die Komponenten des entwickelten Systems von seinen Clients. Die Clients dürfen ausschließlich über die Fassade auf das System zugreifen. Hinter der Fassade gibt es mehrere Komponenten oder Subsysteme, die ihrerseits definierte Schnittstellen aufweisen und als Bausteine das System selbst verkörpern. Die Komponenten sind häufig viel komplexer, als es ein Client benötigt. Sie können in unterschiedlichen Programmiersprachen, mittels verschiedener Frameworks realisiert sein und in heterogenen Laufzeitumgebungen bereitgestellt werden (s. Kapitel [Modularisierung und Architektur](#)). Die Fassade greift ihrerseits auf diese Komponenten zu und verbirgt deren Komplexität vor den Clients. Eine Methode der Fassade kann z.B. eine logische Folge von internen Methodenaufrufen zu einer höherwertigen Funktionalität zusammenführen.

In der Praxis sind Fassaden-Klassen sehr häufig vorzufinden, da nahezu jedes System über die Jahre an Komplexität gewinnt und so irgendwann der Bedarf nach einer Fassade entsteht, um diese interne Komplexität überschaubar zu gestalten. Ein Softwaresystem entwickelt sich i.d.R. organisch weiter, dadurch dass verschiedene Entwickler an verschiedenen Stellen neue Klassen, Beziehungen und Methoden hinzufügen. Das System wird im Laufe seiner kontinuierlichen Weiterentwicklung sukzessive in Komponenten zerlegt, die von unterschiedlichen Teams betreut werden.

Fassaden können auch helfen, wenn später mehrere Komponenten wieder integriert werden sollen, weil sie fachlich ähnliche Funktionalitäten beinhalten. Die Fassade bietet dann eine einheitliche Schnittstelle über die Zugriffe von außen an die existierenden Komponenten delegiert werden können, während diese hinter der Fassade langsam verschmelzen.

Es gibt also zusammenfassend verschiedene Motive eine Fassade einzusetzen:

- Ein Client soll mit einer einfachen Schnittstelle kommunizieren. Die Komplexität hinter der Fassade wird abstrahiert, so dass sich der Client damit nicht auseinandersetzen muss.
- Hinter einer Fassade können die Komponenten grundsätzlich weiterentwickelt werden (insbesondere Refactoring), ohne dass ein Client dies bemerkt. Die Schnittstelle der Fassade soll dabei möglichst stabil bleiben, während ihre Implementierung sich an die Änderungen in den internen Komponenten anpasst.
- Allgemein unterstützt eine Fassade das Prinzip loser Kopplung zwischen Komponenten, um deren Abhängigkeiten voneinander zu verringern. Die lose Kopplung erleichtert die Portierung und die Replikation einzelner Komponenten in einem verteilten System. Portierung heißt, dass eine Komponente unter eine anderen Adresse erreichbar gemacht wird, z.B. weil sich der Host von einem dedizierten Server im eigenen Rechenzentrum in Richtung eines virtuellen Servers bei einem Cloud-Dienstleister ändert. Replikation heißt, dass eine Komponente im verteilten System mehrfach instantiiert wird, um ggf. eine höhere Systemlast tragen zu können.
- Eine Fassade kann auch der Sicherheit dienen. Das System ist einfacher zu schützen, wenn Zugriffe nur über die Fassade erfolgen. Die Clients werden durch die Fassade ihrerseits vor fehlerhafter Anwendung geschützt, d.h. vor anderenfalls möglichen Kombinationen und Parametrisierungen von Methodenaufrufen, die fachlich nicht sinnvoll sind.
- Querschnittliche Belange (engl. *Cross-cutting Concerns*) können von einer Fassade übernommen werden, z.B. Authentifizierung und Autorisierung, Session-Verwaltung, allgemeine Validierungen, Logging oder Monitoring.

Das in der obigen Abbildung dargestellte UML-Komponentendiagramm soll anhand der mit Java 9 eingeführten [Module](#) (= Komponenten) folgend implementiert werden. Die Programmiersprache Java bietet seit ihrer Entstehung die Möglichkeit, die Sichtbarkeit von Klassen, ihren Attributen und Methoden, usw. einzuschränken. Per Default ist die Sichtbarkeit auf Inhalte innerhalb des eigenen Package beschränkt. Der Default kann über die bekannten Modifizierer `public`, `private` und `protected` angepasst werden. Seit Java 9 gibt es auf höherer Abstraktionsebene nun Module, die den Komponenten der Architektur entsprechen und über die ebenfalls die Sichtbarkeit gesteuert werden kann. Diese Module umfassen i.d.R. mehrere Packages und spezifizieren in ihrem Modul-Deskriptor (`module-info.java`) zum einen, von welchen anderen Modulen sie abhängig sind (`requires <<module>>`), und zum anderen, welche ihrer Packages sie an andere Module nach außen freigeben (`exports <<package>>`). Per Konvention werden die Module in Java genau wie Packages kleingeschrieben und mit einer umgekehrten Domain als identifizierendes Prefix bezeichnet. Auf das Domain-Prefix wird hier zur Vereinfachung verzichtet.

Modul Client

```
1 // module-info.java
2 module client {
3     requires mySystem;
4 }
5
6 // Client.java
7 package client;
8 import facade.Facade;
9
10 public class Client {
11     public static void main(String[] args) {
12         System.out.println(Facade.helloFromX());
13         System.out.println(Facade.welcomeHelloFromY());
14         System.out.println(Facade.welcomeHelloFromXY());
15     }
16 }
```

Modul MySystem

```
1 // module-info.java
2 module mySystem {
3     requires componentX;
4     requires componentY;
5     exports facade;
6 }
7
8 // Facade.java
9 package facade;
10 import x.X2;
11 import y.Y1;
12
13 public class Facade {
14     public static String helloFromX() { return X2.hello("X2"); }
15     public static String welcomeHelloFromY() { return Y1.welcome() + Y1.hello(); }
16     public static String welcomeHelloFromXY() { return Y1.welcome() + X2.hello("X2") + Y1.hello(); }
17 }
```

Modul ComponentX

```
1 // module-info.java
2 module componentX {
3     exports x to mySystem;
4 }
5
6 // X2.java
7 package x;
8
9 public class X2 extends X1 {
10     public static String hello(String from) { return "Hello from " + from + ". "; }
11     X3[] x3;
12 }
```

Modul ComponentY

```
1 // module-info.java
2 module componentY {
3     exports y to mySystem;
4 }
5
6 // Y1.java
7 package y;
8
9 public class Y1 {
10     public static String welcome() { return "Welcome! "; }
11     public static String hello() { return "Hello from " + Y1.class.getSimpleName() + ". "; }
12 }
```

- Das Modul **client** kann hier ausschließlich über das Modul **mySystem** auf die als Bausteine enthaltenen Komponenten zugreifen.
- Die Inhalte der Packages in den Modulen **componentX** und **componentY** kann der Client nicht importieren, das diese nur für die Fassade freigegeben sind (**exports x to mySystem**).
- Die Fassade hingegen exportiert die Inhalte ihres Package öffentlich an beliebige interessierte Clients (**exports facade**).
- In der Klasse **Facade** ist zu erkennen, das in den Methoden der Fassade durchaus mehrere Methoden aus den Komponenten kombiniert werden können und deren Parametrisierung eingeschränkt werden kann.

Die gezeigten Code-Beispiele zum Entwurfsmuster Fassade finden sich im Verzeichnis `/patterns/facade` des Modul-Repository.

Proxy

Ein Proxy hat den Zweck, dass ein Client nicht direkt mit einem Zielobjekt kommunizieren kann. Stattdessen läuft die Kommunikation zwischen Client und Zielobjekt stets über den zwischengeschalteten Stellvertreter/Proxy. Dabei wird die ursprüngliche Anfrage des Client ggf. manipuliert, aufgezeichnet, verzögert, gänzlich blockiert oder ähnliches.

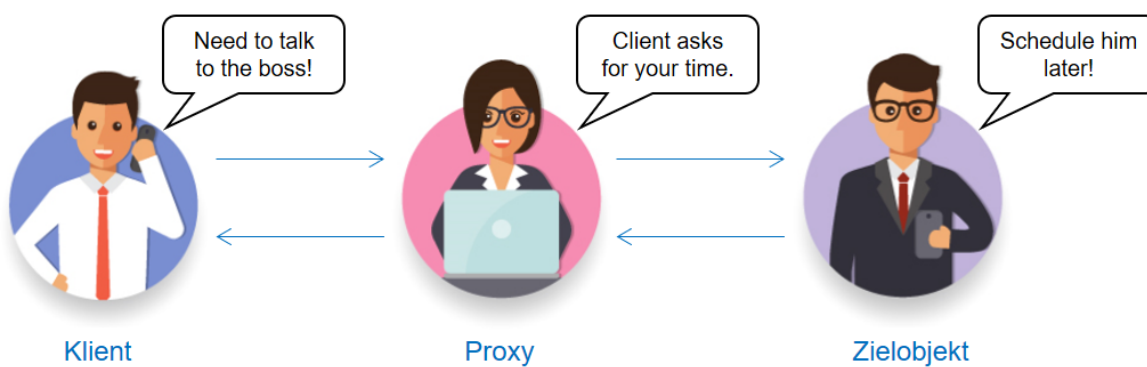
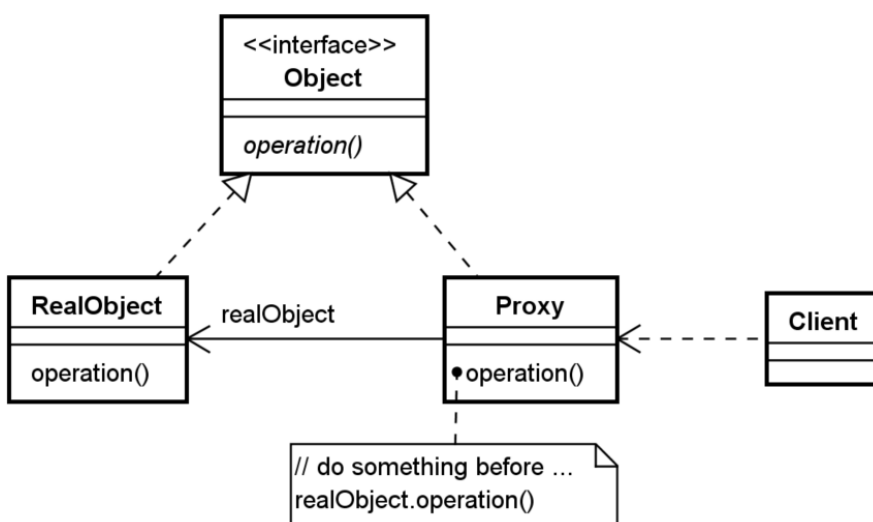


Illustration eines Stellvertreters (Proxy), der die direkte Kommunikation verhindert

Der Proxy ist eine Klasse, die dieselbe Schnittstelle wie das eigentliche Zielobjekt erfüllt und daher an dessen Stelle eingesetzt werden kann. Im folgenden UML-Klassendiagramm wird das Proxy-Muster dargestellt. Der Proxy delegiert die Anfrage des Client an das Zielobjekt `RealObject`, welches ihm zu diesem Zweck bekannt ist (Attribut `Proxy.realObject`, s. Implementierung der Methode `Proxy.operation`). Im Gegensatz zum Adapter verändert der Proxy die Schnittstelle nicht, sondern implementiert sie seinerseits vollständig, wobei eine zusätzliche Funktionalität ergänzt wird, um das Zielobjekt zu entlasten.



Entwurfsmuster Proxy

Das Proxy-Muster bringt zunächst einen eindeutigen Nachteil mit sich: Es entsteht durch die Proxy-Klasse zusätzlicher Code, der die Komplexität im Call-Stack und den verbundenen Aufwand für Test und Wartung erhöht. Für die zusätzliche Kontrollebene, die durch den Proxy gewonnen wird, muss es also gute Gründe geben, die folgend exemplarisch vorgestellt werden. Voraussetzung für die Notwendigkeit einer Proxy-Klasse ist es, dass die dahinterliegende Klasse nicht verändert werden kann oder soll.

- **Schutz-Proxy:** Ein Schutz-Proxy fängt den Zugriff auf das echte Zielobjekt ab, um diesen ggf. nach Prüfung der Autorisierung einzuschränken. Ein solcher Proxy kann eingesetzt werden, wenn die abzusichernde Klasse selbst nicht angepasst werden kann, weil ihr Code nicht zugänglich ist oder Änderungen nicht zulässig sind (z.B. aus lizenzrechtlichen Gründen).
- **Remote-Proxy:** Ein Remote-Proxy ist ein Stellvertreterobjekt auf der Seite des Client, wenn dieser Methoden auf Remote-Objekten aufruft, die in einer anderen Laufzeitumgebung (z.B. einer entfernten JVM) residieren. Die andere Laufzeitumgebung wird in einem anderen Prozess und Adressraum ausgeführt, meistens sogar auf einem anderen Host, und bietet einen *Remote Service* an, der von außen aufgerufen werden kann (s. Kapitel [Remote Method Invocation](#) und [SOAP-Webservices](#)). Der Remote-Proxy ist der Endpunkt für diese Interprozesskommunikation im Adressraum des Client – als Stellvertreter des Remote-Objekts im entfernten Adressraum. Der Proxy übernimmt das Verpacken der Input-Argumente bei einem Methodenaufwurf auf dem Remote-Objekt in ein Serialisierungsformat (z.B. `java.io.Serializable`) und das entsprechende Entpacken des Return-Arguments, wenn die Antwort eingeht. Der Proxy delegiert die erforderliche Netzwerkkommunikation an die Laufzeitumgebung bzw. das Betriebssystem und kümmert sich um Fehlerfälle wie z.B. *Timeouts*.
- **Virtueller Proxy:** Ein virtueller Proxy wird eingesetzt, wenn die Erzeugung des Zielobjekts sehr aufwändig ist, d.h. es werden dazu viele Ressourcen beansprucht. Der Proxy kann in diesem Fall die vollständige Erzeugung des Zielobjekts verzögern oder sie in einzelne Teilschritte zerlegen. Da der Proxy den Zugriff auf das Zielobjekt kontrolliert, kann er dessen einzelne Teile ggf. erst bei Bedarf, d.h. beim ersten Zugriff, erzeugen lassen (= *Lazy Loading*). Ein virtueller Proxy eignet sich wenn das eigentliche Zielobjekt groß und behäbig ist, z.B. beim Einlesen eines komplexen Objekts aus einer sehr großen Datei oder Datenbank. Ein virtueller Proxy lässt sich häufig sinnvoll mit einem Remote-Proxy kombinieren, wenn über eine Netzwerkverbindung mit eingeschränkter Datenübertragungsrate kommuniziert wird, z.B. beim Mobilfunk mit schlechtem Empfang.

Im folgenden Code-Beispiel zur Verdeutlichung des Proxy-Musters besteht die Funktion des Zielobjekts `RealVideo` darin eine Video-Datei aus dem Web zu laden und abzuspielen. Das Laden des Videos findet im Konstruktor der Klasse `RealVideo` statt (Zeile 6). Das Abspielen startet bei Aufruf der Methode `play` (Zeilen 10-14). Das Interface `Video` schreibt nur diese Methode `play` vor, die auch von der Proxy-Klasse `ProxyVideo` implementiert wird. Die Proxy-Klasse verändert das Verhalten des Zielobjekts um zwei Aspekte:

- Das Laden des Videos wird verzögert und findet nicht im Konstruktor, sondern erst direkt vor dem Abspielen in der Methode `play` statt (Lazy Loading).
- Videos, die kürzer sind als 30 Sek., werden gar nicht erst abgespielt, da es sich (in diesem Fall) um Werbung handelt.

RealVideo

```
1 class RealVideo implements Video {
2
3     MediaPlayer player;
4
5     RealVideo(String src) {
6         player = new MediaPlayer(new Media(src));
7     }
8
9     @Override
10    public void play(MediaView mediaView) {
11        mediaView.setMediaPlayer(player);
12        System.out.println("Start playing " + player.getMedia().getSource());
13        player.play();
14    }
15 }
```

Video

```
1 interface Video {
2     void play(MediaView mediaView);
3 }
```

ProxyVideo

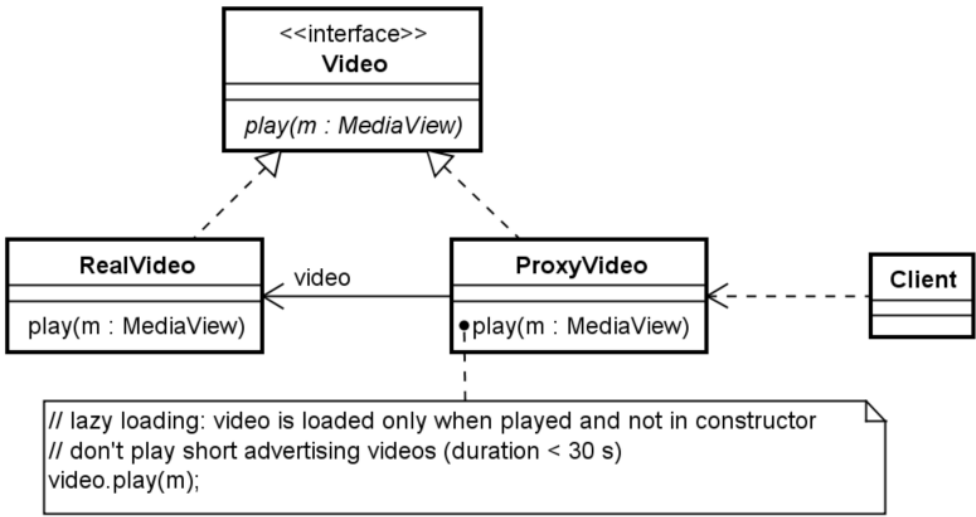
```
1 class ProxyVideo implements Video {
2
3     RealVideo video;
4     String src;
5
6     ProxyVideo(String src) {
7         // video is not loaded here
8         this.src = src;
9     }
10
11    @Override
12    public void play(MediaView mediaView) {
13        // lazy loading >> video is loaded only when played and not in constructor
14        if (video == null) {
15            video = new RealVideo(src);
16            video.player.setOnReady(() -> checkBeforePlay(mediaView));
17        }
18        else { checkBeforePlay(mediaView); }
19    }
20
21    void checkBeforePlay(MediaView mediaView) {
22        // don't play short advertising videos (duration < 30 s)
23        Double duration = video.player.getMedia().getDuration().toSeconds();
24        if (duration < 30) {
25            System.out.printf("Ignored an advertising video %s (%.2f s)\n", src, duration);
26            return;
27        }
28        video.play(mediaView);
29    }
30 }
```


Client

```
1 public class Client extends Application { // the client is a JavaFX application
2
3     final static String VIDEO_URL = "http://commondatastorage.googleapis.com/gtv-videos-bucket/sample/";
4
5     public static void main(String[] args) { launch(args); } // JavaFX application launcher
6
7     @Override
8     public void start(Stage stage) {
9         MediaView mediaView = new MediaView();
10
11         List<String> filenames = List.of("BigBuckBunny.mp4", "ElephantsDream.mp4", "ForBiggerBlazes.mp4", "ForBiggerEscapes.mp4");
12         List<Video> videos = filenames.stream().map(filename -> new ProxyVideo(VIDEO_URL + filename))
13             .collect(Collectors.toList());
14
15         Button b = new Button("Play next video");
16         b.setOnAction(e -> {
17             Video anyVideo = videos.get(new Random().nextInt(videos.size()));
18             anyVideo.play(mediaView); // play random video
19         });
20
21         Scene scene = new Scene(new VBox(mediaView, b));
22         stage.setScene(scene);
23         stage.show();
24     }
25 }
```

Der Client startet eine einfache JavaFX-Anwendung, die jedes Mal, wenn der Anwender auf einen Button klickt, ein zufälliges Video abspielt (Zeilen 15-19 in `Client`). Zuvor wird eine Liste mit `ProxyVideo`-Objekten befüllt (Zeile 12). In dieser Zeile könnte der Konstruktoraufwurf `new ProxyVideo` durch `new RealVideo` ersetzt werden, um den Proxy nicht zu verwenden, und demzufolge die Videos bereits vorab zu laden und die kurzen Werbevideos abzuspielen anstatt sie zu überspringen.

Das folgende UML-Klassendiagramm entspricht dem Code-Beispiel für das Proxy-Muster.



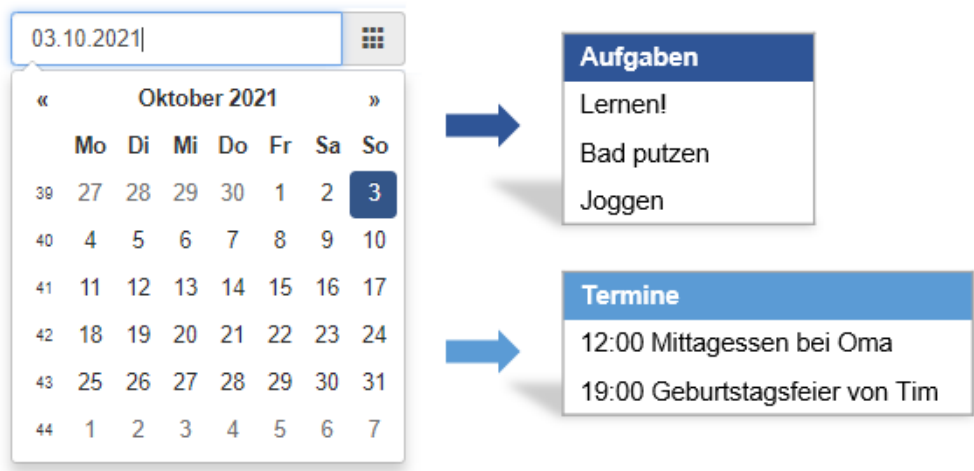
Anwendungsbeispiel für das Proxy-Muster

Die gezeigten Code-Beispiele zum Entwurfsmuster Proxy finden sich im Verzeichnis `/patterns/proxy` des Modul-Repository.

Beobachter

Das Beobachter-Muster (engl. *Observer* oder *Listener*) wird eingesetzt, wenn mehrere Objekte dauerhaft den Zustand eines anderen Objekts beobachten. Wenn sich der Zustand des beobachteten Objekts ändert, soll dieses alle seine Beobachter darüber informieren, damit diese sich ihrerseits reaktiv aktualisieren können. Das Muster entspricht also dem Prinzip: "Don't call us, we'll call you!"

Wir können uns folgendes Beispiel vorstellen: Das beobachtete Objekt ist ein Feld zur Datumsauswahl. Dessen Beobachter sind eine Aufgaben- und eine Terminliste, die ihren jeweiligen Zustand (also die Aufgaben und Termine, die anzuzeigen sind) in Abhängigkeit des ausgewählten Datums ändern.



Beispiel für Beobachter-Muster

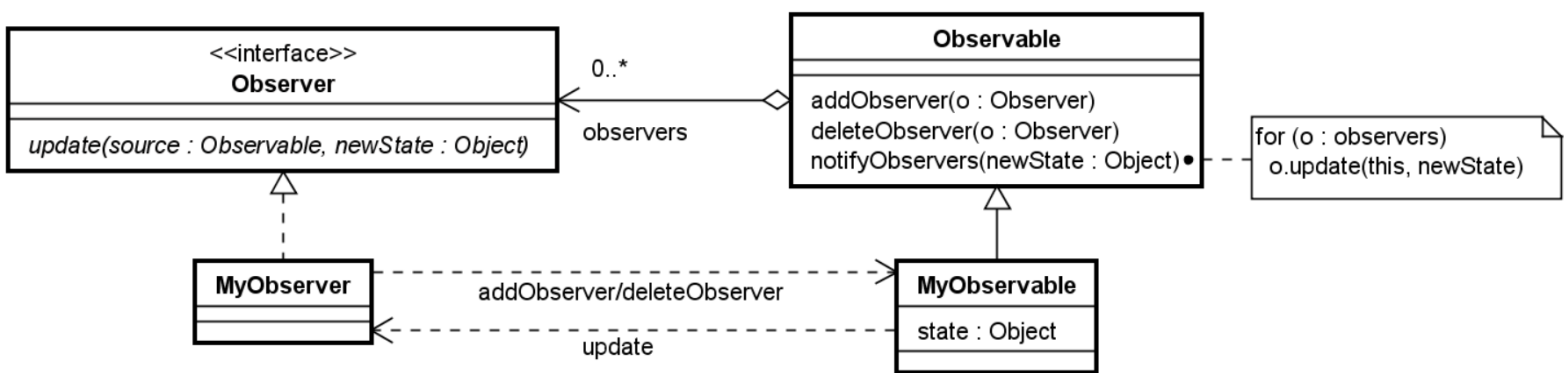
Das beobachtete Objekt (engl. *Observable*) speichert häufig Daten in Form einer *List<T>* oder einer *Map<T,T>*, die im User Interface (UI) an verschiedenen Stellen mittels unterschiedlicher Sichten (engl. *Views*) dargestellt werden, z.B. als Tabelle oder Chart. Diese Sichten im UI sind dann die Beobachter, die ihre Darstellung bei einer Zustandsänderung des Observable aktualisieren müssen. Das grundsätzliche Ziel des Beobachter-Musters ist es also, Observable und Observer voneinander zu entkoppeln, was in der Anwendung häufig der Trennung von Modell und beobachtenden Sichten entspricht.

Die Abhängigkeit zwischen den Objekten kann im Software-Entwurf über unterschiedliche prinzipielle Wege abgebildet werden:

- **Pull:** Die abhängigen Objekte sind aktiv und fragen den Zustand des beobachteten Objekts an. Wenn diese Zustandsanfrage gezielt durch eine bestimmte Interaktion des Anwenders (z.B. Öffnen einer neuen Seite oder eines neuen Dialogs) ausgelöst wird, ist das Pull-Prinzip sinnvoll. Das HTTP-Protokoll funktioniert ausschließlich nach dem Pull-Prinzip. Weniger geeignet ist das Pull-Prinzip, wenn sich der Zustand des Observable häufig ändert und die Beobachter stets den aktuellen Zustand benötigen. Da die Observer in diesem Fall nicht wissen, wann sich der Zustand des Observable ändert, müssen sie in kurzen, regelmäßigen Intervallen nachfragen, was *Polling* genannt wird.
- **Push:** Das beobachtete Objekt ist aktiv und benachrichtigt direkt und unaufgefordert seine Beobachter, indem es deren entsprechende Methoden aufruft. Das ist einfach und effizient, hat aber zur Bedingung, dass die zu informierenden Objekte vorab bekannt sind und sich zur Laufzeit nicht ändern. Da diese beiden Bedingungen in den meisten Anwendungsfällen nicht gelten, werden Pull- und Push-Prinzip zu einem bidirektionalen Kommunikationsmuster kombiniert: *Publish-Subscribe*.
- **Publish-Subscribe:** Die grundlegende Idee ist, die Abhängigkeit dadurch umzukehren, dass interessierte Beobachter ihren Wunsch, bei Zustandsänderungen benachrichtigt zu werden, beim Observable registrieren (*Subscribe*). Alle registrierten Beobachter werden durch das Observable informiert, wenn es zu einer Änderung seines Zustands kommt (*Publish*).

Das Beobachter-Muster wird demzufolge auch *Publish-Subscribe*-Muster genannt. Die Vorstellung hierbei entspricht einem Newsletter- oder Newsfeed-Abonnement: Beobachter abonnieren Nachrichten von Herausgebern, die diese gelegentlich veröffentlichen. Das Observable ist dabei der sendende *Publisher*, und die Observer sind die empfangenden *Subscriber*, die ihr Abonnement auch wieder kündigen können, falls sie sich nicht mehr für die Nachrichten interessieren.

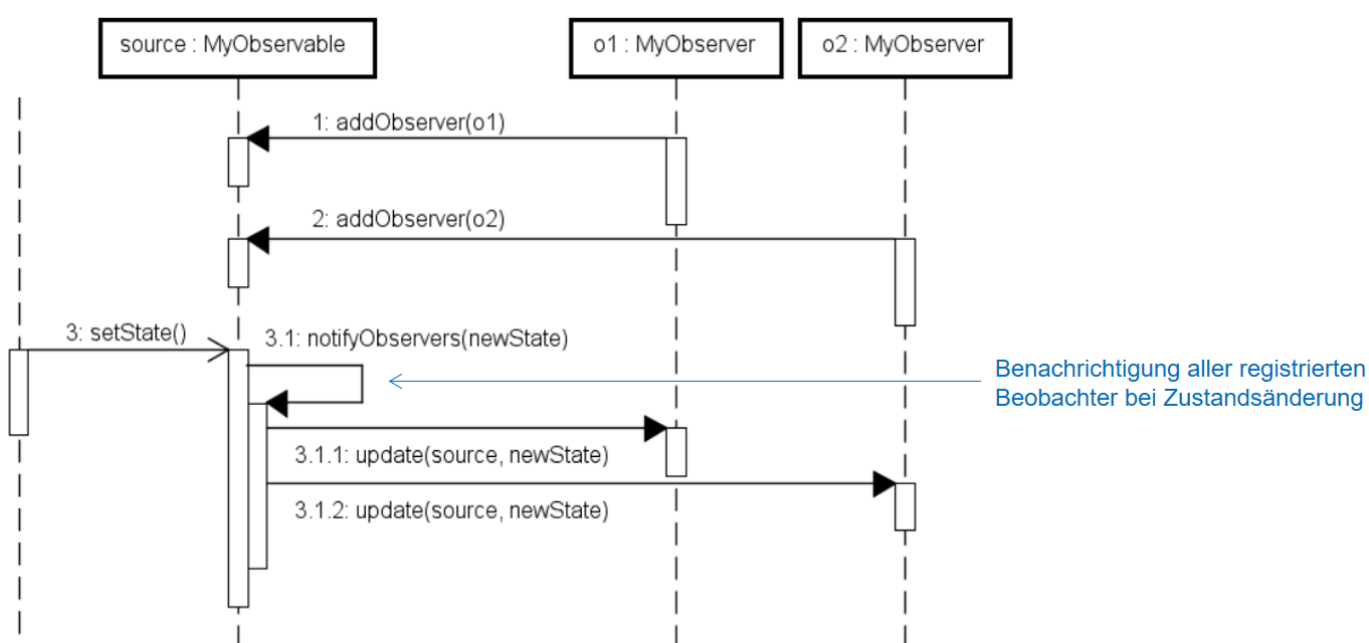
Das folgende UML-Klassendiagramm visualisiert das Beobachter-Muster.



Entwurfsmuster Beobachter

Ein Observable bietet Methoden an, um Beobachter zu registrieren (`addObserver`) und zu entfernen (`deleteObserver`) und verwaltet seine Beobachter in einer entsprechenden Liste (`observers`). Das folgende UML-Sequenzdiagramm verdeutlicht den grundlegenden Ablauf des Beobachter-Musters. Sobald der Zustand des konkreten Observable von außen geändert wird (`setState`), informiert das Observable über die Methode `notifyObservers` alle registrierten konkreten Beobachter, indem es deren Methode `update` aufruft und den neuen Zustand sowie sich selbst als Quelle dieses neuen Zustands weitergibt.

In einer alternativen Variante des Musters kann die Methode `update` auch ohne Argumente dargestellt werden. In diesem Fall muss ein Beobachter nach der Benachrichtigung sich den neuen Zustand selbst über eine entsprechende Methode `getState` vom Observable abholen.



Sequenzdiagramm für das Entwurfsmuster Beobachter

Zur Anwendung des ursprünglichen Beobachter-Musters werden in Java [java.util.Observable](#) und [java.util.Observer](#) bereitgestellt. Jedoch sind beide seit Java 9 als *deprecated* gekennzeichnet und daher nicht weiter zu empfehlen. Das folgende Code-Beispiel demonstriert die Verwendung von `Observable` (Zeile 1) und `Observer` (Zeile 11), wobei `Observer` ein funktionales Interface (engl. [Functional Interface](#)) ist, d.h. das Interface schreibt genau eine Methode vor und kann daher in Lambda-Ausdrücken implementiert werden.

Alle im Folgenden gezeigten Code-Beispiele zum Entwurfsmuster Beobachter finden sich im Verzeichnis `/patterns/observer` des Modul-Repository.

```

1  class MyObservable extends Observable {
2      Object state;
3
4      void setState(Object newState) {
5          this.setChanged();
6          this.notifyObservers(state = newState);
7      }
8
9      public static void main(String[] args) {
10         // Observer is implemented as a lambda expression
11         Observer o1 = (source, newState) -> System.out.printf("Received new state = %s from %s\n", newState, source);
12
13         MyObservable source = new MyObservable();
14         source.addObserver(o1);
15
16         List.of(1, 2, 3).forEach(i -> source.setState(i));
17     }
18 }

```

Das folgende Code-Beispiel demonstriert die Verwendung eines `PropertyChangeListener` als neuere Alternative zu `java.util.Observer`. Anstatt `java.util.Observable` zu erweitern, kann die Liste der registrierten Beobachter und die Methode `notifyObservers` auch eigenständig implementiert werden (Zeilen 3-7). Die Beobachter können in diesem Fall das funktionale Interface [java.beans.PropertyChangeListener](#) implementieren und über dessen Methode ein `PropertyChangeEvent` empfangen (Zeilen 15-17).

```

1 class MySecondObservable {
2     Object state;
3     List<PropertyChangeListener> listeners = new ArrayList<>();
4
5     void notifyListeners(String property, Object oldValue, Object newValue) {
6         listeners.forEach(l -> l.propertyChange(new PropertyChangeEvent(this, property, oldValue, newValue)));
7     }
8
9     void setState(Object newState) {
10        notifyListeners("state", state, state = newState);
11    }
12
13    public static void main(String[] args) {
14        // Observer is implemented as a lambda expression
15        PropertyChangeListener o1 = (PropertyChangeEvent e) ->
16            System.out.printf("Received new state = %s [previous state = %s] from %s\n",
17                e.getNewValue(), e.getOldValue(), e.getSource());
18
19        MySecondObservable source = new MySecondObservable();
20        source.listeners.add(o1);
21
22        List.of(1, 2, 3).forEach(i -> source.setState(i));
23    }
24 }

```

Der `PropertyChangeListener` ist ein spezieller [java.util.EventListener](#). In Java begegnen uns bei der UI-Programmierung häufig spezielle `EventListener` als Teil des Beobachter-Musters, um z.B. verschiedene UI-Komponenten wie Textfelder, Buttons u.ä. oder die Interaktion des Anwenders über Tastatur, Maus u.ä. zu beobachten. In JavaFX wird üblicherweise das Interface [javafx.event.EventHandler](#), das auch eine `EventListener`-Spezialisierung ist, verwendet, um ein anderes Objekt zu beobachten und auf dessen Zustandsänderung zu reagieren:

```

1 interface EventHandler<T extends Event> extends EventListener {
2     void handle(T event);
3 }

```

Der folgende Code zeigt, wie in JavaFX die Tastatureingabe, die Mausbewegung und ein Button beobachtet werden können. Dabei sind `Button` und `Scene` durch JavaFX bereitgestellte Observables.

```

1 Scene scene = new Scene(/* ... */);
2 scene.setOnKeyPressed((KeyEvent e) -> System.out.println("Pressed key: " + e.getCode()));
3
4 Button btn = new Button(/* ... */);
5 btn.setOnMouseEntered((MouseEvent e) -> System.out.println("Moved mouse over button"));
6 btn.setOnAction((ActionEvent e) -> System.out.println("Clicked button"));

```

Die seit Java 9 neue Klasse [java.util.concurrent.Flow](#) stellt die Interfaces [Publisher](#) und [Subscriber](#) bereit, über die das Beobachter-Muster als asynchroner Datenstrom (engl. *Stream*) realisiert werden kann. Der folgende Code demonstriert eine grundlegende Verwendung der Flow-API.

```

1 class MySubscriber implements Subscriber<Integer> {
2     Subscription subscription;
3
4     @Override
5     public void onSubscribe(Subscription subscription) {
6         System.out.println("Subscription startet");
7         this.subscription = subscription;
8         subscription.request(1);
9     }
10
11    @Override
12    public void onNext(Integer item) {
13        System.out.printf("Received new state = %s\n", item);
14        this.subscription.request(1);
15    }
16
17    @Override
18    public void onError(Throwable throwable) { }
19
20    @Override
21    public void onComplete() { System.out.println("Subscription ended"); }
22 }

```

Ein konkreter Subscriber muss die Methoden `onSubscribe`, `onNext`, `onError` und `onComplete` implementieren. Beim initialen Aufruf von `onSubscribe` (Zeile 5) erhält der Subscriber ein Objekt vom Typ `Subscription`, über das er steuern kann, ob er in der Folge weitere Nachrichten erhalten möchte (Methode `request` ist auszuführen, Zeilen 8+14) oder nicht (Methode `cancel`). Falls ein Subscriber weitere Nachrichten erhalten möchte, werden ihm diese über die Methode `onNext` (Zeile 12) zugestellt.

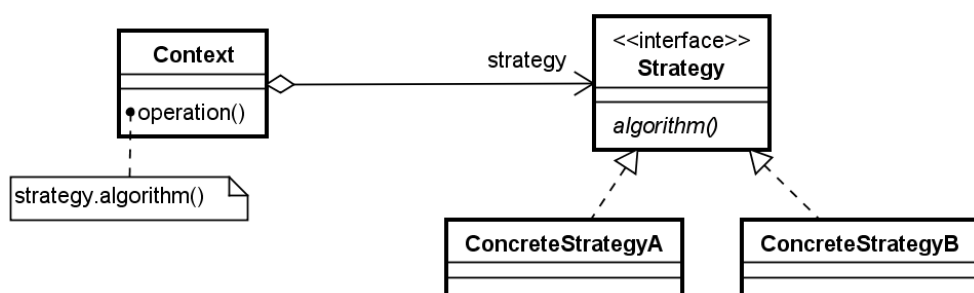
```
1 class MyPublisher {
2     public static void main(String[] args) throws InterruptedException {
3         SubmissionPublisher source = new SubmissionPublisher<Integer>();
4         source.subscribe(new MySubscriber());
5
6         List.of(1, 2, 3).forEach(i -> source.submit(i));
7         source.close();
8         Thread.sleep(100); // wait for termination of subscriber thread
9     }
10 }
```

Die Klasse `SubmissionPublisher` bietet eine Standard-Implementierung des `Publisher`-Interface. Nachrichten, die über die Methode `submit` (Zeile 6) eingereicht werden, sendet der Publisher an alle aktuell registrierten Subscriber, bis er über die Methode `close` (Zeile 7) geschlossen wird. Hervorzuheben ist, dass ein Publisher die Nachrichten asynchron zustellt, d.h. er wartet nicht auf eine Empfangs- oder Verarbeitungsbestätigung der Subscriber, die ihrerseits jeweils in einem parallelen Thread (s. Kapitel [Threads in Java](#)) ablaufen. Jeder aktuelle Subscriber erhält neu übermittelte Nachrichten in derselben Reihenfolge, wie sie versendet werden - außer bei Exceptions (`onError`) oder bei Timeouts. Wenn eine Nachricht über `offer` anstatt `submit` versendet wird, kann ein Timeout definiert werden, falls ein Subscriber zu lange braucht, um die Nachricht entgegenzunehmen. Auf diese Weise können Publisher als nicht blockierende, [reaktive Streams](#) fungieren. Damit der Subscriber-Thread ausreichend Zeit zur Verarbeitung der Nachrichten hat, schläft der Main-Thread im obigen Code eine Weile (Zeile 8). Das ist für die Praxis keine gute Lösung, sondern ein [Anti-Pattern](#), aber es trägt hier zur Erhaltung der Einfachheit des Beispiels bei.

Strategie

Das Entwurfsmuster Strategie (engl. *Strategy* oder *Policy*) beschäftigt sich damit, dass es auch in der Software-Entwicklung unterschiedliche Wege zum gleichen Ziel geben kann. Diese Wege entsprechen Algorithmen, die in Methoden implementiert werden und sich eine gemeinsame Schnittstelle teilen. Vereinfacht betrachtet betont das Strategie-Muster daher lediglich die konsequente Verwendung von Interfaces und die damit verbundene Polymorphie in einer objektorientierten Sprache.

Wenn es alternative Algorithmen als zielführende Wege gibt, soll dem Client zur Laufzeit ermöglicht werden, selbst zu entscheiden, welchen Algorithmus er als Lösungsstrategie verwenden möchte. Die Wahl der optimalen Strategie kann je nach Kontext unterschiedlich ausfallen. Es kann z.B. eine konkrete Strategie A geben, die besonders genaue Ergebnisse liefert, aber im Vergleich zu einer anderen Strategie B deutlich mehr Ressourcen (wie Prozessorzeit oder Speicher) erfordert, was dazu führt, dass ihre Berechnung länger dauert oder teurer wird. Strategie B dagegen benötigt weniger Ressourcen, liefert aber nur annähernd genaue Ergebnisse, die in Abhängigkeit des Kontexts ausreichend sein können oder eben nicht. Das Strategie-Muster erlaubt einen Wechsel der konkreten Strategie zur Laufzeit. Es wird in folgendem UML-Klassendiagramm visualisiert.



Entwurfsmuster Strategie

Der umgebende Kontext der Anwendung (**Context**) implementiert die alternativen Algorithmen nicht selbst, sondern kennt nur ein Interface **Strategy**, das die Signatur für den Algorithmus vorschreibt und zur Laufzeit durch konkrete Strategien (z.B. **ConcreteStrategyA**) ausgeprägt wird. Die Vorteile dieses Musters liegen auf der Hand:

- Der Anwendungskontext wird von den konkreten Strategien entkoppelt und ist nicht von diesen abhängig, d.h. in der Klasse **Context** gibt es keine `import`-Anweisung für eine konkrete Strategie.
- Bei komplexeren Algorithmen, z.B. für optimierte Sortier-, Such- oder Hashverfahren, fördert die Erstellung einer Strategie-Klasse je Algorithmus die Übersichtlichkeit. Der konkrete Algorithmus kann unabhängig von anderen Aspekten getestet und gewartet werden.
- Die Strategien werden wiederverwendbar und können auch in anderen Kontexten eingesetzt werden.
- Neue Strategien können einfach hinzugefügt werden. Es sind nahezu keine Änderungen am bestehenden Code erforderlich.

Das folgende Beispiel soll das Strategie-Muster anhand einer einfachen Implementierung verdeutlichen. Es soll die Distanz zwischen zwei Punkten bestimmt werden, z.B. zur Navigation von einem Startpunkt zu einem Zielpunkt. Die Algorithmen implementieren 3 alternative Distanzfunktionen im zweidimensionalen Raum: [Euklidische Distanz](#), [City-Block-Distanz](#) und [Tschebyschew-Distanz](#). Der Client probiert zur Laufzeit alle 3 Strategien zur Messung der Distanz zwischen den Punkten (0,0) und (3,4) nacheinander aus und erhält mit 5, 7 und 4 jeweils ein unterschiedliches Ergebnis.

Client

```
1 class Client {
2     public static void main(String[] args) {
3         Context context = new Context(new Point2D(0, 0), new Point2D(3, 4));
4
5         context.strategy = new EuclideanDistance(); // set initial strategy
6         System.out.println("Distance to destination is " + context.distance());
7
8         context.strategy = new CityBlockDistance(); // change strategy at runtime
9         System.out.println("Distance to destination is " + context.distance());
10
11        context.strategy = new ChebyshevDistance(); // change strategy at runtime again
12        System.out.println("Distance to destination is " + context.distance());
13    }
14 }
```


Context

```
1 class Context {
2     Point2D from, to;
3     DistanceFunctionStrategy strategy;
4
5     Context(Point2D from, Point2D to) { this.from = from; this.to = to; }
6
7     Double distance() { return strategy.distance(from, to); }
8 }
```

DistanceFunctionStrategy

```
1 interface DistanceFunctionStrategy {
2     Double distance(Point2D a, Point2D b);
3 }
```

EuclideanDistance

```
1 class EuclideanDistance implements DistanceFunctionStrategy {
2
3     @Override
4     public Double distance(Point2D a, Point2D b) {
5         return Math.sqrt(Math.pow(a.getX() - b.getX(), 2) + Math.pow(a.getY() - b.getY(), 2));
6     }
7 }
```

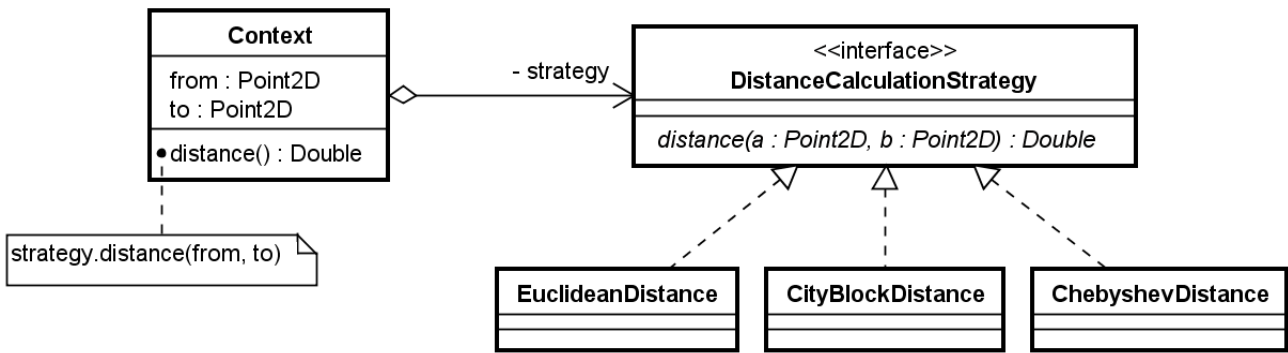
CityBlockDistance

```
1 class CityBlockDistance implements DistanceFunctionStrategy {
2
3     @Override
4     public Double distance(Point2D a, Point2D b) {
5         return Math.abs(a.getX() - b.getX()) + Math.abs(a.getY() - b.getY());
6     }
7 }
```

ChebyshevDistance

```
1 class ChebyshevDistance implements DistanceFunctionStrategy {
2
3     @Override
4     public Double distance(Point2D a, Point2D b) {
5         return Math.max(Math.abs(a.getX() - b.getX()), Math.abs(a.getY() - b.getY()));
6     }
7 }
```

Das folgende UML-Klassendiagramm entspricht dem obigen Code-Beispiel für das Strategie-Muster.



Anwendungsbeispiel für das Strategie-Muster

Dieses Beispiel lässt sich gedanklich einfach auf die Routenoptimierung in einem Navigationssystem übertragen. Dabei kann je nach Kontext nach dem schnellsten, dem kürzesten oder dem günstigsten Weg gesucht werden, d.h. es sollen entweder die Wegzeit, die Wegstrecke oder die Wegkosten minimiert werden. Auch hier gibt es unterschiedliche konkrete Wege mit einheitlicher Schnittstelle zum Erreichen des vorgegebenen Ziels.

Die gezeigten Code-Beispiele zum Entwurfsmuster Strategie finden sich im Verzeichnis /patterns/strategy des Modul-Repository.

Modularisierung und Architektur

Als Software-Architektur bezeichnen wir ein abstraktes Modell, aus dem insbesondere eine strukturierte und hierarchische Anordnung der Bauteile eines Anwendungssystems und eine Beschreibung der Beziehungen dieser Teile untereinander hervorgeht [\[Bal11\]](#). Mit dem Begriff Architektur (lat. *architectura*, Baukunst) verbinden wir zunächst allgemein die Gestaltung von Raum, was in der Praxis häufig dem Entwurf von Bauwerken entspricht. Durch den Architekturentwurf entsteht ein Modell, das Vorgaben für die spätere Konstruktion des Bauwerks macht. In der Softwaretechnik wird der Architekturbegriff mit gleicher Semantik genutzt, wobei die Bauteile entweder generisch als Komponenten (z.B. in der UML) oder als Module (z.B. in Java) bezeichnet werden. Es folgt eine entsprechende Definition der [IEEE](#).

"Architecture is the fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution." (ISO/IEC/IEEE 42010:2011).

Prinzipien des Software Engineering

Der Zweck einer Software-Architektur ist an den allgemeinen Prinzipien des Software Engineering ausgerichtet. Zu diesen Prinzipien zählen nach Ghezzi et al. [\[GJM03\]](#):

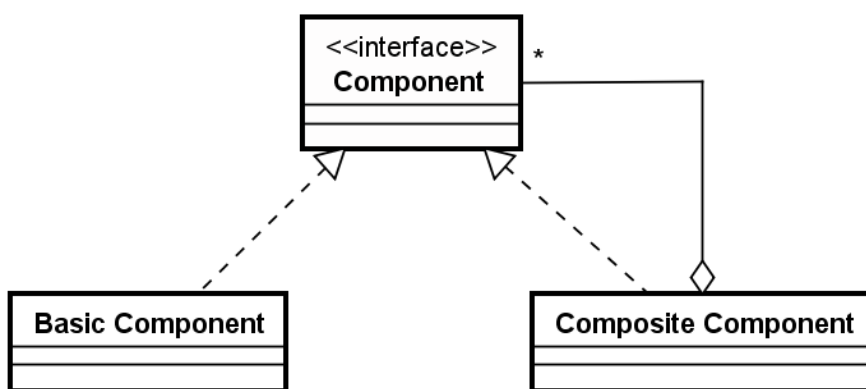
- **Abstraktion:** Eine Architektur soll einen ganzheitlichen Überblick auf ein Anwendungssystem bieten und muss dazu verallgemeinern. Wichtige Aspekte sind zu identifizieren und darzustellen, unwichtige Aspekte auszublenden. Das Abstraktionsniveau ist auf den Zweck der Darstellung angepasst. Abstraktion ist grundsätzlich erforderlich, um Komplexität zu beherrschen.
- **Modularisierung:** Ein komplexes System wird in Komponenten/Module aufgeteilt. Dabei erlaubt Hierarchisierung, dass Module ihrerseits wieder in feinere Teilmodule zerlegt werden können, und Kapselung, dass ein Modul die Details seiner Implementierung in der Sichtbarkeit nach außen bewusst begrenzt. Bei der Aufteilung der Module wird eine hohe Kohäsion und eine geringe Kopplung angestrebt. Hohe Kohäsion heißt, dass innerhalb des Moduls ein enger Zusammenhang zwischen seinen Teilen besteht, z.B. viele Abhängigkeiten der Klassen innerhalb eines Moduls. Geringe Kopplung heißt, dass zwischen den Modulen über deren nach außen sichtbare Schnittstellen möglichst wenig Abhängigkeiten bestehen.
- **Trennung von Zuständigkeiten** (engl. *Separation of Concerns*): Es wird angestrebt, dass ein Modul möglichst für einen abgegrenzten Aspekt zuständig ist. Wenn die Aspekte klar getrennt sind, kann die Verantwortlichkeit für die Entwicklung der Module ebenso aufgeteilt und parallelisiert werden. Da das ganzheitliche Architekturmodell als Kommunikationsmittel für verschiedene Stakeholder dient, sollte es aus unterschiedlichen aspekt-orientierten Sichten betrachtet werden können. Ein früher Vorschlag diesbezüglich ist das ["4+1 Architectural View Model"](#), das folgende 4 zueinander konsistente Sichten auf eine Architektur vorschlägt und diese anschließend in ausgewählten Anwendungsszenarien zur Veranschaulichung zusammenführt (+1):
 - *Logical View:* Die Funktionalität des Systems aus Sicht des Anwenders wird dargestellt, z.B. als Sequenz- oder Klassendiagramm in der UML.
 - *Development View:* Die hierarchische Struktur und Abhängigkeiten des System werden aus Sicht des Entwicklers dargestellt, z.B. als Komponenten- oder Paketdiagramm in der UML.
 - *Process View:* Das dynamische Verhalten des Systems zur Laufzeit wird dargestellt, d.h. insbesondere Parallelität und Synchronisation, z.B. als Aktivitätsdiagramm in der UML.
 - *Physical View:* Die Verteilung der Komponenten auf Laufzeitumgebungen, insbesondere auf virtuelle und dedizierte Server, aus Sicht des für den Betrieb zuständigen Systemadministrators wird dargestellt, z.B. als Verteilungsdiagramm in der UML.
- **Allgemeingültigkeit** (engl. *Generality*): Zu Beginn eines Softwareprojekts stellt sich die Frage, wie individuell die Architektur des zu konstruierenden Systems sein muss. Wenn ein bewährtes Framework bereits als passende Architekturvorlage vorliegt, ist es grundsätzlich empfehlenswert darauf aufzubauen. Bei der Wiederverwendung von erprobten Komponenten kann i.d.R. davon ausgegangen werden, dass der Aufwand für eine eigene Implementierung deutlich höher ausfällt als die Einarbeitungszeit in die API der wiederverwendeten Komponente. Außerdem können durch Wiederverwendung ansonsten häufig auftretende Schwachstellen vermieden werden. Die Vor- und Nachteile eines individuellen Architekturentwurfs gegenüber einer adaptierten Standardarchitektur sind sorgfältig abzuwägen. Es ist zu beachten, dass die Architektur eines neuen Systems sich durch die eigenen Entwurfsentscheidungen manifestiert.
- **Inkrementalität:** Inkrementalität basiert auf der Annahme, dass im Verlauf der Entwicklung Änderungen nicht zu verhindern sind und impliziert damit iteratives Vorgehen. Solange das Anwendungssystem weiterentwickelt wird, ist auch dessen Architektur i.d.R. nicht dauerhaft stabil. Die Evolution eines Anwendungssystems führt zu Unterschieden in der sogenannten präskriptiven und deskriptiven Architektur. Die präskriptive Architektur beschreibt, was durch die Architekten zur Entwurfszeit geplant war, während die deskriptive Architektur ausdrückt, was tatsächlich zur Laufzeit im Betrieb beobachtet werden kann, z.B. durch dynamische Code-Analyse mittels geeigneter [Application Performance Monitoring-Werkzeuge](#) wie [AppDynamics](#), [Dynatrace](#) oder [New Relic](#). Wir sprechen von Software-Erosion, wenn der Soll-Zustand (präskriptiv) und der Ist-Zustand (deskriptiv) der Architektur stark voneinander abweichen.
- **Früherkennung von Änderungen** (engl. *Anticipation of Change*): Aufbauend auf der inkrementellen Entwicklung eines Systems gilt es potentielle zukünftige Änderungen möglichst früh zu antizipieren und in der Architektur zu berücksichtigen. Eine gute

Architektur ist flexibel erweiterbar, aber weiterhin möglichst einfach zu handhaben. Die Ursachen für Änderungen sind sehr verschieden, z.B. Beseitigung von Fehlern (korrektive Wartung), Verbesserung nicht-funktionaler Eigenschaften (perfektive Wartung), Anpassung der Funktionalität wegen sich ändernder Bedingungen (adaptive Wartung).

- **Genauigkeit und Formalität:** Es geht hier nicht darum Kreativität zu verhindern, sondern das Ergebnis kreativer Entwurfsphasen möglichst präzise, z.B. in standardisierten Notationen, zu erfassen. Genauigkeit schafft Vertrauen für einen Entwurf. Formalität ist Genauigkeit in höchstem Maße, wobei jede Organisation bzw. jedes Team hier ein adäquates Maß für sich definieren muss. Es stellen sich dabei Fragen wie: Ist eine Verifikation für bestimmte Algorithmen erforderlich? Wie systematisch werden Testdaten und Testfälle erzeugt? Wie gründlich ist die Dokumentation von Aktivitäten?

Komponentenbasierte Entwicklung

Komponentenbasierte Entwicklung drückt das Paradigma aus, dass Software Engineering zu einer echten Ingenieurwissenschaft macht. Die zentrale Idee besteht in dem oben genannten Prinzip der Modularisierung, d.h. der hierarchischen Aufteilung des Systems in seine Komponenten, und der damit verbundenen Wiederverwendbarkeit von Komponenten in anderen Projekten. Damit entsteht Analogie zu klassischen Ingenieurdisziplinen, in denen es üblich ist, zuvor geprüfte/zugelassene Bauteile und bewährte Verfahren wiederzuverwenden. Für die Wiederverwendbarkeit ist es wichtig, dass jede Komponente möglichst einen klar definierten Aspekt implementiert und kapselt. Es kann mehrere Komponenten geben, die demselben inhaltlichen Zweck dienen. Wenn für diese Komponenten eine gemeinsame Schnittstelle spezifiziert ist, sind sie als alternative Implementierungen leicht austauschbar. Die Verknüpfung von mehreren Basiskomponenten zu einer zusammengesetzten Komponente oder schließlich zu einem Gesamtsystem bleibt eine architektonische Aufgabe, in der sich das [Entwurfsmuster Kompositum](#) wiederfindet.



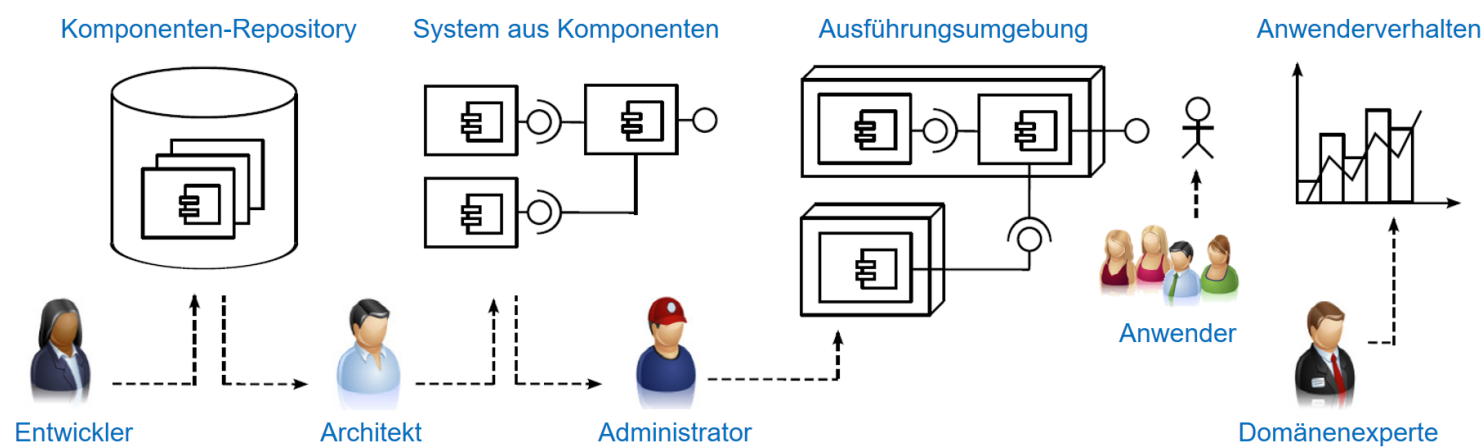
Komponentenbasiertes System als Kompositum

Es stellt sich nun die Frage, was genau eine Software-Komponente ausmacht. Dazu wird folgend eine verbreitete Definition von Clemens Szyperski et al. [\[SGM02\]](#) zitiert und diskutiert.

"A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties." (Clemens Szyperski et al.)

- **Unit of Composition:** Eine Komponente ist Bauteil innerhalb einer hierarchischen Systemarchitektur. Die Granularität einer Komponente ist nicht festgelegt. Eine Komponente könnte nur eine einzelne Methode implementieren – oder ein komplexes Subsystem, das verschiedene andere Komponenten umfasst.
- **Specified Interfaces:** Eine Komponente spezifiziert ihre Schnittstelle nach außen. Andere Komponenten oder ein menschlicher Anwender können nur über diese Schnittstelle mit der Komponente interagieren. Das Verhalten der Komponente ist dabei entsprechend dem sogenannten ["Design-by-Contract-Prinzip"](#) zugesichert. Dazu zählen z.B. die Typisierung von Input- und Return-Argumenten, die Angabe möglicher Exceptions und ggf. spezieller Vor- und Nachbedingungen. Alles was nicht in der Schnittstelle beschrieben ist, stellt sich für den Aufrufer als *Black Box* dar.
- **Explicit Context Dependencies:** Eine Komponente hängt von definierten Prämissen bezüglich ihres Ausführungskontextes ab, die zur Laufzeit permanent erfüllt werden müssen. Diese Abhängigkeiten können z.B. enthalten:
 - Erforderliche Schnittstellen, die zur Laufzeit von anderen Komponenten im System bereitgestellt und implementiert werden (s. [Dependency Injection](#))
 - Verfügbarkeit bestimmter Ressourcen wie Datenbanken oder Dateisystempfade
 - Konkrete Anforderungen an eine Laufzeitumgebung (z.B. JVM-Version), an ein Betriebssystem oder an Treiber zur Kommunikation mit Peripheriegeräten
- **Independent Deployment:** Eine nicht weiter unterteilte Basiskomponente ist durch den Anspruch begrenzt, dass jede Komponente unabhängig von anderen Komponenten bereitgestellt werden kann, was i.d.R. auch unabhängige Lauffähigkeit impliziert. Dieser Anspruch wird durch eine [Microservice-Architektur](#) auf Basis von [Containervirtualisierung](#) (in der Praxis insbesondere [Docker](#)) erreicht. Der Begriff *Service-orientierte Architektur (SOA)* impliziert zwar, dass eine Anwendung zur Entwurfszeit in lose gekoppelte Komponenten gegliedert wurde, die aber typischerweise nicht unabhängig voneinander bereitgestellt werden können.
- **Third-party Composition:** Komponenten, die durch Dritte erstellt worden sind, können wiederverwendet werden. Es erfolgt eine explizite Rollentrennung zwischen Anbieter und Anwender einer Komponente. Ein Anbieter fungiert als Entwickler, der die Funktionalität einer Komponente implementiert, deren Schnittstelle und Abhängigkeiten angibt und diese in einem Repository

veröffentlicht, aus dem zukünftige Anwender die Komponente (ggf. lizenzpflichtig) abrufen können. Ein Anwender kann seinerseits als Architekt arbeiten, indem er verschiedene Basiskomponenten so zusammenstellt, dass eine höherwertige Komponente entsteht, für die er selbst als Anbieter auftritt. Die folgende Abbildung skizziert die verschiedenen Rollen in der komponentenbasierten Entwicklung. Eine Person oder ein Team kann natürlich auch mehrere Rollen gleichzeitig einnehmen.



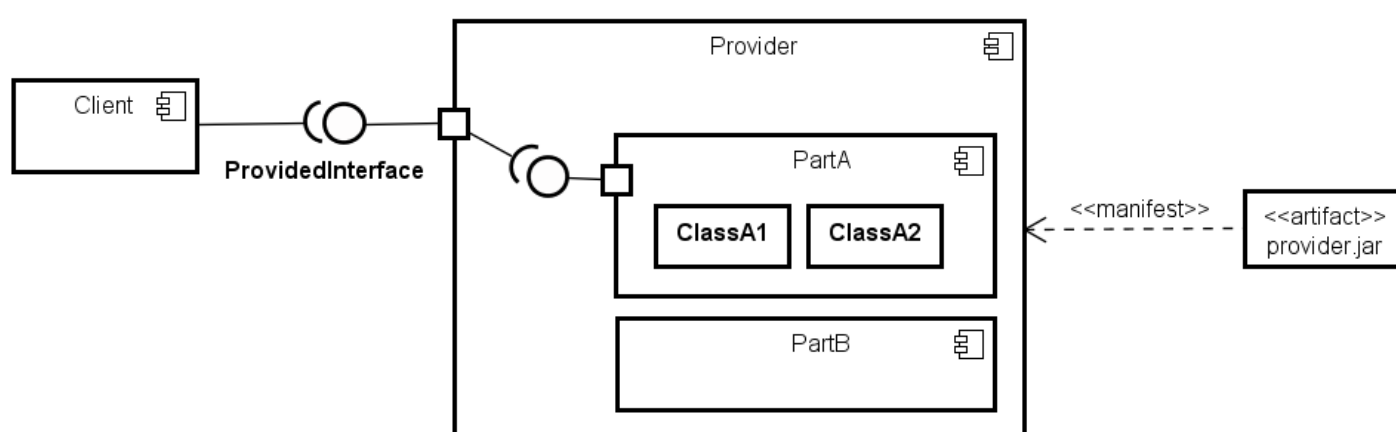
Rollen in der komponentenbasierten Entwicklung

Komponentenmodelle

Im Rahmen der komponentenbasierten Entwicklung werden wir uns in der Praxis für ein spezifisches Komponentenmodell entscheiden. Ein Komponentenmodell spezifiziert, wie Komponenten und deren Komposition textuell (z.B. in einer Programmiersprache) oder grafisch (z.B. in UML-Notation) abgebildet werden. Folgende Definition fasst zusammen, was wir von einem Komponentenmodell erwarten können [LW07].

"A software component model is a definition of (1) the semantics of components, that is, what components are meant to be, (2) the syntax of components, that is, how they are defined, constructed, and represented, and (3) the composition of components, that is, how they are composed or assembled." (Kung-Kiu Lau et al.)

Aktuelle Komponentenmodelle sind z.B. [Microsoft .NET](#), [Java Platform Module System \(JPMS\)](#), [OSGi](#) und [Spring](#). Zu frühen Komponentenmodellen, die heute veraltet sind, gehören [Enterprise JavaBeans \(EJBs\)](#), [OMG CORBA Component Model \(CCM\)](#) und [Microsoft COM](#). Auch die UML bietet ein Komponentenmodell an, das nicht wie die vorherigen auf eine konkrete Implementierung, sondern auf die Spezifikation, Dokumentation und Kommunikation eines Architekturentwurfs ausgerichtet ist. Die folgende Abbildung zeigt wie Komponenten in einem UML-Komponentendiagramm dargestellt werden.



Notation des UML-Komponentendiagramms

Eine Komponente kann in der UML durch andere Teilkomponenten oder durch Klassen realisiert werden. Beides sind im Sinne der UML sogenannte *Classifier*. In der obigen Abbildung wird die Komponente **Provider** durch die Teilkomponenten **PartA** und **PartB** realisiert, wobei **PartA** wiederum durch die Klassen **ClassA1** und **ClassA2** realisiert wird. Weitere Realisierungen sind nicht angegeben. Die Komponente **Provider** wird manifestiert durch das ausführbare Artefakt **provider.jar**. Das Komponentendiagramm enthält keine Information darüber, in welcher Laufzeitumgebung und auf welchem Host dieses Artefakt bereitgestellt wird. Diese Information würde in der Sicht eines zum Gesamtmodell zugehörigen Verteilungsdiagramms dargestellt werden. Die Komponente **Provider** bietet nach außen die Schnittstelle **ProvidedInterface** an, die von der Komponente **Client** genutzt wird. Intern wird diese Schnittstelle in der Teilkomponente **PartA** realisiert. Eine anschauliche Notationsbeschreibung des UML-Komponentendiagramms findet sich in der [Dokumentation zu Microsoft Visual Studio](#) sowie in den UML-Lehrbüchern, die im Kapitel [Objektorientierung und UML](#) referenziert werden.

Java Platform Module System (JPMS)

Im Gegensatz zur UML erlaubt es das [Modulsystem in Java \(JPMS\)](#), nicht, dass Komponenten hierarchisch ineinander verschachtelt werden. In Java können lediglich Packages als Realisierung der Module wie gewohnt hierarchisch untergliedert werden. Auf Ebene der Module können Abhängigkeiten explizit definiert werden (**requires <module>**). Die Sichtbarkeit der modulinternen Realisierung kann ebenso explizit nach außen freigegeben werden (**export <package>**).

Das folgende Code-Beispiel zeigt die beiden Java-Module `client` und `simpleRegression`, wobei letzteres eine einfache lineare Regression implementiert. Als Input für das Regressionsmodell dient eine Datenreihe mit 2D-Koordinaten im JSON-Format, d.h. zusammengehörige x- und y-Werte (Zeilen 13-14). Das Modul `client` ist ausschließlich vom Modul `simpleRegression` abhängig, dessen Abhängigkeiten ihm aber verborgen bleiben (Zeile 3). Es ist damit weitgehend entkoppelt. Der Zugriff auf das Package `regression` ist nur möglich, weil es explizit exportiert wird (Zeile 7 im Modul `simpleRegression`). Das Modul `simpleRegression` ist von folgenden Modulen abhängig: `commons.math3` zum Trainieren des Regressionsmodells, `gson` zum Konvertieren von JSON in Java Objekte und `java.desktop` aufgrund der Verwendung der Klasse `Point` für 2D-Koordinaten (Zeilen 3-5). Weiterhin ergibt sich durch das Modul `gson` eine transitive Abhängigkeit zum Modul `java.sql` (Zeile 6).

Client

```

1 // module-info.java
2 module client {
3     requires simpleRegression;
4 }
5
6 // Client.java
7 package client;
8 import regression.SimpleRegressionModel;
9
10 class Client {
11
12     public static void main(String[] args) {
13         String json = "[{'x': 2, 'y': 4}, {'x': 4, 'y': 3}, {'x': 3, 'y': 6}, {'x': 9, 'y': 7}, {'x': 7, 'y': 8}]";
14         SimpleRegressionModel model = new SimpleRegressionModel(json);
15
16         System.out.println("Regression function: y = f(x) = " + model.getSlope() + "x + " + model.getIntercept());
17         System.out.println("R² = " + model.getDeterminationTokenCoefficient());
18
19         double x = 5;
20         System.out.printf("Prediction: y = f(%.1f) = %.1f", x, model.predict(x));
21     }
22 }

```

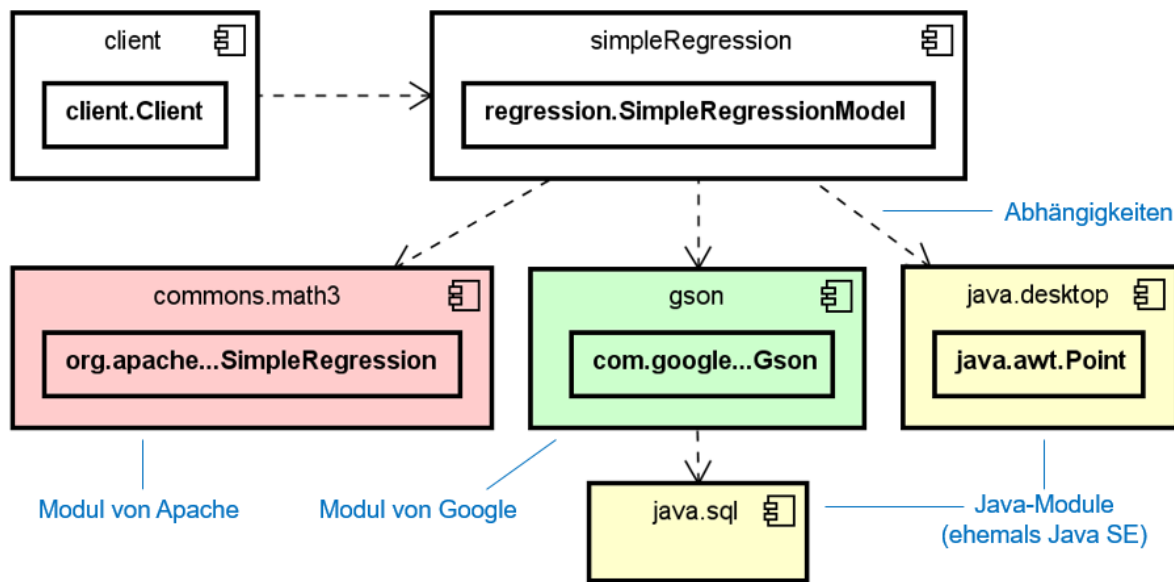
SimpleRegression

```

1 // module-info.java
2 module simpleRegression {
3     requires gson;
4     requires commons.math3;
5     requires java.desktop;
6     requires transitive java.sql; // gson depends on java.sql
7     exports regression; // exports package
8 }
9
10 // SimpleRegressionModel.java
11 package regression;
12 import com.google.gson.Gson;
13 import org.apache.commons.math3.stat.regression.SimpleRegression;
14 import java.awt.Point;
15
16 public class SimpleRegressionModel {
17
18     Gson gson = new Gson(); // dependency to Google GSON
19     SimpleRegression model = new SimpleRegression(); // dependency to Apache Commons Math
20
21     public SimpleRegressionModel(String json) {
22         Point[] points = gson.fromJson(json, Point[].class); // dependency to Java Desktop (Point class)
23         for (Point p : points) model.addData(p.x, p.y);
24     }
25
26     public double getIntercept() { return model.getIntercept(); }
27
28     public double getSlope() { return model.getSlope(); }
29
30     public double getDeterminationTokenCoefficient() { return model.getRSquare(); }
31
32     public double predict(double x) { return model.predict(x); }
33 }

```

Die gezeigten Code-Beispiele zu JPMS finden sich im Verzeichnis `/architecture/jpms` des Modul-Repository. Das folgende UML-Komponentendiagramm veranschaulicht die Abhängigkeiten der Module aus dem obigen Code-Beispiel. Es wird deutlich, wie das JPMS dazu beiträgt, Abhängigkeiten zwischen Komponenten explizit zu gestalten und ggf. einzuschränken.



UML-Komponentendiagramm zur Veranschaulichung des Java Platform Module System (JPMS)

Kapselung auf Ebene von Klassen

Bisher ist nur über Modularisierung und Kapselung auf Ebene von Komponenten gesprochen worden. Im Feinentwurf werden die Komponenten durch Klassen realisiert, die ebenfalls interne Daten- und Verhaltensstrukturen in Form von privaten Attributen und Methoden voneinander verstecken können. Dazu dienen in objektorientierten Sprachen insbesondere die Sicherheitsmodifizierer **private** und **protected**. Wenn auf eine Klasse regelmäßig von außen zugegriffen wird, ist eine stabile Schnittstelle wichtig. Der Zugriff kann in diesem Fall sinnvoll über Getter- und Setter-Methoden gekapselt werden, um die zugrundeliegenden internen Strukturen verändern zu können, ohne den Zugriff von außen zu beeinflussen. Grundsätzlich stellt sich die Frage, vor welchen äußeren Zugriffen die internen Strukturen gekapselt werden sollen:

- Vor Zugriffen aus Methoden in anderen Klassen im selben Modul?
- Oder vor Zugriffen aus anderen Modulen durch externe Anwender, die das eigene Modul als abhängige Bibliothek einsetzen?

Im letzteren Fall kann der Aufwand, eine Veränderung der Schnittstelle durchzusetzen, sehr hoch sein, weil die externen Anwender etwaige Änderungen aus Trägheit ablehnen und ggf. als Anwender abspringen, oder weil sie aufgrund ihrer Vielzahl gar nicht vollständig bekannt sind, usw. Es werden aber auch viele Klassen entwickelt, die nur teamintern für eine kleine Anwendung oder ein kleines Modul genutzt und nicht nach außen über eine API weitergegeben werden. Wenn die Anzahl der Zugriffe auf die Strukturen einer Klasse übersichtlich ist und relativ einfach per Refactoring auch an den abhängigen Stellen geändert werden kann, ist zu überlegen, ob Getter und Setter tatsächlich erforderlich sind. Der strukturelle Aufbau der Klasse soll hier weiterhin einfach bleiben und nicht die Narben der Versionsgeschichte der Klasse abbilden. *Keep it simple!* Getter und Setter bleiben stets sinnvoll, wenn sie zusätzliche Funktionalitäten übernehmen und diese vom Aufrufer auch erwartet werden. Sichtbarkeit kann seit Java 9 aber z.T. sinnvoller auf Modulebene als auf Klassenebene eingeschränkt werden.

Das Projekt [Lombok](#) ermöglicht es in Java, Getter und Setter ohne weitere Funktionalität, die lediglich *Boilerplate-Code* darstellen, über die Verwendung von Annotationen wie **@Getter** und **@Setter** zur Kompilierungszeit zu generieren. In der verwendeten Entwicklungsumgebung muss ein entsprechendes Plugin (s. hier für [IntelliJ IDEA](#)) installiert werden, damit die Lombok-Annotationen auch während der Entwicklung aufgelöst werden können.

Schichtenarchitektur

Die vorgestellten Entwurfsmuster haben Lösungsansätze für den Feinentwurf aufgezeigt, d.h. sie betreffen konkrete Entwurfsentscheidungen für die objektorientierte Programmierung im Inneren einer Komponente. Architekturmuster haben im Unterschied zu den Entwurfsmustern den Anspruch, die grundlegende Struktur und Interaktion zwischen den Komponenten eines Anwendungssystems zu bestimmen. Demzufolge treffen wir Architekturmuster im Entwicklungsprozess in der vorausgehenden Phase des Grobentwurfs an. Grundsätzlich betrachten Architekturmuster auf einer abstrakten Ebene das Zusammenspiel von Komponenten, ohne dabei vorauszusetzen, wie diese Komponenten konkret implementiert werden, z.B. in welcher Programmiersprache. Auf dieser Ebene bleiben Architekturmuster per Definition abstrakt. Das gemeinsame Ziel sämtlicher Architekturmuster ist es, entsprechend dem Prinzip der [Modularisierung](#) Abhängigkeiten zwischen den Komponenten bewusst so zu gestalten, dass eine möglichst geringe Kopplung erreicht wird.

Als Ausgangspunkt wird meistens ein sogenanntes "monolithisches System" angenommen, für das scheinbar keine durchdachte Struktur vorliegt und das uns dem Begriff Monolith entsprechend als ein einzelner großer Baustein erscheint. Ein monolithisches System ist typischerweise bereits seit vielen Jahren im produktiven Einsatz. In der Vergangenheit wurde es kontinuierlich erweitert, bis es seine heutige Komplexität erreicht hat. Es läuft überraschend stabil, aber es ist nicht mehr (oder kaum noch) wartbar, erweiterbar und skalierbar. Unter den Entwicklern wird es als sogenanntes *Legacy-System* bezeichnet, dessen präskriptive Architektur längst erodiert ist. Wenn wir genau in den Monolithen hineinschauen, werden wir nichtsdestotrotz irgendeine Struktur erkennen, aber die Abhängigkeiten der Module oder Klassen sind so stark miteinander verwoben, dass sich keine Komponenten herauslösen lassen. Das [Refactoring](#) des Systems erscheint daher dringend notwendig.

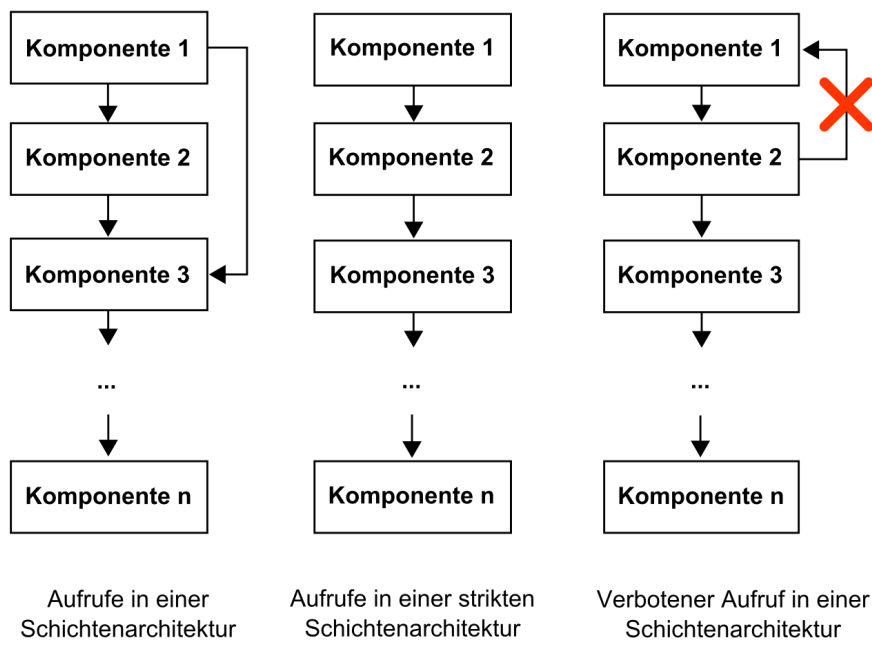


Uluru. Monolith oder nicht? Im allgemeinen Sprachgebrauch: ja. Aus geomorphologischer Perspektive: eher nein, da sich verschiedene, teilweise verwachsene Schichten unterscheiden lassen.

Zusammenfassend ergeben sich folgende Nachteile eines monolithische Systems [\[KB18\]](#):

- Die fehlende Modularisierung führt dazu, dass die Komplexität nicht aufgeteilt und reduziert werden kann.
- Die korrektive Wartung gestaltet sich sehr aufwändig, da eine Änderung an einer bestimmten Stelle im Code eine kaskadierende Folge von weiteren Änderungen an anderen Stellen verursacht. Dadurch gelingt auch eine schrittweise perfektive Wartung nicht, insbesondere die Migration einzelner Aspekte auf eine neue technologische Basis (z.B. andere Laufzeitumgebung, andere Programmiersprache).
- Die Wiederverwendung einzelner Komponenten ist ausgeschlossen.
- Skalierbarkeit im Kontext von Cloud-Plattformen ist stark eingeschränkt.

Die Schichtenarchitektur (auch Schichtenmodell) ist ein einfaches Muster zur Strukturierung eines Softwaresystems, bei dem die einzelnen Aspekte (z.B. Komponenten oder Klassen) eines Systems jeweils unterschiedlichen Schichten (engl. *Tier* oder *Layer*) zugeordnet werden. Es gilt das grundsätzliche Prinzip, dass aus höheren Schichten auf tiefere Schichten zugegriffen werden darf, aber nicht von unten nach oben. Die Abhängigkeiten sind also von oben nach unten gerichtet. Bei einer strikten (oder geschlossenen) Schichtenarchitektur sind ausschließlich Zugriffe auf die nächsttiefere Schicht zulässig, d.h. es dürfen keine Schichten übersprungen werden. Im Gegensatz dazu darf in einer nicht-strikten (oder offenen) Schichtenarchitektur eine beliebige tiefere Schicht aufgerufen werden. Der Kontrollfluss und die Daten müssen dadurch nicht durch zwischenliegende Schichten durchgeschleust werden, was im verteilten System zu einer verbesserten Performance führen kann. Als Nachteil steht diesem Vorteil gegenüber, dass sich der Grad der Kopplung im System erhöht, wenn die tieferen Schichten potentiell von diversen höheren Schichten eingebunden werden und nicht nur von der nächsthöheren Schicht.

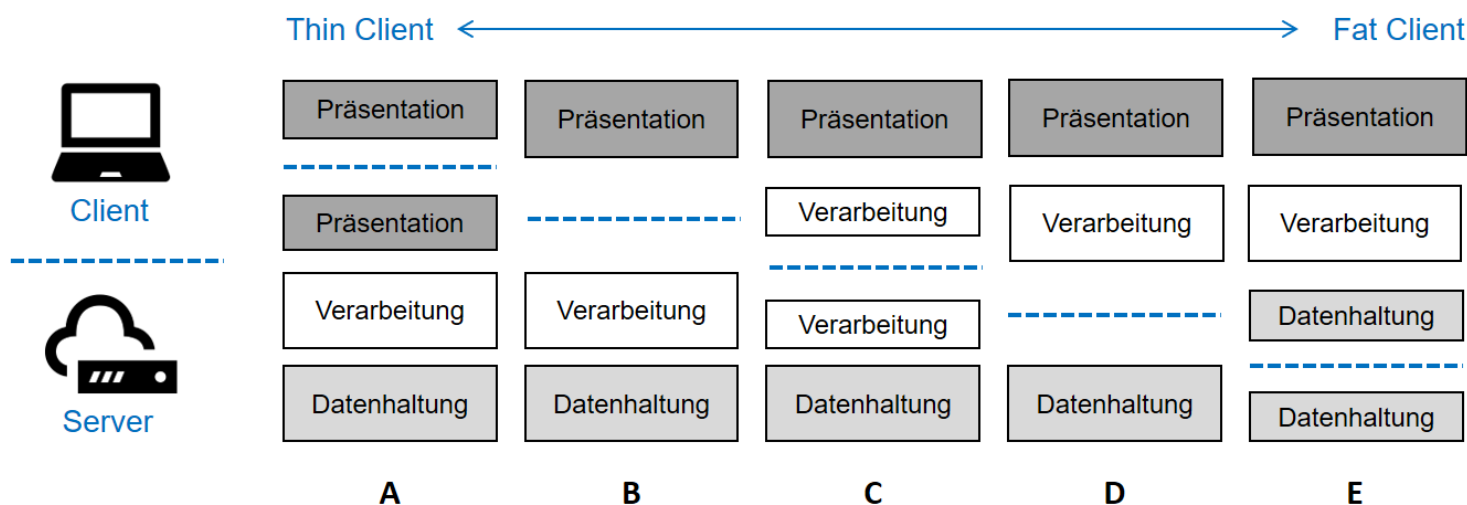


Prinzip der Schichtenarchitektur

Die bekannteste Schichtenarchitektur besteht aus 3 Schichten:

- **Präsentation:** Über diese Schicht interagiert der Anwender mit dem System. Sie beinhaltet das [User Interface \(UI\)](#), dessen Bedienbarkeit den Entwurf dieser Schicht maßgeblich bestimmt. Der Anwender bekommt die Daten in diesem *Frontend* aufbereitet dargestellt und sendet seine Eingaben über diese Schicht an die unterliegenden Schichten.
- **Verarbeitung der Logik:** In dieser Schicht findet die Verarbeitung der Daten gemäß der Anwendungslogik statt, die sich wiederum aus den funktionalen Anforderungen an das System ergibt. Die Eingaben der Anwender werden hier validiert. Ressourcen-intensive Verarbeitungsschritte auf Massendaten sollten parallelisiert ausgeführt werden.
- **Datenhaltung:** Diese Schicht speichert die Daten dauerhaft und stellt eine Schnittstelle bereit, um die Daten abzufragen und zu manipulieren. Dazu wird i.d.R. ein Datenbanksystem als sogenanntes *Backend* verwendet.

Wie die folgende Abbildung zeigt, können diese 3 Schichten in einem verteilten System, das aus Client und Server besteht, je nach Anwendung unterschiedlich zugeordnet werden. Ein *Thin Client* ist ausschließlich für die Präsentation der Daten zuständig. Ein *Fat Client* dagegen übernimmt neben der Präsentation auch die eigentliche Verarbeitung der Daten, d.h. insbesondere Validierung und Transformation, und u.U. sogar die Datenhaltung.



Verschiedene Varianten einer 3-Schichtenarchitektur im verteilten System

Die unterschiedlichen Varianten der 3-Schichtenarchitektur sollen folgend kurz erläutert werden, wobei erst in späteren Kapiteln auf konkrete Frameworks zur Implementierung eingegangen wird. Wir stellen uns dazu eine Web-Anwendung vor, deren Client-Komponenten in JavaScript implementiert sind und innerhalb eines Web-Browsers des Anwenders ausgeführt werden. Die Server-Komponenten seien in Java implementiert. Grundsätzlich sind auch andere Anwendungstypen, z.B. eine Desktop-Anwendung oder eine mobile Anwendung, und andere Programmiersprachen vorstellbar.

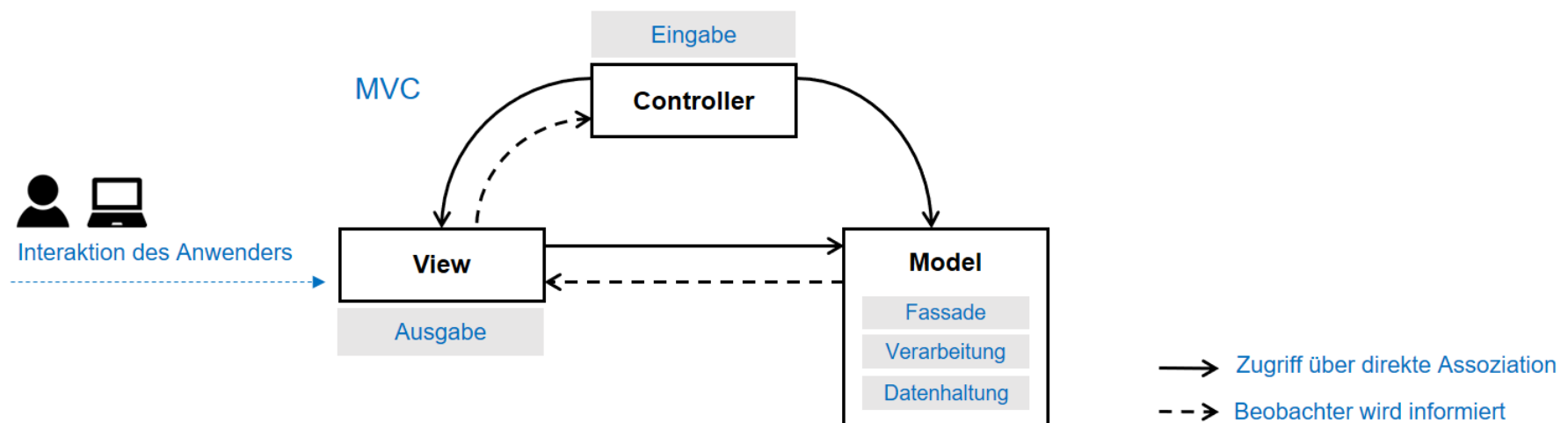
- A** In einer Web-Anwendung finden Verarbeitung und Datenhaltung i.d.R. auf dem Server statt. Viele Frameworks sind darauf ausgerichtet, eine serverseitige [Template-Engine](#) einzusetzen, um HTML-Templates mit Daten aus den verbundenen Datenbanken zu befüllen und HTML-Dokumente an den Client auszuliefern. Im Java-Umfeld zählen zu den aktuell verbreiteten Template-Engines u.a. [Mustache](#), [Thymeleaf](#), [FreeMarker](#) sowie die mittlerweile veralteten [JavaServer Pages \(JSP\)](#). Wenn eine serverseitige Template-Engine eingesetzt wird, erstreckt sich die Präsentationsschicht über Client und Server.
- B** Alternativ dazu kann eine Web-Anwendung so gebaut sein, dass der Server anstatt ganze HTML-Dokumente nur strukturierte Daten im JSON- oder XML-Format an den Client ausliefert. Diese Daten werden durch den Client empfangen, verarbeitet und in das [Document Object Model \(DOM\)](#) des Browsers dynamisch eingearbeitet. In diesem Fall sind Präsentations- und Verarbeitungsschicht über die Schnittstelle zwischen Client und Server klar voneinander getrennt.
- C** Um die Bedienbarkeit für den Anwender durch schnellere Rückmeldungen zu verbessern, bietet es sich an, einfache Validierungen der Anwendereingaben bereits im Client durchzuführen. Andernfalls wäre für jede Validierung eine Anfrage an

den Server erforderlich, deren Antwortzeit von der verfügbaren Datenübertragungsrate abhängig ist. Auf diese Weise wird ein Teil der Anwendungslogik in den Client verschoben, so dass die Verarbeitungsschicht sich auf Client und Server aufteilt.

- D** Es kann auch sämtliche Anwendungslogik einer Web-Anwendung im Client implementiert werden. Lediglich die Datenhaltung wird noch auf einem Server bereitgestellt, um die Daten zwischen mehreren Clients über eine zentrale Stelle austauschen zu können. Jeder Client benötigt in dieser Architektur einen persönlichen Zugang zur Datenhaltungsschicht. Ein entsprechender Dienst, der sich auf reine Datenhaltung und vorausgehende Authentifizierung beschränkt, ist z.B. [Google Firebase](#).
- E** Wenn die Daten ausschließlich zentral auf einem Server gehalten werden, kann die Anwendung nicht ohne aktive Verbindung zum Server zwecks Datenaustausch genutzt werden. Um dem Anwender zu ermöglichen, die Anwendung auch offline zu nutzen, werden Daten auf dem Client zwischengespeichert und bei aktiver Verbindung zum Server periodisch synchronisiert. Bei dieser Architektur erstreckt sich die Datenhaltungsschicht also auf Client und Server gleichermaßen. Die Herausforderung liegt in der Synchronisation der Daten, wenn diese sowohl auf dem Client als auch auf dem Server (indirekt von einem anderen Client) verändert worden sind. Diese Architektur wird häufig für mobile Anwendungen eingesetzt, da diese hinsichtlich der Verbindung zum Server zwischen online und offline schwanken.

Model-View-Controller

Das Architekturmuster *Model-View-Controller* (MVC) ist der klassische Ansatz zur Entkopplung eines grafischen User Interface (GUI) vom Rest des Anwendungssystems. MVC spielt also nur dann eine Rolle, wenn die zu konstruierende Anwendung irgendein nicht weiter spezifiziertes GUI benötigt. Entstanden ist der Begriff bereits 1979 im Kontext der ersten objektorientierten Programmiersprache Smalltalk am [Xerox PARC](#). MVC ist ursprünglich auf Desktop-Anwendungen ausgerichtet, aber nicht darauf festgelegt. Es ist zunächst zu betonen, dass die Komponenten *Model*, *View* und *Controller* nicht mit den Schichten Datenhaltung, Präsentation und Verarbeitungslogik der oben vorgestellten 3-Schichtenarchitektur gleichzusetzen sind. Insbesondere der *Controller* entspricht nicht der Verarbeitungsschicht, sondern bildet vielmehr gemeinsam mit der Datenhaltungsschicht die *Model*-Komponente im MVC-Muster – wie in der folgenden Abbildung verdeutlicht wird. Die Präsentationsschicht wird unterteilt in die grafische Darstellung der Ausgabe (*View*) und die Steuerung der Anwendereingaben (*Controller*).



Architekturmuster Model-View-Controller

Folgend werden die Interaktionen und die Abhängigkeiten der Komponenten im MVC-Muster beschrieben:

- **Model:** Das *Model* benachrichtigt eine oder mehrere *Views*, die als Beobachter registriert sind, sobald sich sein Zustand ändert, damit die *Views* sich entsprechend aktualisieren können. Änderungen am *Model* werden durch Zugriffe eines *Controller* ausgelöst, der Eingabedaten zur Validierung, Verarbeitung und Speicherung an das *Model* weiterreicht.
- **View:** Die *View* präsentiert die Ausgabe des Systems für den Anwender. Heute wird der grundlegende strukturelle Aufbau einer *View* häufig deklarativ formuliert (z.B. in HTML oder XML). Eine *View* verarbeitet selbst keine Daten, sondern benachrichtigt bei Eingaben des Anwenders einen angeordneten *Controller*. Im MVC-Muster wird eine *View* von einem assoziierten *Model* über dessen Zustandsänderungen informiert. Die *View* greift dann direkt auf das *Model* zu, um die darzustellenden Daten abzuholen. Diese direkte Abhängigkeit einer *View* von einem *Model* wird in den neueren Varianten des MVC-Musters, nämlich *Model-View-Presenter (MVP)* und *Model-View-ViewModel (MVVM)* aufgehoben.
- **Controller:** Der *Controller* nimmt die Eingaben des Anwenders entgegen. Wenn der Anwender innerhalb der Anwendung z.B. zu einem anderen Dialog navigiert, steuert der *Controller* die Aktualisierung der *View*. Wenn der Anwender in der *View* dargestellte Daten manipuliert, ruft der *Controller* das assoziierte *Model* auf, um die Eingaben weiterzugeben.

Wenn es zur Umsetzung des MVC-Musters kommt, existiert jedoch kein einheitliches Verständnis über die Zuständigkeiten der einzelnen Komponenten sowie deren Beziehungen und Sichtbarkeit untereinander. Wer nach "MVC" googelt, findet je nach Programmiersprache und Zielumgebung unterschiedliche Varianten. Die Ursachen für diese Variation liegen in der hohen Abstraktion des Musters. Wenn es zur Implementierung kommt, muss das Muster in unterschiedliche Protokolle und Frameworks eingebettet werden. Beispielsweise unterscheidet sich diesbezüglich die Implementierung einer Web-Anwendung mit PHP stark von der Implementierung einer Desktop-Anwendung mit Java oder einer mobilen Anwendung für iOS mit Swift.

Es haben sich dabei insbesondere zwei Varianten herausgebildet, die das Zusammenspiel der Komponenten konkretisieren und sich im Detail vom ursprünglichen MVC unterscheiden. Die grundlegende Idee von MVC bleibt bei beiden Varianten weiterhin gültig: Die fachlichen Teile einer Anwendung, die unabhängig von einem konkreten UI-Framework sind, sollen von der konkreten Ein- und Ausgabe des UI entkoppelt werden.

- **Model-View-Presenter (MVP):** MVP wurde in 2004 in einem [Artikel vom Martin Fowler](#) vorgeschlagen und darin von MVC abgegrenzt. Im MVP-Muster läuft sämtliche Kommunikation zwischen *View* und *Model* über einen zwischengeschalteten *Presenter*, damit keine Abhängigkeit zwischen *View* und *Model* existiert. Der *Presenter* arbeitet nur auf Interfaces von *View* und *Model*, damit diese in Komponententests (engl. [Unit Testing](#)) durch [Mock-Objekte](#) ausgetauscht und unabhängig voneinander getestet werden können. Martin Fowler hat das MVP-Muster später noch in [Supervising Controller](#) und [Passive View](#) untergliedert. In aktuellen Frameworks ist MVP meist begrifflich durch MVVM ersetzt worden.
- **Model-View-ViewModel (MVVM):** MVVM wurde als Begriff in 2005 durch das Framework [Windows Presentation Foundation \(WPF\)](#) innerhalb von Microsoft .NET geprägt. Es ist insbesondere auf UIs ausgerichtet, in denen Teile der *View* deklarativ in HTML oder XML beschrieben werden. Daher haben auch diverse JavaScript-Frameworks wie [Angular](#), [Vue.js](#) und zuerst [Knockout](#) das MVVM-Muster übernommen. Wie bei MVC und MVP werden der Zustand und die Steuerung des Verhaltens der *View* von ihrer grafischen Darstellung getrennt und ins sogenannte *ViewModel* ausgelagert. Das *ViewModel* soll wie der *Presenter* in MVP frei von Abhängigkeiten zu Klassen des konkreten UI-Frameworks sein. *View* und *ViewModel* werden per

Two-Way-Binding (bidirektionaler Beobachter) aneinander gebunden, so dass ihre Zustände nicht voneinander abweichen können. Wie genau das in Java implementiert werden kann, folgt später.



Architekturmuster Model-View-Presenter und Model-View-ViewModel

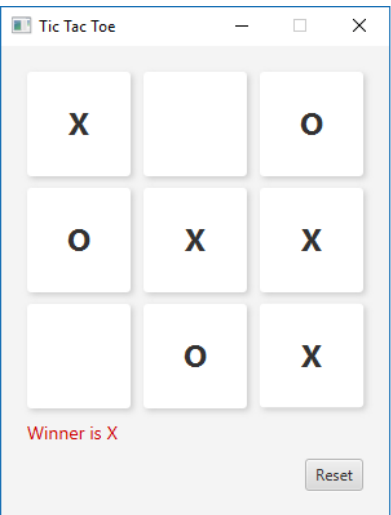
MVC und MVVM in JavaFX

MVC in JavaFX

Die Architekturmuster bleiben ohne konkrete Umsetzung in einem Framework sehr abstrakt. Zur Veranschaulichung von MVC und MVVM sollen diese beiden Muster folgend in JavaFX umgesetzt werden. JavaFX wird hier als populäres UI-Framework für Desktop-Anwendungen in Java exemplarisch ausgewählt. Auf Grundlagen von JavaFX wird an dieser Stelle nur kurz eingegangen. Eine ausführlichere Einführung in JavaFX erfolgt im Kapitel [Desktop-App mit JavaFX](#).

Als Beispiel soll das einfache Spiel [Tic-Tac-Toe](#) implementiert werden. Das UI wird in der folgenden Abbildung dargestellt. Die *View* enthält folgende Attribute, um ihren Zustand zu erfassen, und Methoden zur Steuerung der Anwenderinteraktion:

- Attribut **board**: Spielbrett bestehend 9 Zellen, die jeweils leer oder mit einem X oder einem O belegt sind
- Attribut **winner**: Label, dass ggf. einen Gewinner ausweist
- Methode **selectCell**: Auswählen einer Zelle, um diese im Wechsel mit einem X oder einem O zu markieren
- Methode **reset**: Zurücksetzen des bisherigen Spiels, um ein neues Spiel zu starten



UI der Beispielanwendung Tic-Tac-Toe

Es folgt der Code einer Tic-Tac-Toe-Implementierung, die dem gemäß Dokumentation vorgeschlagen MVC-Muster in JavaFX folgt. Der Code wird anschließend erläutert.

View

```
1  <StackPane xmlns="http://javafx.com/javafx/8.0.121" xmlns:fx="http://javafx.com/fxml/1"
2  fx:controller="controller.TicTacToeController">
3      <stylesheets><URL value="@tictactoe.css" ></URL></stylesheets>
4      <children>
5          <VBox>
6              <GridPane fx:id="board" hgap="10" vgap="10"></GridPane>
7              <Label fx:id="winner" styleClass="winnerMessage"></Label>
8              <HBox alignment="CENTER_RIGHT">
9                  <Button onAction="#reset" text="Reset"></Button>
10             </HBox>
11         </VBox>
12     </children>
</StackPane>
```

Controller

```
1  import javafx.fxml.FXML;
2  import javafx.scene.control.Button;
3  import javafx.scene.control.Label;
4  import javafx.scene.layout.GridPane;
5  // ...
6
7  public class TicTacToeController {
8
9      private Board model; // controller is connected to model here
10
11      @FXML
12      private GridPane board;
13
14      @FXML
15      private Label winner;
16
17      @FXML
18      protected void initialize() {
19          model = new Board();
20          for (int row = 0; row < 3; row++) {
21              for (int col = 0; col < 3; col++) {
22                  Button cell = new Button();
23                  cell.getStyleClass().add("cell");
24                  cell.setOnAction(event -> selectCell(event));
25                  board.add(cell, col, row);
26              }
27          }
28      }
29
30      public void selectCell(ActionEvent event) {
31          Button cell = (Button) event.getSource();
32          int row = GridPane.getRowIndex(cell);
33          int col = GridPane.getColumnIndex(cell);
34
35          Player player = model.markCell(row, col);
36          if (player != null) {
37              cell.setText(player.toString());
38              if (model.getWinner() != null) {
39                  winner.setText("Winner is " + player.toString());
40              }
41          }
42      }
43
44      public void reset(ActionEvent event) {
45          model.restart();
46          winner.setText("");
47          board.getChildren().forEach(node -> ((Button) node).setText(""));
48      }
49  }
```


Model

```
1  enum Player {X, O}
2
3  class Cell {
4      Player value;
5  }
6
7  class Board {
8      Cell[][] cells = new Cell[3][3];
9      Player currentTurn;
10     Player winner;
11
12     public Board() { restart(); }
13
14     /**
15      * Start a new game, i.e. clear the board and the game status.
16      */
17     public void restart() {
18         clearCells();
19         winner = null;
20         currentTurn = Player.X;
21     }
22
23     /**
24      * Mark a cell for the player in turn.
25      * Nothing is done, if row or col are out of range, or the cell is already marked, or the game is finished.
26      *
27      * @param row 0..2
28      * @param col 0..2
29      * @return player who made the turn
30      */
31     public Player markCell(int row, int col) {
32         if (isValid(row, col)) {
33             Player player = currentTurn;
34             cells[row][col].value = currentTurn;
35             if (isWinningMoveByPlayer(currentTurn, row, col)) { // check if game is won
36                 winner = currentTurn;
37             } else {
38                 currentTurn = currentTurn == Player.X ? Player.O : Player.X; // flip current turn
39             }
40             return player;
41         }
42         return null;
43     }
44
45     public Player getWinner() { return winner; }
46
47     void clearCells() {
48         for (int i = 0; i < 3; i++) {
49             for (int j = 0; j < 3; j++) {
50                 cells[i][j] = new Cell();
51             }
52         }
53     }
54
55     boolean isValid(int row, int col) {
56         return !(winner != null || isOutOfBounds(row) || isOutOfBounds(col) || isCellValueAlreadySet(row, col));
57     }
58
59     boolean isOutOfBounds(int i) { return i < 0 || i > 2; }
60
61     boolean isCellValueAlreadySet(int row, int col) { return cells[row][col].value != null; }
62
63     boolean isWinningMoveByPlayer(Player player, int row, int col) {
64         return cells[row][0].value == player && cells[row][1].value == player && cells[row][2].value == player // 3 in a row
65             || cells[0][col].value == player && cells[1][col].value == player && cells[2][col].value == player // 3 in a col
66             || cells[0][0].value == player && cells[1][1].value == player && cells[2][2].value == player // 3 in a diagonal
67             || cells[0][2].value == player && cells[1][1].value == player && cells[2][0].value == player; // other diagonal
68     }
69 }
```

Application

```
1 import javafx.application.Application;
2 // ...
3
4 public class TicTacToeApplicationMVC extends Application {
5
6     @Override
7     public void start(Stage stage) throws Exception {
8         Parent view = FXMLLoader.load(getClass().getResource("view/tictactoe.fxml"));
9         Scene scene = new Scene(view);
10        stage.setScene(scene);
11        stage.show();
12    }
13
14    public static void main(String[] args) { launch(args); }
15 }
```

- **View:** In JavaFX kann das UI sowohl deklarativ beschrieben als auch imperativ aufgebaut werden. Beide Ansätze können auch kombiniert werden. Es ist zu empfehlen, die statische Grundstruktur einer *View*, z.B. den grundlegenden Aufbau eines Formulars, deklarativ zu beschreiben. Die Auszeichnungssprache von JavaFX basiert auf XML und heißt [FXML](#). In obigem Code ist die Grundstruktur der *View* in einer Datei `view/tictactoe.fxml` mittels FXML deklariert.
 - In Zeile 1 wird die *View* mit einer *Controller*-Klasse verknüpft: `fx:controller="controller.TicTacToeController"`. In JavaFX wird die *View* auch im MVC-Muster nicht direkt auf das *Model* zugreifen, sondern indirekt über den *Controller* kommunizieren.
 - In JavaFX kann das Layout einer *View* wie aus der Web-Entwicklung bekannt mittels *Cascading Stylesheets* (CSS) gestaltet werden. In Zeile 2 wird auf ein Stylesheet verwiesen, in dem die verwendeten Style-Klassen definiert sind.
 - Das `GridPane` (Zeile 5), das später die 9 Zellen des Spielbretts enthalten soll, ist hier leer. Die Zellen werden dynamisch bei der Initialisierung des *Controllers* als Kindelemente hinzugefügt.
- **Controller:** JavaFX erzeugt beim Laden der *View* automatisch ein Objekt der verknüpften *Controller*-Klasse. Diese Klasse hat diverse Abhängigkeiten zu spezifischen UI-Elementen von JavaFX, z.B. `javafx.fxml.FXML` und `javafx.scene.control.Button` (Zeilen 1-4).
 - Die Attribute, die mit einer `FXML`-Annotation versehen sind, werden an ein Element in der deklarativen *View* bidirektional gebunden. Die Konvention ist, dass das `Id`-Attribut `fx:id` des zu verbindenden Elements im FXML-Dokument der Bezeichnung des Attributs in der *Controller*-Klasse entspricht und die jeweiligen Typen übereinstimmen. Das Label `<Label fx:id="winner">` (Zeile 6 in *View*) ist z.B. über seine `Id` an das Attribut `Label winner` (Zeile 15 in *Controller*) gebunden. Die Attribute müssen nicht instantiiert werden, die entsprechenden Objekte werden von JavaFX injiziert (Dependency Injection).
 - JavaFX führt die mit einer `FXML`-Annotation versehene Methode `initialize` aus, während das *Controller*-Objekt erzeugt wird. In dieser Methode wird das *Model* instantiiert (Zeile 19), und es werden 9 Buttons dem Spielbrett als Zellen hinzugefügt (Zeilen 20-27). Es bietet sich an, die Grundstruktur einer *View* immer dann dynamisch zu erweitern, wenn (wie hier) eine Menge ähnlicher Elemente hinzugefügt werden soll und dazu mittels einer oder mehrerer Schleifen über die Menge iteriert werden kann.
 - Die Methode `reset` wird über ihre Bezeichnung in der *View* referenziert (Zeile 8: `<Button onAction="#reset">`) und durch JavaFX aufgerufen, wenn der Anwender den Button anklickt.
- **Model:** Im *Model* ist die einfache Logik des Spiels Tic-Tac-Toe implementiert – und das unabhängig davon, mit welchem UI-Framework das Spiel dargestellt wird. In diesem Fall ist das *Model* einem JavaFX-*Controller* (`TicTacToeController` für JavaFX) bekannt, der es nach Eingaben des Anwenders manipuliert. Es ist vorstellbar, dass das Spiel in einem weiteren UI-Framework für andere Zielumgebungen (z.B. Android für Smartphones) implementiert wird. Mehrere *Controller* könnten sich dann ein gemeinsames *Model* teilen.
- **Application:** Die Klasse `TicTacToeApplicationMVC` erweitert die generische Klasse `javafx.application.Application` und startet die Anwendung. In der Methode `start` wird die *View* `view/tictactoe.fxml` initial geladen und angezeigt.

MVVM in JavaFX

Die Beispielanwendung soll jetzt dahingehend umgebaut (oder auch "refaktoriert") werden, dass sie dem MVVM-Muster entspricht. Die Grundidee von MVVM ist es, den gesamten Zustand des UI im *ViewModel* abzubilden. Das *ViewModel* enthält also Attribute für die aktuellen Werte in Textfeldern, Tabellen und anderen UI-Elementen. Auch unscheinbare Aspekte, wie z.B. ob ein Button aktiv oder inaktiv ist, werden im *ViewModel* erfasst. Die *View* präsentiert nur den Zustand des *ViewModel*. Wenn der Zustand durch den Anwender verändert wird, z.B. indem in einem Textfeld getippt wird, delegiert die *View* diese Interaktion umgehend an das *ViewModel*. Entgegengesetzt muss auch eine Änderung, die vom *ViewModel* ausgeht, umgehend in der *View* angezeigt werden. *View* und *ViewModel* müssen also stets synchron gehalten werden.

In JavaFX ermöglichen Implementierungen des Interface [Property](#) sowohl unidirektionales als auch bidirektionales [Data Binding](#). Bidirektionales Binding (auch *Two-Way-Binding* genannt) basiert in JavaFX darauf, dass sich zwei `Property`-Objekte gegenseitig beobachten. Der folgende Code verdeutlicht das Prinzip anhand von zwei `StringProperty`-Objekten.

```

1 StringProperty a = new SimpleStringProperty();
2 StringProperty b = new SimpleStringProperty();
3
4 a.bindBidirectional(b); // two-way-binding
5
6 a.set("Hello");
7 System.out.println(b.get()); // prints "Hello"
8
9 b.set("World!");
10 System.out.println(a.get()); // prints "World!"

```

Die Aufteilung in *View* und *ViewModel* hat folgende Vorteile:

- Designer können das UI hinsichtlich einer einfachen Bedienbarkeit gestalten, ohne Aspekte der Präsentationslogik im *ViewModel* beachten zu müssen. Gleichzeitig können Entwickler die Präsentationslogik implementieren, ohne die konkrete Darstellung der *View* zu kennen.
- Die Testbarkeit wird stark verbessert. Die *View* enthält nahezu keinen Kontrollfluss mehr, der im Unit Testing zu berücksichtigen ist. Ein wichtiger Vorteil ist es, dass das *ViewModel* frei von Abhängigkeiten zu den UI-Elementen von JavaFX ist. Daher müssen auch die *FXML*-Annotationen aus dem *ViewModel* ausgelagert werden, damit das *ViewModel* losgelöst von JavaFX getestet werden kann. Es kann z.B. einfach mittels [JUnit](#) getestet werden, wo vorher noch ein aufwändiger Integrationstest erforderlich war, in dem zunächst die gesamte Anwendung gestartet und Anwenderverhalten simuliert werden musste.

Im folgenden Code wird zur Implementierung des MMVM-Musters das Framework [MvvmFX](#) eingesetzt, das als Zusatz auf dem JavaFX-Standard aufsetzt. Es folgt zunächst wieder der Code, der anschließend erläutert wird.

View

```
1 // TicTacToeView.fxml
2 <StackPane xmlns="http://javafx.com/javafx/8.0.121" xmlns:fx="http://javafx.com/fxml/1" fx:controller="view.TicTacToeView">
3   <stylesheets><URL value="@../tictactoe.css"></URL></stylesheets>
4   <children>
5     <VBox>
6       <GridPane fx:id="board" hgap="10" vgap="10"></GridPane>
7       <Label fx:id="winner" styleClass="winnerMessage"></Label>
8       <HBox alignment="CENTER_RIGHT">
9         <Button onAction="#reset" text="Reset"></Button>
10      </HBox>
11    </VBox>
12  </children>
13 </StackPane>
14
15 // TicTacToeView.java
16 import javafx.fxml.FXML;
17 import javafx.scene.control.Button;
18 import javafx.scene.control.Label;
19 import javafx.scene.layout.GridPane;
20 import de.saxsys.mvvmfx.FxmlView;
21 import de.saxsys.mvvmfx.InjectViewModel;
22 // ...
23
24 public class TicTacToeView implements FxmlView<TicTacToeViewModel>, Initializable {
25
26     @InjectViewModel
27     private TicTacToeViewModel viewModel;
28
29     @FXML
30     private GridPane board;
31
32     @FXML
33     private Label winner;
34
35     @Override
36     public void initialize(URL url, ResourceBundle resourceBundle) {
37         winner.textProperty().bindBidirectional(viewModel.winnerMessage); // winner label is bind to view model
38
39         for (int row = 0; row < 3; row++) {
40             for (int col = 0; col < 3; col++) {
41                 Button cell = new Button();
42                 cell.getStyleClass().add("cell");
43                 cell.setOnAction(event -> selectCell(event));
44                 cell.textProperty().bindBidirectional(viewModel.cells[row][col]); // each cell button is bind to view model
45                 board.add(cell, col, row);
46             }
47         }
48     }
49
50     public void selectCell(ActionEvent event) {
51         Button cell = (Button) event.getSource();
52         viewModel.selectCell(GridPane.getRowIndex(cell), GridPane.getColumnIndex(cell));
53     }
54
55     public void reset(ActionEvent event) {viewModel.reset(); }
56 }
```

ViewModel

```
1  import javafx.beans.property.StringProperty;
2  import de.saxsys.mvvmfx.ViewModel;
3  // ...
4
5  public class TicTacToeViewModel implements ViewModel {
6
7      private Board model; // view model is connected to model here
8
9      public StringProperty[][] cells = new StringProperty[3][3];
10     public StringProperty winnerMessage = new SimpleStringProperty();
11
12     public TicTacToeViewModel() {
13         model = new Board();
14         for (StringProperty[] cellsInRow : cells) {
15             Arrays.setAll(cellsInRow, i -> new SimpleStringProperty());
16         }
17     }
18
19     public void selectCell(int row, int col) {
20         Player player = model.markCell(row, col);
21         if (player != null) {
22             cells[row][col].setValue(player.toString());
23             if (model.getWinner() != null) {
24                 winnerMessage.setValue("Winner is " + player.toString());
25             }
26         }
27     }
28
29     public void reset() {
30         model.restart();
31         winnerMessage.setValue("");
32         for (StringProperty[] cellsInRow : cells) {
33             Arrays.stream(cellsInRow).forEach(cell -> cell.setValue(""));
34         }
35     }
36 }
```

Model

```
1  enum Player {X, O}
2
3  class Cell {
4      Player value;
5  }
6
7  class Board {
8      Cell[][] cells = new Cell[3][3];
9      Player currentTurn;
10     Player winner;
11
12     public Board() { restart(); }
13
14     /**
15      * Start a new game, i.e. clear the board and the game status.
16      */
17     public void restart() {
18         clearCells();
19         winner = null;
20         currentTurn = Player.X;
21     }
22
23     /**
24      * Mark a cell for the player in turn.
25      * Nothing is done, if row or col are out of range, or the cell is already marked, or the game is finished.
26      *
27      * @param row 0..2
28      * @param col 0..2
29      * @return player who made the turn
30      */
31     public Player markCell(int row, int col) {
32         if (isValid(row, col)) {
33             Player player = currentTurn;
34             cells[row][col].value = currentTurn;
35             if (isWinningMoveByPlayer(currentTurn, row, col)) { // check if game is won
36                 winner = currentTurn;
37             } else {
38                 currentTurn = currentTurn == Player.X ? Player.O : Player.X; // flip current turn
39             }
40             return player;
41         }
42         return null;
43     }
44
45     public Player getWinner() { return winner; }
46
47     void clearCells() {
48         for (int i = 0; i < 3; i++) {
49             for (int j = 0; j < 3; j++) {
50                 cells[i][j] = new Cell();
51             }
52         }
53     }
54
55     boolean isValid(int row, int col) {
56         return !(winner != null || isOutOfBounds(row) || isOutOfBounds(col) || isCellValueAlreadySet(row, col));
57     }
58
59     boolean isOutOfBounds(int i) { return i < 0 || i > 2; }
60
61     boolean isCellValueAlreadySet(int row, int col) { return cells[row][col].value != null; }
62
63     boolean isWinningMoveByPlayer(Player player, int row, int col) {
64         return cells[row][0].value == player && cells[row][1].value == player && cells[row][2].value == player // 3 in a row
65             || cells[0][col].value == player && cells[1][col].value == player && cells[2][col].value == player // 3 in a col
66             || cells[0][0].value == player && cells[1][1].value == player && cells[2][2].value == player // 3 in a diagonal
67             || cells[0][2].value == player && cells[1][1].value == player && cells[2][0].value == player; // other diagonal
68     }
69 }
```


Application

```
1  import javafx.application.Application;
2  // ...
3
4  public class TicTacToeApplicationMVVM extends Application {
5
6      @Override
7      public void start(Stage stage) {
8          ViewTuple<TicTacToeView, TicTacToeViewModel> viewTuple = FluentViewLoader.fxmlView(TicTacToeView.class).load();
9          Scene scene = new Scene(viewTuple.getView());
10         stage.setScene(scene);
11         stage.show();
12     }
13
14     public static void main(String[] args) { launch(args); }
15 }
```

- **View:** Nach dem Verständnis des MVVM-Musters besteht die *View* nun aus 2 Dateien:
 - zum einen aus dem deklarativen FXML-Dokument, das den strukturellen Aufbau der *View* beschreibt (Zeilen 2-13),
 - und zum anderen aus der per Two-Way-Binding verknüpften Java-Klasse, die den Zustand und das Verhalten der *View* steuert und die im obigen MVC-Muster in Teilen dem *Controller* entsprach (Zeilen 16-56).

Eine Konvention des MvvmFX-Framework verlangt es, dass beide Dateien im gleichen Verzeichnis liegen und gleich benannt sind, hier demzufolge `view/TicTacToeView.fxml` und `view/TicTacToeView.java`. Das FXML-Dokument bleibt inhaltlich unverändert. Die Java-Klasse enthält die Abhängigkeiten zu den UI-Elementen von JavaFX, z.B. sämtliche `FXML`-Annotationen (Zeilen 16-20). In der Klasse wird sämtliche Logik entfernt, die den Zustand der *View* verändert, und stattdessen an das *ViewModel* delegiert, z.B. in den Methoden `selectCell` (Zeile 52) und `reset` (Zeile 55). Der initiale dynamische Aufbau der *View* (Zeilen 39-47), der alternativ auch in FXML hätte deklariert werden können, gehört weiterhin zur *View*. Das *ViewModel* wird per Dependency Injection injiziert (Zeilen 26-27). Die Attribute, die den Zustand der *View* abbilden, werden im *ViewModel* gespiegelt, und per Two-Way-Binding verknüpft (Zeilen 37 und 44). Dazu wird jeweils eine beobachtbare JavaFX-`StringProperty` genutzt.

- **ViewModel:** Das *ViewModel* hat keine Abhängigkeiten zu UI-Elementen von JavaFX (s. `import`-Anweisungen in Zeilen 1-2 im *ViewModel*). Wie der *Controller* in MVC kennt das *ViewModel* eine Instanz des *Model*, um es zu manipulieren (Zeile 7). Der Zustand der *View* wird über die `StringProperty`-Attribute in den Zeilen 9-10 gespiegelt. Ansonsten enthält das *ViewModel* die Präsentationslogik, die die Anwenderinteraktion steuert, d.h. hier die Methoden `selectCell` und `reset`. Für diese Methoden können einfache *Unit Tests* geschrieben werden – und zwar unabhängig von JavaFX.
- **Model:** Das *Model* ist nicht verändert worden.
- **Application:** Wie zuvor startet die Klasse `TicTacToeApplicationMVVM` in gewohnter Weise die JavaFX-Anwendung. Beim initialen Laden der *View* (Zeile 8) wird die Klasse `FluentViewLoader` von MvvmFX verwendet, um das *ViewModel* an die *View* zu binden. Ab hier verwaltet das Framework beide Objekte in einem sogenannten `ViewTuple`.

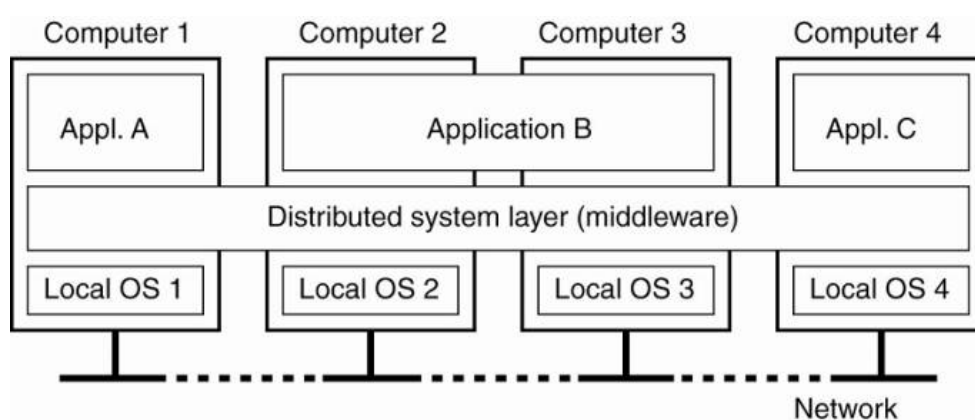
Die gezeigten Code-Beispiele zur Realisierung des MVC- und MVVM-Musters in JavaFX finden sich in den Verzeichnissen `/architecture/mvc` und `/architecture/mvvm` des Modul-Repository.

Kommunikation und Skalierbarkeit im verteilten System

Ein verteiltes System (engl. *Distributed System*) ist eine Menge unabhängiger Computer, die sich ihren Anwendern als ein einziges zusammenhängendes System präsentieren [\[TvS16\]](#). In einem verteilten Anwendungssystem werden also Teilprozesse der Anwendung auf unterschiedlichen Computern ausgeführt.

"A distributed system is a collection of independent computers that appears to its users as a single coherent system." (Andrew Tanenbaum)

Ein Computer kann dabei ganz unterschiedlich ausgeprägt sein, z.B. als handelsübliches Notebook oder Smartphone eines Anwenders, als Einplatinencomputer (z.B. [Raspberry Pi](#), [Arduino](#)) eingebettet in ein Gerät mit speziellen Sensoren/Aktuatoren oder als dedizierter oder virtueller Server in einem Rechenzentrum. Es wird i.d.R. eine sogenannte [Shared Nothing-Architektur](#) angenommen – ein Begriff den Michael Stonebraker bereits 1986 prägte (s. [The Case for Shared Nothing](#)). Die Computer in einem verteilten System werden dabei vereinfacht Knoten genannt. Jeder Knoten verfügt über mind. einen Prozessor, über flüchtigen Arbeitsspeicher (RAM) und über nicht-flüchtigen Sekundärspeicher (HDD, Flash-Speicher). Die Prozessoren führen die Teilprozesse (engl. *Threads*) der verteilten Anwendung aus. Da sich die Knoten keinen gemeinsamen Speicher teilen, kommunizieren sie untereinander über Nachrichten. Die Koordination der Knoten erfolgt also auf Softwareebene und setzt eine Netzwerkverbindung der Knoten voraus. Die Netzwerkverbindung kann u.U. vorübergehend unterbrochen sein.



Struktureller Aufbau eines verteilten Systems nach [\[TvS16\]](#).

In der obigen Abbildung ist zu erkennen, dass alle am System beteiligten Computer mit unterschiedlichen Betriebssystemen (engl. *Operating System*, kurz OS) ausgestattet sein können. Die Komponenten einer verteilten Anwendung können direkt untereinander kommunizieren – wie für Application B in der obigen Abbildung angedeutet. In den folgenden Kapiteln werden wir aktuell geläufige Protokolle für diese Interprozesskommunikation, die die Grenzen eines Computers überwindet, vorstellen, z.B. [RMI](#), [SOAP](#) und [WebSockets](#). Alternativ können die Komponenten in einem verteilten System auch indirekt Nachrichten über eine vermittelnde Middleware austauschen. Neben verteilten Datenbanksystemen bieten sich hier insbesondere zuverlässige *Message Broker* wie [Kafka](#) oder [RabbitMQ](#) an, die auf einen hohen Durchsatz an vermittelten Nachrichten ausgelegt sind.

Eine *Shared Nothing-Architektur* lässt sich leicht skalieren, indem weitere Knoten dem System hinzugefügt werden oder wieder entfernt werden. Die maßgeblichen Motivationsfaktoren bei der Skalierung sind eine Verbesserung der Performance und eine Verbesserung der Datensicherheit durch Replikation.

Kommunikation im verteilten System

Es existiert eine Vielzahl von Protokollen zur Kommunikation in einem verteilten System, d.h. zum Austausch von Nachrichten/Daten zwischen Software-Komponenten, die in Prozessen auf verschiedenen Computern ausgeführt werden und über ein Netzwerk miteinander verbunden sind. Entsprechend den Anforderungen eines konkreten Anwendungsfalls ist ein geeignetes Protokoll als Grundlage für diese Interprozesskommunikation auszuwählen. Eine erste Orientierung bietet das [TCP/IP-Modell](#), das die Kommunikationsprotokolle gemäß der folgenden Abbildung in 4 Schichten untergliedert.

Aufgabe der Schicht	TCP/IP-Modell	OSI-Modell	Einheiten	Ausgewählte Protokolle
Serialisierung der Anwendungsdaten	Anwendungsschicht	7. Anwendung 6. Präsentation 5. Sitzung	Daten	RMI, WebSocket, SOAP, HTTP, FTP, SMTP, SSH, ...
Ende-zu-Ende-Kommunikation zwischen lokalem und entferntem Socket (= IP-Adresse + Port)	Transportschicht	4. Transport	Segmente (TCP) Datagramme (UDP)	TCP, UDP
Routing von Paketen mittels IP-Adressen	Internetschicht	3. Netzwerk	Pakete	IPv4, IPv6
Auflösung von IP- zu MAC-Adressen	Netzzugangsschicht	2. Sicherung	Frames	Ethernet (802.3), Wireless (802.11), ...
Physische Bitübertragung		1. Bitübertragung	Bits	...

Protokolle zur Remote-Kommunikation und das TCP/IP-Schichtenmodell

Aus der Perspektive eines Anwendungsentwicklers möchten wir uns auf die Protokolle der unteren Schichten (Transport-, Internet-, Netzzugangsschicht im TCP/IP-Modell) verlassen und ein adäquates Protokoll auf Ebene der Anwendungsschicht nutzen. Wir verfolgen grundsätzlich das Ziel, dass eine lokale Client-Komponente eine Kommunikationsverbindung zu einer entfernten (engl. *remote*) Server-Komponente aufbaut und anschließend anwendungsspezifische Daten austauschen kann. In den nächsten Kapiteln werden dazu folgende alternative Ansätze zur sogenannten Remote-Kommunikation am Beispiel von Java-Anwendungen vorgestellt:

- [Remote Method Invocation \(RMI\)](#)
- [WebSockets](#)
- [Sockets](#)
- [SOAP-Webservices](#)
- [REST-Webservices](#)

Eine wichtige Entwurfsentscheidung ist, ob die Kommunikation zwischen dem aufrufenden Client und dem Server nach dem *Pull-Prinzip* oder nach dem *Push-Prinzip* erfolgen soll. Der Unterschied zwischen beiden Kommunikationsprinzipien ist bereits im Kapitel zum [Beobachter-Muster](#) vorgestellt worden.

- **Pull-Prinzip:** Der Client erfragt den aktuellen Status einer Ressource beim Server. RMI und insbesondere Webservices sind konzeptionell auf das Pull-Prinzip ausgerichtet. Da sowohl SOAP- als auch REST-basierte Webservices auf dem HTTP-Protokoll aufsetzen, ist nur eine unidirektionale Kommunikation möglich.
- **Push-Prinzip:** Der Server teilt den vorab registrierten Clients kontinuierlich Statusänderungen mit. Sockets und WebSockets sind konzeptionell auf das Push-Prinzip ausgerichtet. Auch bei RMI lässt sich das Beobachter-Muster umsetzen, um eine derartige bidirektionale Kommunikation abzubilden.

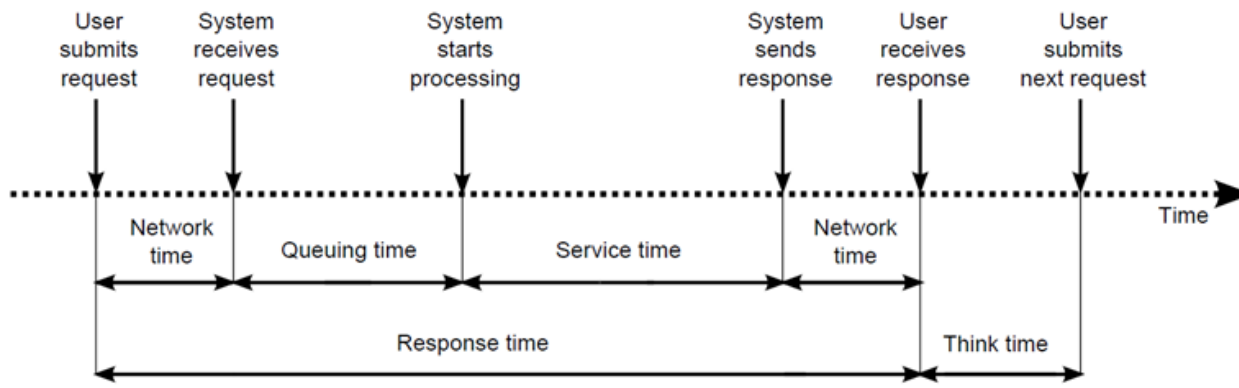
Eine weitere Entwurfsentscheidung ist, ob die Remote-Kommunikation zwischen Client und Server synchron oder asynchron erfolgen soll.

- **Synchrone Kommunikation:** Der Client wartet nach einer Anfrage an den Server auf dessen Antwort und ist währenddessen blockiert. Erst nach dem Empfang der Antwort wird der Kontrollfluss im Client fortgesetzt.
- **Asynchrone Kommunikation:** Der Client sendet eine Anfrage an den Server und setzt anschließend den Kontrollfluss – ohne zu blockieren – unmittelbar fort. Wenn die Antwort des Servers später irgendwann eingeht, wird sie mittels einer Callback-Funktion verarbeitet.

Es ist allgemein anzustreben, dass die Remote-Kommunikation (z.B. in Form von HTTP-Requests) nicht das User Interface eines Client blockiert. Insbesondere bei synchroner Remote-Kommunikation ist darauf zu achten, dass die Anfragen an einen Server nicht aus dem Thread der Anwendung abgesetzt werden, in dem auch die Anwenderinteraktion verarbeitet wird. Auf der Serverseite ist zu berücksichtigen, dass i.d.R. mit mehreren Clients parallel kommuniziert werden soll. Daher ist sowohl bei der Implementierung der Client-Komponente als auch bei der Implementierung der Server-Komponente zu empfehlen, die Remote-Kommunikation in nebenläufige Threads zu verlagern.

Performance

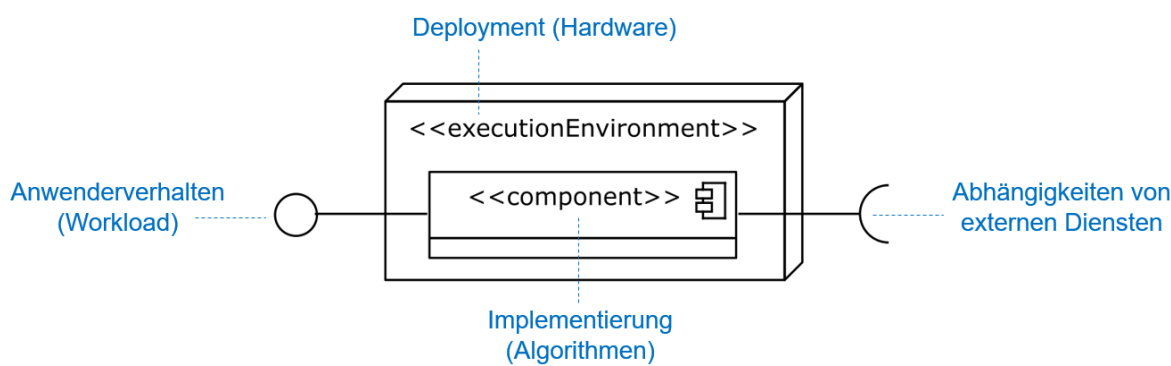
Die Performance eines Anwendungssystems beschreibt den Erfüllungsgrad von spezifizierten Zielen bezüglich der Metriken Antwortzeit und Durchsatz [\[SW01\]](#). Beide Metriken lassen sich einfach beobachten. Die Antwortzeit (engl. *Response Time*) wird i.d.R. aus Sicht der Anwenders gemessen, der eine Anfrage an das System sendet, und kann wie in der folgenden Abbildung dargestellt in Netzwerkzeit, Wartezeit und Bedienzeit zerlegt werden. Jeder dieser Teilaspekte der Antwortzeit kann für sich optimiert werden.



Unterteilung der Antwortzeit eines Systems in Netzwerk-, Warte- und Bedienzeit

"Performance is the degree to which a software system or component meets its objectives for timeliness. There are two important dimensions to software performance: responsiveness and scalability. Responsiveness is the ability of a system to meet its objectives for response time or throughput. Scalability is the ability of a system to continue to meet its response time or throughput objectives as the demand for the software functions increases." (Connie Smith, Lloyd Williams)

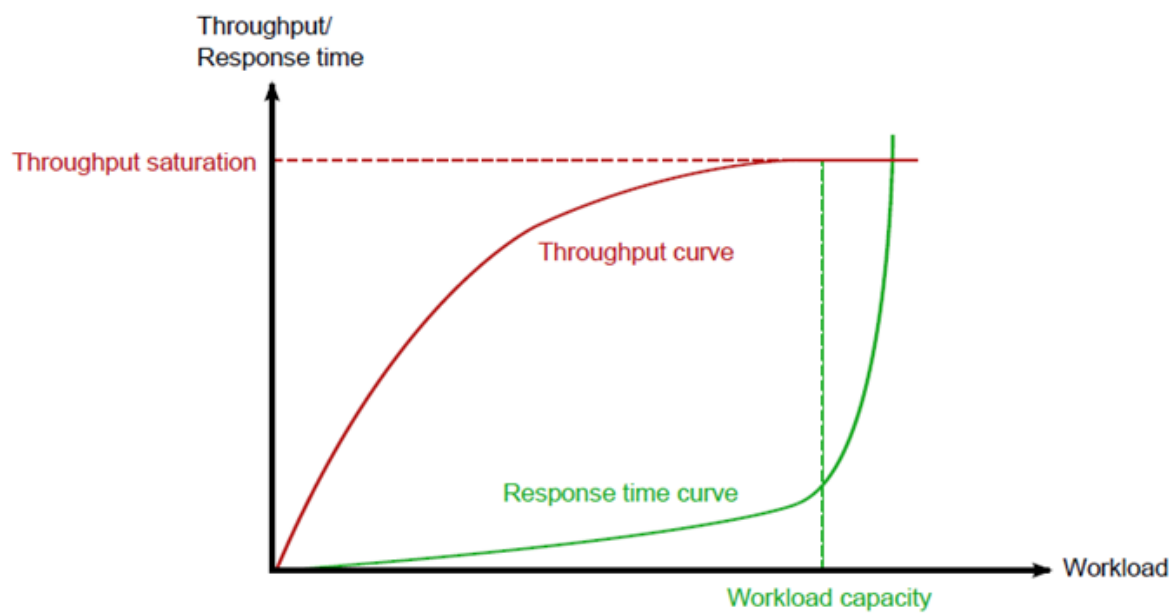
Es ergeben sich folgende Einflussfaktoren auf die Performance einer Komponente zur Laufzeit:



Einflussfaktoren auf die Antwortzeit einer Komponente

- **Implementierung:** In Abhängigkeit der verwendeten Datenstrukturen und Algorithmen kann eine Komponente unterschiedlich effizient implementiert werden.
- **Deployment:** Die Laufzeitumgebung beeinflusst die Performance einer Komponente maßgeblich. Zur Laufzeitumgebung zählen neben den eigentlichen Hardware-Ressourcen auch das Betriebssystem und andere Software-Container. Der einfachste Ansatz die Performance zu verbessern, ist das Prinzip "Kill it with iron", d.h. leistungsfähigere Hardware-Ressourcen einzusetzen.
- **Abhängigkeiten:** Wenn externe Dienste anderer Komponenten aus einer Komponente heraus (insbesondere synchron) aufgerufen werden, wird dadurch die Antwortzeit beeinflusst – z.B. bei Anfragen an Datenbanksysteme oder Webservices. Die Abhängigkeiten einer Komponenten sind i.d.R. vor dem Aufrufer verborgen. Dadurch kann der Aufrufer bei einer Störung nicht unterscheiden, ob die Komponente selbst oder einer der Dienste, von denen sie abhängt, die Störung verursachen.
- **Anwenderverhalten:** Die Performance einer Komponente wird vom Interaktionsverhalten der Anwender beeinflusst. Antwortzeiten und Durchsatz hängen vom aktuellen *Workload* ab, d.h. in welcher Frequenz neue Anfragen eingehen und wie viel Datentransfer und Berechnungsaufwand durch die unterschiedliche Parametrisierung der Anfragen verursacht wird. Eine Komponente sollte möglichst mehrere Anfragen parallel verarbeiten können. Es kann notwendig sein, Prioritäten beim Scheduling der Anfragen zu berücksichtigen.

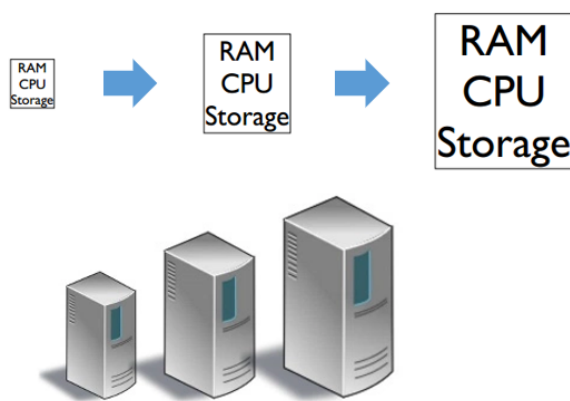
Skalierbarkeit drückt das Verhältnis zwischen der Steigerung der Problemgröße (z.B. mehr Workload) und den zur Lösung zusätzlich benötigten Ressourcen aus. Antwortzeiten und Durchsatz skalieren bei steigendem Workload und beschränkter Ressourcen-Kapazität bis zum Erreichen der Kapazitätsgrenze etwa linear. Anschließend steigen die Antwortzeiten exponentiell, da eine bestimmte Ressource (z.B. CPU, RAM, I/O) aufgrund von Überlastung zum Engpass wird. Dieser typische Effekt ist in der folgenden Abbildung dargestellt. Gleichmaßen erreicht auch der Durchsatz einen Sättigungspunkt, an dem die Komponente oder das System einfach nicht mehr Anfragen pro Zeitintervall verarbeiten kann. In automatisch skalierenden Cloud-Plattformen kann dieser Sättigungspunkt soweit hinausgezögert werden, dass er praktisch nicht erreicht wird. Allgemein ist es aus Gründen der Wirtschaftlichkeit zu empfehlen, die bereitgestellten Hardware-Ressourcen nicht zu stark überzudimensionieren.



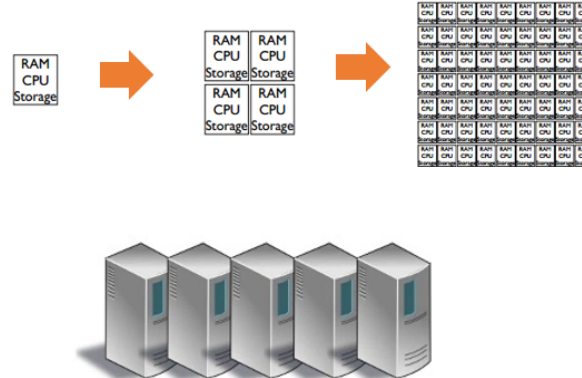
Skalierbarkeit von Antwortzeit und Durchsatz bei beschränkter Ressourcen-Kapazität

Um die Ressourcen-Kapazität eines Systems auszubauen, gibt es zwei grundlegende Ansätze: die vertikale Skalierung (*Scale Up*) und die horizontale Skalierung (*Scale Out*). Bei der vertikalen Skalierung wird ein leistungstarker Großrechner angestrebt, der sukzessiv ausgebaut wird. Dieser Ansatz gilt als vergleichsweise teuer. Außerdem stellt der Großrechner einen möglichen *Single Point of Failure* dar. Daher wird heute horizontale Skalierung durch ein verteiltes System von vielen gleichartigen Knoten favorisiert. In einem derartigen *Cluster* wird der Ausfall von wenigen Knoten als Normalzustand angenommen, der die Verfügbarkeit und die Konsistenz der Daten nicht gefährden darf. Zwar ist ein Cluster komplexer zu beherrschen als ein einzelner Großrechner, kann dafür aber skalierbare Performance und Datensicherheit gleichermaßen gewährleisten.

Vertikale Skalierung (*Scale Up*)



Horizontale Skalierung (*Scale Out*)

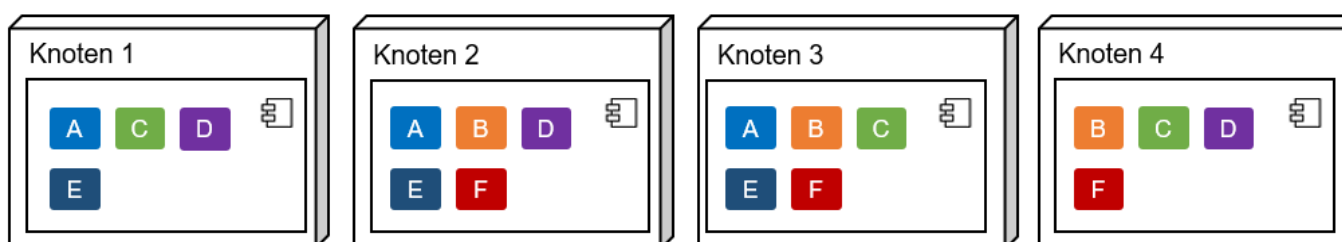


Vertikale und horizontale Skalierung

Datensicherheit durch Replikation

Nehmen wir an, dass wie oben dargestellt ein Cluster von gleichartigen Knoten aufgebaut worden ist, um auch bei hohem Workload die geforderte Performance des Systems zu gewährleisten. Eine Komponente wird innerhalb des Clusters nun mehrfach instantiiert, damit der Dienst, den sie anbietet, ebenso skaliert. Jede Instanz dieser Komponente wird nur einen Ausschnitt der gesamten Datenbasis verarbeiten und auf dem Knoten, auf dem sie ausgeführt wird, speichern. Die Daten werden in einem verteilten Datei- oder Datenbanksystem in sogenannte Partitionen (oder auch *Shards*) zerlegt. Einzelne Datenobjekte/Datensätze werden i.d.R. über eine Hashfunktion einer Partition zugeordnet. Die folgende Abbildung zeigt ein Cluster mit 4 Knoten, auf denen jeweils eine Instanz der gleichen Komponente ausgeführt wird. Die Datenbasis wurde in 6 Partitionen (A-F) unterteilt. Da das System keine Daten verlieren darf, wenn ein Knoten ausfällt, werden die Partitionen repliziert. Im dargestellten Beispiel ist der Replikationsfaktor 3, d.h. von jeder Partition gibt es 3 identische Replikate, so dass das abgebildete System den Ausfall beliebiger 2 Knoten tolerieren kann. Die Anzahl der Knoten und insbesondere der Partitionen ist in der Praxis häufig deutlich größer. Eine größere Anzahl an Partitionen als Knoten vereinfacht die Umverteilung der Partitionen, wenn ein neuer Knoten in den Cluster aufgenommen wird oder ein Knoten ausscheidet.

Cluster mit 4 Knoten, 6 Partitionen (A-F) und Replikationsfaktor 3



Partitionierung und Replikation von Daten im verteilten System

Ein Kompromiss zwischen Verfügbarkeit und Konsistenz der Daten ergibt sich dadurch, dass die Replikate nach einem Schreibzugriff synchronisiert werden müssen. Es stellt sich in einem verteilten System die Frage, ob während der Synchronisation bedingt durch einen Schreibzugriff entweder ein veralteter Zustand gelesen werden darf (d.h. Verfügbarkeit nicht eingeschränkt, aber Konsistenz eingeschränkt) oder das Lesen bis zum Ende der Synchronisation verzögert wird (d.h. Konsistenz nicht eingeschränkt, aber Verfügbarkeit eingeschränkt). In einem verteilten System ist grundsätzlich mit dem Ausfall einzelner Knoten oder der vorübergehenden Unterbrechung der Netzwerkverbindungen zu rechnen, was zu Verzögerungen in der Synchronisation führt. Der Begriff Partitionstoleranz beschreibt, dass das System bis zu einem gewissen Grad tolerant gegenüber einem kurzzeitigen Verfall des Systems in unverbundene Teile sein muss. Der Kompromiss zwischen der Konsistenz (engl. *Consistency*), der Verfügbarkeit (engl. *Availability*) und der Partitionstoleranz (engl. *Partition Tolerance*) in einem verteilten System ist als [CAP-Theorem](#) bekannt. Verteilte Datenbanken werden in diesem Model nicht weiter behandelt. Einen fundierten Überblick liefert das populäre Buch "Designing Data-Intensive Applications" von Martin Kleppmann [\[Kle17\]](#).

Remote Method Invocation (RMI)

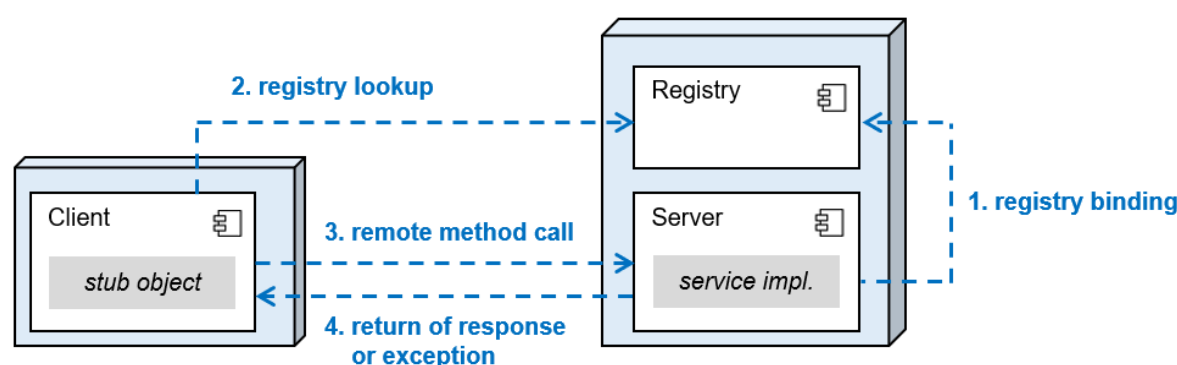
Remote Method Invocation (RMI) bezeichnet eine Java-API für entfernte Methodenaufrufe, die über das Java-Modul [java.rmi](#) zur Verfügung gestellt wird. "Entfernt" bedeutet dabei, dass Methoden auf Objekten aufgerufen werden, die sich im Adressraum (*Heap*) einer anderen *Java Virtual Machine (JVM)* befinden als der Kontext, aus dem der Methodenaufruf erfolgt. Ob die andere JVM auf einem entfernten Computer im Netzwerk oder testweise auf dem lokalen Computer läuft, ist unerheblich. Der entfernte Methodenaufruf soll für den Entwickler möglichst genauso aussehen, wie ein lokaler Methodenaufruf. Es muss lediglich eine spezielle [RemoteException](#) behandelt werden, die geworfen wird, falls die zugrundeliegende Netzwerkverbindung unterbrochen wird.

Auf der Seite des Client wird ein sogenanntes Stub-Objekt erzeugt, das die Aufgabe eines Remote-Proxy übernimmt (s. [Entwurfsmuster Proxy](#)). Das Stub-Objekt ist der Endpunkt für die Interprozesskommunikation im Adressraum des Client – als Stellvertreter für das Remote-Objekt im Adressraum der entfernten JVM. Es übernimmt das Verpacken der Input-Argumente bei einem entfernten Methodenaufruf (engl. *Marshalling*) und das Entpacken des Return-Arguments, sobald die Antwort empfangen wird (engl. *Unmarshalling*). Die Netzwerkkommunikation, die alle Schichten des TCP/IP-Modells durchläuft, wird durch das Stub-Objekt vor dem Entwickler verborgen.

Alle Klassen, deren Objekte als Input- oder Return-Argumente zwischen Client und Server ausgetauscht werden, müssen das Interface [java.io.Serializable](#) implementieren. Diese Objekte werden Datentransferobjekte (engl. [Data Transfer Objects](#), kurz *DTO*) genannt. Die DTO-Klassen beinhalten üblicherweise nur Attribute und keine fachlichen Operationen. Da die DTO-Klassen sowohl dem Client als auch dem Server bekannt sein müssen, bietet es sich an, sie in einem separaten Package abzulegen. Das Package kann Teil eines Moduls sein, von dem Client und Server gemeinsam abhängen, oder sie sind Teil des Server-Moduls und werden explizit für den Client exportiert.

Die RMI-Architektur sieht neben Client und Server konzeptionell noch eine weitere Komponente zur Registrierung von RMI-Services vor: die sogenannte *Registry*. Die Idee ist, dass die Entwickler einer Server-Komponente, über die ein bestimmter RMI-Service angeboten werden soll, diesen Service unter einem sprechenden Namen bei der Registry bekanntmachen. Die Registry dient als allgemein bekannte Anlaufstelle für potentielle Nutzer der RMI-Services. Sie können einen RMI-Service bei der Registry über seinen Namen nachschlagen und erhalten ein zugehöriges Stub-Objekt. Daraus ergibt sich der folgende Ablauf der Kommunikation, der zur Veranschaulichung in der nächsten Abbildung visualisiert ist.

0. Server: Erzeugen eines RMI-Service, `service = java.rmi.server.UnicastRemoteObject.exportObject(new MyServiceImpl())`
1. Server: Bekanntmachen des Service unter einem eindeutigen Namen bei der Registry, `registry.bind("MyServiceName", service)`
2. Client: Nachschlagen des Service bei der Registry und erzeugen eines zugehörigen Stub-Objekts, `stub = registry.lookup("MyServiceName")`
3. Client: Aufruf einer Methode auf dem Stub-Objekt inkl. möglicher Input-Argumente, `stub.myRemoteMethod(...)`
4. Client: Empfangen des Return-Arguments oder einer Exception



Ablauf der Kommunikation bei RMI

In der obigen Abbildung werden die Server- und die Registry-Komponente gemeinsam auf einem Host bereitgestellt, während die Client-Komponente auf einem anderen Host untergebracht ist. Tatsächlich sind alle 3 Komponenten unabhängig voneinander lauffähig und können auf beliebige Hosts verteilt werden.

Es können auch mehrere Clients parallel einen RMI-Service nutzen, da serverseitig die Methodenaufrufe je erzeugtem Client-Stub implizit in einem separaten Thread verarbeitet werden. Es liegt dabei in der Verantwortung des Entwicklers wechselseitigen Ausschluss zu gewährleisten, falls mehrere Clients eine Ressource zeitgleich manipulieren möchten, d.h. kritische Abschnitte sind mittels `synchronized` o.ä. zu schützen (s. Kapitel [Synchronisation von Threads](#)).

Anwendungsbeispiel RMI

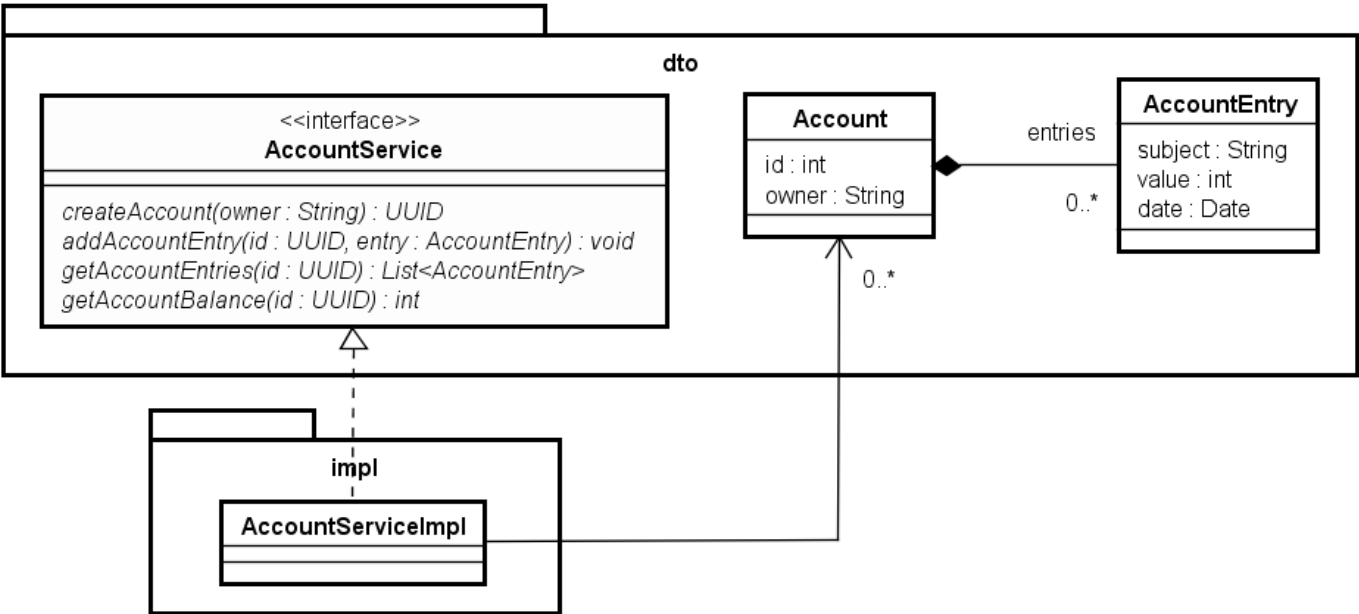
Das folgende Anwendungsbeispiel für RMI bildet einen vereinfachten Bankkontoservice ab. Ein beispielhaftes Bankkonto ([Account](#)) könnte wie folgt aussehen:

Buchungsdatum (date)	Betreff (subject)	Wert (value)
12.01.2021	Gutschrift 1	50
14.01.2021	Gutschrift 2	20
15.01.2021	Lastschrift 1	-30
Saldo (balance)		40

Kontoeinträge
(account entries)

Anwendungsbeispiel Bankkontoservice: Konto und Kontoeinträge

Die grundlegenden Klassen des Beispiels sind in folgendem Klassendiagramm dargestellt. Das Server-Modul stellt einen Bankkontoservice bereit, der die Methoden des Interface `AccountService` implementiert. Im Package `dto` sind neben diesem Interface noch die Klassen `Account` und `AccountEntry` enthalten, auf die der Client ebenfalls zugreifen kann, da sie als Datentransferobjekte genutzt werden. Auf das Package `impl` des Server-Moduls soll der Client nicht zugreifen können.



Klassendiagramm für das Anwendungsbeispiel Bankkontoservice

Der Bankkontoservice umfasst die folgenden Methoden:

- `UUID createAccount(String owner)`: In dieser Methode wird ein neues Bankkonto für den angegebenen Eigentümer erzeugt. Die eindeutige Id des Kontos (vom Typ `java.util.UUID`) wird zurückgegeben.
- `void addAccountEntry(UUID id, AccountEntry entry)`: In dieser Methode wird einem Bankkonto ein neuer Kontoeintrag (`AccountEntry`) hinzugefügt. Ein Kontoeintrag besteht aus einem Betreff, einem Wert und einem Buchungsdatum.
- `List<AccountEntry> getAccountEntries(UUID id)`: Diese Methode gibt alle Kontoeinträge zu einem Konto als Liste zurück.
- `int getAccountBalance(UUID id)`: Diese Methode gibt den Saldo (= Summe der Werte aller Kontoeinträge) eines Kontos zurück.

Im Folgenden wird zunächst der Code des Server-Moduls vorgestellt, anschließend der Code des Client-Moduls.

Modul-Deskriptor

```
1 module rmi_server {
2     requires java.rmi;
3     exports dto;
4 }
```

RegistryStarter

```
1 package impl;
2
3 import java.rmi.registry.Registry;
4 import java.rmi.registry.LocateRegistry;
5 import java.rmi.server.UnicastRemoteObject;
6 import java.rmi.RemoteException;
7 // ...
8
9 public class RegistryStarter {
10
11     public static void main(String[] args) {
12         try {
13             Registry registry = LocateRegistry.createRegistry(1099); // RMI default port is 1099
14             AccountService service = (AccountService) UnicastRemoteObject.exportObject(new AccountServiceImpl(), 0);
15             registry.rebind("AccountService", service);
16
17             System.out.println("RMI registry started and service bound.");
18         }
19         catch (RemoteException e) {
20             e.printStackTrace();
21         }
22     }
23 }
```

AccountService

```
1 package dto;
2
3 import java.rmi.Remote;
4 import java.rmi.RemoteException;
5 // ...
6
7 public interface AccountService extends Remote {
8
9     /**
10      * Creates a new account.
11      * @param owner account owner
12      * @return unique account id
13      * @throws RemoteException
14      */
15     UUID createAccount(String owner) throws RemoteException;
16
17     /**
18      * Adds a new account entry to the list of entries for the given account id.
19      * @param id account id
20      * @param entry new account entry
21      */
22     void addAccountEntry(UUID id, AccountEntry entry) throws RemoteException, AccountNotFoundException;
23
24     /**
25      * Returns the list of all account entries for the given account id.
26      * @param id account id
27      * @return account entries
28      */
29     List<AccountEntry> getAccountEntries(UUID id) throws RemoteException, AccountNotFoundException;
30
31     /**
32      * Calculates the account balance as the sum of all entry values for the given account id.
33      * @param id account id
34      * @return sum of all account entry values
35      */
36     int getAccountBalance(UUID id) throws RemoteException, AccountNotFoundException;
37 }
```

AccountServiceImpl

```
1 package impl;
2
3 import java.rmi.RemoteException;
4 // ...
5
6 public class AccountServiceImpl implements AccountService {
7
8     List<Account> accounts = new ArrayList<>();
9
10    @Override
11    public UUID createAccount(String owner) {
12        Account account = new Account(owner);
13        accounts.add(account);
14        return account.id;
15    }
16
17    @Override
18    public void addAccountEntry(UUID id, AccountEntry entry) throws AccountNotFoundException {
19        getAccountById(id).entries.add(entry);
20    }
21
22    @Override
23    public List<AccountEntry> getAccountEntries(UUID id) throws AccountNotFoundException {
24        return getAccountById(id).entries;
25    }
26
27    @Override
28    public int getAccountBalance(UUID id) throws AccountNotFoundException {
29        return getAccountEntries(id).stream().mapToInt(entry -> entry.value).sum();
30    }
31
32    private Account getAccountById(UUID id) throws AccountNotFoundException {
33        return accounts.stream().filter(a -> a.id.equals(id)).findFirst().orElseThrow(() -> new AccountNotFoundException());
34    }
35 }
```

Account

```
1 package dto;
2 // ...
3
4 public class Account implements Serializable {
5
6     UUID id;
7     String owner;
8     List<AccountEntry> entries;
9
10    public Account(String owner) {
11        this.id = UUID.randomUUID();
12        this.owner = owner;
13        entries = new ArrayList<>();
14    }
15 }
```

AccountEntry

```
1 package dto;
2 // ...
3
4 public class AccountEntry implements Serializable {
5
6     String subject;
7     int value;
8     Date date;
9
10    public AccountEntry(String subject, int value) {
11        this.subject = subject;
12        this.value = value;
13        this.date = new Date();
14    }
15 }
```

AccountNotFoundException

```
1 package dto;
2
3 public class AccountNotFoundException extends Exception { }
```

- **Modul-Deskriptor:** Unser Modul `rmi_server` ist vom Java-Modul `java.rmi` abhängig und exportiert explizit nur das Package `dto` an potentielle Clients.
- **RegistryStarter:** In Zeile 13 wird mittels `LocateRegistry.createRegistry(1099)` eine lokale Registry auf dem Port 1099 gestartet. Dieser Port ist der [registrierte Default-Port für RMI](#). Alternativ könnte mittels `LocateRegistry.getRegistry("10.0.0.2", 1099)` eine bereits vorab auf einem anderen Host gestartete Registry adressiert werden. In Zeile 14 wird mittels `UnicastRemoteObject.exportObject()` der RMI-Service instantiiert und anschließend in Zeile 15 unter dem Namen "AccountService" über die Registry veröffentlicht. Der RMI-Service ist nun für Clients unter diesem Namen erreichbar.
- **AccountService:** Das Interface beschreibt die [Fassade](#) des Server-Moduls und deklariert die Signaturen der oben vorgestellten, fachlichen Methoden des RMI-Service. Es erweitert per Konvention das Interface `java.rmi.Remote`. Jede der Methoden muss eine `java.rmi.RemoteException` werfen dürfen – z.B. für den Fall, dass die Netzwerkverbindung unterbrochen wird, aber auch für andere Fehler während des Nachrichtenaustausches. Dieses Interface kapselt sämtliche entfernte Methodenaufrufe und muss daher auch dem Client zur Verfügung stehen. Da der Client ggf. von Dritten entwickelt wird, ist eine ausführliche Dokumentation sinnvoll.
- **AccountServiceImpl:** Diese Klasse enthält die interne Implementierung des Interface `AccountService`. In Zeile 8 wird hier eine Liste von Konten verwaltet.
- **Account, AccountEntry und AccountNotFoundException:** Die DTO-Klassen sind Teil des exportierten Package `dto`. Jede DTO-Klasse muss das Interface `java.io.Serializable` implementieren, damit die Objekte der Klasse serialisiert, mittels TCP/IP über das Netzwerk übertragen und auf der Seite des Empfängers wieder deserialisiert werden können.

Modul-Deskriptor

```
1 module rmi_client {
2     requires java.rmi;
3     requires rmi_server;
4 }
```

Client

```
1 package client;
2
3 import dto.AccountService;
4 import dto.AccountEntry;
5 import dto.AccountNotFoundException;
6
7 import java.rmi.registry.Registry;
8 import java.rmi.registry.LocateRegistry;
9 // ...
10
11 public class Client {
12
13     public static void main(String[] argv) {
14         try {
15             Registry registry = LocateRegistry.getRegistry("localhost", 1099);
16             AccountService stub = (AccountService) registry.lookup("AccountService");
17
18             // Create an account on the remote side and add some account entries
19             UUID id = stub.createAccount("Hendricks");
20             stub.addAccountEntry(id, new AccountEntry("Credit 1", 50));
21             stub.addAccountEntry(id, new AccountEntry("Credit 2", 20));
22             stub.addAccountEntry(id, new AccountEntry("Debit 1", -30));
23
24             // Query account balance and entry count
25             System.out.println("Account balance = " + stub.getAccountBalance(id));
26             System.out.println("Account entries = " + stub.getAccountEntries(id).size());
27
28         } catch (RemoteException | NotBoundException | AccountNotFoundException e) {
29             e.printStackTrace();
30         }
31     }
32 }
33 }
```

- **Modul-Deskriptor:** Unser Modul `rmi_client` ist vom Java-Modul `java.rmi` und vom Modul `rmi_server` abhängig. Aus letzterem werden die DTO-Klassen und das Interface `AccountService` für die entfernten Methodenaufrufe importiert.

- **Client:** In Zeile 15 wird eine Verbindung zu der Registry aufgebaut, über die der Server seinen RMI-Service veröffentlicht hat. In Zeile 16 wird ein Stub-Objekt `stub` für den RMI-Service mit dem Namen "AccountService" erzeugt. Da wir wissen, dass dieser Service vom Typ `AccountService` ist, sollte der Cast gelingen. Alle folgenden Methodenaufrufe auf dem Objekt `stub` verursachen eine *Remote Method Invocation*, also einen Methodenaufruf auf einem Objekt in einer entfernten JVM, das sich hinter dem lokalen Objekt `stub` verbirgt. In Zeile 19 wird demzufolge ein neues Konto auf dem Server erzeugt und dessen Id zurückgegeben. Anschließend werden serverseitig Kontoeinträge für dieses Konto angelegt. Es folgen noch 2 weitere entfernte Methodenaufrufe, um den Saldo und die Anzahl der Kontoeinträge beim Server abzufragen und auszugeben (Zeilen 25-26). Anhand der `import`-Anweisungen (Zeilen 3-9) ist zu erkennen, dass der Client sowohl von Klassen aus dem Package `dto` des Servers als auch von Klassen aus den RMI-Packages `java.rmi.*` abhängig ist.

Die gezeigten Code-Beispiele zu RMI finden sich im Verzeichnis `/remote/rmi` des Modul-Repository.

Erweiterung des Anwendungsbeispiels RMI um Beobachter

Das Anwendungsbeispiel soll nun um die Möglichkeit erweitert werden, dass ein Client bei neuen Kontoeinträgen auf Konten, die er beobachtet, vom Server benachrichtigt wird. Es soll damit gezeigt werden, wie das [Entwurfsmuster Beobachter](#) mit RMI umgesetzt werden kann. Dazu wird das obige Anwendungsbeispiel im Server-Modul wie folgt erweitert:

AccountService

```

1 package dto;
2
3 import java.rmi.Remote;
4 import java.rmi.RemoteException;
5 // ...
6
7 public interface AccountService {
8
9     /**
10      * Adds an observer to the given account.
11      * @param observer remote account observer
12      * @param id id of observed account
13      * @throws RemoteException
14      */
15     void addObserver(AccountObserver observer, UUID id) throws RemoteException, AccountNotFoundException;
16
17     // ...
18 }

```

AccountServiceImpl

```

1 package impl;
2
3 import java.rmi.RemoteException;
4 // ...
5
6 public class AccountServiceImpl implements AccountService {
7
8     Map<Account, List<AccountObserver>> accountObservers = new ConcurrentHashMap<>();
9
10    @Override
11    public void addObserver(AccountObserver observer, UUID id) throws AccountNotFoundException {
12        Account account = getAccountById(id);
13        accountObservers.putIfAbsent(account, new ArrayList<>());
14        accountObservers.get(account).add(observer); // register observer
15    }
16
17    @Override
18    public void addAccountEntry(UUID id, AccountEntry entry) throws AccountNotFoundException {
19        super.addAccountEntry(id, entry);
20
21        Account account = getAccountById(id);
22        for (AccountObserver observer : accountObservers.get(account)) {
23            try {
24                observer.update(account, entry); // notify observer
25            } catch (RemoteException e) {
26                e.printStackTrace();
27            }
28        }
29    }
30
31    // ...
32 }

```


AccountObserver

```
1 package dto;
2
3 import java.rmi.Remote;
4 // ...
5
6 public interface AccountObserver extends Remote, Serializable {
7
8     /**
9      * Observer will be notified, when a new entry has been added to the observed account.
10      * @param account observed account
11      * @param newEntry new account entry
12      * @throws RemoteException
13      */
14     void update(Account account, AccountEntry newEntry) throws RemoteException;
15 }
```

- **AccountService**: Dem Interface für den RMI-Service wird eine Methode **addObserver** hinzugefügt, über die ein Client sich selbst als Beobachter (**AccountObserver**) für ein bestimmtes Konto registrieren kann. Der Client nennt dem Server die Id des Kontos, das er beobachten möchte.
- **AccountServiceImpl**: Diese Klasse wird um ein Attribut vom Typ **Map<Account, List<AccountObserver>>** erweitert, um die registrierten Beobachter je Konto zu speichern (Zeile 8). Die neue Methode **addObserver** des RMI-Service wird hier implementiert. Neue Beobachter werden in eine Beobachterliste je Konto eingetragen (Zeile 14). Zusätzlich wird die Methode **addAccountEntry** dahingehend erweitert, dass alle registrierten Beobachter eines Kontos benachrichtigt werden, wenn ein neuer Kontoeintrag angelegt wird (Zeilen 22-28). Da die Benachrichtigung jedes einzelnen Client fehlschlagen kann, muss jeweils eine **RemoteException** behandelt werden.
- **AccountObserver**: Dieses neue Interface ist durch den Client zu implementieren. Der Client kann hier individuell auf neue Kontoeinträge reagieren, die ihm entsprechend des Push-Prinzips vom Server mitgeteilt wird. Sobald ein neuer Kontoeintrag auf einem beobachteten Konto hinzugefügt wird und der Server die Methode **update** aufruft (Zeile 24 in **AccountServiceImpl**), wird eine *Remote Method Invocation* ausgelöst – nur dieses Mal in die entgegengesetzte Richtung: der Server ruft eine Methode auf, die auf einem Objekt ausgeführt wird, dass sich innerhalb der JVM des Client befindet.

Auf der Seite des Client wird das Interface **AccountObserver** wie folgt implementiert. Die implementierende Klasse **AccountObserverImpl** muss **java.rmi.server.UnicastRemoteObject** erweitern, um selbst entfernte Methodenaufrufe empfangen zu können.

- **AccountObserverImpl**: In der Implementierung der Methode **update** wird die eingehende Information über einen neuen Kontoeintrag lediglich auf die Konsole ausgegeben (Zeile 13).
- **Client**: Im Client wird zuerst ein neues Konto angelegt (Zeile 20) und ein Beobachter auf diesem Konto registriert (Zeile 24). Anschließend werden neue Kontoeinträge auf dem Server angelegt (Zeilen 27-28), um Benachrichtigungen über diese Einträge in einem nebenläufigen Thread zu provozieren. Wenn der Client im Debug-Modus gestartet und ein Breakpoint in der Methode **update** des Beobachters (Zeile 13 in **AccountObserverImpl**) gesetzt wird, ist zu erkennen, dass diese Methode in einem Thread namens "RMI TCP Connection (...)" in der Thread-Gruppe "RMI Runtime" ausgeführt wird – also parallel zum Thread "main" des Client. In Zeile 31 wird der Beobachter unterbrochen, so dass er nicht mehr auf Updates des Server lauscht.

AccountObserverImpl

```
1 package client;
2
3 import java.rmi.server.UnicastRemoteObject;
4 import java.rmi.RemoteException;
5 // ...
6
7 public class AccountObserverImpl extends UnicastRemoteObject implements AccountObserver {
8
9     public AccountObserverImpl() throws RemoteException { }
10
11     @Override
12     public void update(Account account, AccountEntry newEntry) throws RemoteException {
13         System.out.printf("New entry for %s account observed: %s, %d, %s\n", account.owner, newEntry.subject, newEntry.value,
14         newEntry.date);
15     }
16 }
```

Client

```
1 package client;
2
3 import dto.AccountService;
4 import dto.AccountEntry;
5 import dto.AccountNotFoundException;
6
7 import java.rmi.registry.Registry;
8 import java.rmi.registry.LocateRegistry;
9 // ...
10
11 public class Client {
12
13     public static void main(String[] argv) {
14
15         try {
16             Registry registry = LocateRegistry.getRegistry("localhost", 1099);
17             AccountService stub = (AccountService) registry.lookup("AccountService");
18
19             // Create an account on the remote side
20             UUID id = stub.createAccount("Bachmann");
21
22             // Register an observer for this account
23             AccountObserver observer = new AccountObserverImpl();
24             stub.addObserver(observer, id);
25
26             // Add some account entries so that observer is notified
27             stub.addAccountEntry(id, new AccountEntry("Credit 1", 100));
28             stub.addAccountEntry(id, new AccountEntry("Credit 2", 42));
29
30             // Stop remote availability of the observer
31             UnicastRemoteObject.unexportObject(observer, true);
32
33         } catch (RemoteException | NotBoundException | AccountNotFoundException e) {
34             e.printStackTrace();
35         }
36     }
37
38 }
```

RMI ist eine Java-spezifische API, d.h. Client und Server müssen in der Programmiersprache Java implementiert werden. Es gibt keine Beschreibung der Schnittstelle zwischen Client und Server in einem unabhängigen, strukturierten Dateiformat wie z.B. XML, um passende Stub-Objekte auch für andere Programmiersprachen zu generieren. Im nächsten Kapitel wenden wir uns SOAP-basierten Webservices zu, die genau diese Einschränkung aufheben.

SOAP-Webservices

Da *Remote Method Invocation (RMI)* ein Java-spezifisches Protokoll ist, kann es nicht als programmiersprachenunabhängiger Standard eingesetzt werden. Als allgemeiner Begriff für entfernte Methodenaufrufe zur Interprozesskommunikation hat sich *Remote Procedure Call (RPC)* durchgesetzt. Der Begriff RPC ist mit dem [RFC 707](#) im Jahr 1976 lange vor dem Entstehen von RMI geprägt worden. Die anfänglichen RPC-Implementierungen sind inzwischen veraltet. Daher beschreibt der Begriff RPC heute eher die grundlegende Idee eines Methodenaufrufs auf Objekten in anderen Adressräumen und weniger ein konkretes Protokoll oder eine konkrete Implementierung.

SOAP

Eine konkrete RPC-Implementierung ist hingegen das XML-basierte Protokoll [SOAP](#), das seit 2000 vom [W3C](#) als empfohlener Standard etabliert worden ist. Wie wir noch im Detail sehen werden, definiert SOAP ein XML-Schema zur Repräsentation der auszutauschenden Daten in XML und nutzt die üblichen Protokolle des [TCP/IP-Modells](#) zur Nachrichtenübertragung, auf oberster Ebene i.d.R. HTTPS. Ursprünglich war SOAP ein Akronym für *Simple Object Access Protocol*. Seit der aktuellen [Version 1.2](#) soll SOAP aber nicht mehr als Abkürzung, sondern als Eigenname verstanden werden, da es subjektiv nicht unbedingt einfach (*simple*) ist und nicht ausschließlich dem Zugriff auf Objekte (*object access*) dient. Historisch geht SOAP auf das Protokoll [XML-RPC](#) zurück, das Microsoft und Dave Winer 1998 veröffentlicht haben.

Im Folgenden soll das Bankkontoservice-Anwendungsbeispiel aus dem vorherigen [Kapitel zu RMI](#) in einen SOAP-Webservice überführt werden. Dabei soll die fachliche Funktionalität des Anwendungsbeispiels möglichst nicht verändert werden. Die einzelnen Methoden des Webservice sind:

- `String createAccount(String: owner)`: In dieser Methode wird ein neues Bankkonto für den angegebenen Eigentümer erzeugt. Die eindeutige Id des Kontos (vom Typ `java.util.UUID`) wird in ihrer String-Repräsentation zurückgegeben. Die Klasse `java.util.UUID` selbst kann nicht als Input- oder Return-Argument verwendet werden, da die Klasse keinen leeren Default-Konstruktor besitzt, der für die Objekterzeugung per Reflection API erforderlich ist.
- `void addAccountEntry(String id, AccountEntry entry)`: In dieser Methode wird einem Bankkonto ein neuer Kontoeintrag (`AccountEntry`) hinzugefügt. Ein Kontoeintrag besteht weiterhin aus einem Betreff, einem Wert und einem Buchungsdatum. Der Klasse `AccountEntry` ist ein leerer Default-Konstruktor hinzugefügt worden.
- `AccountEntry[] getAccountEntries(String id)`: Diese Methode gibt alle Kontoeinträge zu einem Konto als Array zurück. Da die Bibliothek [JAXB \(Java Architecture for XML Binding\)](#) nicht ohne Weiteres Schnittstellen wie z.B. `java.util.List` verarbeiten kann, ist das Return-Argument der Methode ein Array. JAXB übernimmt in der Java-Implementierung (s. unten) implizit die Serialisierung von Java-Objekten in eine XML-Repräsentation und die Deserialisierung in die andere Richtung.
- `int getAccountBalance(String id)`: Diese Methode gibt den Saldo (= Summe der Werte aller Kontoeinträge) eines Kontos zurück.

Es folgen entfernte Methodenaufrufe mittels SOAP für die ersten beiden Methoden des Webservice, nämlich `String createAccount(String: owner)` und `void addAccountEntry(String id, AccountEntry entry)`. Es wird jeweils exemplarisch ein HTTP-Request und die zugehörige HTTP-Response dargestellt.

SOAP-RPC createAccount

```
1  ---[HTTP request]---
2  Accept: text/xml, multipart/related
3  Connection: keep-alive
4  Host: localhost:8080
5  User-agent: JAX-WS RI 2.3.1
6  Content-type: text/xml; charset=utf-8
7  Soapaction: "http://de.oncampus.patterns.soap/AccountService/createAccountRequest"
8  <?xml version='1.0' encoding='UTF-8'?>
9  <S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
10     <S:Body>
11         <ns2:createAccount xmlns:ns2="http://de.oncampus.patterns.soap/">
12             <owner>Hendricks</owner>
13         </ns2:createAccount>
14     </S:Body>
15 </S:Envelope>
16
17 ---[HTTP response 200]---
18 Content-type: text/xml; charset=utf-8
19 <?xml version='1.0' encoding='UTF-8'?>
20 <S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
21     <S:Body>
22         <ns2:createAccountResponse xmlns:ns2="http://de.oncampus.patterns.soap/">
23             <return>ea6f53f3-5381-4061-b0c9-8ad9bde2c523</return>
24         </ns2:createAccountResponse>
25     </S:Body>
26 </S:Envelope>
```

SOAP-RPC addAccountEntry

```
1  ---[HTTP request]---
2  Accept: text/xml, multipart/related
3  Connection: keep-alive
4  Host: localhost:8080
5  User-agent: JAX-WS RI 2.3.1
6  Content-type: text/xml; charset=utf-8
7  Soapaction: "http://de.oncampus.patterns.soap/AccountService/addAccountEntryRequest"
8  <?xml version='1.0' encoding='UTF-8'?>
9  <S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
10     <S:Body>
11         <ns2:addAccountEntry xmlns:ns2="http://de.oncampus.patterns.soap/">
12             <accountId>ea6f53f3-5381-4061-b0c9-8ad9bde2c523</accountId>
13             <entry>
14                 <subject>Credit 1</subject>
15                 <value>50</value>
16             </entry>
17         </ns2:addAccountEntry>
18     </S:Body>
19 </S:Envelope>
20
21 ---[HTTP response 200]---
22 Content-type: text/xml; charset=utf-8
23 <?xml version='1.0' encoding='UTF-8'?>
24 <S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
25     <S:Body>
26         <ns2:addAccountEntryResponse xmlns:ns2="http://de.oncampus.patterns.soap/"></ns2:addAccountEntryResponse>
27     </S:Body>
28 </S:Envelope>
```

Der Vorteil von SOAP ist, dass APIs für nahezu jede geläufige Programmiersprache existieren. Dadurch ist es möglich, dass Client und Server unabhängig voneinander in verschiedenen Programmiersprachen implementiert werden und trotzdem Nachrichten untereinander austauschen können. Die auszutauschenden Daten werden dazu automatisiert in SOAP-Nachrichten verpackt und mittels HTTP/HTTPS ausgetauscht. Jede SOAP-Nachricht ist ein gültiges XML-Dokument, das gegenüber einem zugehörigen XML-Schema validiert werden kann (vgl. Zeile 10: Referenz auf XML-Schema von SOAP Version 1.1 unter <http://schemas.xmlsoap.org/soap/envelope/>). Demzufolge enthält jede SOAP-Nachricht als Wurzelement einen *Envelope*, der als Kindelemente einen optionalen *Header* und einen *Body* umfasst. Der *Body* einer SOAP-Nachricht enthält den eigentlichen Methodenaufruf inkl. der Argumente.

Webservices und WSDL

Als nächstes betrachten wir den generischen Begriff *Webservice*, der im Allgemeinen einen nicht weiter spezifizierten Mechanismus zur Maschine-zu-Maschine-Kommunikation auf Basis von HTTP/HTTPS über eine Netzwerkverbindung beschreibt. Während eine Webanwendung also ein User Interface besitzt, das im Browser dargestellt wird, bietet ein Webservice nur eine API ohne UI. Wenn SOAP als Protokoll für einen Webservice eingesetzt wird, sprechen wir explizit von einem SOAP-Webservice. Das W3C konzentriert sich in der folgenden Definition für Webservices auf genau diese SOAP-Variante, wobei auch auf alternative Webservice-Implementierungen hingewiesen wird.

"There are many things that might be called 'web services' in the world at large. However, [...] we will use the following definition: A web service is a software system designed to support interoperable machine-to-machine interaction over a network. It has an interface described in a machine-processable format (specifically WSDL). Other systems interact with the web service in a manner prescribed by its description using SOAP-messages, typically conveyed using HTTP with an XML serialization in conjunction with other web-related standards." (W3C).

Der Definition ist zu entnehmen, dass die Schnittstelle zwischen Client und Server im Falle eines SOAP-Webservice in einem strukturierten Format namens [WSDL \(Web Service Description Language\)](#) beschrieben wird. WSDL ist ebenfalls ein W3C-Standard, der im Wesentlichen auf XML-Schemas basiert. Ein WSDL-Dokument wird vom Webservice-Anbieter unter einer URI veröffentlicht. Es wird i.d.R. generiert und nicht händisch erstellt. Ein Client kann das WSDL-Dokument parsen und sich passende DTO-Klassen erzeugen lassen, um anschließend gültige SOAP-Nachrichten an den Webservice-Anbieter senden zu können. Es folgt ein Beispiel für ein WSDL-Dokument, das die Methoden des Webservice für das obige Anwendungsbeispiel beschreibt.

WSDL-Dokument

```
1 <!-- Generated by JAX-WS RI (http://javaee.github.io/metro-jax-ws). RI's version is JAX-WS RI 2.3.1 -->
2 <definitions xmlns="http://schemas.xmlsoap.org/wsdl/" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
3   xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:tns="http://de.oncampus.patterns.soap/" name="AccountServiceFactory" ...>
4
5   <types> <xsd:schema> <xsd:import schemaLocation="http://localhost:8080/AccountService?xsd=1"></xsd:import> </xsd:schema> </types>
6
7   <message name="createAccount"> <part name="owner" type="xsd:string"></part> </message>
8   <message name="createAccountResponse"> <part name="return" type="xsd:string"></part> </message>
9
10  <message name="addAccountEntry">
11    <part name="accountId" type="xsd:string"></part>
12    <part name="entry" type="tns:accountEntry"></part>
13  </message>
14  <message name="addAccountEntryResponse"></message>
15
16  <message name="getAccountEntries"> <part name="accountId" type="xsd:string"></part> </message>
17  <message name="getAccountEntriesResponse"> <part name="return" type="tns:accountEntryArray"></part> </message>
18
19  <message name="getAccountBalance"> <part name="accountId" type="xsd:string"></part> </message>
20  <message name="getAccountBalanceResponse"> <part name="return" type="xsd:int"></part> </message>
21  ...
22 </definitions>
```

Referenziertes XML-Schema mit Typen

```
1 <!-- Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.3.1 -->
2 <xs:schema xmlns:tns="http://de.oncampus.patterns.soap/" xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0">
3   <xs:complexType name="accountEntry">
4     <xs:sequence>
5       <xs:element name="subject" type="xs:string" minOccurs="0"></xs:element>
6       <xs:element name="value" type="xs:int"></xs:element>
7       <xs:element name="date" type="xs:dateTime" minOccurs="0"></xs:element>
8     </xs:sequence>
9   </xs:complexType>
10  <xs:complexType name="accountEntryArray">
11    <xs:sequence>
12      <xs:element name="item" type="tns:accountEntry" minOccurs="0" maxOccurs="unbounded"></xs:element>
13    </xs:sequence>
14  </xs:complexType>
15  ...
16 </xs:schema>
```

Zusammenfassend sind folgende Aspekte bezüglich des WSDL-Formats bemerkenswert:

- Jedes WSDL-Dokument ist ein wohlgeformtes und gültiges XML-Dokument. Der Bezug zum zugehörigen XML-Schema zur Validierung ist zu Beginn angegeben (Zeile 2: `xmlns="http://schemas.xmlsoap.org/wsdl/"`).
- Eigene komplexe Datentypen werden in einem separaten XML-Schema deklariert und eingebettet (Zeile 5). Das XML-Schema, das die eigenen Typen (z.B. `AccountEntry`) deklariert, ist oben im zweiten Tab dargestellt.
- Für jede Methode werden die Input-Argumente und das Return-Argument spezifiziert (Zeilen 7-20). Die Bezeichner der Methoden und Argumente können vom Anbieter des Webservice sinnvoll gewählt werden.

JAX-WS (Java API for XML Web Services)

In Java wird durch den [JSR 224](#) eine API namens JAX-WS für SOAP-Webservices definiert, deren Referenzimplementierung (RI) das Framework [JAX-WS RI](#) ist. Maintainer für JAX-WS ist seit dem [Entfernen der Java EE-Module aus dem JDK](#) das Projekt [Metro](#). Wir werden im Folgenden sowohl ein Client-Modul als auch ein Server-Modul mittels des Frameworks JAX-WS RI implementieren, so dass beide Module eine Abhängigkeit zum [JAX-WS RI Runtime Bundle](#) aufweisen werden. Die benötigten Annotationen von JAX-WS befinden sich im Package `javax.jws`. Es folgt zunächst der Code des Server-Moduls.

WebServiceStarter

```
1 package impl;
2
3 import javax.xml.ws.Endpoint;
4
5 public class WebServiceStarter {
6
7     public static void main(String[] args) throws Exception {
8         System.setProperty("com.sun.xml.ws.transport.http.HttpAdapter.dump", "true"); // dump SOAP messages to console
9
10        Endpoint endpoint = Endpoint.publish("http://localhost:8080/AccountService", new AccountServiceImpl());
11        System.out.println("Hit enter to stop web service endpoint.");
12        System.in.read();
13        endpoint.stop();
14    }
15 }
```

AccountServiceImpl

```
1 package impl;
2
3 import javax.jws.WebService;
4 import javax.jws.WebMethod;
5 import javax.jws.WebParam;
6 // ...
7
8 @WebService(name = "AccountService", serviceName = "AccountServiceFactory")
9 @SOAPBinding(style = SOAPBinding.Style.RPC)
10 public class AccountServiceImpl {
11
12     List<Account> accounts = new ArrayList<>();
13
14     @WebMethod
15     public String createAccount(@WebParam(name = "owner") String owner) {
16         Account account = new Account(owner);
17         accounts.add(account);
18         return account.id.toString();
19     }
20
21     @WebMethod
22     public void addAccountEntry(@WebParam(name = "accountId") String id, @WebParam(name = "entry") AccountEntry entry)
23         throws AccountNotFoundException {
24         getAccountById(id).entries.add(entry);
25     }
26
27     @WebMethod(operationName = "getAccountEntries")
28     public AccountEntry[] getEntries(@WebParam(name = "accountId") String id) throws AccountNotFoundException {
29         List<AccountEntry> entries = getAccountById(id).entries; // JAXB can't process interfaces
30         return entries.toArray(new AccountEntry[entries.size()]);
31     }
32
33     @WebMethod
34     public int getAccountBalance(@WebParam(name = "accountId") String id) throws AccountNotFoundException {
35         return Stream.of(getEntries(id)).mapToInt(entry -> entry.value).sum();
36     }
37
38     @WebMethod(exclude = true)
39     public void deleteAccount(@WebParam(name = "accountId") String id) throws AccountNotFoundException {
40         accounts.remove(getAccountById(id));
41     }
42
43     Account getAccountById(String id) throws AccountNotFoundException {
44         return accounts.stream().filter(a -> a.id.toString().equals(id)).findFirst()
45             .orElseThrow(() -> new AccountNotFoundException());
46     }
47 }
```


Account

```
1 package dto;
2 // ...
3
4 public class Account {
5
6     UUID id;
7     String owner;
8     List<AccountEntry> entries;
9
10    public Account(String owner) {
11        this.id = UUID.randomUUID();
12        this.owner = owner;
13        entries = new ArrayList<>();
14    }
15 }
```

AccountEntry

```
1 package dto;
2 // ...
3
4 public class AccountEntry {
5
6     String subject;
7     int value;
8     Date date;
9
10    public AccountEntry() { this.date = new Date(); }
11
12    public AccountEntry(String subject, int value) {
13        super();
14        this.subject = subject;
15        this.value = value;
16    }
17 }
```

AccountNotFoundException

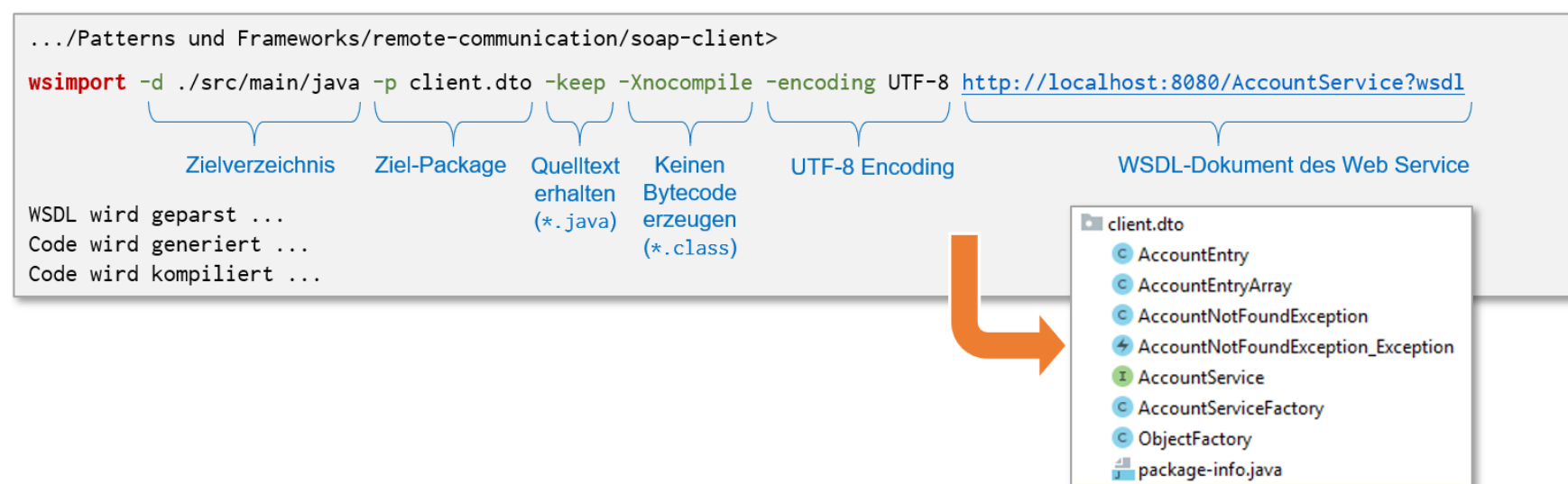
```
1 package dto;
2
3 public class AccountNotFoundException extends Exception { }
```

- **WebServiceStarter**: In Zeile 10 wird der Webservice gestartet und unter dem Endpunkt <http://localhost:8080/AccountService> veröffentlicht. In Zeile 13 wird der Webservice wieder beendet. Die Systemeigenschaft, die in Zeile 8 aktiviert wird, sorgt lediglich dafür, dass alle ausgetauschten SOAP-Nachrichten auf die Konsole ausgegeben werden.
- **AccountServiceImpl**: Die Klasse implementiert den SOAP-Webservice. Im Gegensatz zu RMI ist ein zugehöriges Interface, das dem Client zur Verfügung gestellt werden muss, nicht unbedingt erforderlich. Jede Klasse, die als Webservice instantiiert werden soll, benötigt die Annotation **WebService** (Zeile 8).
 - Die optionale Annotation **@SOAPBinding(style = SOAPBinding.Style.RPC)** sorgt u.a. dafür, dass im generierten WSDL-Dokument Basisdatentypen wie Integer und String auf die entsprechenden Basisdatentypen von XML-Schema abgebildet werden, also z.B. **xsd:int** und **xsd:string**.
 - Jede öffentliche Methode (Sichtbarkeit **public**) wird in die Schnittstelle zum Client, also in das WSDL-Dokument, aufgenommen – außer sie ist mit **@WebMethod(exclude = true)** annotiert. Private Methoden werden nicht in das WSDL-Dokument aufgenommen. Aus eben diesen Gründen sind die Methoden **deleteAccount** (Zeilen 38-41) und **getAccountById** (Zeilen 43-46) nicht im obigen WSDL-Dokument wiederzufinden, während alle anderen Methoden der Klasse **AccountServiceImpl** im WSDL-Dokument enthalten sind.
 - Mittels der Annotationen **@WebMethod(operationName = "...")** und **@WebParam(name = "...")** können Methoden bzw. ihre Argumente gezielt nach außen gerichtet bezeichnet werden. Im obigen WSDL-Dokument wird z.B. die Methode **getEntries** exemplarisch in **getAccountEntries** umbenannt (Zeilen 27-28). Die Methodenargumente explizit zu bezeichnen, ist bei JAX-WS RI sinnvoll, da diese im WSDL-Dokument andernfalls schlicht **arg0**, **arg1**, ... heißen.
 - **Account**, **AccountEntry** und **AccountNotFoundException**: Die serverseitigen DTO-Klassen haben sich im Vergleich zum RMI-Anwendungsbeispiel nicht grundlegend geändert. Das Interface **java.io.Serializable** muss nicht mehr implementiert werden, da die Serialisierung und Deserialisierung in das XML-Format nun über die Bibliothek **JAXB** erfolgt.

Die gezeigten Code-Beispiele zu SOAP-Webservices finden sich im Verzeichnis /remote/soap des Modul-Repository.

Damit ein Client diesen Webservice nutzen kann, benötigt er passende DTO-Klassen. JAX-WS sieht vor, dass diese DTO-Klassen mittels des Kommandozeilenwerkzeugs **wsimport** generiert werden. Bis Java 10 war **wsimport** Teil des JDK, seit Java 11 ist es zusammen mit den Java EE-Modulen ausgelagert worden. Heute kann es auf der [Projektseite von JAX-WS](#) heruntergeladen werden.

Die folgende Abbildung illustriert die Generierung der DTO-Klassen für einen Client mittels `wsimport`. Die Parameter werden in der [Dokumentation zu wsimport](#) genauer erläutert.



Generieren der Datentransferobjekte (DTO) mittels `wsimport`

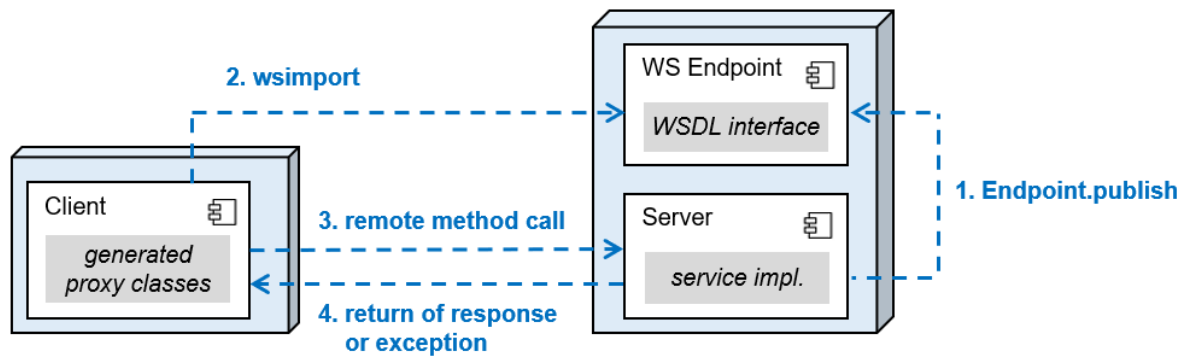
Anschließend kann der Client die generierten DTO-Klassen verwenden, um SOAP-Nachrichten an den Webservice zu senden und Antworten zu empfangen. Im folgenden Code des Clients wird das Stub-Objekt in Zeile 12 erzeugt. Auf diesem Stub-Objekt werden danach die folgenden RPCs ausgeführt: Es wird ein neues Konto angelegt (Zeile 14), neue Kontoeinträge hinzugefügt (Zeilen 16-24) und der Saldo und die Anzahl der Kontoeinträge abgefragt (Zeilen 26-27).

```
1 package client;
2
3 import client.dto.AccountService;
4 import client.dto.AccountServiceFactory;
5 import client.dto.AccountEntry;
6 // ...
7
8 public class Client {
9
10     public static void main(String[] args) {
11         try {
12             AccountService stub = new AccountServiceFactory().getAccountServicePort(); // create remote proxy
13
14             String id = stub.createAccount("Hendricks");
15
16             AccountEntry e1 = new AccountEntry();
17             e1.setSubject("Credit 1");
18             e1.setValue(50);
19             stub.addAccountEntry(id, e1);
20
21             AccountEntry e2 = new AccountEntry();
22             e2.setSubject("Credit 2");
23             e2.setValue(30);
24             stub.addAccountEntry(id, e2);
25
26             System.out.println("Account balance = " + stub.getAccountBalance(id));
27             System.out.println("Account entries = " + stub.getAccountEntries(id).getItem().size());
28         } catch (AccountNotFoundException_Exception e) {
29             e.printStackTrace();
30         }
31     }
32 }
```

Bezüglich der generierten DTO-Klassen ist zu beachten, dass diese nicht 1:1 den serverseitigen DTO-Klassen entsprechen. Es fehlen z.B. geeignete Konstruktoren, obwohl diese in den serverseitigen DTO-Klassen vorhanden sind. So müssen im obigen Code z.B. die Attribute zu einem Kontoeintrag einzeln gesetzt werden (Zeilen 17-18), weil kein entsprechender Konstruktor generiert worden ist. Es bietet sich an, die generierten DTO-Klassen über einen [Adapter](#) an die Bedürfnisse des Clients anzupassen.

In der folgenden Abbildung wird der Ablauf der Kommunikation bei SOAP-Webservices (hier speziell für JAX-WS) zusammengefasst:

1. Der Anbieter (= Server) veröffentlicht seine Webservice-Implementierung unter einer bestimmten URI als Endpunkt – bei JAX-WS z.B. mittels `javax.xml.ws.Endpoint.publish(...)`.
2. Ein Client greift auf das veröffentlichte WSDL-Dokument zu, um sich passende Zugriffsklassen generieren zu lassen – bei JAX-WS z.B. mittels `wsimport`.
3. Der Client sendet mit Hilfe der generierten Zugriffsklassen SOAP-Nachrichten im XML-Format an den Server und wartet anschließend auf eine SOAP-Nachricht als Antwort des Servers. SOAP-Webservices sind auf synchrone Kommunikation ausgelegt.



Ablauf der Kommunikation bei SOAP-Webservices

SOAP-Webservices sind unabhängig von einer konkreten Laufzeitumgebung oder Programmiersprache. Das ist ein gewichtiger Vorteil zur Entkopplung der Komponenten in einem komplexen Softwaresystem. Der Preis dafür ist ein Performance-Nachteil gegenüber Protokollen wie RMI, die eine effizientere, binäre Serialisierung unterstützen. Bei starker Last kann mit RMI ein höherer Durchsatz an Nachrichten erreicht werden als mit SOAP-Webservices, da bei letzteren sämtliche Datentransferobjekte stets zwischen ihrer objektorientierten Repräsentation und XML transformiert werden müssen. Ob die Entkopplung von Komponenten oder die Maximierung des Nachrichtendurchsatzes im Fokus stehen, hängt von den konkreten Anforderungen eines Projekts ab.

WS-*

Im Kontext von SOAP und WSDL sind noch diverse weitere W3C/OASIS-Spezifikationen – insbesondere zur sicheren und zuverlässigen Kommunikation über Organisationsgrenzen hinaus – entstanden, die wir heute zusammenfassend und durchaus etwas despektierlich als [WS-*](#) bezeichnen. Diese Spezifikationen wie WS-Security, WS-SecureConversation, WS-ReliableMessaging, WS-Policy, WS-Discovery, usw. nehmen der grundlegenden Idee "XML über HTTP" ihre Einfachheit und haben zu einer Komplexität geführt, die zusammen mit dem Datenformat JSON das Aufkommen von [REST Webservices](#) (s. nächstes Kapitel) als populäre Alternative zu SOAP begünstigte. SOAP-Webservices sind in großen Unternehmen weiterhin verbreitet, da sie lange von Marktführern wie Microsoft und IBM vorangetrieben worden sind. Letztlich gilt aber auch für den Entwurf von Webservices, dass nach einer einfachen Klarheit in der technischen Lösung gestrebt wird. Keith Ballinger vergleicht die historische Entwicklung der Wahrnehmung von SOAP und REST unter Software-Entwicklern in seinem Artikel ["Simplicity and Utility, or, Why SOAP Lost"](#) sehr anschaulich und endet mit: "Simple is important. Perhaps the most important thing. [...] YAGNI trumps future extensibility." *YAGNI* (*You Aren't Gonna Need It*) ist ein bekanntes Prinzip des [Extreme Programming \(XP\)](#), das besagt Funktionalität erst zu implementieren, wenn sie auch benötigt wird.

Verzeichnisdienst UDDI

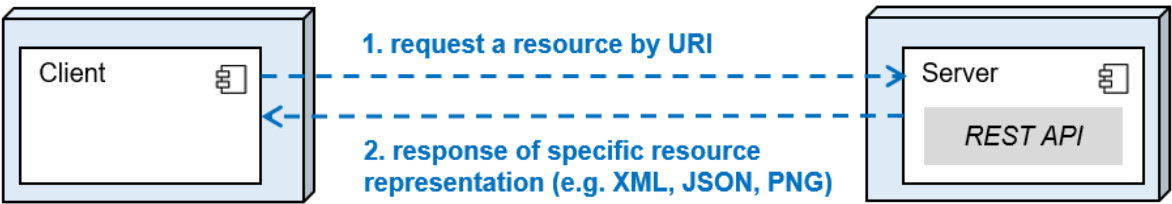
Ähnlich der Registry bei RMI war auch für SOAP-Webservices ein zentraler Verzeichnisdienst vorgesehen, über den Webservices publiziert werden sollten. Die grundlegende Idee war, dass sich aus diesem Verzeichnisdienst ein weltweiter Marktplatz für Webservices ähnlich des [Apple App Store](#) oder des [Google Play Store](#) für mobile Anwendungen entwickelt. Dieser Verzeichnisdienst hieß [UDDI \(Universal Description, Discovery and Integration\)](#), hat sich in der Praxis aber nicht durchgesetzt. Bereits 2005 haben die initialen Unterstützer IBM, Microsoft und SAP ihren UDDI-Verzeichnisdienst geschlossen.

REST-Webservices

REST-Webservices basieren wie SOAP-Webservices auf dem HTTP-Protokoll, über das sämtliche Nachrichten zwischen Client und Server ausgetauscht werden. Damit ist ein REST-Webservice auch grundlegend auf synchrone Kommunikation ausgelegt. Die Nachrichten werden üblicherweise ebenso wie bei SOAP in einem standardisierten, textbasierten und damit technologieunabhängigen Dateiformat wie [XML](#) oder [JSON](#) ausgetauscht. Demzufolge sind auch REST-Webservices unabhängig von einer konkreten Laufzeitumgebung oder Programmiersprache, so dass Client und Server in ganz unterschiedlichen Sprachen implementiert werden können und als Komponenten klar voneinander abgegrenzt sind. Dies führt wie gewünscht zu einer losen Kopplung der Komponenten in der Systemarchitektur.

Prinzipien von REST

Der Begriff REST steht für *Representational State Transfer* und geht auf die [Dissertation von Roy Fielding](#) im Jahr 2000 zurück. Die wesentliche Idee besteht darin, alle Funktionalität über eindeutig adressierbare Ressourcen bereitzustellen – und nicht wie bei RPC-Ansätzen (SOAP, RMI) über Methoden. Der prinzipielle Ablauf der Kommunikation zwischen Client und Server ist in der folgenden Abbildung skizziert.



Ablauf der Kommunikation bei REST-Webservices

Zur Adressierung einer Ressource dient eine [URI \(Unique Resource Identifier\)](#), die allgemein nach dem Muster `http://<host>:<port>/<context-path>/<resource-path>` aufgebaut ist.

- Die URI für die Ressource eines einzelnen Bankkontos mit der Id 1 könnte sein: `http://example.com/banking/accounts/1`
- Auch Mengen sind Ressourcen. So ergibt sich als URI für die Menge aller Bankkonten entsprechend:
`http://example.com/banking/accounts`

REST ist kein Protokoll sondern ein Architekturstil, der auf Konventionen basiert. Es gibt keine Schnittstellenbeschreibungssprache ([Interface Definition Language](#), kurz *IDL*) wie z.B. WSDL bei SOAP-Webservices. Eine wichtige Konvention ist die Einhaltung eines *Uniform Interface*, demzufolge die HTTP-Methoden je Ressource wie folgt abzubilden sind:

- **GET**: Lesen des aktuellen Zustands einer Ressource.
- **POST**: Erzeugen einer neuen Ressource.
- **PUT**: Ersetzen einer bestehenden Ressource als Ganzes.
- **PATCH**: Aktualisieren einer bestehenden Ressource in Teilen, d.h. Ändern ausgewählter Attribute.
- **DELETE**: Löschen einer bestehenden Ressource.

Es ergeben sich folgende typische Operationen auf einer Ressource, die bei einer Anfrage an der URI entsprechend der gewählten HTTP-Methode ausgeführt werden. Bezüglich der leeren Zellen der Tabelle werden i.d.R. keine Operationen von einem REST-Webservice unterstützt, da z.B. das Löschen einer Menge von Ressourcen oder deren vollständiges Ersetzen durch eine andere Menge nicht benötigt werden.

Ressource	GET	POST	PUT	PATCH	DELETE
Menge, z.B. <code>/accounts</code>	Ausgabe einer Liste aller Elemente der Menge. Es werden i.d.R. nicht alle Attribute der Elemente zurückgegeben, aber mind. deren URIs. Die Liste ist ggf. mittels weiterer Parameter des HTTP-Request sortiert, gefiltert und/oder in paginierte Abschnitte unterteilt.	Erzeugen eines neuen Elements in der Menge, dessen URI im Header oder Body der HTTP-Response zurückgegeben wird.			
Element, z.B. <code>/accounts/1</code>	Lesen aller Attribute des Elements.		Ersetzen des Elements an der angegebenen URI. Erzeugen des Elements, falls es nicht existiert.	Aktualisieren von ausgewählten Attributen des Elements.	Löschen des Elements.

Die Ressourcen können vom Server in unterschiedlichen Repräsentationen an einen Client ausgeliefert werden. Die geläufigsten Repräsentationen sind die beiden strukturierten Datenformate XML und JSON, aber eine Ressource könnte auch in HTML, in einem spezifischen Bildformat (JPG, PNG, ...), o.ä. repräsentiert werden. Ein Client kann über den HTTP-Header [Accept](#) angeben, welches Format er in der HTTP-Response erhalten möchte. Falls möglich, liefert der Server genau dieses Format aus, andernfalls ein möglichst ähnliches. Es folgt eine Auswahl häufig verwendeter Formate und ihrer [MIME-Typen](#) nach [RFC 6838](#).

Format	MIME-Type
JSON	application/json
XML	application/xml
HTML	text/html
Unstrukturierter Text	text/plain
Spezifische Bildformate	image/jpeg, image/png, image/gif, ...
Spezifische Audioformate	audio/mpeg, audio/wav, audio/webm, ...
Spezifische Videoformate	video/mp4, video/ogg, video/webm, ...
Unspezifisches Binärformat	application/octet-stream

Bei einer GET-Anfrage an die Ressource `/accounts/1` könnte der Body der HTTP-Response, der die Attribute des Kontos mit der Id 1 darstellt, je nach angeforderter Repräsentation exemplarisch wie folgt aussehen:

XML (application/xml)

```
1  <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2  <account>
3    <id>1</id>
4    <owner>Hendricks</owner>
5    <entries>
6      <subject>Credit 1</subject>
7      <value>10</value>
8      <date>2021-01-30T15:00:00.000+01:00</date>
9    </entries>
10   <entries>
11     <subject>Credit 2</subject>
12     <value>20</value>
13     <date>2021-01-31T15:00:00.000+01:00</date>
14   </entries>
15 </account>
```

JSON (application/json)

```
1  {
2    "id": 1,
3    "owner": "Hendricks",
4    "entries": [
5      {
6        "subject": "Credit 1",
7        "value": 10,
8        "date": 1612015200
9      },
10     {
11       "subject": "Credit 2",
12       "value": 20,
13       "date": 1612101600
14     }
15   ]
16 }
```

HTML (text/html)

```
1  <html>
2  <body>
3    <div id="id"> <label>Account Id</label> 1 </div>
4    <div id="owner"> <label>Owner</label> Hendricks </div>
5    <div id="balance"> <label>Balance</label> 30 </div>
6    <table>
7      <tr> <th>Date</th> <th>Subject</th> <th>Value</th> </tr>
8      <tr> <td>2021-01-30</td> <td>Credit 1</td> <td>10</td> </tr>
9      <tr> <td>2021-01-31</td> <td>Credit 2</td> <td>20</td> </tr>
10   </table>
11 </body>
12 </html>
```


Bei POST-, PUT- und PATCH-Anfragen wird der Client seinerseits Daten in einem bestimmten Format im Body des HTTP-Request an den Server übermitteln wollen. Dazu kann er über den HTTP-Header [Content-Type](#) den Server informieren, in welchem Format er diese Daten sendet.

Weitere Konventionen eines REST-Webservice sind:

- **Cachebarkeit:** Der lesende Zugriff mittels GET ist seitenfrei, d.h. der Zustand keiner Ressource wird verändert. Dadurch können Clients und Intermediäre (Proxys) die Antworten auf GET-Anfragen cachen, solange dieses nicht explizit durch einen entsprechenden Eintrag im HTTP-Header ([Cache-Control](#)) unterbunden wird. Das Caching kann die Performance der Anwendung verbessern und eingeschränkte Offline-Funktionalität ermöglichen.
- **Idempotenz:** Zugriffe auf eine Ressource per GET, PUT und DELETE sind idempotent, d.h. wenn mehrere Zugriffe hintereinander mit den gleichen Parametern ausgeführt werden, ist der serverseitige Zustand derselbe wie bei einem einzelnen Zugriff. Idempotenz erleichtert die Fehlertoleranz, da ein Client im Zweifel, wenn er unsicher ist, ob eine GET-, PUT- oder DELETE-Anfrage den Server erreicht hat, diese Anfrage einfach wiederholen kann. Für POST- und PATCH-Anfragen wird per Konvention keine Idempotenz gefordert.
- **Zustandslosigkeit:** Bei einem REST-Webservice ist die Client-Server-Kommunikation grundsätzlich zustandslos, d.h. der Server speichert keinen Kontext des Client zwischen dessen einzelnen HTTP-Requests. Es gibt also serverseitig keine *Sessions*. Das schließt allerdings nicht aus, dass sämtliche HTTP-Requests vom Server in einer Logdatei protokolliert werden. Jeder Client muss nun selbst den Zustand seiner Session vorhalten und bei jedem HTTP-Request ggf. ein Token zur Autorisierung an den Server mitsenden, falls dieses für den Zugriff auf eine geschützte Ressource erforderlich ist. Demzufolge gibt es i.d.R. als Teil eines REST-Webservice eine bestimmte Ressource zur Authentifizierung, bei der ein Client initial ein entsprechendes Token erhalten kann. Weiter unten gehen wir zu diesem Zweck genauer auf [JSON Web Tokens \(JWT\)](#) ein

JAX-RS (Java API for RESTful Web Services)

Um REST-Webservices in Java zu implementieren, ist zunächst die in [JSR 370](#) spezifizierte API namens JAX-RS maßgeblich. Die Referenzimplementierung für JAX-RS ist das Framework [Jersey](#), das wir für das folgende Anwendungsbeispiel verwenden werden. Alternative Implementierungen für JAX-RS sind [RESTEasy](#), [Apache CXF](#) und [Restlet](#). Auch mit dem [Spring Framework](#) lässt sich einfach ein REST-Webservice in Java implementieren, der aber nicht unbedingt konform zu JAX-RS sein muss, wenn die Implementierung den Empfehlungen von [Spring MVC](#) folgt.

Wie die meisten aktuellen Java-Frameworks arbeitet auch JAX-RS intensiv mit Annotationen. Diese Annotationen aus dem Package `javax.ws.rs` werden anhand der folgenden Code-Beispiele sukzessive erklärt. Der Code bezieht sich auf ein Anwendungsbeispiel, in dem erneut – wie in den vorherigen Kapiteln – Bankkonten über eine API verwaltet werden sollen. Um den REST-Webservice zu realisieren, wird passend zur Datenmodellklasse `Account` eine zugehörige Ressourcenklasse `AccountResource` entworfen, die über die oben vorgestellten HTTP-Methoden eine Menge von Konten verwaltet. Diese Klasse verwaltet die Konten in einer Map, deren Schlüssel die Konto-Ids sind (Zeile 12). Diese Map ist noch flüchtig und wird erst im nächsten Kapitel in einer relationalen Datenbank gespeichert (s. Kapitel [Objekt-Relationales Mapping](#)). Da die Klasse `AccountResource` insgesamt relativ umfangreich ist, werden zunächst nur die Signaturen ihrer Methoden dargestellt und deren Implementierungen anschließend schrittweise ergänzt und erläutert.

WebServiceStarter

```
1  import com.sun.net.httpserver.HttpServer;
2  // ...
3
4  public class WebServiceStarter {
5
6      public static void main(String[] args) throws Exception {
7          ResourceConfig rc = new ResourceConfig().packages("resources");
8          HttpServer server = JdkHttpServerFactory.createHttpServer(URI.create("http://localhost:8080/"), rc);
9          System.out.println("Hit enter to stop HTTP server.");
10         System.in.read();
11         server.stop(0);
12     }
13 }
```


AccountResource

```
1 package resources;
2
3 import javax.ws.rs.core.Response;
4 import javax.ws.rs.Path;
5 import javax.ws.rs.GET;
6 import javax.ws.rs.POST;
7 // ...
8
9 @Path("/accounts")
10 public class AccountResource {
11
12     final static Map<Integer, Account> accounts = new ConcurrentHashMap<>();
13
14     @GET
15     @Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
16     public Collection<Account> getAccounts() { /* ... */ }
17
18     @POST
19     @Consumes({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
20     public Response postAccount(Account account) { /* ... */ }
21
22     @GET @Path("/{id}")
23     public Response getAccount(@PathParam("id") int id) { /* ... */ }
24
25     @PUT @Path("/{id}")
26     public Response putAccount(@PathParam("id") int id, Account account) { /* ... */ }
27
28     @PATCH @Path("/{id}")
29     public Response patchAccount(@PathParam("id") int id, Account patchedAccount) { /* ... */ }
30
31     @DELETE @Path("/{id}")
32     public Response deleteAccount(@PathParam("id") int id) { /* ... */ }
33 }
```

Account

```
1 package model;
2
3 import javax.xml.bind.annotation.XmlRootElement;
4 // ...
5
6 @XmlRootElement // required for XML binding
7 public class Account {
8
9     public static AtomicInteger nextId = new AtomicInteger(1);
10
11     public int id;
12     public String owner;
13     public List<AccountEntry> entries;
14
15     public int balance() {
16         if (entries == null) return 0;
17         return entries.stream().mapToInt(entry -> entry.value).sum();
18     }
19 }
```

AccountEntry

```
1 package model;
2
3 public class AccountEntry {
4
5     public String subject;
6     public int value;
7     public Date date;
8
9     public AccountEntry() {
10         this.date = new Date();
11     }
12 }
```

Erläuterungen zu den Annotationen aus dem Package `javax.ws.rs` in dem obigen Code-Beispiel:

- **@Path**: In der Klasse `WebServiceStarter` wird ein HTTP-Server gestartet, dem eine Ressourcen-Konfiguration als Argument übergeben wird (s. `WebServiceStarter`, Zeilen 7-8). Mittels der Ressourcen-Konfiguration wird Jersey angewiesen alle Klassen im Package `resources` dahingehend zu analysieren, ob sie mit einer Annotation `@Path` versehen sind (z.B. `AccountResource`,

Zeile 9). Falls diese Annotation vorhanden ist, handelt es sich um eine REST-Ressourcenklasse, die unter dem entsprechenden Pfad erreichbar gemacht wird.

- **@GET, @POST, etc.:** Die einzelnen Methoden einer Ressourcenklasse reagieren nur auf spezifische HTTP-Methoden, wenn sie mit einer entsprechenden Annotation versehen sind, z.B. **@GET** (Zeile 14) oder **@POST** (Zeile 18).
- **@PathParam:** Der Pfad einer Ressourcenklasse kann durch eine entsprechende Annotation an ihren Methoden dynamisch erweitert werden, z.B. **@Path("{id}")** (Zeile 23). Die angehängte Id wird von Jersey aus der URI extrahiert und mittels der Annotation **@PathParam("id")** als Methodenargument injiziert (Zeile 24). Hier wird implizit ein Cast von **String** auf **int** durchgeführt, der auch fehlschlagen kann.
- **@Consumes** und **@Produces:** An den Methoden einer Ressourcenklasse kann mittels der Annotationen **@Consumes** (Zeile 19) und **@Produces** (Zeile 15) explizit angegeben werden, in welchen Formaten die annotierten Methoden Eingaben verarbeiten bzw. Ausgaben erzeugen können. Diese Annotationen sind insbesondere dann notwendig, wenn je nach Eingabeformat (HTTP-Header **Content-Type**) oder angefordertem Ausgabeformat (HTTP-Header **Accept**) eine unterschiedliche Methode der Ressourcenklasse ausgeführt werden soll.

Die gezeigten Code-Beispiele zu REST-Webservices finden sich im Verzeichnis `/remote/rest` des Modul-Repository.

HTTP-Response

Ein REST-Webservice sollte einem Client grundsätzlich mit [HTTP-Status-Codes](#) antworten, die dem vereinbarten Standard entsprechen. Die folgende Tabelle zeigt eine Auswahl von gebräuchlichen HTTP-Status-Codes, wobei der Bereich 2xx stets für erfolgreiche HTTP-Requests steht, 3xx für Weiterleitungen, 4xx für Fehler des aufrufenden Client und 5xx für Fehler im Server.

HTTP-Status-Code	Beschreibung
200	<i>OK</i> , generischer Code für eine erfolgreiche Anfrage
201	<i>Created</i> , neue Ressource erfolgreich angelegt
204	<i>No Content</i> , Anfrage erfolgreich, aber es wird kein Inhalt zurückgegeben
301	<i>Moved Permanently</i> , Ressource ist dauerhaft unter anderer URI erreichbar
304	<i>Not Modified</i> , unverändert, d.h. Cache muss nicht aktualisiert werden
400	<i>Bad Request</i> , generischer Code für einen Fehler des Client in der Parametrisierung der Anfrage
403	<i>Forbidden</i> , Zugriff auf die Ressource ist verboten
404	<i>Not Found</i> , für die URI konnte keine Ressource gefunden werden
500	<i>Internal Server Error</i> , generischer Code für einen nicht weiter behandelten Fehler im Server
503	<i>Service Unavailable</i> , Service ist vorübergehend nicht zu erreichen

Die beiden Methoden im folgenden Code-Beispiel geben eine Liste aller Konten bzw. ein einzelnes Konto zurück. Der Rückgabotyp der Methode **getAccounts** (Pfad `/accounts`) ist **Collection<Account>** (Zeile 2). Für diese Methode wird die HTTP-Response implizit durch das JAX-RS-Framework zusammengestellt. Die Liste der Konten wird ohne Zutun des Entwicklers in eine JSON- oder XML-Repräsentation transformiert und in ein Objekt vom Typ **javax.ws.rs.core.Response** eingebettet, dessen Status-Code 200 ist. Bei der zweiten Methode **getAccount** (Pfad z.B. `/accounts/1`) wird das **Response**-Objekt explizit zusammengestellt (Zeilen 10 und 12). Die Klasse **Response** bietet dazu statische Methoden um den Status-Code (Methode **status**), den Body (Methode **entity**) und spezifische Header-Attribute (Methode **header**) zu setzen.

```
1  @GET
2  public Collection<Account> getAccounts() {
3      return accounts.values(); // return code is 200
4  }
5
6  @GET @Path("{id}")
7  public Response getAccount(@PathParam("id") int id) {
8      Account account = accounts.get(id);
9      if (account == null) {
10         return Response.status(Response.Status.NOT_FOUND).build(); // return code is 404
11     }
12     return Response.status(Response.Status.OK).entity(account).build(); // return code is 200
13 }
```

Im Body eines HTTP-Request und einer HTTP-Response wird i.d.R. JSON oder XML von einem REST-Webservice empfangen bzw. von diesem versendet. Das JAX-RS-Framework übernimmt implizit die Transformation von Java-Objekten in JSON/XML und andersherum. Dazu wird jeweils eine entsprechende Bibliothek benötigt. Diese Bibliotheken sind üblicherweise [Jackson](#) für das JSON-Binding und [JAXB](#) für das XML-Binding.

Semantische Konvention von POST, PUT und PATCH

Die folgenden drei Methoden zeigen Implementierungen für die HTTP-Methoden POST, PUT und PATCH, die eine Ressource anlegen bzw. verändern. Dabei sind die im Folgenden erläuterten Unterschiede relevant.

- **POST:** Beim Einfügen einer neuen Ressource wird deren eindeutige Id vom Server vergeben. Daher ist der Pfad `/accounts`. Die neu vergebene Id muss dem Client im Response-Body und/oder Response-Header mitgeteilt werden (Zeile 5).

```
1  @POST
2  public Response postAccount(@NotNull Account account, @Context UriInfo uriInfo) {
3      boolean validId = account.id > 0 && accounts.get(account.id) == null;
4      if (!validId) {
5          account.id = Account.nextId.getAndIncrement();
6      }
7      accounts.put(account.id, account);
8      URI uri = uriInfo.getAbsolutePathBuilder().path(Integer.toString(account.id)).build(); // append new id to URI
9      return Response.created(uri).entity(account).build(); // return code is 201
10 }
```

- **PUT:** Es kann nur eine bereits bestehende Ressource ersetzt werden. Falls bisher keine Ressource unter der in der URI genannten Id zu finden ist, wird eine neue Ressource angelegt (s. Aufruf von `postAccount` in Zeile 6). Die bestehende Ressource wird vollständig ersetzt (Zeile 8).

```
1  @PUT @Path("{id}")
2  public Response putAccount(@PathParam("id") int id, @NotNull Account account, @Context UriInfo uriInfo) {
3      boolean exists = accounts.get(id) != null;
4      account.id = id;
5      if (!exists) {
6          return postAccount(account, uriInfo);
7      } else {
8          accounts.put(id, account);
9          return Response.ok(account).build(); // return code is 200
10     }
11 }
```

- **PATCH:** Es kann nur eine bereits bestehende Ressource aktualisiert werden. Falls keine Ressource unter der in der URI genannten Id zu finden ist, wird der entsprechende Status-Code 404 zurückgegeben (Zeile 6). Nur die im Body des HTTP-Request angegebenen Attribute der Ressource werden aktualisiert. Alle anderen Attribute bleiben unverändert erhalten (Zeilen 8-13).

```
1  @PATCH @Path("{id}")
2  public Response patchAccount(@PathParam("id") int id, @NotNull Account patchedAccount) {
3      Account account = accounts.get(id);
4      boolean exists = account != null;
5      if (!exists) {
6          return Response.status(404).build(); // return code is 404
7      } else {
8          if (patchedAccount.owner != null) {
9              account.owner = patchedAccount.owner;
10         }
11         if (patchedAccount.entries != null) {
12             account.entries = patchedAccount.entries;
13         }
14         return Response.ok(account).build(); // return code is 200
15     }
16 }
```

Das Anlegen oder Verändern einer Ressource kann grundsätzlich fehlschlagen, falls die Validierung einzelner Attribute fehlschlägt. Exemplarisch werden einige typische Bedingungen für Validierungsschritte aufgezählt.

- Ein Attribut darf nicht leer sein: "Es soll sich kein Anwender ohne Angabe einer E-Mail-Adresse registrieren."
- Ein Attribut muss ein bestimmtes Format oder einen Wertebereich einhalten: "Die E-Mail-Adresse muss einem regulären Ausdruck entsprechen, u.a. ein @-Symbol enthalten."
- Ein Attribut muss eindeutig sein: "Es sollen nicht mehrere Anwender mit derselben E-Mail-Adresse registriert sein."

Zur Validierung bieten sich die [Bean Validation](#)-Annotationen an. Im obigen Code-Beispiel wird exemplarisch bei jeder Methode die selbsterklärende Annotation `@NotNull` verwendet.

Der Unterschied zwischen PUT und PATCH soll noch anhand eines Beispiels verdeutlicht werden. Zur Vorbereitung wird über folgende POST-Anfrage ein neues Konto angelegt.

```

1  ---[HTTP POST request]---
2  Request URL: http://localhost:8080/accounts
3  Content-type: application/json; charset=utf-8
4  {"owner": "A", "entries": [{"value": 10}, {"value": 20}]}
5
6  -----[HTTP response 201]---
7  Content-type: application/json; charset=utf-8
8  {"id": 1, "owner": "A", "entries": [{"subject": null, "value": 10}, {"subject": null, "value": 20}]}
9

```

Anschließend wird der Eigentümer des Kontos zunächst mittels einer PATCH-Anfrage verändert.

```

1  ---[HTTP PATCH request]---
2  Request URL: http://localhost:8080/accounts/1
3  Content-type: application/json; charset=utf-8
4  {"owner": "Patched"}
5
6  -----[HTTP response 200]---
7  Content-type: application/json; charset=utf-8
8  {"id": 1, "owner": "Patched", "entries": [{"subject": null, "value": 10}, {"subject": null, "value": 20}]}
9

```

Zum Vergleich wird der Eigentümer des Kontos danach mittels einer PUT-Anfrage verändert. Wie in der jeweiligen HTTP-Response zu erkennen ist, verändert PATCH die Kontoeinträge nicht, während PUT dafür sorgt, dass keine Kontoeinträge mehr existieren, da im Body des HTTP-Request auch keine übergeben worden sind.

```

1  ---[HTTP PUT request]---
2  Request URL: http://localhost:8080/accounts/1
3  Content-type: application/json; charset=utf-8
4  {"owner": "Changed"}
5
6  -----[HTTP response 200]---
7  Content-type: application/json; charset=utf-8
8  {"id": 1, "owner": "Changed", "entries": null}
9

```

Repräsentation einer Ressource in einem Binärformat

Das folgende Code-Beispiel zeigt zwei Ressourcen-Methoden, die beide auf GET-Anfragen bezüglich des Pfads `/users/<id>` reagieren, aber jeweils eine Ausgabe in unterschiedlichem Format erzeugen. Während die erste Methode `getUser` JSON oder XML als Ausgabe produziert, erzeugt die zweite Methode `getUserImage` eine Binärdatei, die das User-Profilbild enthält. Das JAX-RS-Framework wertet den HTTP-Header `Accept` aus, um zu entscheiden, welche der beiden Methoden ausgeführt werden soll. Dabei soll die Rückgabe in einem Format erfolgen, das möglichst der Anforderung des Client entspricht. Es kann allerdings sein, dass der Client z.B. `Accept: image/jpeg` sendet, aber eine Antwort mit `Content-Type: image/png` erhält, da das User-Profilbild serverseitig im PNG-Format vorliegt.

```

1  @Path("/users")
2  public class UserResource {
3
4      @GET @Path("{id}")
5      @Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
6      public Response getUser(@PathParam("id") int id) {
7          User user = getUserById(id);
8          return Response.ok(user).build();
9      }
10
11     @GET @Path("{id}")
12     @Produces({MediaType.APPLICATION_OCTET_STREAM, "image/png", "image/jpeg"})
13     public Response getUserImage(@PathParam("id") int id) throws IOException {
14         User user = getUserById(id);
15         InputStream is = new ByteArrayInputStream(user.image);
16         String type = URLConnection.guessContentTypeFromStream(is); // get MIME type
17         String ext = type.substring(type.lastIndexOf("/") + 1); // get file extension
18         return Response.ok(is).type(type)
19             .header("Content-Disposition", "inline; filename=\"" + user.name + "." + ext + "\"").build();
20     }
21
22     private User getUserById(int id) {
23         Optional<User> user = UserService.queryById(id); // query user object from database
24         if (user.isPresent()) { return user.get(); }
25         throw new NotFoundException("User not found.");
26     }
27 }

```

Behandlung von Exceptions in JAX-RS

Wenn innerhalb einer Ressourcen-Methode eine unbehandelte `RuntimeException` (z.B. eine `NullPointerException`) auftritt, wird diese durch das JAX-RS-Framework gefangen und "verschluckt", so dass keine Ausgabe auf der Konsole oder im Log erscheint. JAX-RS sieht vor, dass der Entwickler gezielt sogenannte *ExceptionHandler* registriert, die auf spezifische Exceptions reagieren und diese auswerten, um eine aussagekräftige HTTP-Response an den aufrufenden Client zu senden. Das folgende Code-Beispiel zeigt exemplarisch drei verschiedene Implementierungen des `ExceptionHandler`-Interface.

- Der `ValidationExceptionHandler` behandelt Exceptions, die während der Validierung der [Bean Validation](#)-Annotationen auftreten, d.h. jede `ConstraintValidationException`.
- Der `JsonProcessingExceptionHandler` behandelt Exceptions, die während der Transformation von JSON-Input aus dem HTTP-Request-Body in Java-Objekte auftreten (z.B. unbekannte Attribute), d.h. jede `JsonProcessingException`.
- Der `GenericExceptionHandler` reagiert auf jede beliebige `RuntimeException`. Falls es sich um eine konkrete `ClientException` (Status-Code 4xx) handelt, soll die Fehlermeldung lediglich an den Client weitergereicht werden. Falls nicht, wird der Exception-Stacktrace protokolliert und der generische Status-Code 500 ausgegeben.

ValidationExceptionHandler

```

1  import javax.ws.rs.ext.Provider;
2  import javax.ws.rs.ext.ExceptionMapper;
3  import javax.validation.ConstraintViolationException;
4  // ...
5
6  @Provider
7  public class ValidationExceptionHandler implements ExceptionMapper<ConstraintViolationException> {
8
9      @Override
10     public Response toResponse(ConstraintViolationException e) {
11         ConstraintViolation violation = e.getConstraintViolations().stream().findFirst().get();
12         String message = violation.getPropertyPath() + " " + violation.getMessage();
13         return Response.status(Response.Status.BAD_REQUEST).entity(message).type("text/plain").build();
14     }
15 }

```

JsonProcessingExceptionMapper

```
1 import javax.ws.rs.ext.Provider;
2 import javax.ws.rs.ext.ExceptionMapper;
3 import com.fasterxml.jackson.core.JsonProcessingException;
4 // ...
5
6 @Provider
7 public class JsonProcessingExceptionMapper implements ExceptionMapper<JsonProcessingException> {
8
9     @Override
10    public Response toResponse(JsonProcessingException e) {
11        return Response.status(Response.Status.BAD_REQUEST).entity(e.getMessage()).type("text/plain").build();
12    }
13 }
```

GenericExceptionMapper

```
1 import javax.ws.rs.ext.Provider;
2 import javax.ws.rs.ext.ExceptionMapper;
3 // ...
4
5 @Provider
6 public class GenericExceptionMapper implements ExceptionMapper<RuntimeException> {
7
8     @Override
9    public Response toResponse(RuntimeException e) {
10        if (e instanceof ClientErrorException) {
11            int status = ((ClientErrorException) e).getResponse().getStatus();
12            return Response.status(status).entity(e.getMessage()).type("text/plain").build();
13        }
14        else {
15            e.printStackTrace();
16            return Response.status(Response.Status.INTERNAL_SERVER_ERROR).build();
17        }
18    }
19 }
```

Testen eines REST-Webservice

Für automatisierte Unit-Tests bietet Jersey die Klasse `JerseyTest` (enthalten in der Bibliothek [Jersey Test Framework Core](#)), die durch eine eigene Testklasse erweitert werden kann (Zeile 7). Das Jersey-Test-Framework startet automatisch einen HTTP-Server unter Port 9998, auf dem die zu testende Ressource bereitgestellt wird (Zeile 11) und anschließend die Testfälle ausgeführt werden. Mittels der Methode `target` der `JerseyTest`-Klasse können Anfragen an den REST-Webservice aus den Testfällen heraus erzeugt werden. Ansonsten gelten die üblichen Konvention für [JUnit](#)-Tests. Dazu gehört u.a., dass der gemeinsame initiale Zustand für alle Testfälle durch eine Methode mit der Annotation `@BeforeEach` hergestellt wird (Zeilen 14-20). Das folgende Code-Beispiel zeigt zwei exemplarische Testfälle für PUT (Zeilen 22-30) und DELETE (Zeilen 32-39).

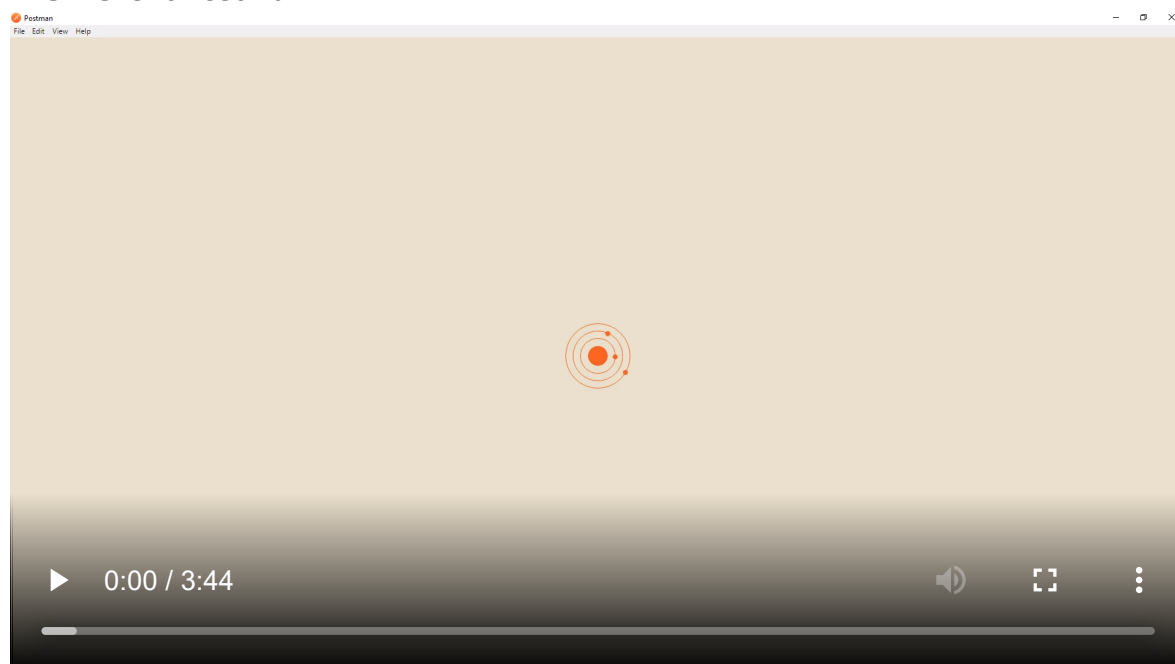

```

1  import org.glassfish.jersey.test.JerseyTest;
2  import org.junit.jupiter.api.Test;
3  import org.junit.jupiter.api.TestInstance;
4  // ...
5
6  @TestInstance(Lifecycle.PER_CLASS)
7  public class AccountResourceTest extends JerseyTest {
8
9      @Override
10     protected Application configure() {
11         return new ResourceConfig(AccountResource.class);
12     }
13
14     @BeforeEach
15     public void prepareTest() {
16         target().path("accounts").request().delete(); // delete all accounts
17         String a = "{\"owner\": \"A\", \"entries\": [{\"value\": 20}, {\"value\": -10}]}";
18         target("accounts/1").request().put(Entity.json(a)); // put a test account with id = 1
19         // ...
20     }
21
22     @Test
23     public void testPutAccount() {
24         String a = "{\"owner\": \"Changed\"}";
25         Response res = target("accounts/1").request().put(Entity.json(a));
26         assertEquals(200, res.getStatus());
27         Account account = res.readEntity(Account.class);
28         assertEquals("Changed", account.owner);
29         assertEquals(0, account.balance());
30     }
31
32     @Test
33     public void testDeleteAccount() {
34         Response res = target("accounts/1").request().delete();
35         assertEquals(204, res.getStatus());
36
37         res = target("accounts/1").request().get(Response.class);
38         assertEquals(404, res.getStatus());
39     }
40
41     // ...
42 }

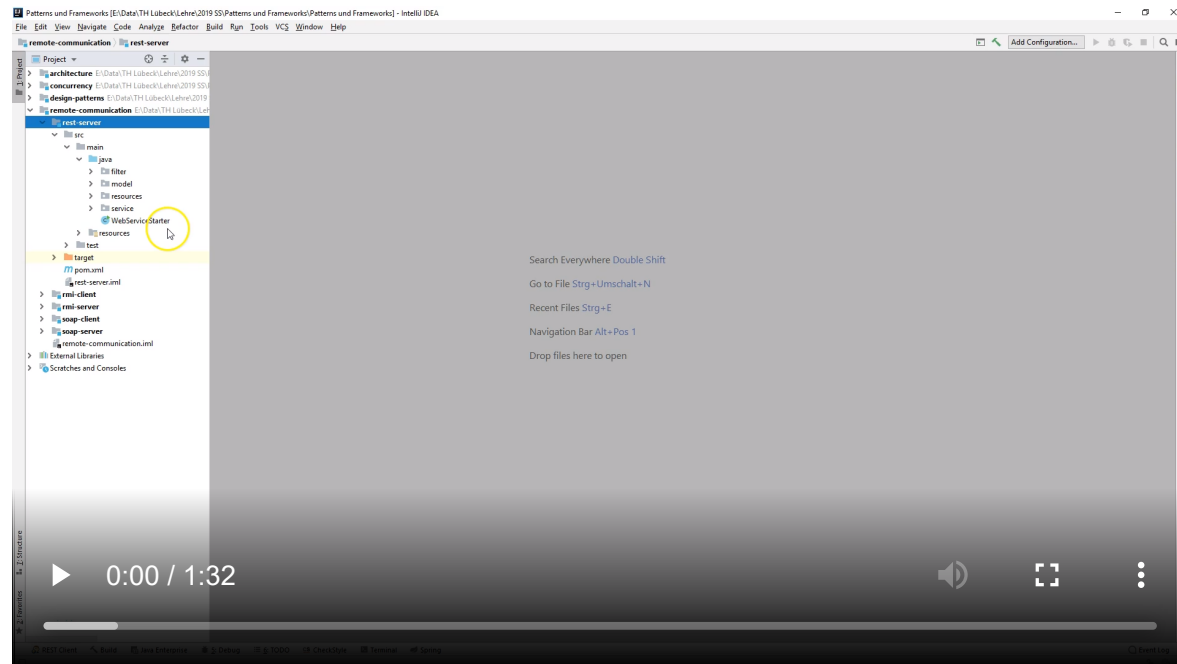
```

Zur Entwicklungszeit ist es wichtig, die REST-API einfach stichprobenartig testen zu können. Die erstellten Testfälle sollen zum Austausch mit anderen Entwicklern und zur Versionierung gespeichert werden können, und sie sollen wiederholt und automatisch ausführbar sein. Dazu gibt es mehrere, relativ ähnliche Werkzeuge, die entweder als Desktop-Anwendung oder als Browser-Plugin installiert werden können – z.B. [Postman](#), [Insomnia](#) und [Advanced REST Client](#). Alternativ ist es auch möglich die REST-API direkt aus der Entwicklungsumgebung (IDE) zu testen. Die beiden folgenden Videos zeigen kurz, wie die entwickelte REST-API zur Kontenverwaltung exemplarisch mittels Postman und IntelliJ IDEA getestet werden können.

REST-Client Postman



REST-Client in IntelliJ IDEA

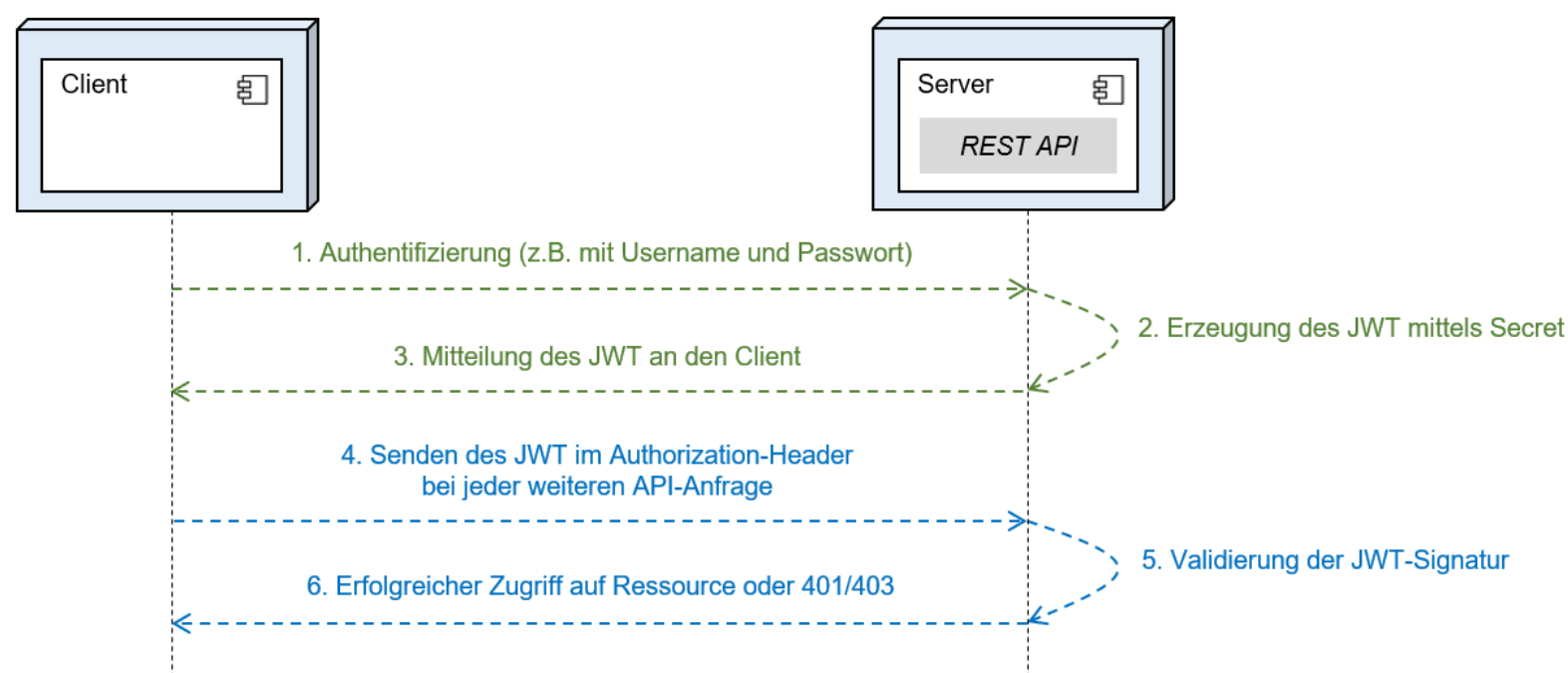


JWT (JSON Web Token)

REST-Webservices sind per Konvention zustandslos, d.h. auf dem Server werden keine Sessions für die aktuell verbundenen Clients verwaltet. Demzufolge muss jeder Client selbst seine Session verwalten und mit jedem HTTP-Request ein Token o.ä. an den Server senden, über das er authentifiziert und für den Zugriff autorisiert werden kann. Eine standardisierte Möglichkeit zur Erzeugung derartiger Tokens findet sich in [RFC 7519](https://tools.ietf.org/html/rfc7519), in dem sogenannte [JSON Web Token \(JWT\)](https://tools.ietf.org/html/rfc7519) eingeführt werden.

"JSON Web Token (JWT) is a compact, URL-safe means of representing claims to be transferred between two parties. The claims in a JWT are encoded as a JSON object that is used as the payload of a JSON Web Signature (JWS) structure or as the plaintext of a JSON Web Encryption (JWE) structure, enabling the claims to be digitally signed or integrity protected with a Message Authentication Code (MAC) and/or encrypted." (Internet Engineering Task Force).

Der Kommunikationsablauf zwischen Client und Server bei Verwendung von JWT wird in der folgenden Abbildung dargestellt.



Ablauf der Kommunikation bei Authentifizierung mittels JWT

1. Der Anwender authentifiziert sich beim Server, z.B. in dem er seinen Usernamen und sein Passwort in ein Login-Formular einträgt und dieses absendet.
2. Der Server generiert ein individuelles Token für den Client. Zur Signatur des Token wird ein *Secret* verwendet, das ausschließlich dem Erzeuger des Token bekannt ist. Das Token besteht aus 3 Teilen: dem Header, dem eigentlichen Inhalt genannt *Payload* und der Signatur.
 - **Header:** Der Header enthält die Information, welcher Hash-/Verschlüsselungsalgorithmus eingesetzt wird, z.B. `{"alg": "HS256"}`.
 - **Payload:** Der Payload enthält ein gültiges JSON-Objekt, das den sogenannten *Claim* abbildet. Der Claim drückt eine Anwenderrolle aus, die der Client mittels des Token für sich beansprucht und die ihm bei gültiger Signatur auch zugestanden wird. In dem unten dargestellten Screenshot der Webseite jwt.io ist der Claim exemplarisch `{"sub": 123, "name": "John Doe", "iat": 1612083600}`. Es gibt einige verbreitete Claim-Attribute, die im RFC 7519 genannt werden, wie z.B. *Subject (sub)*, *Issued At (iat)* und *Expiration Time (exp)*.
 - **Signatur:** Während Header und Payload lediglich Base64-encodiert werden und damit in Klartext vorliegen, enthält die Signatur ein verschlüsseltes *Secret*. Dieses Geheimnis ist bei einem Hash-Algorithmus wie [HMAC](https://en.wikipedia.org/wiki/HMAC) mit [SHA-256](https://en.wikipedia.org/wiki/SHA-256) (kurz HS256) eine entsprechend lange Zeichenkette (z.B. 32 Zeichen bei SHA-256). Alternativ besteht das Geheimnis in einem

- Der Server sendet das Token an den Client, der es sicher vor dem Zugriff von Dritten verwahrt. Das Token mit Header `{"alg": "HS256"}`, Payload `{"sub": 123, "name": "John Doe", "iat": 1612083600}` und Secret `my-secret` wäre z.B. `eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiEYMywibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNjEyMDgzNjAwfQ.FQVhKVWAzvW-Pje0FIB30ITulrRwcl9Mm8HpOinMYyc.`
- Der Client sendet das Token mit jedem weiteren HTTP-Request im HTTP-Header `Authorization` an den Server.
- Der Server verifiziert das Token, bevor er den Zugriff auf die angeforderte Ressource zulässt. Falls es sich um kein gültiges Token handelt, sollte der Status-Code *Unauthorized (401)* zurückgegeben werden. Falls es sich zwar um ein gültiges Token handelt, aber der authentifizierte Anwender nicht über die notwendigen Rechte verfügt, um auf die Ressource zuzugreifen, sollte der Status-Code *Forbidden (403)* zurückgegeben werden. Das Token kann durch ein *Expiration Time*-Attribut in seiner Gültigkeit eingeschränkt werden. Aufgrund der prinzipiellen Zustandslosigkeit der REST-API kann der Server ein einmal ausgestelltes Token aber nicht sofort für ungültig erklären (z.B. Logout erzwingen). Als Workaround könnte ein Token einer serverseitigen Blacklist hinzugefügt werden, was aber der Idee der Zustandslosigkeit widerspricht.
- Bei autorisiertem Zugriff auf die Ressource wird die eigentliche Anfrage mit einer entsprechenden HTTP-Response bedient.

Interaktive Einführung in JWT auf jwt.io

Seite 96

AuthenticationResource

```
1  @Path("/login")
2  public class AuthenticationResource {
3
4      @POST
5      @Produces(MediaType.TEXT_PLAIN)
6      public String login(User userClaim) {
7          Optional<User> user = UserService.queryByCredentials(userClaim.name, userClaim.passwordHash); // lookup user in database
8          if (user.isPresent()) {
9              try {
10                 // map claim object to JSON
11                 JWTClaim claim = new JWTClaim(String.valueOf(user.get().id), user.get().name);
12                 String claimJson = new ObjectMapper().writeValueAsString(claim);
13
14                 // create JWS object with claim as payload
15                 JWSObject jwsObject = new JWSObject(new JWSHeader(JWSAlgorithm.HS256), new Payload(claimJson));
16
17                 // use a secret key for HS256 and sign the JWS object using HMAC
18                 byte[] secret = AuthenticationFilter.SECRET.getBytes();
19                 jwsObject.sign(new MACSigner(secret));
20
21                 // provide token to the client
22                 return Response.ok( jwsObject.serialize() ).build();
23
24             } catch (JsonProcessingException | JOSEException e) {
25                 throw new RuntimeException(e.getMessage());
26             }
27         }
28         return Response.status(401).entity("Login failed").build();
29     }
30 }
```

AuthenticationFilter

```
1  import javax.ws.rs.container.ContainerRequestFilter;
2  import javax.ws.rs.ext.Provider;
3  // ...
4
5  @Provider
6  @Priority(Priorities.AUTHENTICATION)
7  public class AuthenticationFilter implements ContainerRequestFilter {
8
9      public static final String SECRET = "my-very-private-secret:32-symbols=256-bit";
10     static final String TOKEN_TYPE = "Bearer";
11
12     @NameBinding @Retention(RetentionPolicy.RUNTIME)
13     public @interface Secured { }
14
15     @Override
16     public void filter(ContainerRequestContext request) {
17         // validate HTTP authorization header
18         String authHeader = request.getHeaders().getFirst(HttpHeaders.AUTHORIZATION);
19         if (authHeader == null || !authHeader.startsWith(TOKEN_TYPE + " ")) {
20             request.abortWith(Response.status(401).entity("JWT not found in HTTP header").build());
21         }
22         // extract the token from the HTTP authorization header
23         String jwt = authHeader.substring(TOKEN_TYPE.length()).trim();
24
25         // validate JWT signature
26         try {
27             JWSObject jwsObject = JWSObject.parse(jwt);
28             JWSVerifier verifier = new MACVerifier(SECRET);
29             if (jwsObject.verify(verifier)) {
30                 String payload = jwsObject.getPayload().toString();
31                 JWTClaim claim = new ObjectMapper().readValue(payload, JWTClaim.class);
32                 System.out.println("User verified: " + claim.name);
33                 return;
34             }
35             else {
36                 request.abortWith(Response.status(401).entity("Invalid JWT").build());
37             }
38         } catch (ParseException | JOSEException | IOException e) {
39             request.abortWith(Response.status(400).entity(e.getMessage()).build());
40         }
41     }
42 }
```

UserResource

```
1  @Path("/users")
2  public class UserResource {
3
4      @Secured
5      @GET
6      public Collection<User> getUsers() {
7          return UserService.queryAllUsers();
8      }
9
10     // ...
11 }
```

JWTClaim

```
1  public class JWTClaim {
2
3      // registered JWT claim names, see https://tools.ietf.org/html/rfc7519#section-4.1
4      String sub; // subject
5      String name;
6      Date iat; // issued at
7      Date exp; // expiration time
8
9      public JWTClaim(String sub, String name) {
10         this.sub = sub;
11         this.name = name;
12         Calendar cal = Calendar.getInstance();
13         this.iat = cal.getTime();
14         cal.add(Calendar.HOUR, 1); // tokens are valid for 1 hour
15         this.exp = cal.getTime();
16     }
17 }
```

- **AuthenticationResource**: Diese Ressourcen-Klasse implementiert eine Methode zur Authentifizierung, die über den Pfad `/login` erreichbar ist. Die Ausgabe ist bei erfolgreichem Login ein JSON Web Token (Zeile 22), bei nicht erfolgreichem Login ein entsprechender Fehler (Zeilen 25+28). Falls die übergebenen Credentials stimmen (Zeilen 7-8), werden folgende Schritte ausgeführt:
 - Zusammenstellen des Claim-Objekts inkl. *Expiration Time* (Zeilen 11-12).
 - Erzeugen eines JWS-Objekts (s. [RFC 7515: JSON Web Signature](https://tools.ietf.org/html/rfc7515)), das den Claim als Payload enthält und den verwendeten Verschlüsselungsalgorithmus angibt (Zeile 15).
 - Verschlüsseln des JWS-Objekts mit einem *Secret* (Zeilen 18-19).
 - Serialisieren des JWS-Objekts in das JWT-Format (Zeile 22).
- **AuthenticationFilter**: Das JWT, das der Client bei weiteren Anfragen im HTTP-Header mitsendet, soll bei gesicherten Ressourcen-Methoden geprüft werden. Die Sicherung einer Ressourcen-Methode erfolgt über eine eigene Annotation, die hier `@Secured` heißen soll und in den Zeilen 12-13 der Klasse **AuthenticationFilter** deklariert wird. Diese Klasse implementiert das Interface **ContainerRequestFilter** und funktioniert ihrem Namen entsprechend als Filter vor jedem Aufruf einer Methode mit der Annotation `@Secured`. Die Verifikation des JWT erfolgt in drei Schritten:
 - Zunächst wird das JWT als sogenanntes [Bearer Token](#) aus dem HTTP-Header extrahiert (Zeilen 17-23).
 - Anschließend wird aus dem JWT wieder ein JWS-Objekt erzeugt (Zeile 27) und dieses mit demselben *Secret* verifiziert, das auch zur Signatur verwendet worden ist (Zeilen 28-29).
 - Nachdem das JWS-Objekt verifiziert ist, kann der Claim inhaltlich ausgewertet werden. Ist der Anwender berechtigt den Ressourcenzugriff durchzuführen? Ist das Token ggf. schon abgelaufen? Im Beispiel wird der Filter in Zeile 33 erfolgreich verlassen und der Ressourcenzugriff erlaubt.
- **UserResource**: Die Klasse enthält eine Ressourcen-Methode, die exemplarisch mit `@Secured` annotiert ist, um den Zugriff auf diese Methode mittels der **AuthenticationFilter**-Klasse abzusichern.

Object-Relational Mapping (ORM)

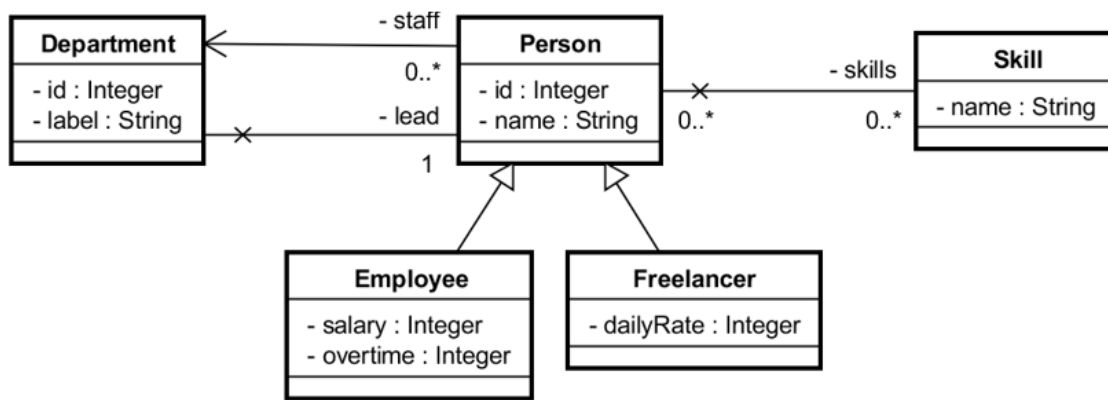
Als *Object-Relational Mapping* (kurz *ORM*) bezeichnen wir in der Softwareentwicklung die Abbildung der Objekte aus einer objektorientierten Programmiersprache auf ein relationales Datenbankschema. In der Praxis ist diese objektrelationale Abbildung in einer Vielzahl von Projekten zu bewerkstelligen, da die Entscheidung für eine objektorientierte Programmiersprache als auch für ein relationales Datenbanksystem sehr verbreitet ist. Zwischen dem objektorientierten und dem relationalen Datenmodell ergeben sich gewisse Unterschiede, die als [Object-Relational Impedance Mismatch](#) bezeichnet werden. Zwei typische Unterschiede zwischen objektorientiertem und relationalem Modell sind die Abbildung von N:N-Beziehungen und die Abbildung von Vererbungsbeziehungen.

Die Abweichungen zwischen den Modellen werden am Beispiel der folgenden Abbildung verdeutlicht.

- Im objektorientierten Modell links besteht eine N:N-Beziehung zwischen den Klassen **Person** und **Skill**, die im relationalen Modell rechts durch eine zusätzliche Beziehungstabelle **skill_acquisition** mit Fremdschlüsseln dargestellt werden muss.
- Die Vererbungsbeziehung zwischen der allgemeinen Oberklasse **Person** und ihren Unterklassen **Employee** und **Freelancer** im objektorientierten Modell kann im relationalen Modell in drei bekannten, unterschiedlichen Varianten abgebildet werden. In diesem Fall wird die Vererbungshierarchie in einer gemeinsamen Tabelle **person** zusammengefasst (Variante *Single Table*).
- Explizit erlaubte oder ausgeschlossene Navigierbarkeit – wie zwischen den Klassen **Person** und **Department** – kann im relationalen Modell gar nicht dargestellt werden.

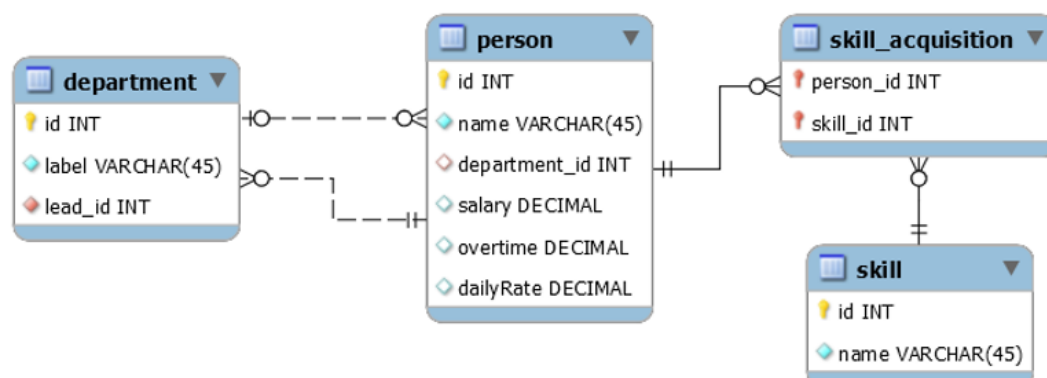
Objektorientiertes Modell

Klassen bilden mittels Assoziationen und Spezialisierung *komplexe* Datenstrukturen



Relationales Modell

Tabellen bilden mittels Fremdschlüsseln *flache* Datenstrukturen



Object-Relational Impedance Mismatch

Einem ORM-Framework kommt nun die Aufgabe zu, die Abbildung zwischen objektorientiertem und relationalem Modell auf Ebene der Programmiersprache durch Annotationen (oder früher durch XML-Konfigurationsdateien) zu ermöglichen. Es ergeben sich allgemein folgende Anforderungen an ein ORM-Framework:

- Klassen, ihre Attribute, ihre Assoziationen und ihre Spezialisierungen abbilden
- Kein SQL in Klassen des Datenmodells
- Mittels Assoziationen des objektorientierten Modells in Anfragen an die Datenbank navigieren
- Einzelne Objekte und Mengen von Objekten über Anfragen ermitteln
- Identität von Objekten gewährleisten
- Transaktionen auf Ebene von Objekten
- Verbesserte Performance durch Caching

Für viele aktuelle Programmiersprachen existieren ORM-Frameworks, wie z.B. das [Entity Framework](#) für .NET-Sprachen, [Doctrine](#) für PHP oder [SQLAlchemy](#) für Python. In Java gibt es durch [JSR 317](#) und [JSR 338](#) eine standardisierte API für die objektrelationale Abbildung im Package `javax.persistence`, die sogenannte *Java Persistence API (JPA)*. Für die JPA existieren verschiedene

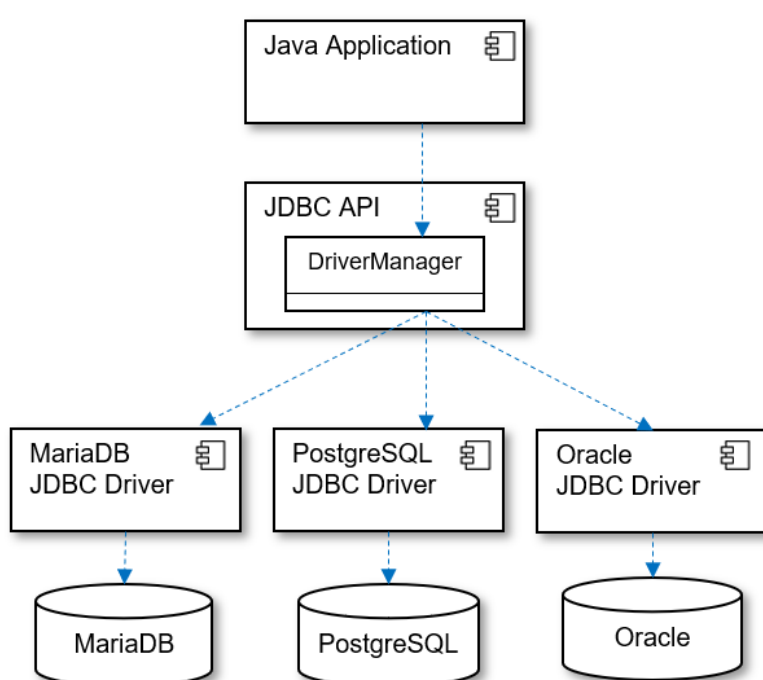
Implementierungen: die Referenzimplementierung [EclipseLink](#), [OpenJPA](#) und insbesondere [Hibernate](#). Hibernate ist seit vielen Jahren in der professionellen Praxis am stärksten verbreitet und hat selbst maßgeblich zur Weiterentwicklung der JPA beigetragen. Jede dieser Implementierungen unterstützt die [geläufigen relationalen Datenbanksysteme](#) (u.a. [MariaDB](#), [MySQL](#), [PostgreSQL](#), [Oracle](#), [Microsoft SQL Server](#)) sowie ihre z.T. spezifischen SQL-Funktionen und -Datentypen. Die Verwendung der JPA kann also dazu beitragen, den *Lock-in-Effekt* zugunsten des Datenbanksystem-Herstellers, der üblicherweise mit der Entscheidung für ein konkretes Datenbanksystem einhergeht, abzuschwächen.

Wenn die JPA in einem Projekt eingesetzt werden soll, wird als Abhängigkeit eine JPA-Implementierung (z.B. [EclipseLink JPA](#)) und ein JDBC-Treiber für das verwendete Datenbanksystem (z.B. [MariaDB Java Client](#)) benötigt. JPA baut intern auf den Grundlagen von [JDBC \(Java Database Connectivity\)](#) auf, um dem Entwickler eine API mit höherem Abstraktionsgrad anzubieten. Um die Unterschiede zwischen der JDBC-API und der JPA-API zu verdeutlichen, werden im Folgenden die Grundlagen von JDBC wiederholt.

Java Database Connectivity (JDBC)

JDBC ist die seit langer Zeit standardisierte API für den Zugriff auf relationale Datenbanksysteme aus Java. JDBC ist implementiert im Java-Modul [java.sql](#). Es gibt JDBC-Treiber von nahezu jedem Hersteller eines relationalen Datenbanksystems (z.B. [MariaDB](#), [MySQL](#), [PostgreSQL](#), [Oracle](#), [Microsoft SQL Server](#)). Der typische Ablauf eines Zugriffs mittels JDBC auf die Datenbank erfolgt in folgenden Schritten:

- Herstellung einer Verbindung zur Datenbank
- Senden eines SQL-Statement, das dynamisch zusammengestellt wird
- Verarbeitung der Datensätze, die in einem `ResultSet`-Objekt zurückgeliefert werden
- Mapping zwischen relationalen und objektorientierten Datenstrukturen



Zugriff auf ein relationales Datenbanksystem per JDBC

Der Verbindungsaufbau zur Datenbank (DB) erfolgt über die Klasse `java.sql.DriverManager`. Es wird implizit auf dem Java-Klassenpfad des Projekts nach einem passenden JDBC-Treiber für das angegebene Protokoll in der URL zum Verbindungsaufbau gesucht. Das im folgenden Code-Beispiel angegebene Protokoll `jdbc:mariadb` kann z.B. durch den Treiber `org.mariadb.jdbc.Driver` bedient werden.

```
1 Connection conn = DriverManager.getConnection("jdbc:mariadb://localhost:3306/test?user=foo&password=pass");
2 conn.setAutoCommit(false);
```

Der URL können initial Konfigurationsparameter mitgegeben werden, z.B. die Credentials für den DB-User (`user=foo&password=pass`), eine Zeitzone (`serverTimezone=CET`), eine Einschränkung auf verschlüsselte Kommunikation mit der DB (`useSSL=true`) oder der Hinweis, dass auch mehrere SQL-Statements gleichzeitig gesendet werden können (`allowMultiQueries=true`). Diese Parameter sind z.T. spezifisch für ein konkretes Datenbanksystem und werden in der jeweiligen Hersteller-Dokumentation beschrieben, z.B. [URL-Parameter für MariaDB](#). Wenn die Verbindung zur Datenbank gelingt, wird ein `java.sql.Connection`-Objekt erzeugt. Mittels des `Connection`-Objekts können `Statement`-Objekte erzeugt werden, über die beliebige SQL-Statements ausgeführt werden können. Die SQL-Statements können dynamisch als String zusammengestellt werden, wobei darauf zu achten ist, [SQL-Injections](#) zu vermeiden. Insbesondere Anführungszeichen müssen [escaped](#) werden, damit über diese nicht der angedachte Zweck eines SQL-Statement verändert werden kann.

```
1 Statement stmt = conn.createStatement();
2 stmt.execute("CREATE TABLE department(id INT KEY AUTO_INCREMENT, label VARCHAR(50))");
3 stmt.execute("CREATE TABLE person(id INT KEY AUTO_INCREMENT, name VARCHAR(50), dept_id INT)");
```

Im nächsten Code-Beispiel werden mehrere SQL-Statements zu einer Transaktion gebündelt, die erst mit dem abschließenden Commit (Zeile 9) beendet wird. Eine Transaktion ist insbesondere dann sinnvoll, wenn ein komplexes Objekt persistiert werden soll, das im relationalen Modell Insert- bzw. Update-Statements für mehrere voneinander abhängige Datensätze in verschiedenen Tabellen verursacht. Im Code-Beispiel soll eine Abteilung (Department) inkl. der ihr zugeordneten Personen gespeichert werden, wobei die jeweiligen Ids automatisch durch das Datenbanksystems vergeben werden.

```
1 Department d = new Department("R&D");
2 d.getStaff().add( new Person("Hendricks") );
3 d.getStaff().add( new Person("Gilfoyle") );
4
5 int deptId = stmt.executeUpdate("INSERT INTO department(label) VALUES('" + d.getLabel() + "')", Statement.RETURN_GENERATED_KEYS);
6 for (Person p : d.getStaff()) {
7     stmt.execute("INSERT INTO person(name, dept_id) VALUES('" + p.getName() + "', " + deptId + ")");
8 }
9 conn.commit();
```

Das folgende Code-Beispiel zeigt ein PreparedStatement, das zum einen bei wiederholter Ausführung effizient ist, da das SQL-Statement vorab geparkt und validiert werden kann, und zum anderen mehr Sicherheit bietet, da durch die typsichere Parameterwertzuweisung SQL-Injections verhindert werden.

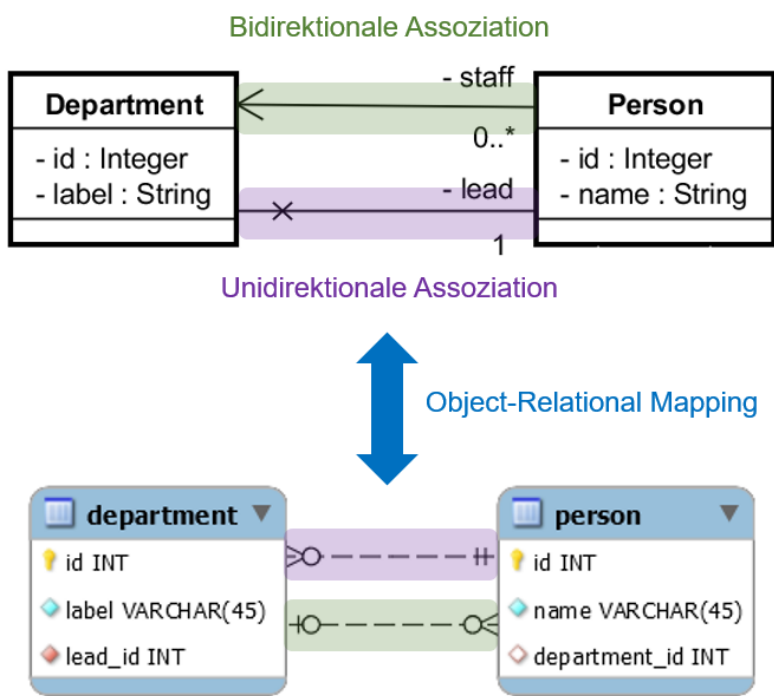
```
1 PreparedStatement prepStmt = conn.prepareStatement(
2     "SELECT p.id, p.name FROM person p JOIN department d ON p.dept_id = d.id WHERE d.label = ?");
3 prepStmt.setString(1, "R&D");
4 ResultSet res = prepStmt.executeQuery();
```

Über das zurückgegebene ResultSet einer SQL-Anfrage kann iteriert werden, um die einzelnen Datensätze zu verarbeiten. Die Werte aus dem ResultSet werden wieder in einer objektorientierten Datenstruktur zusammengeführt – wie im folgenden Code-Beispiel anhand der Klasse Person (Zeilen 2-3) exemplarisch gezeigt wird. Bei einem großen Projekt mit einer Vielzahl von Klassen ist dieses Mapping zwischen objektorientiertem und relationalem Modell mit viel Aufwand verbunden. Ein ORM-Framework automatisiert diesen Schritt, indem es intern zunächst leere Objekte instantiiert (Default-Konstruktor erforderlich) und deren Attribute dann mittels der Reflection API befüllt.

```
1 while (res.next()) {
2     Person p = new Person(res.getString("name"));
3     p.setId(res.getInt("id"));
4     System.out.println(">>> " + p.getId() + ": " + p.getName());
5 }
```

ORM mit der Java Persistence API (JPA)

Das Mapping mittels Annotationen zwischen objektorientiertem und relationalem Modell soll anhand der beiden Beispielklassen Person und Department erklärt werden, die über eine bidirektionale Assoziation (Attribut Department department in Klasse Person ↔ Attribut List<Person> staff in Klasse Department) und eine unidirektionale Assoziation (Attribut Person lead in Klasse Department) miteinander verbunden sind. Sämtliche Annotationen kommen aus dem Package javax.persistence.



Object-Relational Mapping

Klassen ohne JPA-Annotationen

```
1 public class Department {
2     int id;
3     String label;
4     Set<Person> staff;
5     Person lead;
6 }
7
8 public class Person {
9     int id;
10    String name;
11    Department department;
12 }
```

SQL-Statements

```
1 CREATE TABLE department (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     label VARCHAR(45) NOT NULL,
4     lead_id INT NOT NULL,
5     FOREIGN KEY (lead_id) REFERENCES person(id)
6 );
7
8 CREATE TABLE person (
9     id INT PRIMARY KEY AUTO_INCREMENT,
10    name VARCHAR(45) NOT NULL,
11    dept_id INT,
12    FOREIGN KEY (dept_id) REFERENCES department(id)
13 );
```

Klassen mit JPA-Annotationen

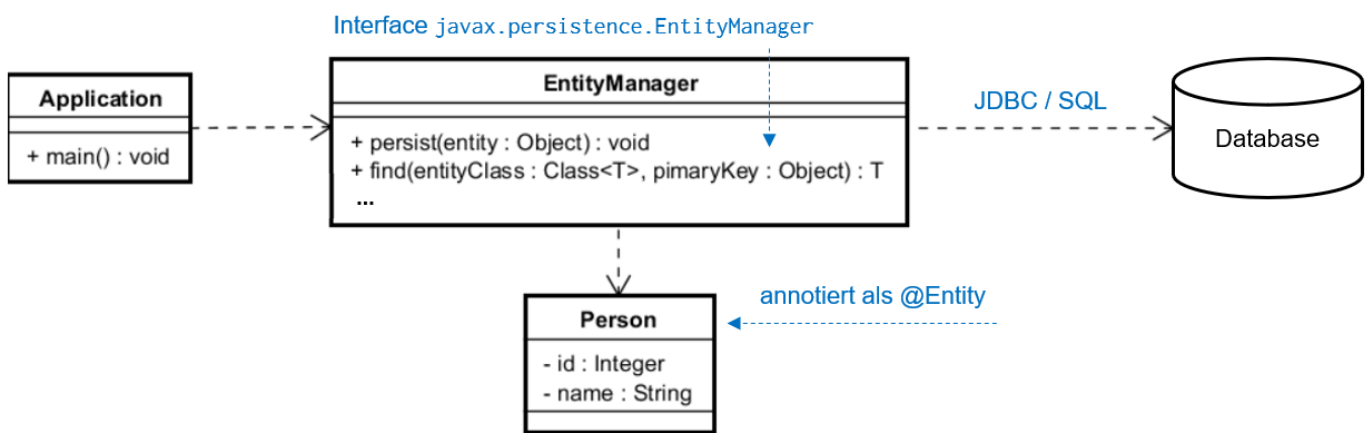
```
1 import javax.persistence.Entity;
2 // ...
3
4 @Entity
5 public class Department {
6
7     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
8     int id;
9
10    @Column(name = "dept_label", nullable = false, length = 45)
11    String label;
12
13    @OneToMany(mappedBy = "department") // bidirectional relationship
14    Set<Person> staff;
15
16    @ManyToOne @JoinColumn(name = "lead_id", nullable = false) // unidirectional relationship
17    Person lead;
18 }
19
20 @Entity
21 public class Person {
22
23     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
24     int id;
25
26     @Column(nullable = false, length = 45)
27     String name;
28
29     @ManyToOne @JoinColumn(name = "dept_id") // bidirectional relationship
30     Department department;
31 }
```

- **@Entity:** Jede Klasse, deren Objekte mittels JPA geschrieben oder gelesen werden sollen, müssen mit der Annotation **@Entity** versehen werden. Ein Objekt, das im JPA-Persistenzkontext bekannt ist, wird als Entität (engl. *Entity*) bezeichnet.
- **@Id:** Das Attribut, das als Primärschlüssel fungieren soll, ist mit der Annotation **@Id** zu kennzeichnen. Für numerische Ganzzahl-Datentypen kann ein eindeutiger Wert generiert werden (**@GeneratedValue**). Für jede Entity ist ein Primärschlüssel verpflichtend. Zusammengesetzte Primärschlüssel sind auch möglich.
- **@Column:** Die Annotation **@Column** ist optional. Auch Attribute ohne diese Annotation werden persistiert. Wenn Attribute explizit nicht persistiert werden sollen, sind sie mit der Annotation **@Transient** zu kennzeichnen. Mittels **@Column** kann die Spalte im relationalen Modell umbenannt und der Wertebereich des Attributs gezielt eingeschränkt werden, z.B. durch Angabe von **nullable=true**, **unique=true**, **length=<n>**, **precision=<n>** oder **scale=<n>**.

- **@OneToMany** und **@ManyToOne**: Die beiden Annotationen kennzeichnen jeweils eine Seite einer 1:N-Beziehung. Auf der Seite von **@ManyToOne** wird immer ein singuläres Entity-Attribut erwartet (keine Collection). Die Bezeichnung der Fremdschlüsselspalte (**@JoinColumn**) kann hier optional konfiguriert werden. Auf Seite von **@OneToMany** wird immer ein Collection<Entity>-Attribut erwartet und im Fall einer bidirektionalen Beziehung kann die Steuerung der Beziehung hier an die andere Seite delegiert werden. Im Code-Beispiel kennzeichnet z.B. **@OneToMany(mappedBy="department") Set<Person> ...** (Zeile 13), dass diese bidirektionale Beziehung zwischen einer Abteilung und den ihr zugeordneten Personen über das Attribut **department** in der Klasse **Person** gesteuert wird. Bei einer unidirektionalen Beziehung gibt es keine entsprechende Gegenseite.
- **@OneToOne** und **@ManyToMany**: Entsprechend den eben genannten Annotationen für 1:N-Beziehungen gibt es auch Annotationen für 1:1-Beziehungen (**@OneToOne**) und N:N-Beziehungen (**@ManyToMany**). Bei einer N:N-Beziehung lassen sich mittels der **@ManyToMany**-Annotation auch die Bezeichnungen der erforderlichen Beziehungstabelle und ihrer Fremdschlüsselspalten konfigurieren.

JPA-Persistenzkontext

Der JPA-Persistenzkontext je Session wird durch ein sogenanntes **EntityManager**-Objekt verwaltet. Wie in der folgenden Abbildung dargestellt wird, erfolgt in JPA jeder Lese- oder Schreibzugriff auf die Datenbank über Methoden des **EntityManager**. Das Pendant zum eher abstrakten Begriff **EntityManager** wird in Hibernate etwas sprechender als **Session** bezeichnet. Das Interface **org.hibernate.Session** erweitert dementsprechend das Interface **javax.persistence.EntityManager**. Der **EntityManager** verwaltet also sämtliche Entitäten, die während einer Session mit der Datenbank ausgetauscht werden.



EntityManager in JPA

Das folgende Code-Beispiel zeigt, wie ein **EntityManager** über eine zugehörige **Fabrik** erzeugt werden kann. Die Fabrik selbst ist ein Singleton, das nur einmal im Anwendungskontext benötigt wird.

```
1 EntityManagerFactory emf = Persistence.createEntityManagerFactory("mariadb-localhost");
2 EntityManager em = emf.createEntityManager();
```

Bei der Erzeugung der Fabrik wird der Name einer sogenannten **Persistence Unit** angegeben, hier z.B. **mariadb-localhost**. Per Konvention wird diese **Persistence Unit** in einer XML-Konfigurationsdatei namens **META-INF/persistence.xml** spezifiziert und vom JPA-Framework dort nachgeschlagen. Es folgt die exemplarische Spezifikation der **Persistence Unit** namens **mariadb-localhost**, in der u.a. die URL für den Verbindungsaufbau per JDBC und die Credentials für den DB-User angegeben sind. Zu berücksichtigen ist, dass die Eigenschaft **<property name="javax.persistence.schema-generation.database.action" value="drop-and-create"/>** (Zeile 9) dafür sorgt, dass sämtliche Tabellen in der Datenbank bei jedem Anwendungsstart neu erzeugt werden. Das ist natürlich nur zur Entwicklungszeit eine sinnvolle Konfiguration.

```
1 <persistence-unit name="mariadb-localhost">
2   <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
3   <exclude-unlisted-classes>false</exclude-unlisted-classes>
4   <properties>
5     <property name="javax.persistence.jdbc.driver" value="org.mariadb.jdbc.Driver"></property>
6     <property name="javax.persistence.jdbc.url" value="jdbc:mariadb://localhost:3306/test"></property>
7     <property name="javax.persistence.jdbc.user" value="foo"></property>
8     <property name="javax.persistence.jdbc.password" value="pass"></property>
9     <property name="javax.persistence.schema-generation.database.action" value="drop-and-create"></property>
10    <property name="eclipselink.logging.level.sql" value="FINE"></property>
11  </properties>
12 </persistence-unit>
```

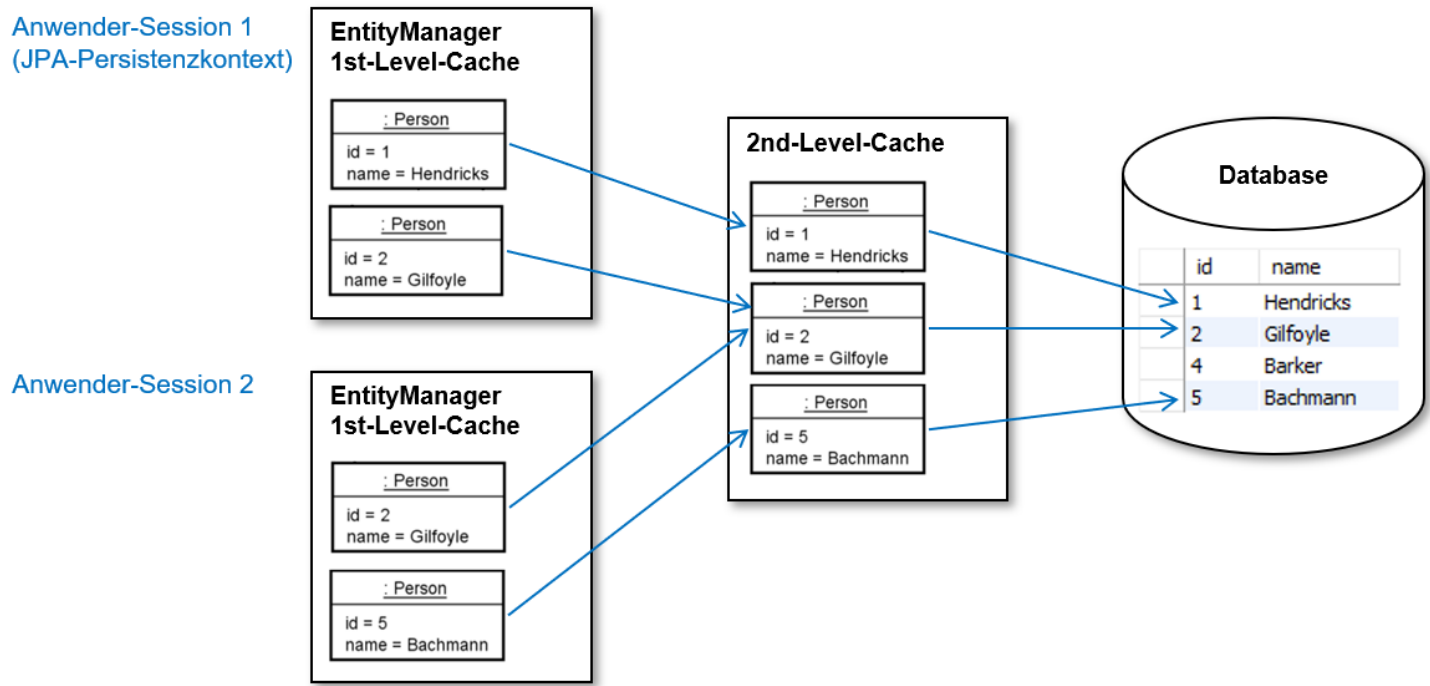
Das Interface **EntityManager** bietet eine Vielzahl von Methoden zur Synchronisation von Entitäten mit der Datenbank über den JPA-Persistenzkontext. Die folgende Tabelle stellt eine Auswahl der grundlegenden Methoden vor.

Type	Method and description
boolean	contains(Object entity) Check if the instance is a managed entity instance belonging to the current persistence context.

T	<code>find(Class<T> entityClass, Object primaryKey)</code> Find by primary key.
void	<code>persist(Object entity)</code> Make an instance managed and persistent.
T	<code>merge(T entity)</code> Merge the state of the given entity into the current persistence context.
void	<code>remove(Object entity)</code> Remove the entity instance.
void	<code>refresh(Object entity)</code> Refresh the state of the instance from the database, overwriting changes made to the entity, if any.
void	<code>detach(Object entity)</code> Remove the given entity from the persistence context, causing a managed entity to become detached.
void	<code>clear()</code> Clear the persistence context, causing all managed entities to become detached.

Eine Entität (= Objekt einer Entitätsklasse) ist entweder im Zustand *managed*, d.h. aktuell durch den JPA-Persistenzkontext zur Synchronisation mit der Datenbank verwaltet, oder im Zustand *detached*, d.h. losgelöst aus dem JPA-Persistenzkontext. Der EntityManager synchronisiert die Entitäten auf dem Java-Heap im Zustand *managed* mit deren Abbild in der relationalen Datenbank. Die dazu erforderlichen Schreibzugriffe per SQL erfolgen spätestens beim Commit einer Transaktion oder können manuell mittels der Methode `flush` forciert werden. Mittels der Methoden `persist` oder `merge` können Entitäten in den Zustand *managed* gebracht werden. Auch die Methode `find` gibt eine Entität im Zustand *managed* zurück. Mittels der Methoden `detach` und `clear` können Entitäten in den Zustand *detached* gebracht werden, ohne sie vorher aus der Datenbank zu löschen. Die Methode `remove` löscht eine Entität aus der Datenbank und bringt sie ebenfalls in den Zustand *detached*. Wenn die Datensätze in der Datenbank außerhalb des JPA-Kontexts verändert werden, geraten die zugehörigen Entitäten in einen inkonsistenten Zustand und können über die Methode `refresh` durch Lesezugriffe auf die Datenbank aktualisiert werden.

Neben dem JPA-Persistenzkontext je EntityManager/Session, der als 1st-Level-Cache angesehen werden kann, existiert i.d.R. ein gemeinsamer 2nd-Level-Cache, den sich alle EntityManager-Instanzen teilen. Zur Veranschaulichung dient die folgende Abbildung. Selbst wenn aus einer Anwender-Session noch nicht auf eine bestimmte Entität zugegriffen worden ist, kann eine Anfrage ggf. aus dem 2nd-Level-Cache bedient werden, so dass keine SQL-Anfrage an die Datenbank gesendet werden muss, falls diese Entität kürzlich in einer parallelen Session angefragt worden ist.



Caching in JPA

Das folgende Code-Beispiel zeigt wie beide Caches explizit geleert werden können.

```
1 | em.clear(); // clear 1st level cache
2 | em.getEntityManagerFactory().getCache().evictAll(); // clear 2nd level cache
```

Transaktionen in JPA

In dem folgenden Code-Beispiel wird ein `Department`-Objekt mit einem zugeordneten `Person`-Objekt persistiert. Zu beachten ist, dass jeder schreibende Zugriff auf die Datenbank explizit in eine Transaktion eingebettet werden muss.


```

1 | Department dept = new Department("R&D");
2 | Person person = new Person("Hendricks", dept); // person is associated with dept
3 |
4 | em.getTransaction().begin();
5 | em.persist(dept);
6 | em.persist(person);
7 | em.getTransaction().commit();

```

Der Versuch nur das `Person`-Objekt ohne das abhängige `Department`-Objekt zu persistieren, misslingt mit einer aussagekräftigen Fehlermeldung:

```

1 | em.getTransaction().begin();
2 | em.persist(person); // try to persist person without persisting associated department in the same transaction
3 | em.getTransaction().commit();

```

>>> Exception in thread "main" javax.persistence.RollbackException: java.lang.IllegalStateException: During synchronization a new object was found through a relationship that was not marked cascade PERSIST: model.Department@xxxxxxx.

Eine Transaktion kann mittels `em.getTransaction().rollback()` abgebrochen werden. Im Fehlerfall kann eine `RollbackException` gefangen und behandelt werden. Es ist davon auszugehen, dass mehrere Transaktionen parallel ausgeführt werden, wenn mehrere Anwender zeitgleich mit einer Anwendung interagieren.

Kaskadierende Zugriffe auf Entitäten und Lazy Loading

Voneinander abhängige Objekte können auch als ein zusammengehöriges Aggregat behandelt werden. Dabei werden Attribute mit komplexem Datentyp (d.h. Datentyp entspricht einer anderen Entitätsklasse wie in Zeile 6 des folgenden Code-Beispiels) kaskadierend gespeichert (`CascadeType.PERSIST`), aktualisiert (`CascadeType.MERGE`), aus der Datenbank gelöscht (`CascadeType.REMOVE`), neu geladen (`CascadeType.REFRESH`) oder aus dem Persistenzkontext gelöst (`CascadeType.DETACH`). Dazu muss die entsprechende Assoziation zwischen den Klassen als kaskadierend hinsichtlich einer oder mehrerer Methoden des EntityManager eingestellt werden (Zeile 4).

Person

```

1 | @Entity
2 | public class Person {
3 |
4 |     @ManyToOne(cascade = CascadeType.ALL) // ALL includes PERSIST, MERGE, REMOVE, REFRESH, DETACH
5 |     @JoinColumn(name = "dept_id")
6 |     Department department;
7 |
8 |     // ...
9 | }

```

Application

```

1 | public class Application {
2 |
3 |     public static void main(String[] args) {
4 |
5 |         // connect to database
6 |         EntityManagerFactory emf = Persistence.createEntityManagerFactory("mariadb-localhost");
7 |         EntityManager em = emf.createEntityManager();
8 |
9 |         // create example objects
10 |        Department dept = new Department("R&D");
11 |        Person person = new Person("Hendricks", dept);
12 |
13 |        // persist objects to database
14 |        em.getTransaction().begin();
15 |        em.persist(person); // persists person and implicitly associated department
16 |        em.getTransaction().commit();
17 |
18 |        // close session
19 |        em.close();
20 |    }
21 | }

```

Entsprechend den Schreibzugriffen kann auch das Lesen eines komplexen Objekts aus der Datenbank schrittweise in Teilen (`FetchType.LAZY`) oder als Ganzes (`FetchType.EAGER`) erfolgen. Der Default ist das einzelne Entity-Attribut stets *eager* (`@OneToOne`- und `@ManyToOne`-Assoziationen) und `Collection<Entity>`-Attribut stets *lazy* geladen werden (`@OneToMany`- und `@ManyToMany`-

Assoziationen). Das folgende Code-Beispiel zeigt die Klasse `Department`, deren Attribut `lead` vom Typ `Person` *eager* geladen wird, während das Attribut `staff` vom Typ `Set<Person>` *lazy* geladen wird. Die angegebenen `FetchType`-Werte entsprechen hierbei dem Default und wären daher nicht notwendig.

```
1 | @Entity
2 | public class Department {
3 |
4 |     @ManyToOne(fetch = FetchType.EAGER)
5 |     @JoinColumn(name = "lead_id")
6 |     Person lead;
7 |
8 |     @OneToMany(fetch = FetchType.LAZY, mappedBy = "department")
9 |     Set<Person> staff;
10 |
11 |     // ...
12 | }
```

Beim *Lazy Loading* wird das entsprechende Attribut erst bei einem späteren Zugriff befüllt. Im folgenden Code-Beispiel löst die Zeile 1 eine oder zwei SQL-Anfragen aus: zum einen wird stets der gesuchte Datensatz aus der Tabelle `department` gelesen, zum anderen der zugehörige Abteilungsleiter ermittelt, falls das Attribut `lead` in der Klasse `Department` gesetzt ist. Das Laden der zugeordneten Personen im Attribut `staff` durch eine SQL-Anfrage auf die Tabelle `person` erfolgt erst beim ersten Zugriff auf eine Methode dieses Attributs in Zeile 8. Bis dahin ist das Attribut nicht instantiiert (vgl. Zeilen 5-6). Im Debug-Modus kann das Lazy Loading nicht ohne Weiteres beobachtet werden, da der Debugger selbst das Laden der Attribute verursacht, um deren aktuellen Status anzeigen zu können.

```
1 | Department dept = em.find(Department.class, 1); // finds person with id = 1
2 | // causes department query: SELECT id, label, lead_id FROM department WHERE id = ?
3 | // causes department lead query, if lead_id is not null: SELECT id, name, dept_id FROM person WHERE id = ?
4 |
5 | System.out.println(">>> persons in department: " + dept.getStaff());
6 | // >>> persons in department: {IndirectSet: not instantiated}
7 |
8 | System.out.println(">>> persons in department: " + dept.getStaff().size());
9 | // causes department staff query: SELECT id, name, dept_id FROM person WHERE dept_id = ?
10 | // >>> persons in department: 10
```

Die Vorteile von Lazy Loading sind eine Verkürzung der initialen Ladezeit und ein zunächst geringerer Hauptspeicherbedarf. Allerdings kann das verzögerte Laden später zu einem ungewollten Zeitpunkt während der Anwenderinteraktion geschehen.

Java Persistence Query Language (JPQL)

Das `EntityManager`-Interface bietet neben der Methode `find`, um ein Objekt über seinen Primärschlüssel aus der Datenbank zu lesen, noch weitere Methoden für Datenbankabfragen.

Type	Method and description
Query	<code>createNativeQuery(String sqlString)</code> Create an instance of Query for executing a native SQL statement, e.g., for update or delete.
StoredProcedureQuery	<code>createStoredProcedureQuery(String procedureName)</code> Create an instance of StoredProcedureQuery for executing a stored procedure in the database.
Query	<code>createQuery(String qlString)</code> Create an instance of Query for executing a Java Persistence query language statement.
TypedQuery<T>	<code>createQuery(String qlString, Class<T> resultClass)</code> Create an instance of TypedQuery for executing a Java Persistence query language statement.
TypedQuery<T>	<code>createQuery(CriteriaQuery<T> criteriaQuery)</code> Create an instance of TypedQuery for executing a criteria query.

Dazu zählen eine Methode für native SQL-Anfragen und eine Methode zum Aufruf von *Stored Procedures*. Bevorzugt werden aber Anfragen über die *Java Persistence Query Language (JPQL)*, die ihrerseits stark an SQL angelehnt ist. JPQL orientiert sich in seiner deklarativen Syntax mit `SELECT ... FROM ... WHERE ... GROUP BY ... ORDER BY ...` an dem gleichen Gerüst an Schlüsselworten für eine Anfrage wie SQL. Viele Operatoren (z.B. `LIKE`, `IN`) und Aggregatfunktionen (z.B. `COUNT`, `SUM`, `AVG`) verhalten sich ebenfalls wie in SQL. Ein wichtiger Unterschied ist, dass in der `FROM`-Klausel nicht die zugrundeliegende Tabelle, sondern die objektorientierte Klasse adressiert wird. Demzufolge ist auch die Navigation entlang von Assoziationen über den Operator `.` möglich, welcher bei der Übersetzung in eine SQL-Anfrage implizit einen Join verursachen kann. Zur Veranschaulichung dienen die folgenden Beispiele.

```

1  -- Select all persons and their associated department label (causes a join of the person table and the department table)
2  SELECT p.id, p.name, p.department.label FROM Person p;
3
4  -- Select the number of persons per department label (causes a join of the person table and the department table)
5  SELECT p.department.label, COUNT(p) FROM Person p GROUP BY p.department.label;
6
7  -- Select the head of the sales department (result is of type Person)
8  SELECT d.lead FROM Department d WHERE d.label = "Sales";

```

Aus einer Java-Anwendung heraus kann JPQL in ungetypten Anfragen (`javax.persistence.Query`) oder getypten Anfragen (`javax.persistence.TypedQuery<T>`) verwendet werden. Bei getypten Anfragen entspricht die Rückgabe der JPQL-Anfrage einer vorab erstellten Klasse des objektorientierten Datenmodells. Bei einer ungetypten Anfrage ist der Typ der Rückgabe nicht definiert. Das folgende Code-Beispiel veranschaulicht eine ungetypte und eine getypte JPQL-Anfrage. Optional kann eine Anfrage parametrisiert werden (Zeilen 8-9) und hinsichtlich der Anzahl der zurückgegebenen Objekte beschränkt werden (Zeile 3). Wenn max. 1 Objekt in der Rückgabe erwartet wird, ist die Methode `getSingleResult` zu verwenden (Zeile 10), ansonsten die Methode `getResultList` (Zeile 4).

```

1  // Untyped query
2  Query q = em.createQuery("SELECT p.id, p.name, p.department.label FROM Person p");
3  q.setMaxResults(100);
4  List<Object[]> result = q.getResultList();
5  result.forEach(object -> System.out.println( Arrays.toString(object) ));
6
7  // Typed query
8  TypedQuery<Person> q = em.createQuery(String jpql = "SELECT d.lead FROM Department d WHERE d.label = :dept", Person.class);
9  q.setParameter("dept", "Sales");
10 Person leadSales = q.getSingleResult();

```

Als Alternative zu einer deklarativen JPQL-Anfrage, die als String an die Methode `createQuery` übergeben wird, gibt es noch die sogenannte *Criteria API*, über die eine JPQL-Anfrage imperativ zusammengestellt werden kann. Zum Vergleich wird die letztere der beiden obigen JPQL-Anfragen im nächsten Code-Beispiel mittels der Criteria API formuliert. Für Entwickler, die SQL-Anfragen gewohnt sind, erscheint der Code, der durch die Criteria API entsteht, vergleichsweise unübersichtlich.

```

1  // Above typed query in Criteria API
2  CriteriaBuilder cb = em.getCriteriaBuilder();
3  CriteriaQuery cq = cb.createQuery(Person.class);
4  Root a = cq.from(Department.class);
5  ParameterExpression<String> p = cb.parameter(String.class);
6  cq.select(a.get("lead")).where(cb.equal(a.get("label"), p));
7  TypedQuery<Person> q = em.createQuery(cq);
8  q.setParameter(p, "Sales");
9  Person leadSales = q.getSingleResult();

```

Nebenläufige Transaktionen in JPA

Bei parallelen Anwender-Sessions mit schreibenden Datenbankzugriffen sind Transaktionen, die die [ACID-Eigenschaften](#) erhalten, erforderlich, um insbesondere sogenannte [Lost Updates](#) zu vermeiden. Die folgende Abbildung zeigt ein potientes Lost Update, da beide Transaktionen parallel zueinander dieselbe Entität verändern möchten.



Lost Update bei nebenläufigen Transaktionen in JPA

Voraussetzung für die Konflikterkennung bei nebenläufigen Transaktionen in JPA ist ein Ganzzahl-Attribut in der betroffenen Entity-Klasse, das mit der Annotation `@Version` versehen ist. Über dieses Attribut gelingt es dem JPA-Framework unterschiedliche Versionen einer Entität voneinander zu unterscheiden. Bezüglich des Sperrverhaltens ist der Default sogenanntes *Optimistic Locking*. Hierbei ist die Annahme, dass Konflikte zwischen nebenläufigen Transaktionen selten auftreten. Konflikte werden nicht vorab durch das Setzen expliziter Sperren verhindert, sondern erst zum Zeitpunkt des Commit erkannt. Eine Transaktion wird beim Commit ggf. zurückgerollt, falls sie Entitäten gelesen oder verändert hat, deren Zustand während der Transaktionslaufzeit durch eine andere Transaktion verändert und schneller erfolgreich *committed* worden ist. Es entsteht also ein Wettlauf zum Commit, der zu kurzen Transaktionen diszipliniert. Das folgende Code-Beispiel für *Optimistic Locking* zeigt, wie eine Transaktion (EntityManager `em1`) durch eine andere Transaktion (EntityManager `em2`) überholt wird und dadurch beim Commit auf eine `RollbackException` (verursacht durch eine `OptimisticLockException`) stößt.

Alternativ kann beim Zugriff auf eine Entität explizit eine exklusive Sperre gesetzt werden, die bis zur Freigabe der Sperre beim Commit verhindert, dass andere Transaktionen die gesperrte Entität verändern können. Dieses Sperrverhalten wird als *Pessimistic Locking* bezeichnet. Es schränkt die mögliche Parallelität stark ein. Bei einer extensiven Verwendung von Sperren sollten Grundlagen aus dem Datenbankbereich wie das [2-Phasen-Sperrprotokoll](#) berücksichtigt werden. Das folgende Code-Beispiel für *Pessimistic Locking* zeigt, wie eine durch eine Sperre verzögerte Transaktion (EntityManager `em2`) auf eine `RollbackException` (verursacht durch einen *Lock-Wait-Timeout*) stoßen kann.

Optimistic Locking in JPA

```
1 // create 2 concurrent sessions
2 EntityManagerFactory emf = Persistence.createEntityManagerFactory("...");
3 EntityManager em1 = emf.createEntityManager();
4 EntityManager em2 = emf.createEntityManager();
5
6 // create managed object
7 em1.getTransaction().begin();
8 Department a = new Department("A");
9 a = em1.merge(a);
10 em1.getTransaction().commit();
11
12 // provoke concurrency conflict: transaction 1 starts, but is not committed
13 em1.getTransaction().begin();
14 Department a1 = em1.find(Department.class, a.getId());
15 a1.setLabel("A1");
16
17 // transaction 2 overtakes transaction 1
18 em2.getTransaction().begin();
19 Department a2 = em2.find(Department.class, a.getId());
20 a2.setLabel("A2");
21 em2.getTransaction().commit();
22
23 // transaction 1 is cancelled, if @Version attribute exists in entity class
24 try {
25     em1.getTransaction().commit();
26 } catch (RollbackException e) {
27     e.printStackTrace();
28 }
```

```
>>> javax.persistence.RollbackException caused by: javax.persistence.OptimisticLockException
The object [model.Department@xxxxxxxx] cannot be updated because it has changed or been deleted since it was last read.
```

Pessimistic Locking in JPA

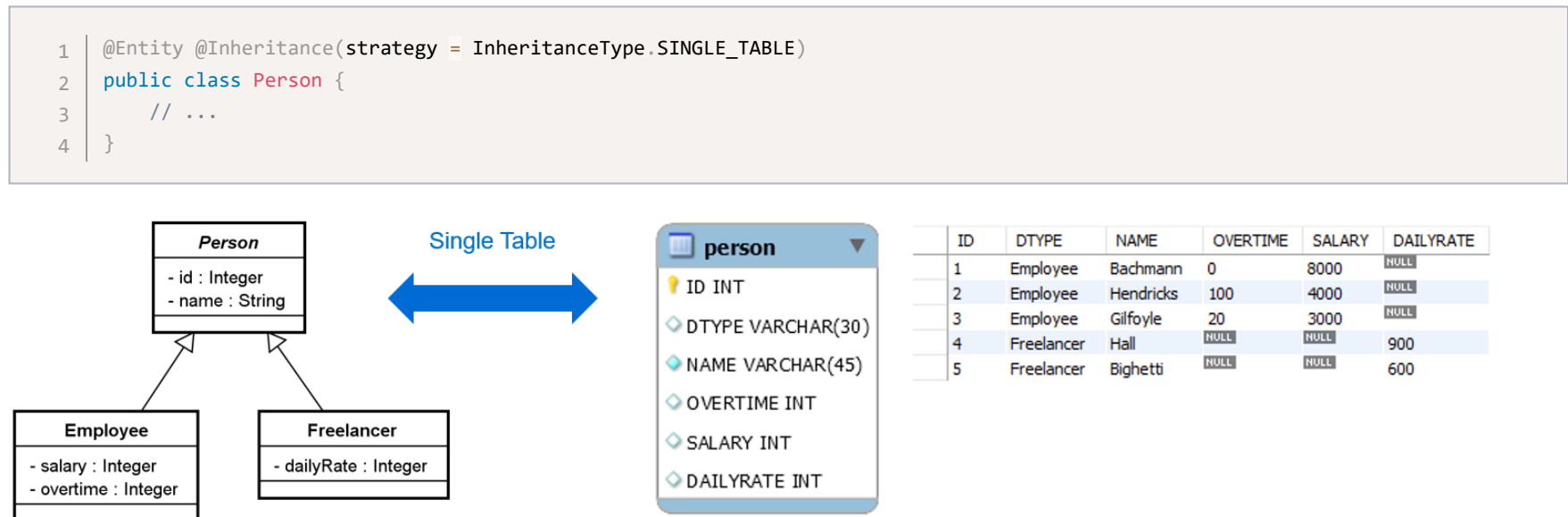
```
1 // create 2 concurrent sessions
2 EntityManagerFactory emf = Persistence.createEntityManagerFactory("...");
3 EntityManager em1 = emf.createEntityManager();
4 EntityManager em2 = emf.createEntityManager();
5
6 // create managed object
7 em1.getTransaction().begin();
8 Department a = new Department("A");
9 a = em1.merge(a);
10 em1.getTransaction().commit();
11
12 // provoke concurrency conflict: transaction 1 starts, but is not committed
13 em1.getTransaction().begin();
14 Department a1 = em1.find(Department.class, a.getId(), LockModeType.PESSIMISTIC_READ); // lock entity
15 a1.setLabel("A1");
16
17 // transaction 2 overtakes transaction 1
18 em2.getTransaction().begin();
19 em2.createNativeQuery("SET innodb_lock_wait_timeout = 3").executeUpdate(); // decrease lock-wait-timeout (default is 50 s)
20 Department a2 = em2.find(Department.class, a.getId());
21 a2.setLabel("A2");
22
23 // commit of transaction 2 is blocked and cancelled after lock-wait-timeout, as transaction 1 has set a lock
24 try {
25     em2.getTransaction().commit();
26 } catch (RollbackException e) {
27     e.printStackTrace();
28 }
29
30 // finally commit transaction 1
31 em1.getTransaction().commit();
```

```
>>> javax.persistence.RollbackException caused by: java.sql.SQLException
Lock wait timeout exceeded; try restarting transaction.
```

Vererbung in JPA

Zur Abbildung von Vererbungsbeziehungen im relationalen Modell gibt es drei geläufige Ansätze, die in der JPA als *Single Table* (`InheritanceType.SINGLE_TABLE`), *Joined Subclass* (`InheritanceType.JOINED`) und *Table per Class* (`InheritanceType.TABLE_PER_CLASS`) bezeichnet werden. Die Vor- und Nachteile der unterschiedlichen Abbildungen von Vererbungshierarchien sollen im Folgenden erläutert werden.

- **Single Table:** Bei dieser Vererbungsstrategie entsteht lediglich eine breite Tabelle für die komplette Vererbungshierarchie. Eine Diskriminatorspalte wird benötigt, um zu erfassen, von welchem konkreten Typ ein Datensatz ist. Die Tabelle ist bei einer Vererbungshierarchie mit vielen Unterklassen und vielen Attributen in den Unterklassen ggf. dünn besetzt, d.h. die Datensätze haben in vielen Attributen keine sinnvollen Ausprägungen (viele NULL-Werte).



Object-Relational Mapping: Vererbungsstrategie Single Table

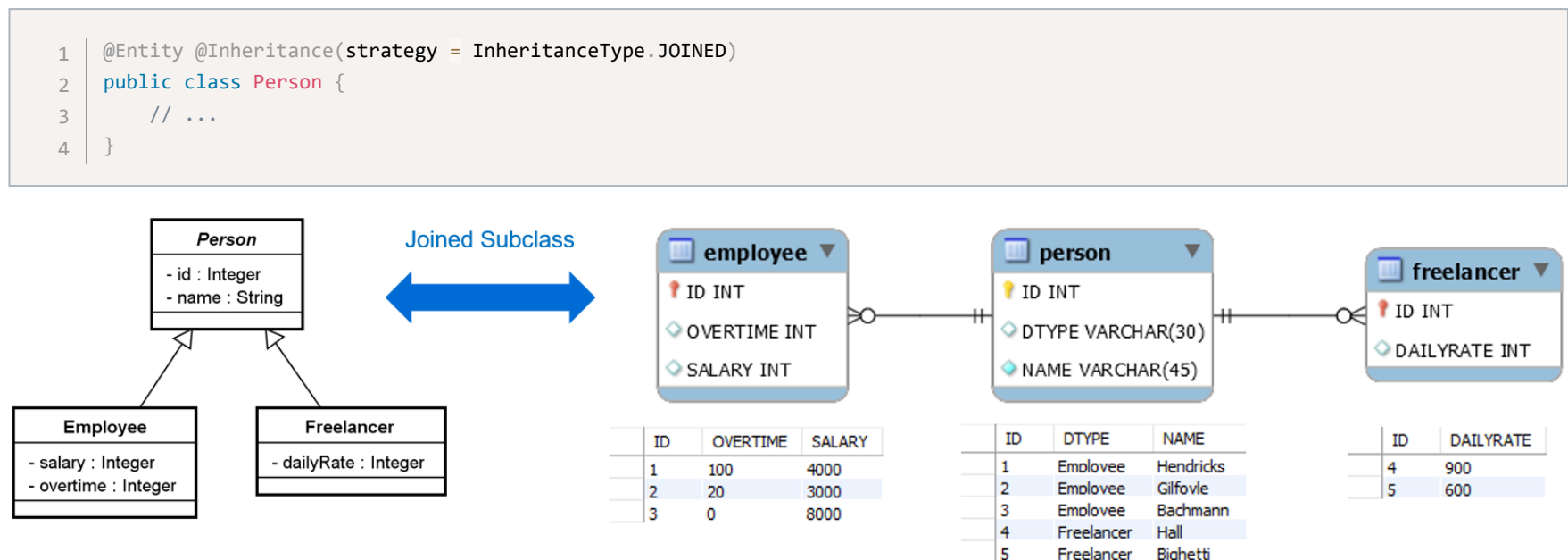
Vorteile:

- Zugriffe betreffen nur eine Tabelle und erfordern keine Joins.
- Polymorphe Anfragen sind einfach (z.B. Namen aller Personen, egal ob Angestellter oder Freelancer).

Nachteile:

- Bei vielen Klassen in der Vererbungshierarchie wird die Tabelle schnell sehr breit und unübersichtlich.
- Bei Anfragen nach einem bestimmten Typ (z.B. Namen aller Freelancer) wird die ganze Tabelle gelesen.

- **Joined Subclass:** Bei dieser Vererbungsstrategie entsteht eine Tabelle je Klasse einer Vererbungshierarchie – auch für abstrakte Oberklassen entsteht jeweils eine Tabelle. Jede Tabelle enthält nur die zusätzlichen Attribute der Unterklasse. Auf die Attribute der Oberklasse wird über einen Fremdschlüssel verwiesen, der gleichzeitig Primärschlüssel der Tabelle ist. Eine Diskriminatorspalte wird in der Tabelle der allgemeinsten Oberklasse benötigt.



Object-Relational Mapping: Vererbungsstrategie Joined Subclass

Vorteile:

- Entspricht objektorientiertem Modell und 3. relationaler Normalform → leicht erweiterbar.
- Polymorphe Anfragen sind einfach (z.B. Namen aller Personen, egal ob Angestellter oder Freelancer).

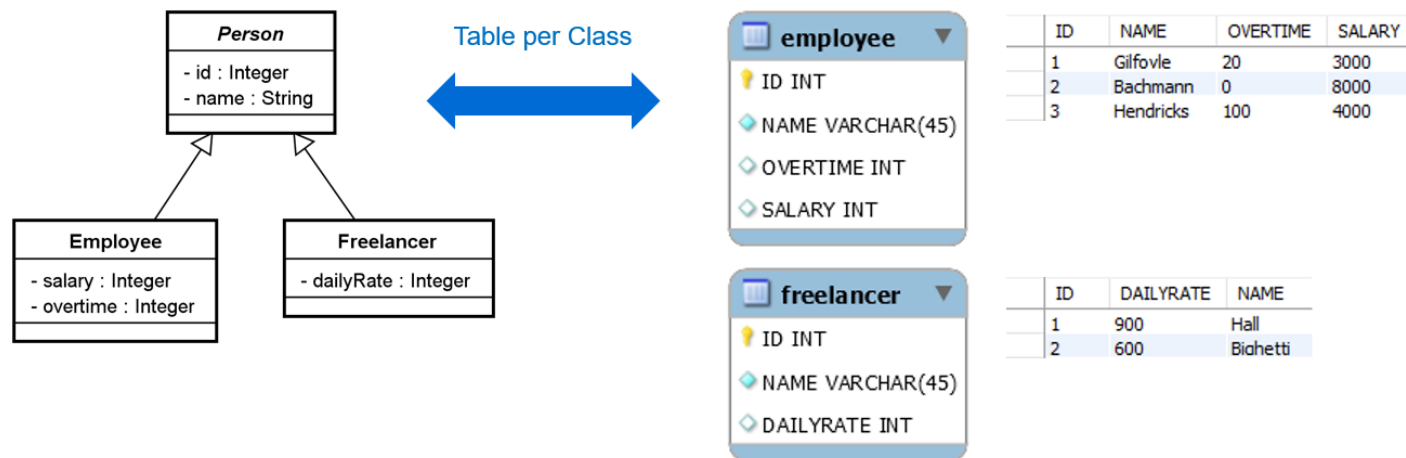
Nachteile:

- Zugriffe betreffen i.d.R. mehrere Tabellen und erfordern Joins (z.B. Name und Gehalt eines Angestellten) → komplexe SQL-Anfragen und bei sehr vielen Datensätzen u.U. schlechtere Performance.
- Bei Anfragen nach einem bestimmten Typ (z.B. Namen aller Freelancer) wird die ganze Tabelle der Oberklasse gelesen.

- **Table per Class:** Bei dieser Vererbungsstrategie entsteht eine Tabelle für jede konkrete Klasse einer Vererbungshierarchie. Jede Tabelle enthält nicht nur die Attribute der konkreten Unterklasse, sondern auch sämtliche Attribute ihrer Oberklassen. Dadurch

gibt es im relationalen Modell keine Fremdschlüssel. Es sind keine Joins erforderlich.

```
1 | @Entity @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
2 | public class Person {
3 |     // ...
4 | }
```



Object-Relational Mapping: Vererbungsstrategie Table per Class

Vorteile:

- Zugriffe betreffen nur einzelne Tabellen und erfordern keine Joins → einfache SQL-Anfragen und gute Performance.
- Anfragen nach einem bestimmten Typ (z.B. Namen aller Freelancer) sind einfach.

Nachteile:

- Polymorphe Anfragen sind umständlich und erfordern die Vereinigung mehrerer Tabellen (z.B. Namen aller Personen).

Entitätsklassen in REST-Ressourcen

Wenn die Objekte der Entitätsklassen direkt über eine mit JAX-RS gebaute REST-API ausgegeben werden (ohne sie explizit in DTO-Klassen zu transformieren), ist darauf zu achten, dass Zyklen bei bidirektionalen Beziehungen unterbrochen werden. Im folgenden Code-Beispiel soll eine Anfrage an den Endpunkt `/departments/<id>` nicht die Personen ausgeben, die einer Abteilung zugeordnet sind. Demzufolge ist das Attribut `Set<Person> staff` mit der Annotation `@JsonIgnore` versehen (Zeilen 10-11 in Klasse `Department`). Auf der anderen Seite dieser Beziehung in der Klasse `Person` ist das Attribut `Department department` exemplarisch mit der Annotation `@JsonIgnoreProperties({"label", "staff"})` versehen (Zeilen 10-11 in Klasse `Person`). Dadurch wird die Ausgabe unter dem Pfad `/persons/<id>` zwar die Abteilung einer Person beinhalten, aber nur mit den Attributen, die nicht explizit ausgeschlossen werden.

Entitätsklasse Department

```
1 | @Entity
2 | public class Department {
3 |
4 |     @Id
5 |     int id;
6 |
7 |     String label;
8 |
9 |     @OneToMany(mappedBy = "department") // bidirectional relationship
10 |    @JsonIgnore
11 |    Set<Person> staff;
12 | }
```

HTTP-Request `/departments/<id>`

```
1 | ---[HTTP GET request]---
2 | Request URL: http://localhost:8080/departments/1
3 |
4 | ---[HTTP response 200]---
5 | Content-type: application/json; charset=utf-8
6 | {
7 |     "id": 1,
8 |     "label": "R&D"
9 | }
```


Entitätsklasse Person

```
1  @Entity
2  public class Person {
3
4      @Id
5      int id;
6
7      String name;
8
9      @ManyToOne @JoinColumn(name = "dept_id") // bidirectional relationship
10     @JsonIgnoreProperties({"label", "staff"})
11     Department department;
12 }
```

HTTP-Request `/persons/<id>`

```
1  ---[HTTP GET request]---
2  Request URL: http://localhost:8080/persons/2
3
4  ---[HTTP response 200]---
5  Content-type: application/json; charset=utf-8
6  {
7      "id": 2,
8      "name": "Gilfoyle",
9      "department": {
10         "id": 1
11     }
12 }
```

Die gezeigten Code-Beispiele zum objekt-relationalen Mapping mittels JPA finden sich im Verzeichnis `/remote/orm` des Modul-Repository.

REST und Reactive Streams in Spring

Das [Spring](#)-Framework ist bereits seit vielen Jahren ein beliebtes Framework für komplexe Java-Anwendungen im betrieblichen Umfeld. Die erste Version von Spring hat Rod Johnson bereits in 2002 vorgestellt. Das Spring-Framework hat sich im Laufe der Jahre mehrfach gewandelt und sich an moderne Programmierparadigmen, Entwurfs- und Architekturmuster angepasst. Heute wird Spring von [Pivotal Software](#) weiterentwickelt.

Inversion of Control

Im Kapitel zu [Dependency Injection](#) ist bereits das grundlegende Paradigma eines jeden Frameworks genannt worden, nämlich *Inversion of Control (IoC)*. Ein Framework gibt einer Anwendung eine grundlegende Struktur vor, also eine Architektur, und steuert den Kontrollfluss der Anwendung. Der Anwendungsentwickler implementiert fachliche Methoden, die er beim Framework an vordefinierten Stellen registriert. Das Framework übernimmt die Steuerung des Kontrollflusses zur Laufzeit und wird die fachlichen Methoden/Anweisungen – ausgelöst durch bestimmte Ereignisse – aufrufen. IoC heißt also, dass die eigenen fachlichen Methoden/Anweisungen in definierte Callback-Methoden eingebettet und dort passiv aufgerufen werden, anstatt dass der Anwendungsentwickler aktiv den Kontrollfluss der Anwendung steuert. Das Spring-Framework hat maßgeblich zur Verbreitung des IoC-Paradigmas beigetragen. Das Spring-Framework selbst übernimmt die Aufgabe des IoC-Containers und verwaltet sogenannte *Beans*. Alle Klassen, die mit der Spring-Annotation `@Component` oder einer ihrer Spezialisierungen wie `@Service`, `@Repository` oder `@Controller` versehen sind, werden von Spring automatisch als Beans erkannt und registriert. Die registrierten Beans können zur Laufzeit über das Singleton-Objekt `ApplicationContext` erreicht werden (s. Kapitel [Dependency Injection](#)).

REST und ORM mit Spring

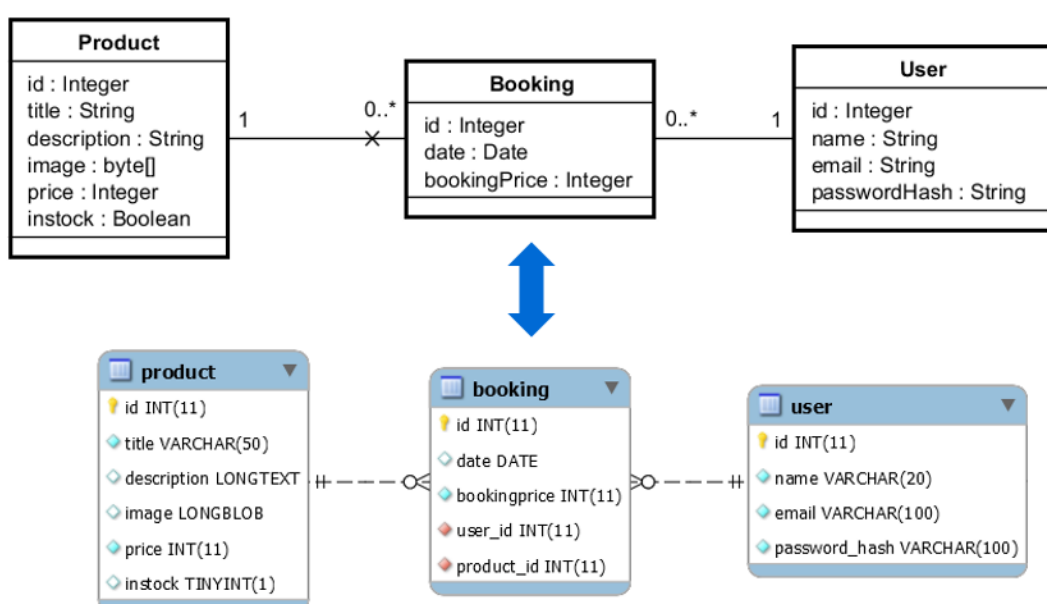
Es soll im Folgenden eine REST-API mit [Spring Data](#) entwickelt werden, die die verwalteten Objekte mittels JPA in einem relationalen Datenbanksystem speichert. Als Datenbanksystem wird MariaDB ausgewählt. Damit ergeben sich für das Projekt folgende Abhängigkeiten:

- [Spring Boot Data REST Starter](#)
- [Spring Boot Data JPA Starter](#)
- [MariaDB Java Client](#)
- optional [Lombok](#)

Bevor wir im Folgenden eigene Beispiele vorstellen, soll an dieser Stelle auf die guten Tutorials auf der Spring-Webseite hingewiesen werden:

- [Einstieg in Spring Data JPA](#)
- [Einstieg in Spring Data REST](#)

Die zu entwickelnde REST-API soll Objekte zu den folgenden Entitätsklassen bereitstellen: `User`, `Product` und `Booking`. Der Zusammenhang der Klassen ist in der nächsten Abbildung veranschaulicht. Die API bildet die Datenhaltung für eine minimale Shop-Anwendung ab, in der authentifizierte Anwender (`User`) Produkte (`Product`) aus einem Produktkatalog auswählen und für diese beliebig viele Buchungen (`Booking`) anlegen und wieder entfernen können. Die hier entwickelte API soll in den folgenden Kapiteln zu den UI-Frameworks als gemeinsame Basis genutzt werden, d.h. die später vorgestellten Clients in [JavaFX](#) und [jQuery/Angular](#) greifen jeweils auf diese API zu.



Datenmodell der entwickelten REST-API mit Spring Data

Product

```
1  @Entity @Getter @Setter
2  public class Product extends IdentifiedEntity {
3
4      @Column(length = 50, nullable = false)
5      String title;
6
7      @Lob
8      String description;
9
10     @Lob
11     @JsonIgnore
12     byte[] image;
13
14     @Column(nullable = false)
15     Integer price;
16
17     Boolean instock;
18 }
```

Booking

```
1  @Entity @Getter @Setter
2  public class Booking extends IdentifiedEntity {
3
4      @Temporal(TemporalType.DATE)
5      @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-dd", timezone = "Europe/Berlin")
6      Date date = new Date();
7
8      @Column(name = "bookingprice", nullable = false)
9      Integer bookingPrice;
10
11     @ManyToOne
12     @JoinColumn(name = "user_id")
13     User user;
14
15     @ManyToOne
16     @JoinColumn(name = "product_id")
17     Product product;
18 }
```

User

```
1  @Entity @Getter @Setter
2  public class User extends IdentifiedEntity {
3
4      @Column(length = 20, nullable = false)
5      String name;
6
7      @Column(length = 100, nullable = false)
8      String email;
9
10     @Column(name = "password_hash", length = 100, nullable = false)
11     String passwordHash;
12
13     @OneToMany(mappedBy = "user")
14     List<Booking> bookings;
15 }
```

IdentifiedEntity

```
1  @MappedSuperclass @Getter @Setter
2  public abstract class IdentifiedEntity {
3
4      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5      Integer id;
6  }
```

Der einfachste Weg mit *Spring Data* eine REST-API zu erzeugen, ist es das [CrudRepository](#)-Interface zu erweitern. In dem folgenden Code-Beispiel wird dieses Vorgehen demonstriert, indem das leere Interface `BookingRepository<Booking, Integer>` das generische `CrudRepository<T, ID>` erweitert und dadurch die in der [CrudRepository-Dokumentation](#) genannten Methoden erhält. Das Framework erstellt zur Laufzeit eine passende Implementierung für das `BookingRepository<Booking, Integer>`, wobei `Booking` die zu verwaltende Entitätsklasse angibt und `Integer` den Typ des Id-Attributs dieser Klasse.

```

1 | import org.springframework.data.repository.CrudRepository;
2 | // ...
3 |
4 | public interface BookingRepository extends CrudRepository<Booking, Integer> {
5 | }

```

Für jedes konkrete `CrudRepository` werden automatisch sogenannte *CRUD*-Endpunkte (*Create*, *Read*, *Update* und *Delete*) in der REST-API erzeugt, d.h. in diesem Fall werden unter dem Pfad `/bookings` die HTTP-Methoden POST und GET unterstützt und unter dem Pfad `/bookings/<id>` die HTTP-Methoden GET, PUT und DELETE. Folgend wird ein HTTP-Request und die zugehörige HTTP-Response der mit *Spring Data REST* generierten REST-API dargestellt. Es wird mittels POST eine neue Buchung angelegt, wobei auf bereits vorab existierende Ressourcen – wie hier das gebuchte Produkt und der buchende User – über deren URI verwiesen werden kann.

```

1 | ---[HTTP POST request]---
2 | Request URL: http://localhost:8080/bookings
3 | Content-type: application/json; charset=utf-8
4 | {
5 |     "product": "http://localhost:8080/products/1",
6 |     "user": "http://localhost:8080/users/2",
7 |     "bookingPrice": 300
8 | }
9 |
10 | ---[HTTP response 201]---
11 | Content-type: application/hal+json; charset=utf-8
12 | Location: http://localhost:8080/bookings/1
13 | {
14 |     "date": "2019-12-31",
15 |     "bookingPrice": 300,
16 |     "_links": {
17 |         "self": {"href": "http://localhost:8080/bookings/1"},
18 |         "user": {"href": "http://localhost:8080/bookings/1/user"},
19 |         "product": {"href": "http://localhost:8080/bookings/1/product"}
20 |     }
21 | }

```

Der Body der HTTP-Response enthält zwar das Datum und den Preis zu einer Buchung, aber nicht die neu vergebene Id und keine weiteren Attribute des referenzierten Produkts und des referenzierten Users. Die Id wird über den HTTP-Header `Location` bekanntgegeben. Die Referenzen auf Produkt und User können über die angegebenen Links aufgelöst werden. *Spring Data REST* erlaubt alternativ die Erstellung von [Projektionen](#) für Entitätsklassen, über die deren Ausgabe angepasst werden kann. Das folgende Code-Beispiel ergänzt die Ausgabe einer Buchung um die Details des gebuchten Produkts, so dass diese nicht über einen weiteren HTTP-Request angefragt werden müssen.

BookingInlineProduct

```

1 | @Projection(name = "inlineProduct", types = {Booking.class})
2 | public interface BookingInlineProduct {
3 |     String getDate();
4 |     int getBookingPrice();
5 |     Product getProduct();
6 | }

```

BookingRepository

```

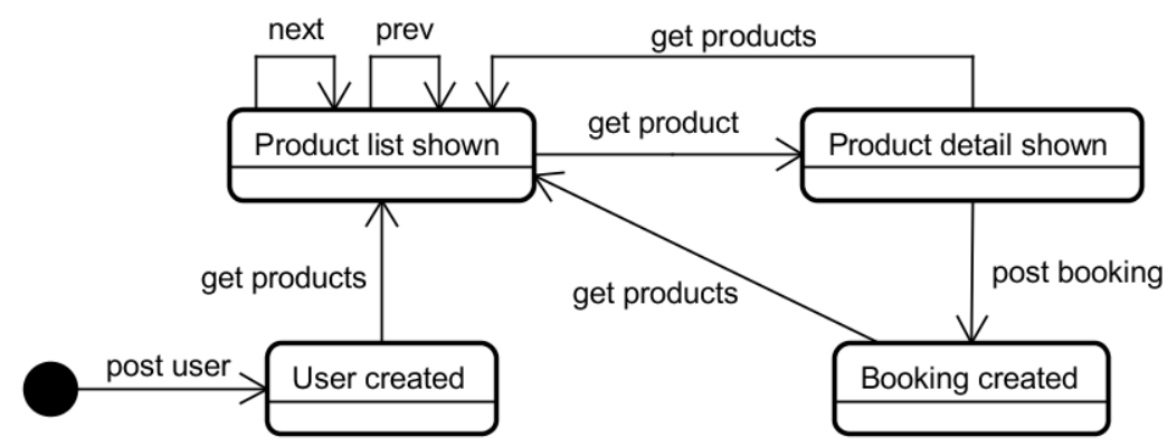
1 | @RepositoryRestResource(excerptProjection = BookingInlineProduct.class)
2 | public interface BookingRepository extends CrudRepository<Booking, Integer> {
3 | }

```

HTTP-Request `/bookings/<id>`

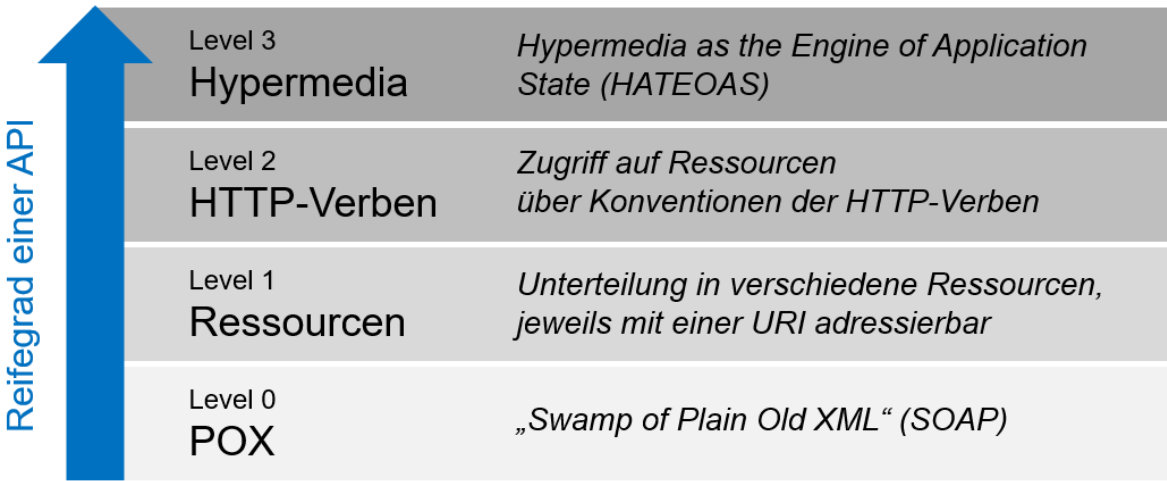
```
1  ---[HTTP GET request]---
2  Request URL: http://localhost:8080/bookings/1?projection=inlineProduct
3
4  ---[HTTP response 200]---
5  Content-type: application/hal+json; charset=utf-8
6  {
7    "id": 1,
8    "date": "2019-12-31",
9    "product": {
10     "title": "Azzam",
11     "description": "Die Azzam ist mit einer Länge von 180 m die längste private Mega-Yacht der Welt. [...]",
12     "price": 181,
13     "instock": true
14   },
15   "bookingPrice": 300,
16   "_links": {
17     "self": { "href": "http://localhost:8080/bookings/1" },
18     "booking": { "href": "http://localhost:8080/bookings/1?projection", "templated": true },
19     "user": { "href": "http://localhost:8080/bookings/1/user" },
20     "product": { "href": "http://localhost:8080/bookings/1/product" }
21   }
22 }
```

Dass die HTTP-Response nicht nur die Daten selbst, sondern auch weiterführende Links zur Navigation innerhalb der API enthält, entspricht dem sogenannten [HATEOAS](#)-Prinzip: *Hypermedia As The Engine Of Application State*. Da der Client bei einer REST-API (im Gegensatz zur WSDL bei SOAP-Webservices) keine formale Schnittstellenbeschreibung erhält, sind die Entwickler, die eine REST-API aufrufen möchten, auf eine gute Dokumentation angewiesen, in der die mögliche Parametrisierung der API-Endpunkte, die über die Konventionen hinausgeht, beschrieben wird. Die grundlegende Idee von HATEOAS ist es, diese Dokumentation in die API selbst einzubetten. Daher werden neben den angefragten Daten auch Links zurückgegeben, die potentielle Transitionen ausgehend von der aktuellen Anfrage zur nächsten Anfrage angeben. Die REST-API wird auf diese Weise als Zustandsautomat betrachtet, durch den mittels Hypermedia (= Repräsentation einer Ressource inkl. Links) navigiert werden kann. Die nächste Abbildung zeigt einen kleinen Ausschnitt einer REST-API als Zustandsautomat.



REST-API als Zustandsautomat

In der Repräsentation einer Ressource im HTML-Format sind wir Links über die Attribute `href` oder `src` gewohnt. Gemäß HATEOAS sind vergleichbare *Hyperlinks* auch bei anderen Formaten wie JSON oder XML zu ergänzen. Entsprechend hat Leonard Richardson ein nach ihm benanntes Reifegrad-Modell für REST-APIs entwickelt, das in der folgenden Abbildung dargestellt ist.



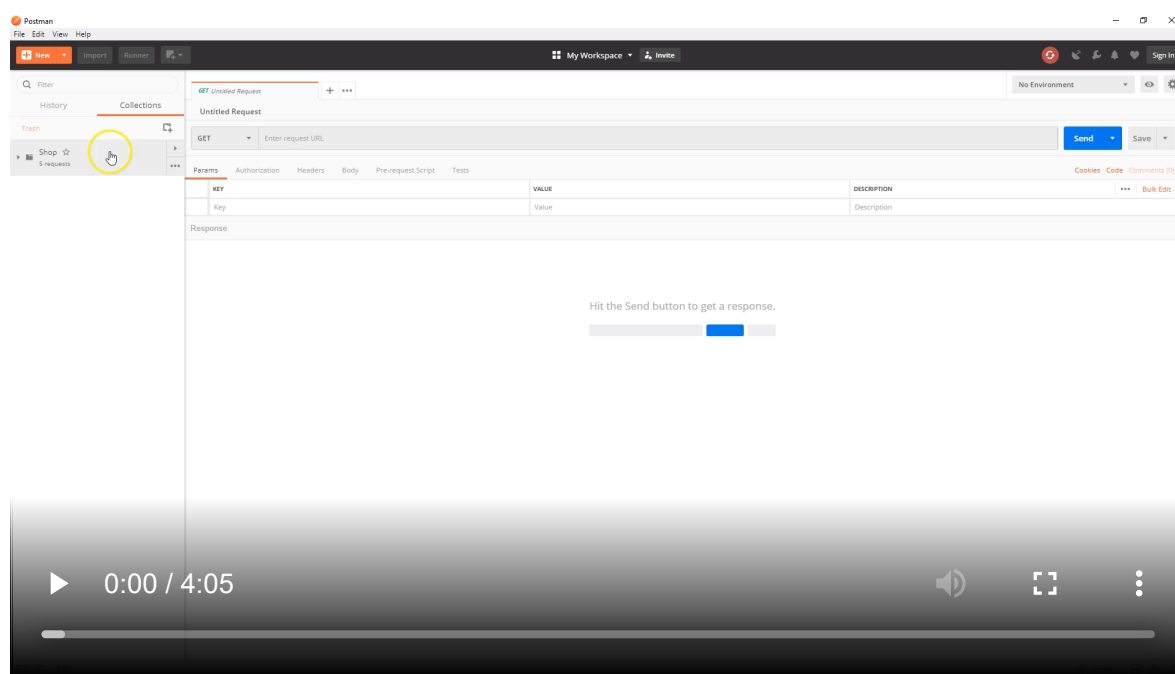
Richardson Maturity Model für API-Design

In einem Interface, das das `CrudRepository` erweitert, können zusätzlich eigene Methoden spezifiziert werden. Für diese *Query Methods* existieren leicht interpretierbare Bezeichnungskonventionen, entsprechend derer eine passende Implementierung durch das Framework generiert wird. Falls das Repository mittels JPA an eine relationale Datenbank gebunden ist, werden z.B. je nach

Methodenbezeichnung bestimmte JPQL-Anfragen erzeugt. In der [Dokumentation zu Spring Data JPA](#) ist genau beschrieben, wie die *Query Methods* zu bezeichnen sind und welche JPQL-Anfrage sich daraus ergibt. Das im nächsten Code-Beispiel dargestellte Interface `ProductRepository` ergänzt z.B. Methoden zur Filterung der ausgegebenen Produkte über ihren Titel, ihre Beschreibung (Volltextsuche) und ihren Preis (Angabe einer Obergrenze).

```
1 import org.springframework.data.repository.PagingAndSortingRepository;
2 // ...
3
4 public interface ProductRepository extends PagingAndSortingRepository<Product, Integer> {
5
6     @RestResource(path = "byTitle")
7     List<Product> findByTitle(@Param(value = "title") String title);
8
9     @RestResource(path = "inDescription")
10    List<Product> findByDescriptionContainingIgnoreCase(@Param(value = "pattern") String pattern);
11
12    @RestResource(path = "maxPrice")
13    List<Product> findByPriceLessThanEqual(@Param(value = "price") Integer price);
14 }
```

Das folgende Video demonstriert die resultierende REST-API dieses Kapitels, indem exemplarische Anfragen mittels Postman ausgeführt werden.



Testen einer mit Spring Data generierten REST-API

Eine Erweiterung des `CrudRepository` ist das oben bereits verwendete [PagingAndSortingRepository](#), das entsprechend seiner Bezeichnung erlaubt, die Menge der Elemente, die mittels der GET-Methode für eine Collection-Ressource zurückgegeben werden, über URL-Parameter in paginierte Abschnitte zu teilen und zu sortieren. Der folgend dargestellte HTTP-Request enthält URL-Parameter zur Paginierung und zur Sortierung, so dass in der zugehörigen HTTP-Response nur 3 von 15 Produkten ausgegeben werden. Die weiterführenden Links zur Navigation durch die API gemäß des HATEOAS-Prinzips sind deutlich zu erkennen.

```

1  ---[HTTP GET request]---
2  Request URL: http://localhost:8080/products?sort=title&page=0&size=3
3
4  ---[HTTP response 200]---
5  Content-type: application/hal+json; charset=utf-8
6  {
7      "_embedded": {
8          "products": [
9              {
10                 "title": "Al Mirqab", "description": "Die Al Mirqab wurde 2004 in Auftrag [...]", "price": 133, "instock": true,
11                 "_links": { "self": { "href": "http://localhost:8080/products/13" } }
12             },
13             {
14                 "title": "Al Said", "description": "Die Al Said ist mit einer Länge von 155 m [...]", "price": 155, "instock": true,
15                 "_links": { "self": { "href": "http://localhost:8080/products/4" } }
16             },
17             {
18                 "title": "Al Salamah", "description": "Die Al Salamah ist eine 139 m lange Yacht [...]", "price": 139, "instock": false,
19                 "_links": { "self": { "href": "http://localhost:8080/products/6" } }
20             }
21         ]
22     },
23     "_links": {
24         "first": { "href": "http://localhost:8080/products?page=0&size=3&sort=title,asc" },
25         "next": { "href": "http://localhost:8080/products?page=1&size=3&sort=title,asc" },
26         "last": { "href": "http://localhost:8080/products?page=4&size=3&sort=title,asc" },
27         "search": { "href": "http://localhost:8080/products/search" }
28     },
29     "page": { "size": 3, "totalElements": 15, "totalPages": 5, "number": 0 }
30 }

```

Die automatisch von Spring generierten Ressourcen-Methoden für ein `CrudRepository` können über verschiedene Wege angepasst werden. Es kann eine Klasse angelegt werden, die das Interface implementiert und die Methoden überschreibt. Alternativ kann eine Klasse, versehen mit der Annotation `@RestController`, angelegt und auf dem Pfad der Ressource registriert werden. Beide Wege erlauben das Verhalten der HTTP-Methoden individuell anzupassen. Im folgenden Code-Beispiel überschreibt die Klasse `UserController` die auf dem Pfad `/users` registrierten Methoden für POST und GET des Interface `UserRepository` (Zeilen 4-5). Bei einer GET-Anfrage werden z.B. nur noch die Namen der vorhandenen User ohne weitere Attribute der Klasse ausgegeben (Zeilen 22-26). Die Methode `findByName` im Interface `UserRepository` ist ohnehin nicht Teil der REST-API, da sie nicht mit einer Annotation `@RestResource` versehen ist. Im `UserRepository` werden explizit noch die Methoden GET und DELETE für den Pfad `/users/{id}` deaktiviert.

UserController

```

1  import org.springframework.web.bind.annotation.RestController;
2  // ...
3
4  @RestController
5  @RequestMapping("/users")
6  public class UserController {
7
8      @Autowired
9      private UserRepository userRepository;
10
11      @Autowired
12      BCryptPasswordEncoder bCryptPasswordEncoder;
13
14      // registers a new user
15      @PostMapping
16      public void register(@RequestBody User user) {
17          user.setPasswordHash(bCryptPasswordEncoder.encode(user.getPasswordHash()));
18          userRepository.save(user);
19      }
20
21      // provides only the user names
22      @GetMapping
23      public List<String> getUserNames() {
24          Stream<User> users = StreamSupport.stream(userRepository.findAll().spliterator(), false);
25          return users.map(User::getName).sorted().collect(Collectors.toList());
26      }
27  }

```


UserRepository

```
1 public interface UserRepository extends CrudRepository<User, Integer> {
2     User findByName(@Param(value = "name") String name);
3
4     @Override
5     @RestResource(exported = false)
6     Optional<User> findById(Integer id);
7
8     @Override
9     @RestResource(exported = false)
10    void delete(User user);
11 }
```

HTTP-Request `/users`

```
1 ---[HTTP GET request]---
2 Request URL: http://localhost:8080/users
3
4 ---[HTTP response 200]---
5 Content-type: application/json; charset=utf-8
6 [ "Alice", "Bob" ]
```

HTTP-Request `/users/<id>`

```
1 ---[HTTP GET request]---
2 Request URL: http://localhost:8080/users/1
3
4 ---[HTTP response 405]---
```

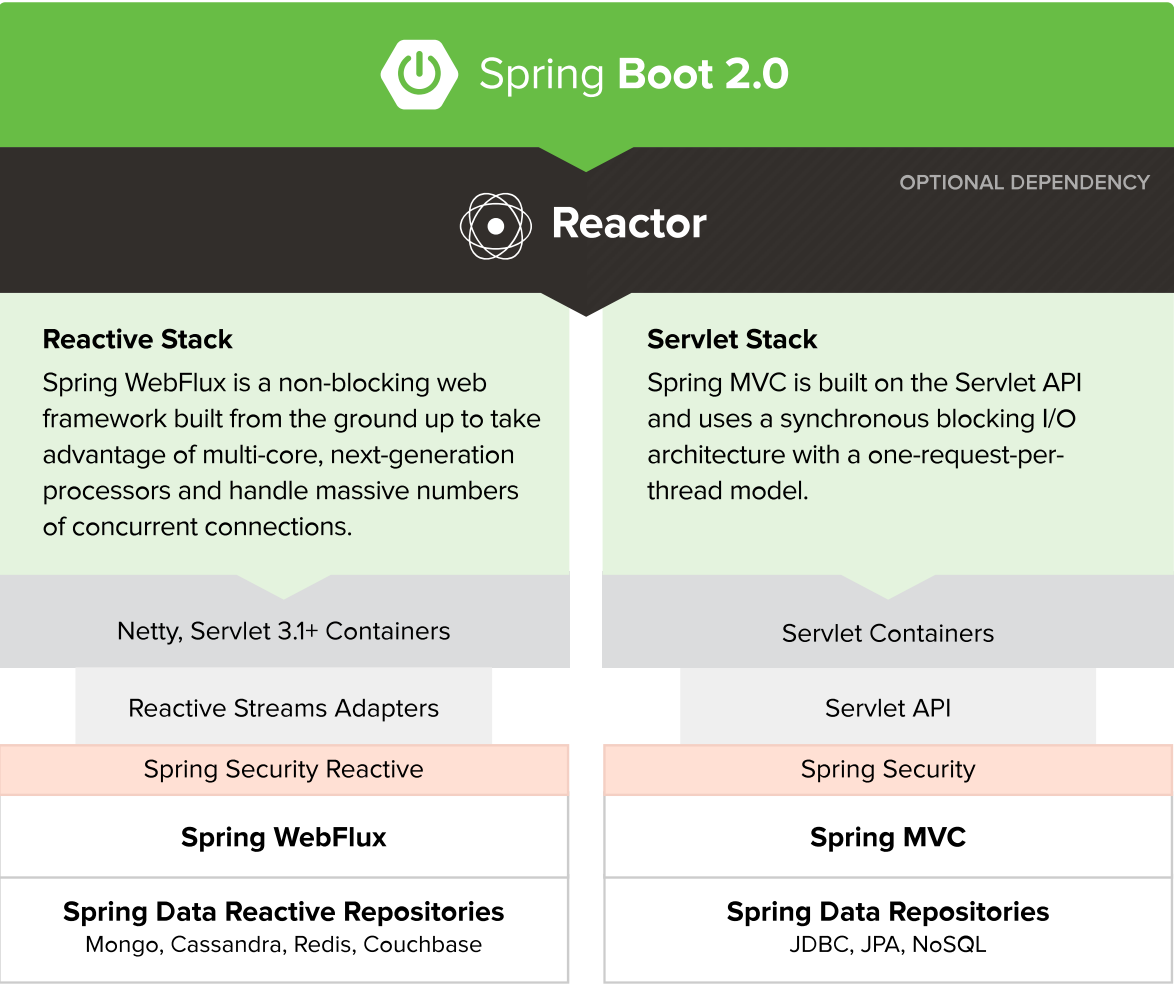
Für ausgewählte Endpunkte der entwickelten REST-API kann der Zugriff mittels [Spring Security](#) abgesichert werden. Falls JSON Web Tokens (JWT) zur Zugangskontrolle eingesetzt werden sollen, findet sich im [Blog von Auth0](#) eine umfassende Einführung, an der sich auch die Implementierung im Package `shop.security` des begleitenden Projekts orientiert. Es entstehen dadurch Abhängigkeiten des Projekts zu [Spring Boot Security Starter](#) und [Java JWT](#).

Reaktive Web-Anwendungen mit Spring

Das ursprüngliche Spring-Framework zur Entwicklung von Web-Anwendungen auf Basis der [Servlet API](#) nennt sich [Spring MVC](#). Neu in der aktuellen Version 5 des Spring-Frameworks ist die Unterstützung der [Reactive Streams API](#) auf Basis der Bibliothek [Reactor](#). Dieser alternative Architekturansatz nennt sich [Spring WebFlux](#) und ist auf asynchrone und nicht-blockierende Kommunikation ausgerichtet. Die folgende Definition aus der Spring-Dokumentation erklärt, was wir unter dem Begriff [Reaktive Programmierung](#) verstehen.

"The term, "reactive," refers to programming models that are built around reacting to change — network components reacting to I/O events, UI controllers reacting to mouse events, and others. In that sense, non-blocking is reactive, because, instead of being blocked, we are now in the mode of reacting to notifications as operations complete or data becomes available. There is also another important mechanism that we on the Spring team associate with "reactive" and that is non-blocking back pressure. In synchronous, imperative code, blocking calls serve as a natural form of back pressure that forces the caller to wait. In non-blocking code, it becomes important to control the rate of events so that a fast producer does not overwhelm its destination." (Spring Docs).

Voraussetzung ist eine reaktive Datenquelle, die als *Publisher* bei ihr registrierte *Subscriber* über neue oder veränderte Daten benachrichtigen kann, wie z.B. [MongoDB](#), [Redis](#) oder [Cassandra](#). JDBC ist nicht darauf ausgelegt. Reaktive Programmierung ist insbesondere für Anwendungsfälle sinnvoll, in denen kontinuierliche, unbegrenzte Datenströme verarbeitet und in einem User Interface dargestellt werden müssen. Die folgende Abbildung stellt Spring MVC und Spring WebFlux einander gegenüber.



Gegenüberstellung von Spring MVC und Spring WebFlux
Pivotal Software: [Spring Framework 5](https://spring.io/), <https://spring.io/>.

Spring WebFlux übernimmt die Begriffe *Flux* und *Mono* aus der Bibliothek [Reactor](#). Ein *Flux* ist ein generischer Datenstrom (*Stream*) der 0..N Elemente emittieren kann, ein *Mono* hingegen nur 0..1 Element. Die Klassen `reactor.core.publisher.Flux` und `reactor.core.publisher.Mono` implementieren das Interface `org.reactivestreams.Publisher`, das lediglich die Methode `void subscribe(Subscriber subscriber)` spezifiziert. Ein Nachrichtenempfänger (*Subscriber*) kann sich demzufolge bei einem *Flux*- oder einem *Mono*-Objekt registrieren, um kontinuierlich mit neuen Nachrichten versorgt zu werden. Spring ermöglicht es, *Flux*- oder *Mono*-Objekte einem Client über eine API zur Verfügung zu stellen, und erzeugt dazu eine *EventSource* gemäß des Protokolls [Server-Sent Events \(SSE\)](#). Das folgende Code-Beispiel verdeutlicht wie mittels *Spring WebFlux* ein endlicher Datenstrom (*Stream*) erzeugt wird, in dem verzögert bis zu einem bestimmten Zielwert hochgezählt wird. Der Datenstrom endet, wenn der Zielwert erreicht ist.

CounterController

```
1 import reactor.core.publisher.Flux;
2 // ...
3
4 @RestController
5 @RequestMapping("/count")
6 public class CounterController {
7
8     @GetMapping(path =("/{number}", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
9     @CrossOrigin
10    public Flux<Integer> countToNumber(@PathVariable("number") int number) {
11
12        Flux<Integer> numbers = Flux.create(sink -> {
13
14            Runnable counter = () -> {
15                for (int i = 1; i <= number; i++) {
16                    sink.next(i); // send number to client
17                    try {
18                        TimeUnit.SECONDS.sleep(1); // delay counting
19                    } catch (InterruptedException e) { }
20                }
21                sink.complete(); // counting is completed, stream ends
22            };
23
24            new Thread(counter).start();
25        });
26        return numbers;
27    }
28 }
```

counter.html

```
1 <html>
2   <head>
3     <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
4     <script>
5       $(document).ready(function() {
6         var source = new EventSource("http://localhost:8080/count/10");
7         source.onmessage = function(event) {
8           $("#count").append(event.data + " ... ");
9         }
10        source.onerror = function(event) {
11          source.close(); // client tries to reconnect if source is not closed
12          $("#count").append("finished! ");
13        }
14      });
15    </script>
16  </head>
17  <body>
18    <label id="count"></label>
19  </body>
20 </html>
```

- Die Rückgabe der API unter dem Pfad `/count/<n>` ist ein `Flux<Integer>` (Zeile 10). Der zurückgegebene MIME-Typ ist explizit angegeben als `text/event-stream` (Zeile 8).
- In Zeile 24 wird ein Thread gestartet, dessen `Runnable` eine For-Schleife von 1..n durchläuft (Zeilen 15-20). Je Iteration wird über die Senke des Flux eine Zahl emittiert (Zeile 16) und anschließend für 1 Sek. pausiert (Zeilen 17-19).
- In Zeile 21 wird der Datenstrom beendet. Er ist daher endlich.
- Der Client in `counter.html` baut eine Verbindung zur EventSource auf (Zeile 6). Die EventSource emittiert Nachrichten an den Client über die EventHandler-Funktion (Zeilen 7-9), die auf dem Attribut `onmessage` registriert ist.
- Wenn der Datenstrom serverseitig geschlossen wird oder in anderen Fehlerfällen, wird die EventSource geschlossen (Zeilen 10-13). Andernfalls würde der Client gemäß SSE-Protokoll versuchen die Verbindung neu aufzubauen.

Im nächsten Code-Beispiel soll die Quelle eines Datenstroms ein MongoDB-Datenbanksystem sein. Jedes Mal, wenn neue Dokumente in eine Collection innerhalb der MongoDB-Datenbank eingefügt werden, soll die serverseitige Komponente auf Basis von *Spring WebFlux* einen registrierten Client darüber benachrichtigen. Die Dokumente könnten z.B. Tweets sein, die aus einer unendlichen externen Quelle bezogen werden. Die Architektur entspricht damit durchgängig dem reaktiven Programmiermodell.

TweetController

```
1 import reactor.core.publisher.Flux;
2 // ...
3
4 @RestController
5 public class TweetController {
6
7     @Autowired
8     TweetRepository tweetRepository;
9
10    @GetMapping("/tweets") @CrossOrigin
11    public Flux<Tweet> getTweets() {
12        return tweetRepository.findTweetsBy(); // return infinite stream
13    }
14 }
```

TweetRepository

```
1 import org.springframework.data.mongodb.repository.ReactiveMongoRepository;
2 import org.springframework.data.mongodb.repository.Tailable;
3 // ...
4
5 public interface TweetRepository extends ReactiveMongoRepository<Tweet, String> {
6
7     @Tailable
8     Flux<Tweet> findTweetsBy(); // infinite stream as annotated by @Tailable
9 }
```

Tweet

```
1  import org.springframework.data.mongodb.core.mapping.Document;
2  // ...
3
4  @Document(collection = Tweet.COLLECTION_NAME) @Getter @Setter
5  public class Tweet {
6
7      public final static String COLLECTION_NAME = "tweet";
8
9      @Id
10     String id;
11     String text;
12     Date date;
13
14     public Tweet(String text) {
15         this.text = text;
16         this.date = new Date();
17     }
18 }
```

TweetStarter

```
1  @SpringBootApplication
2  public class TweetStarter implements CommandLineRunner {
3
4      @Autowired
5      TweetRepository tweetRepository;
6
7      @Autowired
8      ReactiveMongoTemplate reactiveMongoTemplate;
9
10     public static void main(String[] args) {
11         SpringApplication.run(TweetStarter.class, args);
12     }
13
14     @Override
15     public void run(String... args) throws Exception {
16
17         // create a capped collection which supports tailable cursors, limited to 5 documents here
18         reactiveMongoTemplate.dropCollection(Tweet.COLLECTION_NAME).block();
19         reactiveMongoTemplate.createCollection(Tweet.COLLECTION_NAME,
20             CollectionOptions.empty().capped().size(4096).maxDocuments(5)).block();
21
22         // prepare HTTP request for new lorem ipsum tweet
23         HttpClient client = HttpClient.newHttpClient();
24         String loremIpsumApi = "https://baconipsum.com/api/?type=all-meat&sentences=1&start-with-lorem=1&format=text";
25         HttpRequest request = HttpRequest.newBuilder().uri(URI.create(loremIpsumApi)).build();
26
27         // post a new tweet every 3 seconds
28         Runnable changePrices = () -> {
29             try {
30                 HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
31                 Tweet tweet = new Tweet(response.body());
32                 tweetRepository.save(tweet).block();
33             } catch (InterruptedException | IOException e) { }
34         };
35         ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
36         scheduler.scheduleAtFixedRate(changePrices, 0, 3, TimeUnit.SECONDS);
37     }
38 }
```

```

1 <html>
2   <head>
3     <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
4     <script>
5       $(document).ready(function() {
6         var source = new EventSource("http://localhost:8080/tweets");
7         source.onmessage = function(event) {
8           var tweet = JSON.parse(event.data);
9           var time = new Date(tweet.date).toString().substring(0, 8);
10          $("#tweet-list").append("<li>" + time + " <b>" + tweet.text + "</b></li>");
11        }
12        source.onerror = function(event) {
13          source.close();
14          console.log("Event source closed.")
15        }
16      });
17    </script>
18  </head>
19  <body>
20    <ul id="tweet-list"></ul>
21  </body>
22 </html>

```

- **TweetController:** Als Teil der API wird ein Datenstrom unter dem Pfad `/tweets` registriert (Zeilen 10-13). Es wird eine Methode aus dem zugehörigen `TweetRepository` aufgerufen, um auf die Datenbank zuzugreifen.
- **TweetRepository:** Das Interface erweitert `ReactiveMongoRepository<Tweet, String>` und ist damit ein spezielles reaktives Repository. JDBC-Datenquellen können nicht als reaktives Repository eingesetzt werden. Bei einer Anfrage an die Datenbank gibt die Java-API von MongoDB einen Cursor vom Typ `com.mongodb.DBCursor` zurück, über den ähnlich einem `java.sql.ResultSet` iteriert werden kann, um das Ergebnis der Anfrage zu verarbeiten. Per Default schließt MongoDB diesen Cursor, wenn über alle seine Objekte iteriert worden ist. Damit würde aber auch der Stream beendet. MongoDB erlaubt es für sogenannte *Capped Collections* einen *Tailable Cursor* zu verwenden, der geöffnet bleibt, nachdem alle ursprünglich zurückgegebenen Objekte verarbeitet worden sind. Die Methoden eines `ReactiveMongoRepository` aus *Spring Data MongoDB* können mit der Annotation `@Tailable` versehen werden, damit sie einen derartigen nicht abgeschlossenen, unendlichen Datenstrom zurückgeben, d.h. das zurückgegebene `Flux`-Objekt ist noch nicht beendet. Nur in diesem Fall wird über die API eine SSE-EventSource veröffentlicht. Die Bezeichnung der Annotation `@Tailable` ist übrigens an den Linux-Befehl `tail -f` angelehnt.
- **TweetStarter:** In der überschriebenen Methode `run` wird zuerst die verwendete Collection in eine auf max. 5 Dokumente begrenzte *Capped Collection* gewandelt (Zeilen 18-20). Anschließend wird ein HTTP-Request vorbereitet (Zeilen 23-25), der in einer Endlosschleife alle 3 Sekunden ausgeführt wird (Zeile 36), um den Text für einen *fleischlastigen Lorem Ipsum-Tweet* anzufragen (Zeile 30) und diesen in der *Capped Collection* zu speichern (Zeilen 31-32).
- **tweets.html:** Der Client in `tweets.html` baut eine Verbindung zur EventSource auf (Zeile 6). Die EventSource emittiert neue Tweets an den Client über die EventHandler-Funktion (Zeilen 7-11), innerhalb derer die Tweets dem DOM hinzugefügt werden.

Die gezeigten Code-Beispiele zum Umgang mit reaktiven Streams in Spring finden sich im Verzeichnis `/remote/spring-webflux` des Modul-Repository.

Sockets

Eine sehr grundlegende Methode zur bidirektionalen Interprozesskommunikation zwischen verteilten Software-Komponenten ist die Verwendung von *Internet-Sockets*. Ein Internet-Socket ist ein interner Endpunkt aus Sicht einer Anwendung zum Senden und Empfangen von Daten in einem Netzwerk auf Basis des Internet-Protokolls (IP). Der Begriff *Socket* selbst geht ursprünglich auf den [RFC 147](#) von 1971 zurück, der sich noch auf das ARPA-Netzwerk bezieht. Ein Socket wird einer Anwendung vom Betriebssystem auf Anforderung unter einem bestimmten Port bereitgestellt. Das Betriebssystem verwaltet die aktuell aktiven, lokalen Sockets und die zugehörigen Remote-Sockets (IP-Adressen und Ports). Internet-Sockets lassen sich in *Stream Sockets* (TCP) und *Datagram Sockets* (UDP) unterscheiden. Im Folgenden soll die [Java-API für Stream Sockets](#) vorgestellt werden, die im Package [java.net](#) implementiert und damit Bestandteil des Moduls `java.base` ist.

Das Code-Beispiel zeigt eine einfache Chat-Anwendung ähnlich des Beispiels im Kapitel [WebSockets](#).

ChatServer

```
1  import java.net.ServerSocket;
2  import java.net.Socket;
3  import java.io.BufferedReader;
4  import java.io.PrintWriter;
5  // ...
6
7  public class ChatServer {
8
9      static Set<PrintWriter> clientWriters = new HashSet<>();
10
11     public static void main(String[] args) {
12         try {
13             ServerSocket serverSocket = new ServerSocket(5000);
14
15             while (true) {
16                 // wait for new client connection
17                 Socket clientSocket = serverSocket.accept();
18                 System.out.println(">>> Client connected from " + clientSocket.getRemoteSocketAddress());
19
20                 // create output writer
21                 PrintWriter writer = new PrintWriter(clientSocket.getOutputStream());
22                 clientWriters.add(writer);
23
24                 // create input reader
25                 BufferedReader reader = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
26
27                 // start concurrent message handler
28                 new Thread(new BroadcastMessageHandler(reader)).start();
29             }
30         } catch (Exception e) {
31             e.printStackTrace();
32         }
33     }
34
35     static void broadcast(String message) {
36         for (PrintWriter writer : clientWriters) {
37             writer.println(message);
38             writer.flush();
39         }
40     }
41 }
```

BroadcastMessageHandler

```
1  public class BroadcastMessageHandler extends MessageHandler {
2
3      public BroadcastMessageHandler(BufferedReader reader) {
4          super(reader);
5      }
6
7      @Override
8      public void onMessage(String message) {
9          super.onMessage(message);
10         ChatServer.broadcast(message);
11     }
12 }
```


MessageHandler

```
1 public class MessageHandler implements Runnable {
2
3     BufferedReader reader;
4
5     public MessageHandler(BufferedReader reader) {
6         this.reader = reader;
7     }
8
9     @Override
10    public void run() {
11        String message;
12        try {
13            while ((message = reader.readLine()) != null) {
14                onMessage(message);
15            }
16        } catch (IOException e) {
17            e.printStackTrace();
18        }
19    }
20
21    public void onMessage(String message) {
22        System.out.println(">>> Received: '" + message + "'");
23    }
24 }
```

ChatClient

```
1 public class ChatClient {
2
3     static Socket socket;
4     static PrintWriter writer;
5
6     public static void main(String[] args) {
7         try {
8             // connect to server
9             socket = new Socket("127.0.0.1", 5000);
10            System.out.println(">>> Client connected using port " + socket.getLocalPort());
11
12            // create output writer
13            writer = new PrintWriter(socket.getOutputStream());
14
15            // create input reader
16            BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
17
18            // start concurrent message handler
19            new Thread(new MessageHandler(reader)).start();
20
21        } catch (IOException e) {
22            e.printStackTrace();
23        }
24
25        // Wait for messages to send to the server
26        while (true) {
27            System.out.println(">>> Write your message:");
28            Scanner scanner = new Scanner(System.in);
29            writer.println(scanner.next());
30            writer.flush();
31        }
32    }
33 }
```

- **ChatServer:** Das serverseitige Socket wird in Zeile 13 unter Port 5000 geöffnet. In der folgenden Endlosschleife wartet der Server auf Clients, die eine Verbindung aufbauen möchten (Zeile 17). Für jeden neuen Client wird ein **PrintWriter** (Zeile 21) und ein **BufferedReader** (Zeile 25) erzeugt, um neue Nachrichten über einen **OutputStream** zu senden bzw. über einen **InputStream** zu empfangen. Anschließend wird ein **BroadcastMessageHandler** erzeugt und an einen nebenläufigen Thread übergeben (Zeile 28).
- **BroadcastMessageHandler:** Der **BroadcastMessageHandler** ist ein spezieller **MessageHandler**, der seine Oberklasse nur dahingehend erweitert, dass er beim Empfang einer Nachricht, die Methode **ChatServer.broadcast** aufruft. In dieser Methode wird eine eingehende Nachricht an alle Clients über deren jeweiligen **PrintWriter** verteilt.
- **MessageHandler:** Der allgemeinere **MessageHandler** implementiert das Interface **Runnable** (s. Kapitel [Threads in Java](#)). In der Methode **run** wird in Zeile 13 in einer Endlosschleife auf eingehende Nachrichten gewartet. Diese werden in der Methode **onMessage** verarbeitet (Zeilen 21-23).

- **ChatClient:** Der Client versucht sich in Zeile 9 mit dem serverseitigen Socket zu verbinden. Im Erfolgsfall wird ein clientseitiges Socket zurückgegeben, für das das Betriebssystem einen freien Port auswählt (Zeile 10). Wie beim Server werden anschließend ein `PrintWriter` und ein `BufferedReader` erzeugt sowie ein `MessageHandler` in einem nebenläufigen Thread gestartet. Ab Zeile 26 beginnt eine Endlosschleife, in der auf Eingaben des Anwenders gewartet wird, die als Nachrichten an den Server versendet werden (Zeilen 29-30).

Über einen `ObjectOutputStream` können auch Objekte versendet werden, die das `Serializable`-Interface unterstützen. Sender und Empfänger müssen die entsprechenden Klassen zu den versendeten Objekten untereinander austauschen. Der Empfänger kann über `instanceof` ermitteln, von welchem Typ das empfangene Objekt ist.

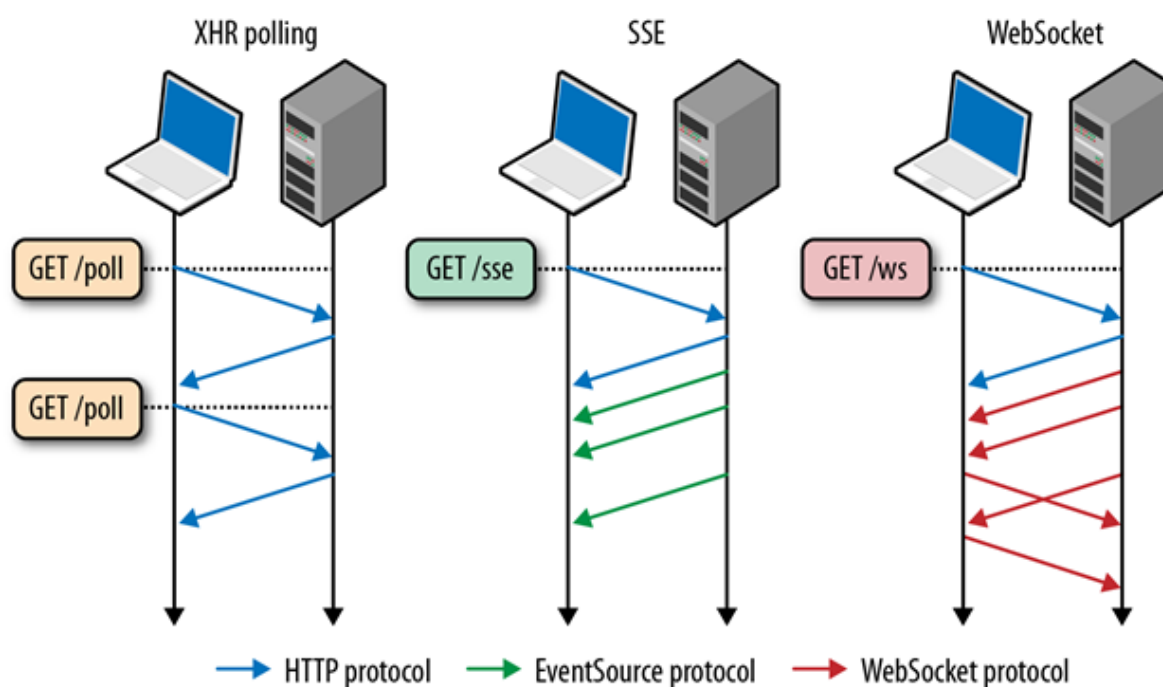
```
1 | Object object = inputStream.readObject();
2 | if(object instanceof Message) {
3 |     Message message = (Message) object;
4 | }
```

Die gezeigten Code-Beispiele zu Sockets finden sich im Verzeichnis `/remote/socket` des Modul-Repository.

WebSockets

Das WebSocket-Protokoll ist ein Kommunikationsprotokoll, das wie HTTP auf einer TCP-Verbindung zwischen Client und Server aufbaut. Allerdings ist es im Gegensatz zu HTTP auf eine bidirektionale, [full-duplex](#) Kommunikation ausgelegt, d.h. sowohl der Client kann dem Server Nachrichten senden als auch gleichzeitig der Server dem Client. Das WebSocket-Protokoll basiert auf wechselseitigem *Publish-Subscribe* zwischen Client und Server (s. Kapitel [Entwurfsmuster Observer](#)) und ist dabei auf Web-Anwendungen ausgerichtet. Ein Client, der häufig in JavaScript oder TypeScript implementiert ist, baut eine Verbindung zum Server auf und registriert einen *EventHandler*, um kontinuierlich Nachrichten vom Server zu empfangen. Gelegentlich sendet aber auch der Client seinerseits Nachrichten an den Server.

Die folgende Abbildung veranschaulicht den Unterschied zwischen den Protokollen WebSocket, HTTP (mittels [Fetch API](#) oder [XHR](#)) und [Server-Sent Events \(SSE\)](#). Letzteres Protokoll ist auf unidirektionale Kommunikation ausgelegt, wobei der Client ausschließlich Nachrichten vom Server empfängt, aber seinerseits keine Nachrichten senden kann.



Ablauf der Kommunikation via HTTP/XHR, SSE und WebSocket
[Ilya Grigorik: High Performance Browser Networking, O'Reilly, 2013.](#)

Das WebSocket-Protokoll wurde 2008 als Teil der HTML5-Spezifikation durch die [WHATWG](#) und das [W3C](#) eingeführt. Die Weiterentwicklung als eigenständiges, von HTTP unabhängiges Protokoll wurde 2010 in den [RFC 6455](#) verlagert. Google Chrome war in 2009 der erste Browser der WebSockets unterstützte. Heute unterstützen [alle gängigen Browser](#) das grundlegende WebSocket-Interface. Es folgt ein Ausschnitt der wichtigsten Attribute und Methoden des WebSocket-Interface, das sehr überschaubar ausfällt.

```
1 interface WebSocket : EventTarget {
2     void send(USVString data); // client sends messages to server
3     void send(Blob data);
4     void close(optional unsigned short code, optional USVString reason);
5
6     attribute EventHandler onmessage; // client receives messages from server
7     attribute EventHandler onopen;
8     attribute EventHandler onerror;
9     attribute EventHandler onclose;
10
11     // ...
12 }
```

Das folgende Code-Beispiel demonstriert wie das WebSocket-Interface prinzipiell in JavaScript einzusetzen ist.

```
1 var websocket = new WebSocket('ws://example.com/websocket/endpoint'); // connect to websocket server endpoint
2
3 websocket.send('Hello World!'); // send message to server
4
5 websocket.onmessage = function(event) { // register listener for messages from server
6     console.log(event.data);
7 };
```

Ein typisches Anwendungsbeispiel für WebSockets ist ein Chat für mehrere Anwender. Die Funktionsweise von WebSockets soll im Folgenden anhand eines sehr einfachen Chats demonstriert werden. Es wird dazu eine Client-Komponente in JavaScript und eine Server-Komponente in Java implementiert.

client.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
5    <script src="https://cdnjs.cloudflare.com/ajax/libs/moment.js/2.24.0/moment.min.js"></script>
6    <script>
7      $(function() {
8        var websocket = new WebSocket('ws://localhost:8025/chat');
9
10       $('#sendMessage').click(function () {
11         var json = {user: $('#user').val(), text: $('#text').val()};
12         websocket.send(JSON.stringify(json));
13         $('#text').val('');
14       });
15
16       websocket.onopen = function(event) { console.log("WebSocket connection is open now."); };
17
18       websocket.onmessage = function(event) {
19         var res = JSON.parse(event.data);
20         if (sessions.indexOf(res.sessionId) < 0) { sessions.push(res.sessionId); }
21
22         var user = res.user.length > 0 ? res.user : 'Anonym';
23         var time = moment.unix(res.time / 1000).fromNow();
24         var newMessage =
25           '<div class="card">' +
26             '<span class="user">' + user + '</span>' +
27             '<span class="time">' + time + '</span>' +
28             '<hr>' + res.text +
29             '</div>';
30         $('#messages').append(newMessage);
31       };
32
33       websocket.onerror = function(event) { console.log(event); };
34     });
35   </script>
36 </head>
37 <body>
38   <h3>MyChat</h3>
39   <input id="user" type="text">
40   <textarea id="text" type="text" rows="2"></textarea>
41   <button id="sendMessage">Senden</button>
42   <div id="messages"></div>
43 </body>
44 </html>
```

ChatEndpoint

```
1  import javax.websocket.server.ServerEndpoint;
2  import javax.websocket.OnMessage;
3  import javax.websocket.Session;
4  import com.fasterxml.jackson.databind.ObjectMapper;
5  // ...
6
7  @ServerEndpoint("/chat")
8  public class ChatEndpoint {
9
10     private final static ObjectMapper mapper = new ObjectMapper();
11
12     @OnMessage
13     public void onMessage(String inputJson, Session s) {
14         try {
15             // add session id to message
16             ChatMessage message = mapper.readValue(inputJson, ChatMessage.class);
17             message.sessionId = s.getId();
18             String outputJson = mapper.writeValueAsString(message);
19
20             // broadcast message
21             for (Session session : s.getOpenSessions()) {
22                 session.getBasicRemote().sendText(outputJson);
23             }
24         } catch (IOException e) {
25             e.printStackTrace();
26         }
27     }
28
29     @OnOpen
30     public void onOpen(Session s) {
31         System.out.println("Client connected: " + s.getId());
32     }
33
34     @OnClose
35     public void onClose(Session s) {
36         System.out.println("Connection closed: " + s.getId());
37     }
38 }
```

ChatStarter

```
1  import org.glassfish.tyrus.server.Server;
2  // ...
3
4  public class ChatStarter {
5
6      public static void main(String[] args) throws IOException, DeploymentException {
7          Server server = new Server("localhost", 8025, "/", null, ChatEndpoint.class);
8          server.start();
9
10         System.out.println("Hit enter to stop server.");
11         System.in.read();
12         server.stop();
13     }
14 }
```

ChatMessage

```
1  public class ChatMessage {
2      String user;
3      Date time = new Date();
4      String text;
5      String sessionId;
6  }
```

In Java wird die API für WebSockets durch [JSR 356 \(Java API for WebSocket\)](#) spezifiziert. Die Referenzimplementierung in Java heißt [Tyrus](#). Dementsprechend ergibt sich eine Abhängigkeit zur Bibliothek [Tyrus Grizzly Server Container](#), um einen WebSocket-Endpoint mittels Tyrus im Kontext eines [Grizzly-Servers](#) zu starten.

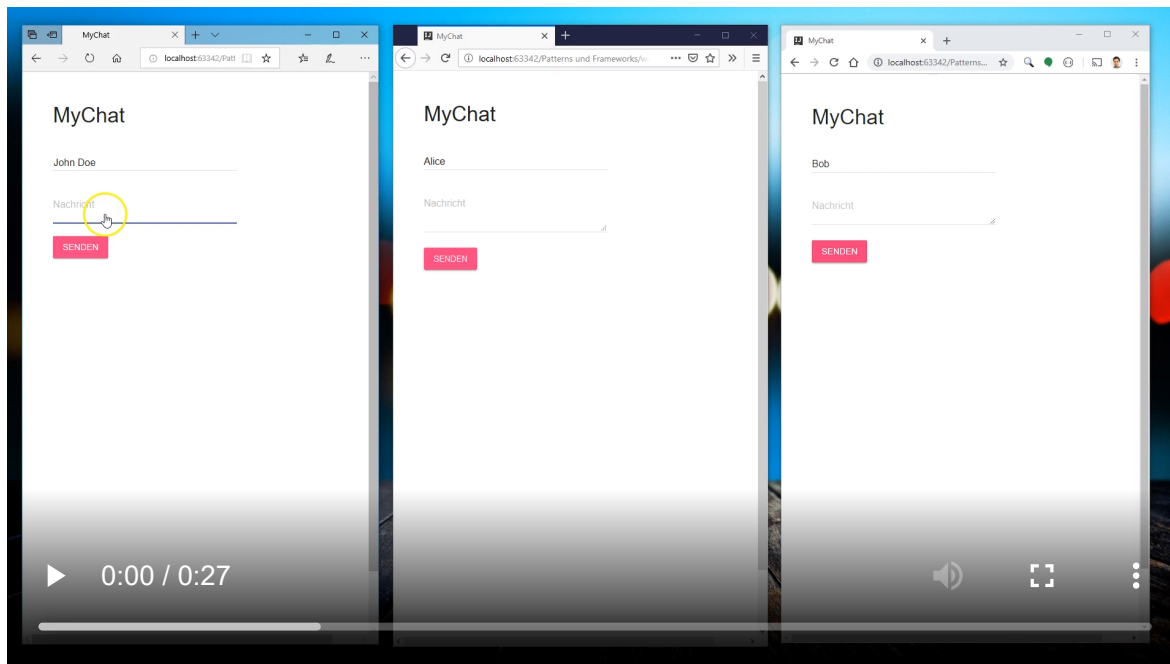
- **ChatEndpoint** und **ChatStarter**: In Zeile 7 der Klasse **ChatStarter** wird der Server auf Port 8025 gestartet und die Klasse **ChatEndpoint** als Endpoint übergeben. Der Endpoint ist unter dem Pfad **/chat** erreichbar (s. Annotation **@ServerEndpoint** in Zeile 6 der Klasse **ChatEndpoint**). Über die Annotationen **@OnMessage**, **@OnOpen** und **@OnClose** können entsprechende EventHandler-Methoden registriert werden. In der Implementierung der Methode **onMessage** wird die eingehende JSON-Nachricht mittels [Jackson](#) in ein **ChatMessage**-Objekt transformiert (Zeile 15), dem anschließend die eindeutige Session-Id des

Absenders hinzugefügt wird (Zeile 16). Anschließend wird die `ChatMessage` im JSON-Format an alle offenen Sessions versendet (Zeilen 20-22).

- **client.html:** Der Client ist in einer einzelnen HTML-Datei implementiert. Diese HTML-Datei enthält insbesondere eingebetteten JavaScript-Code (Zeilen 6-35 in `client.html`), über den mit dem serverseitigen WebSocket-Endpunkt kommuniziert wird. In Zeile 8 wird die Verbindung zum Server aufgebaut (`var websocket = new WebSocket(url)`) und serverseitig eine Session für den Client initialisiert. Das erfolgreiche Öffnen dieser bidirektionalen Kommunikationsverbindung wird dem Client durch den Server bestätigt (Zeile 16). Auch im Fehlerfall wird der Client informiert (Zeile 33). In Zeile 18 wird über das Attribut `websocket.onMessage` eine Funktion als EventHandler zur Verarbeitung eingehender Nachrichten registriert. In dieser Funktion wird für jede eingehende Nachricht ein neues HTML-Element konstruiert (Zeilen 22-29) und dem aktuellen [Document Object Model \(DOM\)](#) hinzugefügt (Zeile 30).

Die gezeigten Code-Beispiele zu WebSockets finden sich im Verzeichnis `/remote/websocket` des Modul-Repository.

Das folgende Video zeigt die gerenderte HTML-Seite des Chats in drei Browsern. Es werden entsprechend drei Anwender simuliert, die sich untereinander Nachrichten senden. Es muss aufgrund der Verwendung des WebSocket nicht mittels *Polling* beim Server erfragt werden, ob es neue Nachrichten gibt. Stattdessen werden neue Nachrichten, sobald sie dem Server zugestellt werden, als Broadcast an alle aktiven Clients verteilt.



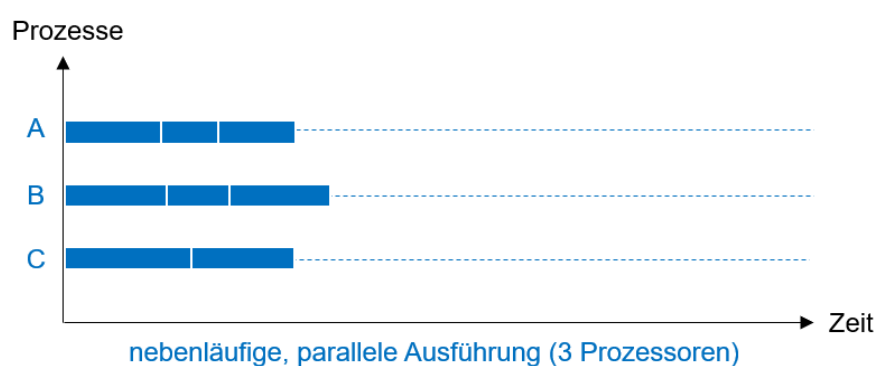
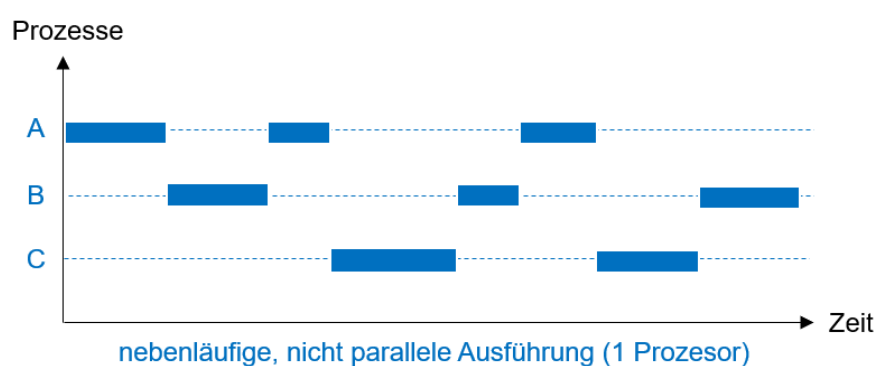
Chat-Anwendung zur Demonstration eines bidirektionalen WebSocket

Threads in Java

Parallelität und Nebenläufigkeit

Die parallele Verarbeitung von Prozessen setzt voraus, dass mehrere Prozessoren (oder Prozessorkerne) zur Verfügung stehen, da jeder Prozessor nur einen Prozess gleichzeitig ausführen kann. Ob die Prozessoren in einem einzigen Computer verbaut sind oder sich auf mehrere Computer verteilen, ist dabei nicht relevant. In der Anwendungsentwicklung hat sich zusätzlich der Begriff Nebenläufigkeit (engl. *Concurrency*) etabliert, der potentielle Parallelität beschreibt. Während der Anwendungsentwicklung ist häufig nicht bekannt, auf welcher Hardware die Komponenten der Anwendung später ausgeführt werden. Das umfasst auch die Kenntnis darüber, wie viele Prozessoren tatsächlich zur Verfügung stehen. Demzufolge wird eine Anwendung i.d.R. nur auf potentielle Parallelität ausgelegt. Die Anwendung könnte auch auf einem Computer mit nur einem Prozessor ausgeführt werden. Dieser Prozessor wird dann mehrere Prozesse verzahnt ausführen. Das Betriebssystem steuert die Verschränkung (engl. *Interleaving*) der nebenläufigen Prozesse. Prozesse mit höherer Priorität bekommen mehr Prozessorzeit zugewiesen. Auch in dem verbreiteten Lehrbuch "Principles of Concurrent and Distributed Programming" von M. Ben-Ari werden die Begriffe Parallelität und Nebenläufigkeit wie folgt abgegrenzt [\[Ben05\]](#).

*"Parallel System: A system in which the execution of several programs overlap in time by running on separate processors.
Concurrent Program: A concurrent program is a set of processes that can be executed in parallel. Thus, a concurrent program does not necessarily overlap its computation, but may do so." (M. Ben-Ari)*

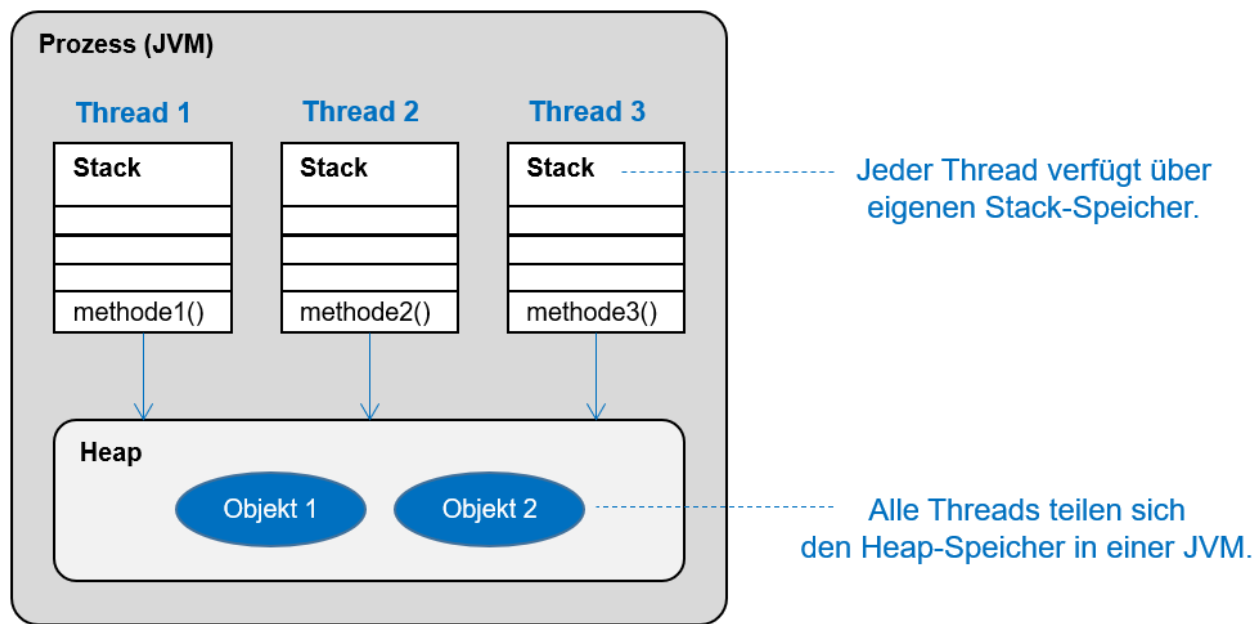


Parallelität und Nebenläufigkeit von Prozessen

Prozesse und Threads

Ein Prozess bezeichnet allgemein den gerichteten Ablauf eines Geschehens. In der Softwareentwicklung ist das ablaufende Geschehen die Ausführung eines programmierten Kontrollflusses. Aus Sicht des Betriebssystems ist ein Prozess ein Container für mehrere zusammengehörige *Threads* einer Anwendung. Ein Thread ist ein einzelner nebenläufiger Ausführungsfaden innerhalb des Prozesses einer Anwendung. Threads sind demzufolge leichtgewichtige Teilprozesse und effizienter in der Erzeugung. Das Betriebssystem führt jeden Prozess unter einem bestimmten Betriebssystem-User mit ggf. eingeschränkten Rechten aus.

Beim Start eines Prozesses weist das Betriebssystem ihm einen Adressraum (= Teil des Hauptspeichers) zu, auf dem der Prozess exklusiv arbeiten kann. Die Threads innerhalb eines Prozesses teilen sich gemeinsam diesen Adressraum. Eine Java-Anwendung wird in einer *Java Virtual Machine (JVM)* als Laufzeitumgebung ausgeführt, die den Prozess aus Sicht des Betriebssystems darstellt. Der JVM wird ein begrenzter Adressraum für alle Objekte, die über Konstruktoraufrufe erzeugt werden, zur Verfügung gestellt. Diesen Adressraum bezeichnen wir in Java als *Heap*. Falls kein Heap-Speicher mehr für neue Objekte allokiert werden kann, wird ein [OutOfMemoryError](#) von der JVM geworfen. In Java wird Heap-Speicher, der durch Objekte belegt ist, die nicht mehr über Referenzen aus dem Kontrollfluss erreicht werden können, automatisch durch einen *Garbage Collection Thread* freigegeben. Während sich die Threads einer Anwendung also den Heap teilen, hat jeder Thread einen eigenen *Stack*, um seinen individuellen Kontrollfluss zu verwalten. Die folgende Abbildung soll diese Beziehung zwischen mehreren Threads und ihrem gemeinsamen Heap in einer JVM verdeutlichen.



Heap und Stack in einer JVM

Ausführen nebenläufiger Threads

In Java kann jede Klasse, die das funktionale Interface `java.lang.Runnable` implementiert als nebenläufiger Thread ausgeführt werden. Der Kontrollfluss des Threads beginnt in der Methode `run` des `Runnable` (Zeile 4-7 im folgenden Code-Beispiel). Um ein `Runnable` zu starten, kann ein `Thread`-Objekt erzeugt werden, dem das `Runnable` übergeben wird (Zeile 10). Über die Methode `start` kann der Thread gestartet werden (Zeile 11). Dadurch wird die Methode `run` implizit nebenläufig ausgeführt. Später werden wir weitere Möglichkeiten kennenlernen, um ein `Runnable` auszuführen.

```

1  class MyRunnable implements Runnable {
2
3      @Override
4      public void run() {
5          String message = "Hello from " + Thread.currentThread().getName(); // output ist "Hello from Thread-0"
6          System.out.println(message);
7      }
8
9      public static void main(String[] args) {
10         Thread thread = new Thread(new MyRunnable());
11         thread.start();
12         System.out.println("Hello from " + Thread.currentThread().getName()); // output ist "Hello from main"
13     }
14 }

```

Die Klasse `Thread` implementiert selbst das Interface `Runnable`. Es könnte daher auch einfach die Klasse `Thread` wie folgt erweitert werden, wenn dadurch die fachliche Vererbungshierarchie nicht gestört wird.

```

1  class MyThread extends Thread {
2
3      @Override
4      public void run() {
5          String message = "Hello from " + getName(); // output ist "Hello from Thread-0"
6          System.out.println(message);
7      }
8
9      public static void main(String[] args) {
10         Thread thread = new MyThread();
11         thread.start();
12         System.out.println("Hello from " + Thread.currentThread().getName()); // output ist "Hello from main"
13     }
14 }

```

Da `Runnable` ein funktionales Interface ist, kann es auch in einem Lambda-Ausdruck verwendet werden.

```

1  Runnable runnable = () -> System.out.println("Hello from concurrent thread");
2  Thread thread = new Thread(runnable);
3  thread.start();

```

Im folgenden Code-Beispiel werden in der Klasse `MyLoopThread` 3 Threads gestartet, die jeweils die Zahlen von 1 bis 100 hochzählen und ausgeben. Anhand der Ausgaben lässt sich das zufällige *Thread-Interleaving* beobachten. Es ist nicht vorhersehbar, welcher Thread zuerst fertig wird. In der Klasse `MyLoopSleepThread` wird das vorherige Beispiel nur darum erweitert, dass jeder Thread nach der Ausgabe einer Zahl für 10 ms wartet (Zeilen 9-11). Durch diese bewusste Verzögerung zählen zwar alle Threads mit hoher Wahrscheinlichkeit im Takt, aber nicht garantiert. Die Korrektheit einer Anwendung sollte nie von einem derartigen Aufruf der Methode `sleep` abhängen.

MyLoopThread

```
1 class MyLoopThread extends Thread {
2
3     MyLoopThread(String name) { super(name); }
4
5     @Override
6     public void run() {
7         IntStream.range(0, 100).forEach(i -> System.out.println(getName() + "\t" + i));
8     }
9
10    public static void main(String[] args) {
11        for (int i = 0; i < 3; i++) {
12            Thread thread = new MyLoopThread("Loop" + i);
13            thread.start();
14        }
15    }
16 }
```

MyLoopSleepThread

```
1 class MyLoopSleepThread extends Thread {
2
3     MyLoopSleepThread(String name) { super(name); }
4
5     @Override
6     public void run() {
7         IntStream.range(0, 100).forEach(i -> {
8             System.out.println(getName() + "\t" + i);
9             try {
10                 sleep(10); // sleep for 10 ms
11             } catch (InterruptedException e) {}
12         });
13     }
14
15    public static void main(String[] args) {
16        for (int i = 0; i < 3; i++) {
17            Thread thread = new MyLoopSleepThread("Loop" + i);
18            thread.start();
19        }
20    }
21 }
```

Ein `Thread` wird beendet, wenn er das Ende der Methode `run` des ausgeführten `Runnable` erreicht. Es ist durchaus üblich, dass Threads dauerhaft auf bestimmte Ereignisse warten und diese verarbeiten, bis eine Abbruchbedingung erfüllt wird. Das erste der beiden folgenden Code-Beispiele zeigt, wie ein derartiges sanftes, kontrolliertes Beenden eines Threads am Ende einer Iteration herbeigeführt werden kann. Das zweite Code-Beispiel zeigt dagegen, dass ein Thread auch abrupt mit sofortiger Wirkung von außen unterbrochen werden kann, indem die Methode `interrupt` auf dem `Thread`-Objekt ausgeführt wird. Die aktuelle Iteration wird hierbei nicht mehr vollständig ausgeführt, sondern unmittelbar abgebrochen. Methoden wie z.B. `sleep`, die einen Thread in einen wartenden Zustand versetzen, werfen i.d.R. eine [InterruptedException](#), falls sie von einem anderen Thread von außen, durch Aufruf der Methode `interrupt`, unterbrochen worden sind.

Thread über Bedingung beenden

```
1  class MyCancelledThread extends Thread {
2
3      private boolean keepRunning = true;
4
5      public void cancel() {
6          this.keepRunning = false;
7      }
8
9      @Override
10     public void run() {
11         while (keepRunning) {
12             // do something meaningful
13         }
14     }
15
16     public static void main(String[] args) throws InterruptedException {
17         MyCancelledThread thread = new MyCancelledThread();
18         thread.start();
19
20         // after a while
21         thread.cancel(); // call custom method to cancel the thread
22     }
23 }
```

Thread-Interruption

```
1  class MyInterruptedThread extends Thread {
2
3      @Override
4      public void run() {
5          while (!isInterrupted()) {
6              // do something meaningful
7          }
8      }
9
10     public static void main(String[] args) throws InterruptedException {
11         MyInterruptedThread thread = new MyInterruptedThread();
12         thread.start();
13
14         // after a while
15         thread.interrupt(); // thread interruption
16     }
17 }
```

In Java können Threads eine Priorität zwischen 1 und 10 zugewiesen bekommen. Einem Thread wird je nach Priorität mehr oder weniger CPU-Zeit zugestanden. Das folgende Code-Beispiel führt 10 Threads mit jeweils unterschiedlicher Priorität aus (Zeilen 14-18). Jeder Thread belastet die CPU, indem er eine Variable inkrementiert (Zeilen 7-8). Nach 10 s werden alle Threads abgebrochen (Zeilen 20+33). Die Ausgabe in den Zeilen 23-31 zeigt exemplarisch, wie viel CPU-Zeit den Threads zugestanden worden ist. Bei jeder Ausführung wird die Ausgabe sich leicht unterscheiden. Es ist zu berücksichtigen, dass auf dem Betriebssystem weitere Prozesse ausgeführt werden, sodass die theoretisch verfügbar CPU-Zeit nicht allein auf die Java-Threads verteilt werden kann. Die Ausgabe könnte auf einem Computer mit 4 Prozessorkernen wie folgt aussehen:

```
CPU time of Thread-0: 187 ms
CPU time of Thread-1: 296 ms
CPU time of Thread-2: 750 ms
CPU time of Thread-3: 1078 ms
CPU time of Thread-4: 2203 ms
CPU time of Thread-5: 2593 ms
CPU time of Thread-6: 4875 ms
CPU time of Thread-7: 5046 ms
CPU time of Thread-8: 6765 ms
CPU time of Thread-9: 6859 ms
Total CPU time: 30652 ms
Available processors: 4
```

```

1  class MyPriorityThread extends Thread {
2
3      MyPriorityThread(int priority) { setPriority(priority); }
4
5      @Override
6      public void run() {
7          BigInteger i = BigInteger.ZERO;
8          while (!isInterrupted()) i = i.add(BigInteger.ONE); // do something to utilize CPU
9      }
10
11     public static void main(String[] args) throws InterruptedException {
12
13         // create threads with different priorities
14         List<Thread> threads = new ArrayList<>();
15         for (int i = 1; i <= 10; i++) {
16             threads.add(new MyPriorityThread(i));
17         }
18         threads.forEach(t -> t.start()); // start all threads
19
20         sleep(10_000); // threads utilize CPU for 10 s
21
22         // query CPU time for each thread
23         ThreadMXBean tmxb = ManagementFactory.getThreadMXBean();
24         long totalCpuTime = 0;
25         for (Thread t : threads) {
26             long cpuTime = tmxb.getThreadCpuTime(t.getId()) / 1_000_000;
27             System.out.println("CPU time of " + t.getName() + ": " + cpuTime + " ms");
28             totalCpuTime += cpuTime;
29         }
30         System.out.println("Total CPU time: " + totalCpuTime + " ms");
31         System.out.println("Available processors: " + Runtime.getRuntime().availableProcessors());
32
33         threads.forEach(t -> t.interrupt()); // interrupt threads
34     }
35 }

```

Scheduling von Threads

Es ergibt sich häufig die Anforderung, dass ein Runnable nicht sofort, sondern zu einem bestimmten Zeitpunkt in der Zukunft ausgeführt werden soll, oder dass ein Runnable periodisch in bestimmten Intervallen wiederholt werden soll. Eine derartige zeitliche Ablaufsteuerung von Prozessen bezeichnen wir allgemein als *Scheduling*. Im Folgenden werden zwei Varianten für das Scheduling von Runnables in Java gezeigt. In beiden Code-Beispielen entsteht bei Ausführung die folgende Ausgabe:

```

Timed event 1    12:01:02.689008600
Timed event 2    12:01:03.673393100
Timed event 3    12:01:04.673484800
Timed event 4    12:01:05.673454700
Timed event 5    12:01:06.673461800

```

TimerTask

```

1  import java.util.Timer;
2  import java.util.TimerTask;
3
4  class MyTimerTask extends TimerTask {
5
6      int i = 0;
7
8      @Override
9      public void run() {
10         System.out.println("Timed event " + (++i) + "\t" + LocalTime.now());
11     }
12
13     public static void main(String[] args) throws InterruptedException {
14         Timer timer = new Timer();
15         timer.scheduleAtFixedRate(new MyTimerTask(), 500, 1_000); // schedule timed task
16
17         Thread.sleep(5_000);
18         timer.cancel(); // cancel timer after 5 s
19     }
20 }

```

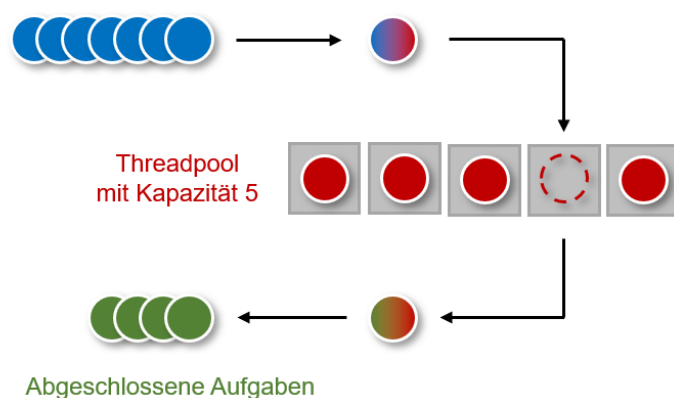
ScheduledExecutorService

```
1 import java.util.concurrent.ScheduledExecutorService;
2 import java.util.concurrent.Executors;
3 // ...
4
5 class MyScheduledExecutorService {
6
7     public static void main(String[] args) throws InterruptedException {
8         AtomicInteger i = new AtomicInteger(0);
9         Runnable runnable = () -> System.out.println("Timed event " + (i.incrementAndGet()) + "\t" + LocalTime.now());
10
11         ScheduledExecutorService scheduler = new ScheduledThreadPoolExecutor(1);
12         scheduler.scheduleAtFixedRate(runnable, 500, 1_000, TimeUnit.MILLISECONDS); // schedule timed task
13
14         Thread.sleep(5_000);
15         scheduler.shutdown(); // cancel timer after 5 s
16     }
17 }
```

- **Timer:** In Zeile 15 wird ein [Timer](#) erzeugt, über den mittels der Methode `scheduleAtFixedRate` ein `TimerTask`-Objekt, also eine zeitgesteuerte Aufgabe, wiederholt ausgeführt werden kann (Zeile 16). Das zweite Input-Argument gibt die Verzögerung (in ms) der ersten Ausführung an, das dritte Input-Argument das Intervall (in ms) für die wiederholte Ausführung. Die Ausführung endet erst mit dem Abbruch des `Timer` (Zeile 19). Die Klasse `TimerTask` implementiert das Interface `Runnable`.
- **ScheduledExecutorService:** In Zeile 11 wird ein [ScheduledExecutorService](#) erzeugt, dem in diesem Fall genau ein Thread zur Ausführung der über ihn gesteuerten Runnables zur Verfügung steht. In Zeile 12 wird, entsprechend des vorherigen Timer-Beispiels, die wiederholte Ausführung eines Runnable gestartet. Die initiale Verzögerung beträgt wiederum 500 ms und das Intervall zwischen den Ausführungen 1 s. Der `ScheduledExecutorService` kann mittels der Methode `shutdown` beendet werden (Zeile 15).

Das Interface `ScheduledExecutorService` erweitert das allgemeine funktionale Interface [Executor](#), das lediglich die Methode `void execute(Runnable command)` spezifiziert, um ein Runnable nebenläufig auszuführen. Eine konkrete `Executor`- und `ScheduledExecutorService`-Implementierung ist die Klasse `ScheduledThreadPoolExecutor`, der im Konstruktor die Kapazität für einen Thread-Pool mitgegeben werden kann. Die Threads werden nun nicht mehr für jede Runnable über einen Konstruktoraufwurf `new Thread(...)` instantiiert, sondern aus einem begrenztem Thread-Pool wiederverwendet. Ein Thread-Pool bietet sich insbesondere für den typischen Anwendungsfall an, bei dem ein Server die Aufgaben (engl. *Tasks*) mehrerer Clients bedient. Das Thread-Pool-Muster vermeidet die ineffiziente Erzeugung eines neuen Threads je Aufgabe und schützt vor Ressourcenüberlastung, z.B. bei *Denial-of-Service*-Angriffen. Die Warteschlange, die zeitweise vor einem Thread-Pool entsteht, kann über unterschiedliche Strategien abgebaut werden, wobei *First-In-First-Out (FIFO)* häufig naheliegend und pragmatisch ist. Die folgende Abbildung verdeutlicht die Funktionsweise eines Thread-Pools.

Aufgaben-Warteschlange



Funktionsweise eines Thread-Pools

In dem folgenden Code-Beispiel wird die begrenzte Kapazität eines Thread-Pools demonstriert. Es sollen 7 Runnables gestartet werden (Zeilen 18-19). Der Thread-Pool lässt aber nur die Ausführung von 3 nebenläufigen Threads zu (Zeile 1), sodass die letzten 4 Runnables warten müssen, bis wieder ein Thread freigegeben wird. Da die Runnables jeweils zufällig lange arbeiten (Zeilen 8-11), ist nicht vorhersehbar, in welchem der 3 Threads die letzten 4 Runnables ausgeführt werden. Eine mögliche Ausgabe könnte wie folgt aussehen:

```
Task 2 worked for 2522 ms in pool-1-thread-2. Total time: 2538 ms
Task 3 worked for 2617 ms in pool-1-thread-3. Total time: 2633 ms
Task 1 worked for 2916 ms in pool-1-thread-1. Total time: 2932 ms
Task 4 worked for 1734 ms in pool-1-thread-2. Total time: 4317 ms
Task 6 worked for 1596 ms in pool-1-thread-1. Total time: 4529 ms
Task 5 worked for 2552 ms in pool-1-thread-3. Total time: 5185 ms
Task 7 worked for 1498 ms in pool-1-thread-2. Total time: 5816 ms
```



```

1  ExecutorService executor = Executors.newFixedThreadPool(3); // thread pool restricted to 3 concurrent threads
2  AtomicInteger uniqueTaskId = new AtomicInteger(1);
3  LocalDateTime startTime = LocalDateTime.now();
4
5  // specify runnable
6  Runnable runnable = () -> {
7      int id = uniqueTaskId.getAndIncrement();
8      int sleepTime = ThreadLocalRandom.current().nextInt(1000, 3000); // random sleep time
9      try {
10         Thread.sleep(sleepTime); // simulate work
11     } catch (InterruptedException e) { }
12
13     long totalTime = startTime.until(LocalDateTime.now(), ChronoUnit.MILLIS);
14     System.out.println("Task " + id + " worked for " + sleepTime + " ms in " + Thread.currentThread().getName() + ". Total time: "
15 + totalTime + " ms");
16 };
17
18 // execute 7 runnables
19 for (int i = 7; i > 0; i--) { executor.execute(runnable); }
    executor.shutdown();

```

Thread-lokale Variablen

Eine Variable, die als Attribut eines Objekts (oder einer Klasse) auf dem Heap angelegt ist, kann von mehreren konkurrierenden Threads zeitgleich verändert werden, wodurch leider Inkonsistenzen in den Daten entstehen können. Der exklusive Zugriff auf derartige Variablen durch das Schlüsselwort `synchronized` wird im nächsten Kapitel zur [Synchronisation von Threads](#) im Detail vorgestellt. Im folgenden Code-Beispiel sichert das Schlüsselwort `synchronized` an der Methode `increment` (Zeile 7) zu, dass nur ein Thread zurzeit diese Methode ausführen kann. Variablen, die von einem Basisdatentyp sind und auf dem Stack gespeichert werden, liegen grundsätzlich je Thread vor. Sollen Objekte auf dem Heap je Thread instantiiert werden, bietet sich ein [ThreadLocal](#)-Objekt an. Das folgende Code-Beispiel demonstriert anschaulich, den Unterschied zwischen einer gemeinsamen Variable (Zeile 3) und einer Thread-lokalen Variable (Zeile 5). Der Aufruf der Methode `yield` in Zeile 20 löst einen Hinweis an den Scheduler aus, dass der aktive Thread bereit ist, seinen zugewiesenen Prozessor abzugeben.

```

1  class MyThreadLocal {
2
3      Integer common = 0; // shared resource of multiple threads
4
5      ThreadLocal<Integer> local = ThreadLocal.withInitial(() -> 0); // thread-local resource
6
7      synchronized void increment() {
8          common++;
9          local.set(local.get() + 1);
10
11         System.out.println(Thread.currentThread().getName() + ": common = " + common + ", local = " + local.get());
12     }
13
14     public static void main(String[] args) {
15         MyThreadLocal resource = new MyThreadLocal();
16
17         Runnable runnable = () -> {
18             for (int i = 0; i < 3; i++) {
19                 resource.increment();
20                 Thread.currentThread().yield(); // current thread is willing to yield its current use of a processor
21             }
22         };
23
24         // start 3 concurrent threads
25         for (int i = 0; i < 3; i++) {
26             new Thread(runnable).start();
27         }
28     }
29 }

```

Die Ausgabe könnte z.B. wie folgt aussehen:

```

Thread-0: common = 1, local = 1
Thread-2: common = 2, local = 1
Thread-1: common = 3, local = 1
Thread-0: common = 4, local = 2
Thread-1: common = 5, local = 2
Thread-0: common = 6, local = 3
Thread-2: common = 7, local = 2
Thread-1: common = 8, local = 3
Thread-2: common = 9, local = 3

```

Die gezeigten Code-Beispiele zum Umgang mit Threads in Java finden sich im Verzeichnis `/concurrency` des Modul-Repository.

Synchronisation von Threads

Der Zugriff auf gemeinsame Ressourcen, z.B. auf Objekte in Java, aus mehreren nebenläufigen Threads heraus, kann ohne Synchronisation zu inkonsistenten, ungewollten Zuständen führen. Dies tritt insbesondere dann auf, wenn mehrere Anweisungen im Kontrollfluss als eine atomare Aktion ausgeführt werden sollen, deren Effekt unbeeinflusst von Anweisungen sein muss, die in nebenläufigen Threads ausgeführt werden. Ähnliche Probleme treten im Umfeld von Datenbanken auf, wenn Transaktionen nicht ausreichend isoliert voneinander sind (s. [ACID-Prinzip](#)).

Race Conditions

Ein typisches Beispiel für ein Nebenläufigkeitsproblem sind sogenannte *Race Conditions*. Diese beschreiben in der Programmierung eine Konstellation, in der der abschließend erreichte Zustand mehrerer nebenläufig ausgeführter Anweisungen von der zeitlichen Verschränkung (engl. *Interleaving*) der Threads durch den Scheduler ist. Race Conditions, die während der Entwicklungszeit nicht erkannt worden sind, können später Fehler verursachen, die im Betrieb nur sehr selten auftreten und deren Ursache im Nachhinein schwer zu lokalisieren ist. Derartige Fehler scheinen nicht reproduzierbar zu sein, da sie nur bei ungünstiger Verschränkung der nebenläufigen Threads auftreten. Das Debugging von Race Conditions ist demzufolge äußerst aufwändig.

Das folgende Beispiel stellt 2 nebenläufige Threads dar, die dieselben einfachen Anweisungen ausführen. Es wird zunächst die serielle Ausführung der Threads nacheinander und anschließend eine verschränkte Ausführung abgebildet. Die am Ende erreichten Zustände unterscheiden sich, da eine potentielle Race Condition vorliegt. Im Fall der verschränkten Ausführung ist eine der beiden Zustandsänderungen an der gemeinsamen Ressource verloren gegangen. Wir sprechen von einem [Lost Update](#).

Thread 1	Thread 2		Zustand der gemeinsamen Ressource
Lesen des Zustands (0)		←	0
Verändern des Zustands (+1)			0
Schreiben des Zustands (1)		→	1
	Lesen des Zustands (1)	←	1
	Verändern des Zustands (+1)		1
	Schreiben des Zustands (2)	→	2

Serielle Ausführung der Anweisungen

Thread 1	Thread 2		Zustand der gemeinsamen Ressource
Lesen des Zustands (0)		←	0
	Lesen des Zustands (0)	←	0
Verändern des Zustands (+1)			0
	Verändern des Zustands (+1)		0
Schreiben des Zustands (1)		→	1
	Schreiben des Zustands (1)	→	1

Verschränkte Ausführung der Anweisungen

Im folgenden Code-Beispiel werden 100 Threads gestartet (Zeilen 29-31), von denen jeder auf eine gemeinsame Ressource vom Typ `int` zugreift (Zeile 5) und diese 100x inkrementiert (Zeilen 22-24). Da die Ressource mit 0 initialisiert ist, sollte in der letzten Zeile demzufolge ein abschließend erreichter Zustand der Ressource von $100 \times 100 = 10.000$ ausgegeben werden. Da in dieser Implementierung aber eine Race Condition vorliegt, wird dieser Zustand praktisch i.d.R. nicht erreicht. Einige Updates der Ressource gehen verloren.

```

1  import java.util.concurrent.CountDownLatch;
2
3  class LostUpdate {
4
5      int resource = 0; // shared resource
6
7      // this method is the critical section
8      void incrementResourceValue() {
9          int t = resource;           // read resource value
10         t++;                         // increment resource value
11         resource = t;               // write resource value
12     }
13
14     public static void main(String[] args) throws InterruptedException {
15         new LostUpdate().runThreads(100);
16     }
17
18     void runThreads(int n) throws InterruptedException {
19         CountDownLatch doneSignal = new CountDownLatch(n);
20
21         Runnable runnable = () -> {
22             for (int i = 0; i < 100; i++) { // each thread increments the resource value 100 times
23                 incrementResourceValue();
24             }
25             doneSignal.countDown(); // send a signal that this thread is completed
26         };
27
28         // start all threads
29         for (int i = 0; i < n; i++) {
30             new Thread(runnable).start();
31         }
32
33         // wait for all threads to complete
34         doneSignal.await();
35
36         System.out.println("Resource value = " + resource);
37     }
38 }

```

Es existiert ein sogenannter kritischer Abschnitt (engl. *Critical Section*), indem sich nur ein Thread zurzeit befinden darf, damit keine Lost Updates auftreten. In dem obigen Code-Beispiel ist die Methode `incrementResourceValue` der kritische Abschnitt (Zeilen 8-12). Das Lesen, Inkrementieren und Schreiben des Zustands der Ressource muss als eine atomare Aktion ausgeführt werden und darf nicht durch eine Verschränkung der Threads unterbrochen werden. Eine naheliegende Idee ist es, den kritischen Abschnitt auf eine einzelne Anweisung zu reduzieren, z.B. indem die Methode `incrementResourceValue` wie folgt umgestaltet wird:

```

1  void incrementResourceValue() {
2      resource++; // read, increment, and write resource value
3  }

```

Aber auch diese äußerst einfache Anweisung ist nicht *thread-safe*, d.h. sie ist keine unteilbare Aktion, da sie im kompilierten Bytecode in mehrere Instruktionen zerlegt wird, die wiederum durch eine unglückliche Verschränkung nebenläufiger Threads nicht direkt nacheinander ausgeführt werden könnten. Der Bytecode der obigen Methode ist folgend dargestellt, wobei die Instruktionen der Anweisung `resource++` sich von `getfield` bis `putfield` erstrecken. Der Bytecode einer kompilierten Java-Klasse, z.B. `MyClass.class`, lässt sich mittels `javap -c MyClass.class` anzeigen.

```

1  void incrementResourceValue();
2      Code:
3      0: aload_0
4      1: dup
5      2: getfield      #2          // Field resource:I
6      5: iconst_1
7      6: iadd
8      7: putfield      #2          // Field resource:I
9      10: return

```

Wechselseitiger Ausschluss für kritische Abschnitte

Das Blockieren von Threads am Anfang von kritischen Abschnitten bezeichnen wir allgemein als wechselseitigen Ausschluss (engl. *Mutual Exclusion*, kurz *Mutex*). Das Verfahren des wechselseitigen Ausschlusses lässt sich in folgende drei Phasen unterteilen:

1. Ein Thread erlangt eine Sperre (engl. *Lock*) für einen kritischen Abschnitt, in dem der Zustand einer gemeinsamen Ressource verändert wird.

2. Ein Thread durchläuft den kritischen Abschnitt und blockiert währenddessen andere Threads, die den kritischen Abschnitt in dieser Zeit ebenfalls betreten möchten.
3. Ein Thread gibt die erlangte Sperre wieder frei und informiert die blockierten Threads darüber.

In Java muss das Schlüsselwort `synchronized` verwendet werden, um eine Methode oder einen beliebigen Anweisungsblock als kritischen Abschnitt zu kennzeichnen. Es gewährleistet, dass nur ein Thread gleichzeitig den kritischen Abschnitt betreten und durchlaufen darf. Erst durch die Verwendung des Schlüsselworts `synchronized` ist die folgende Methode tatsächlich *thread-safe*.

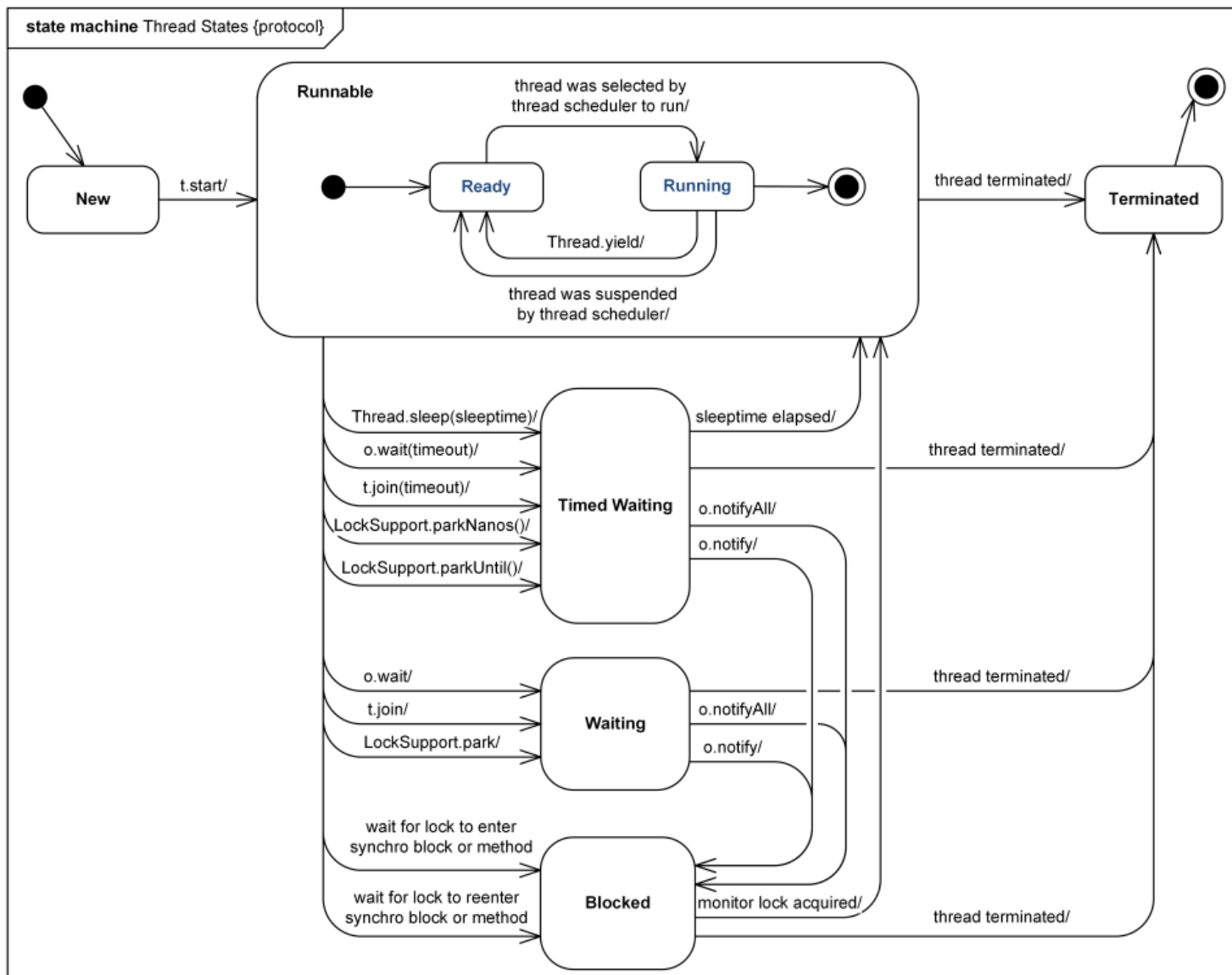
```
1 synchronized void incrementResourceValue() {  
2     resource++;  
3 }
```

Während sich ein Thread im kritischen Abschnitt befindet, sichert die JVM durch eine Sperre zu, dass weitere Threads am Anfang des kritischen Abschnitts blockiert werden. Dadurch wechseln die Threads ggf. aus dem Zustand **Runnable** in den Zustand **Blocked**. Eine Sperre wird in Java je Objekt gesetzt, d.h. mehrere `synchronized`-Methoden und -Blöcke, die sich auf dasselbe Objekt beziehen, sind nicht unabhängig voneinander, sondern teilen sich eine gemeinsame Sperre. Das folgende Code-Beispiel zeigt beispielhaft eine Klasse mit zwei `synchronized`-Methoden und einem `synchronized`-Block. Wenn ein Objekt X dieser Klasse instantiiert wird und ein Thread auf diesem Objekt X die Methode `a` aufruft, so kann kein anderer Thread die Methoden `a`, `b` und den `synchronized`-Block in Methode `c` betreten. Auf einem weiteren Objekt Y dieser Klasse hingegen wären dadurch keine Sperren gesetzt und die `synchronized`-Methoden und -Blöcke könnten weiterhin aufgerufen werden, ohne zu blockieren.

```
1 class SharedResource {  
2  
3     // synchronized method  
4     synchronized void a() {  
5         // ...  
6     }  
7  
8     // another synchronized method  
9     synchronized void b() {  
10        // ...  
11    }  
12  
13    // method containing a synchronized block  
14    void c() {  
15        // ...  
16        synchronized (this) {  
17            // ...  
18        }  
19        // ...  
20    }  
21 }
```

Thread-Zustände in der JVM

Das folgende UML-Zustandsdiagramm visualisiert die möglichen Zustände eines Threads innerhalb der JVM sowie die möglichen Transitionen zwischen diesen Zuständen. Sperren mittels `synchronized` gewährleisten den wechselseitigen Ausschluss für kritische Abschnitte und können damit Race Conditions verhindern. Die JVM versetzt die Threads ggf. automatisch in den Zustand **Blocked**, falls auf eine Sperre, bedingt durch `synchronized`, zu warten ist: "*Thread is blocked for synchronization*".



Zustände und Transitionen von Threads in der JVM

[Java Thread States and Life Cycle](#)

Mittels **synchronized** können aber keine anwendungsspezifischen Bedingungen formuliert werden. Häufig ergibt sich in der Praxis die Situation, dass ein Thread auf ein bestimmtes Ereignis warten muss, bevor er sinnvoll weiterarbeiten kann. Er soll von einem anderen Thread benachrichtigt werden können, wenn dieser das Ereignis herbeigeführt hat, das aus dem Wartezustand befreit. Der grundlegende Mechanismus für das Warten auf Benachrichtigungen von anderen Threads kann in Java über die Methoden **wait** und **notify** abgebildet werden, die in der allgemeinen Oberklasse [java.lang.Object](#) implementiert sind und damit jedem Objekt zur Verfügung stehen. Wenn ein Thread die Methode **wait** auf einem Objekt aufruft, wird er in den Zustand **Waiting** versetzt: "Thread is waiting for notification". Erst wenn ein anderer Thread die Methode **notify** oder die Methode **notifyAll** auf demselben Objekt aufruft, wird der wartende Thread wieder aus dem Wartezustand befreit und in den Zustand **Runnable** versetzt. Die Methode **wait** kann auch mit einem Timeout-Argument versehen werden. In diesem Fall wird der Thread, wie bei einem Aufruf der Methode **Thread.sleep**, in den Zustand **Timed Waiting** versetzt. Über [java.lang.management.ThreadInfo](#) kann grundsätzlich ausgewertet werden, wie viel Zeit ein Thread in welchem der Zustände verbracht hat.

Der Zustand **Runnable** ist in die Zustände **Ready** und **Running** untergliedert. Wenn es mehr nebenläufige Threads als Prozessoren gibt, steuert der Scheduler den Wechsel der Threads zwischen diesen Zuständen. Im Zustand **Running** ist einem Thread tatsächlich ein Prozessor zugewiesen, so dass der Kontrollfluss des Threads abgearbeitet werden kann. Ein Thread im Zustand **Ready** wartet darauf, dass der Scheduler ihm im Zuge des Interleaving einen Prozessor zuweist.

Erzeuger-Verbraucher-Problem

Die Verwendung der Methoden **wait** und **notify** lässt sich über das sogenannte [Erzeuger-Verbraucher-Problem](#) motivieren. Wir stellen uns dazu folgendes Szenario vor:

- Es sind mehrere Erzeuger-Threads (unten implementiert in der Klasse **Producer**) aktiv, die generische Waren – oder in der folgenden Implementierung vereinfacht Textnachrichten – produzieren und diese für interessierte Verbraucher bereitstellen.
- Weiterhin sind mehrere Verbraucher-Threads (unten implementiert in der Klasse **Consumer**) aktiv, die ihrerseits die bereitgestellten Waren bzw. Nachrichten konsumieren.
- Die Erzeuger und Verbraucher tauschen die Nachrichten über einen Speicher aus, der in seiner Kapazität begrenzt ist und die gemeinsam genutzte Ressource darstellt. Beim Zugriff auf den Speicher ist also wechselseitiger Ausschluss zu gewährleisten. Nach welchem Prinzip (FIFO, LIFO, etc.) die Nachrichten im Speicher eingelagert und ausgeliefert werden, ist nicht wichtig. Entscheidend ist, dass der Speicher begrenzt ist, weil dadurch anwendungsspezifische Wartebedingungen für die Erzeuger- und Verbraucher-Threads entstehen. Ein Erzeuger kann nur Nachrichten bereitstellen, wenn der Speicher nicht voll ist. Ein Verbraucher kann nur Nachrichten konsumieren, wenn der Speicher nicht leer ist. Falls der Speicher also voll ist, wird ein Erzeuger, der eine neue Nachricht erzeugt hat, die Methode **wait** auf dem Speicher-Objekt aufrufen. Aus diesem Wartezustand kann ein Erzeuger befreit werden, wenn ein Verbraucher irgendwann eine vorhandene Nachricht aus dem Speicher entnimmt und damit einen freien Platz für die neue Nachricht des Erzeugers schafft. Die Verbraucher müssen dazu bei der Entnahme von

Nachrichten aus dem Speicher, die Methode `notify` auf dem Speicher-Objekt aufrufen, um wartende Erzeuger zu befreien. Falls der Speicher leer ist, wird ein Verbraucher die Methode `wait` auf dem Speicher-Objekt aufrufen, da er aktuell keine Nachricht entnehmen kann. Der Verbraucher wird später entsprechend von einem Erzeuger aus diesem Wartezustand befreit, der die Methode `notify` auf dem Speicher-Objekt aufruft, wenn er eine neue Nachricht bereitgestellt hat. Eine derartige gemeinsam genutzte Ressource, die wechselseitigen Ausschluss in ihren Zugriffsmethoden zusichert und ggf. blockiert bis eine bestimmte Bedingung erfüllt ist, wird in der Informatik auch als [Monitor](#) bezeichnet.

Es folgt eine beispielhafte Implementierung des Erzeuger-Verbraucher-Problems. Der Speicher ist in der Klasse `BoundedBufferSync` implementiert und hat hier eine Kapazität von lediglich 3 Nachrichten. Es werden in der Klasse `ProducerConsumerStarter` jeweils 2 Erzeuger- und Verbraucher-Threads gestartet.

Producer

```
1 class Producer extends Thread {
2
3     BoundedBuffer buffer;
4
5     Producer(int id, BoundedBuffer buffer) {
6         super("Producer " + id);
7         this.buffer = buffer;
8     }
9
10    @Override
11    public void run() {
12        while (true) {
13            try {
14                sleep(ThreadLocalRandom.current().nextInt(0, 100)); // sleep
15                buffer.put(UUID.randomUUID().toString()); // put a message to the buffer
16            } catch (InterruptedException e) { }
17        }
18    }
19 }
```

Consumer

```
1 class Consumer extends Thread {
2
3     BoundedBuffer buffer;
4
5     Consumer(int id, BoundedBuffer buffer) {
6         super("Consumer " + id);
7         this.buffer = buffer;
8     }
9
10    @Override
11    public void run() {
12        while (true) {
13            try {
14                sleep(ThreadLocalRandom.current().nextInt(0, 100)); // sleep
15                buffer.take(); // take a message from the buffer
16            } catch (InterruptedException e) { }
17        }
18    }
19 }
```

BoundedBuffer

```
1 interface BoundedBuffer {
2
3     // put a new message to the buffer, waiting if necessary for space to become available
4     void put(String message) throws InterruptedException;
5
6     // retrieve and remove a message from the buffer, waiting if necessary until a new message becomes available
7     String take() throws InterruptedException;
8 }
```

BoundedBufferSync

```
1  import java.util.Queue;
2  import java.util.LinkedList;
3
4  class BoundedBufferSync implements BoundedBuffer {
5
6      int capacity;
7      Queue<String> queue = new LinkedList<>();
8
9      BoundedBufferSync(int capacity) {
10         this.capacity = capacity;
11     }
12
13     public synchronized void put(String message) throws InterruptedException {
14         while (queue.size() == capacity) {
15             wait();
16         }
17         queue.add(message);
18         System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
19         notify();
20     }
21
22     public synchronized String take() throws InterruptedException {
23         while (queue.isEmpty()) {
24             wait();
25         }
26         String message = queue.poll(); // retrieves and removes head of queue
27         System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
28         notify();
29         return message;
30     }
31 }
```

ProducerConsumerStarter

```
1  class ProducerConsumerStarter {
2
3      public static void main(String[] args) {
4
5          BoundedBuffer buffer = new BoundedBufferSync(3); // select bounded buffer implementation here
6
7          for (int i = 1; i <= 2; i++) {
8              new Producer(i, buffer).start();
9              new Consumer(i, buffer).start();
10         }
11     }
12 }
```

Es ist anzumerken, dass die Methode `notify` nur einen zufälligen Thread aus der Menge, der aktuell auf die betroffene Sperre wartenden Threads, aus dem Wartezustand befreit. Dadurch ist keine Fairness unter den wartenden Erzeugern und Verbrauchern gewährleistet. Das folgende Beispiel widmet sich der Herausforderung fairen Wartens, gemäß des Prinzips einer FIFO-Warteschlange, zu realisieren. Wir stellen uns dazu eine Parkgarage mit einer beschränkten Kapazität an Stellplätzen vor. In diese Parkgarage fahren Autos ein und aus. Wenn alle Stellplätze belegt sind, müssen neu ankommende Autos an einer Schranke vor der Parkgarage in einer FIFO-Warteschlange warten. Die Parkgarage ist die gemeinsam genutzte Ressource für mehrere nebenläufige Threads. Die Autos werden als Threads implementiert, die in einer Endlosschleife folgende Aktivitäten ausführen (Zeilen 22-27 in Klasse `Car`):

- herumfahren (Methode `cruise` in Klasse `Car`),
- einfahren in die Parkgarage (Methode `enter` in Interface `ParkingGarage`),
- parken (Methode `park` in Klasse `Car`),
- ausfahren aus der Parkgarage (Methode `leave` in Interface `ParkingGarage`).

Car

```
1 class Car extends Thread {
2
3     ParkingGarage garage; // shared resource
4
5     public static void main(String[] args) {
6         int places = 3; // number of slots in parking garage
7         int cars = 7;   // number of cars
8
9         ParkingGarage garage = new UnfairParkingGarageImpl(places); // select implementation of parking garage here
10        for (int i = 1; i <= cars; i++) {
11            new Car("Car " + i, garage).start();
12        }
13    }
14
15    public Car(String name, ParkingGarage garage) {
16        super(name);
17        this.garage = garage;
18    }
19
20    @Override
21    public void run() {
22        while (true) {
23            cruise();
24            garage.enter(this);
25            park();
26            garage.leave(this);
27        }
28    }
29
30    void cruise() { idle(30); } // cruise 0 to 30 ms
31
32    void park() { idle(100); } // cruise 0 to 100 ms
33
34    void idle(int maxSleepTime) {
35        try {
36            TimeUnit.MILLISECONDS.sleep(ThreadLocalRandom.current().nextInt(0, maxSleepTime));
37        }
38        catch (InterruptedException e) { }
39    }
40 }
```

ParkingGarage

```
1 interface ParkingGarage {
2
3     // car which enters the parking garage
4     void enter(Car car);
5
6     // car which leaves the parking garage
7     void leave(Car car);
8 }
```

Der wechselseitige Ausschluss der Threads beim Einfahren und Ausfahren der Parkgarage ist sicherzustellen, d.h. die Methoden `enter` und `leave` der gemeinsamen Ressource `ParkingGarage` müssen in der jeweiligen Implementierung des Interface mittels `synchronized` als kritische Abschnitte gekennzeichnet werden (siehe unten Zeilen 11+25 in der Klasse `UnfairParkingGarage`). Im folgenden Code-Beispiel werden 2 alternative Implementierungen des Interface `ParkingGarage` dargestellt. In der Implementierung `UnfairParkingGarage` benachrichtigt ein Auto, das die Parkgarage verlässt, zufällig eines der wartenden Autos vor der Schranke und ermöglicht ihm das Einfahren in die Parkgarage. Dies geschieht durch den Aufruf der Methode `notify` in Zeile 28. Wenn die nummerierte Ankunft an der Schranke vor der Parkgarage nicht der ebenfalls nummerierten Einfahrt in die Parkgarage entspricht – also das FIFO-Prinzip verletzt ist, wird dies durch eine Ausgabe über `System.err` in Zeile 20 kenntlich gemacht. Dieser Fall wird bei der Ausführung des Programms relativ selten auftreten, kann aber beobachtet werden.

Die Implementierung `FairParkingGarage` unterscheidet sich leicht, um die Fairness beim Warten abzubilden – insbesondere in den Zeilen 16 und 32.

- In Zeile 32 werden mittels der Methode `notifyAll` alle der wartenden Autos statt nur ein zufälliges benachrichtigt.
- In Zeile 16 wird die Bedingung für das Warten erweitert. Es muss nicht nur gewartet werden, wenn keine Stellplätze frei sind (`places == 0`), sondern auch, wenn das Auto noch nicht an vorderster Position der Warteschlange angelangt ist. Es wird dazu gezählt, wie viele Autos bereits an der Schranke vor der Parkgarage angekommen sind (`arrivalCount`, Zeile 12) und wie viele Autos bereits in die Parkgarage eingefahren sind (`entranceCount`, Zeile 22). Die Wertzuweisung in die Variable `arrival` (Zeile 12) entspricht daher der Vergabe einer laufend inkrementierten Wartenummer – ähnlich des Wartebereichs in vielen Ämtern in Deutschland. Ein vor der Schranke wartendes Auto wird zwar jedes Mal erweckt, wenn ein anderes Auto die Parkgarage

verlässt, kann aber nur vorrücken und nicht einfahren (`arrival - 1 != entranceCount`, Zeile 16), bis alle übrigen wartenden Autos mit niedrigerer Wartenummer eingefahren sind. Auf diese Weise wird die Fairness beim Zugriff auf die gemeinsame Ressource eingehalten.

UnfairParkingGarage

```
1  class UnfairParkingGarageImpl implements ParkingGarage {
2      int places;
3      int arrivalCount = 0;
4      int entranceCount = 0;
5
6      UnfairParkingGarageImpl(int places) {
7          this.places = places;
8      }
9
10     @Override
11     public synchronized void enter(Car car) {
12         int arrival = ++arrivalCount;
13         while (places == 0) {
14             log(car.getName() + " stops at barrier [waiting cars = " + (arrivalCount - entranceCount) + "].");
15             try {
16                 wait();
17             } catch (InterruptedException e) { }
18         }
19         int entrance = ++entranceCount;
20         log(car.getName() + " enters parking garage [arrival = " + arrival + ", entrance = " + entrance + "].",
21             arrival == entrance ? System.out : System.err);
22         places--;
23     }
24
25     @Override
26     public synchronized void leave(Car car) {
27         log(car.getName() + " leaves parking garage.");
28         places++;
29         notify();
30     }
31
32     void log(String msg, PrintStream printStream) { printStream.println(msg); }
33
34     void log(String msg) { log(msg, System.out); }
35 }
```

FairParkingGarage

```
1 class FairParkingGarageImpl implements ParkingGarage {
2     int places;
3     int arrivalCount = 0;
4     int entranceCount = 0;
5
6     FairParkingGarageImpl(int places) {
7         this.places = places;
8     }
9
10    @Override
11    public synchronized void enter(Car car) {
12        int arrival = ++arrivalCount;
13        if (places == 0) {
14            log(car.getName() + " stops at barrier [waiting cars = " + (arrivalCount - entranceCount) + "].");
15        }
16        while (arrival - 1 != entranceCount || places == 0) {
17            try {
18                wait();
19            } catch (InterruptedException e) { }
20        }
21        int entrance = ++entranceCount;
22        log(car.getName() + " enters parking garage [arrival = " + arrival + ", entrance = " + entrance + "].",
23            arrival == entrance ? System.out : System.err);
24        places--;
25        notifyAll();
26    }
27
28    @Override
29    public synchronized void leave(Car car) {
30        log(car.getName() + " leaves parking garage.");
31        places++;
32        notifyAll();
33    }
34
35    void log(String msg, PrintStream printStream) { printStream.println(msg); }
36
37    void log(String msg) { log(msg, System.out); }
38 }
```

Es stellt sich beim genauen Hinsehen die Frage, warum auch am Ende der Methode `enter` die Methode `notifyAll` aufgerufen wird (Zeile 25). Dazu müssen wir uns vorstellen, dass die Parkgarage voll belegt ist, während mehrere Autos vor der Schranke warten. Nun verlassen direkt hintereinander mehrere Autos die Parkgarage, sodass wieder einige Stellplätze frei werden. Der gemeinsame kritische Abschnitt der Methoden `enter` und `leave` sichert zu, dass während des Ausfahrens eines Autos nicht gleichzeitig ein anderes Auto einfahren kann. Anschließend gelingt es dem vordersten Auto in der Warteschlange in die Parkgarage einzufahren. Nach dem Einfahren muss es die anderen wartenden Autos mittels `notifyAll` benachrichtigen (Zeile 25), da noch weitere Plätze frei sind und ansonsten nicht wieder belegt werden würden.

Im Package `java.util.concurrent` finden wir heute lange etablierte Interfaces und Klassen, die eine entwicklerfreundlichere API für die grundlegenden Muster zur Synchronisation von Threads bieten. Dazu zählen z.B. die Interfaces `Lock` als Alternative zu `synchronized` und `Condition` als Alternative zu den Methoden `wait`, `notify` und `notifyAll`. Blockierende FIFO-Warteschlangen bieten sämtliche Implementierungen des Interface `BlockingQueue`. Das folgende Code-Beispiel demonstriert wie der oben dargestellte, begrenzte und blockierende Speicher in der Klasse `BoundedBufferSync` alternativ über `Lock`- und `Condition`-Objekte (siehe `BoundedBufferLock`) bzw. über eine `ArrayBlockingQueue` (siehe `BoundedBufferBlockingQueue`) realisiert werden kann.

BoundedBufferSync

```
1  import java.util.Queue;
2  import java.util.LinkedList;
3
4  class BoundedBufferSync implements BoundedBuffer {
5
6      int capacity;
7      Queue<String> queue = new LinkedList<>();
8
9      BoundedBufferSync(int capacity) {
10         this.capacity = capacity;
11     }
12
13     public synchronized void put(String message) throws InterruptedException {
14         while (queue.size() == capacity) {
15             wait();
16         }
17         queue.add(message);
18         System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
19         notify();
20     }
21
22     public synchronized String take() throws InterruptedException {
23         while (queue.isEmpty()) {
24             wait();
25         }
26         String message = queue.poll(); // retrieves and removes head of queue
27         System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
28         notify();
29         return message;
30     }
31 }
```


BoundedBufferLock

```
1  import java.util.concurrent.locks.Lock;
2  import java.util.concurrent.locks.ReentrantLock;
3  import java.util.concurrent.locks.Condition;
4  // ...
5
6  class BoundedBufferLock implements BoundedBuffer {
7
8      int capacity;
9      Queue<String> queue = new LinkedList<>();
10
11      Lock lock = new ReentrantLock();
12      Condition notFull = lock.newCondition();
13      Condition notEmpty = lock.newCondition();
14
15      BoundedBufferLock(int capacity) {
16          this.capacity = capacity;
17      }
18
19      public void put(String message) throws InterruptedException {
20          lock.lock(); // enter critical section
21          try {
22              while (queue.size() == capacity) {
23                  notFull.await();
24              }
25              queue.add(message);
26              System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
27              notEmpty.signal();
28          } finally {
29              lock.unlock(); // leave critical section
30          }
31      }
32
33      public String take() throws InterruptedException {
34          lock.lock(); // enter critical section
35          try {
36              while (queue.isEmpty()) {
37                  notEmpty.await();
38              }
39              String message = queue.poll(); // retrieves and removes head of queue
40              System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
41              notFull.signal();
42              return message;
43          } finally {
44              lock.unlock(); // leave critical section
45          }
46      }
47  }
```

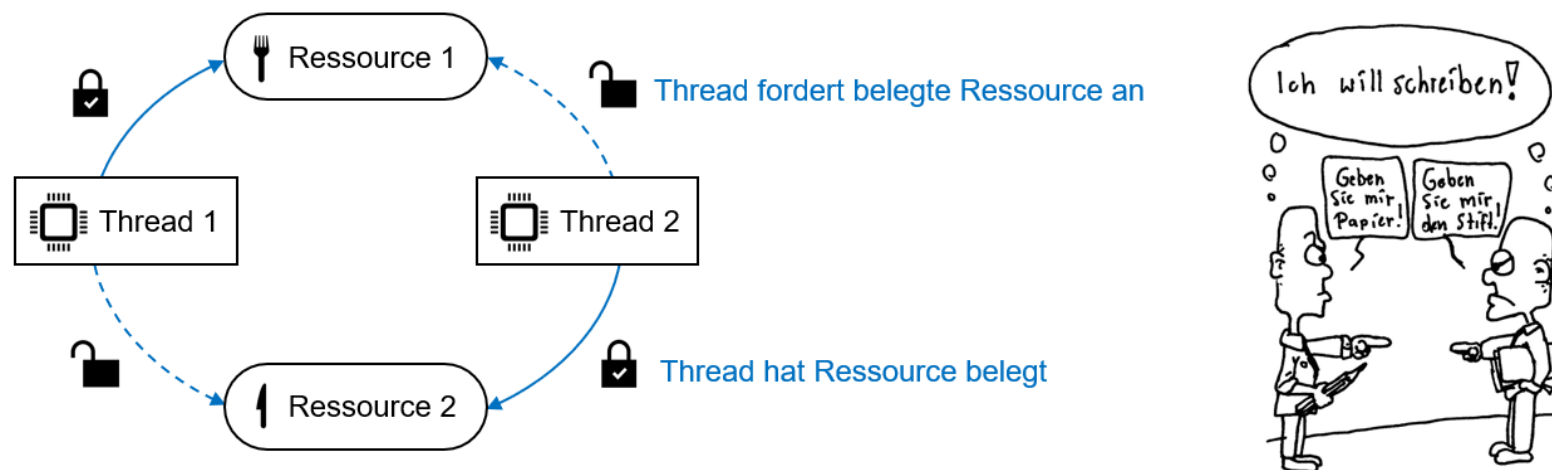
BoundedBufferBlockingQueue

```
1  import java.util.concurrent.BlockingQueue;
2  import java.util.concurrent.ArrayBlockingQueue;
3
4  class BoundedBufferBlockingQueue implements BoundedBuffer {
5
6      int capacity;
7      BlockingQueue<String> queue;
8
9      public BoundedBufferBlockingQueue(int capacity) {
10         queue = new ArrayBlockingQueue<>(capacity);
11     }
12
13     @Override
14     public void put(String message) throws InterruptedException {
15         queue.put(message);
16         System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
17     }
18
19     @Override
20     public String take() throws InterruptedException {
21         String message = queue.take();
22         System.out.printf("%s: %s\t [%d messages in buffer]\n", Thread.currentThread().getName(), message, queue.size());
23         return message;
24     }
25 }
```

Die gezeigten Code-Beispiele zur Synchronisation von Threads in Java finden sich im Verzeichnis /concurrency des Modul-Repository.

Deadlock und Starvation

Wenn ein Thread nicht nur eine, sondern mehrere Sperren auf exklusiv nutzbare Ressourcen anfordert, kann es zu einer Verklemmung (engl. *Deadlock*) zwischen den nebenläufigen Threads kommen, die um den Zugriff auf die Ressourcen konkurrieren. Ein Deadlock bezeichnet den Zustand, bei dem eine zyklische Wartekette zwischen mehreren Threads eingetreten ist, wobei jeder beteiligte Thread auf die Freigabe von Sperren auf Ressourcen wartet, die ein anderer beteiligter Thread bereits exklusiv erlangt hat. Im einfachsten Fall sind, wie in der folgenden Abbildung visualisiert, nur 2 Threads und 2 Ressourcen beteiligt.



Verklemmung von Threads beim Zugriff auf exklusiv nutzbare Ressourcen

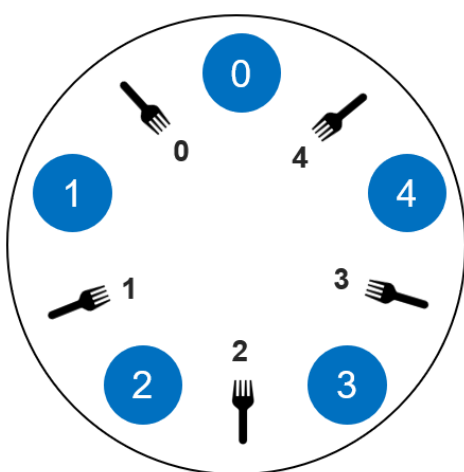
Für einen Deadlock müssen folgende Bedingungen erfüllt sein. Wenn nur eine Bedingung aufgehoben werden kann, wird auch der Deadlock aufgelöst.

1. Ressourcen sind nur exklusiv unter wechselseitigem Ausschluss nutzbar.
2. Belegte Ressourcen können einem Thread nicht entzogen werden.
3. Threads, die bereits eine Ressource belegt haben, fordern weitere an.
4. Es entsteht eine zyklische Wartekette zwischen den Threads. Jeder Thread belegt eine Ressource, die ein anderer Thread in der Wartekette anfordert.

Zur Vermeidung von Deadlocks kann ein optimistischer oder ein pessimistischer Ansatz des Sperrens von Ressourcen verfolgt werden. Beim *Optimistic Locking* ist die grundlegende Annahme, dass Deadlocks aufgrund geringer Konkurrenz beim exklusiven Zugriff auf Ressourcen selten auftreten. Es wird periodisch geprüft, ob ein Deadlock eingetreten ist. Falls ein Deadlock, also eine zyklische Wartekette, erkannt wird, wird einem der beteiligten Threads eine Sperre und damit der Zugriff auf eine Ressource entzogen. Damit wird die zweite der obigen Bedingungen gelöst. Die Transaktion, die aktuell im betroffenen Thread ausgeführt wird, muss zurückgerollt und neugestartet werden (= *Rollback* einer Transaktion).

Beim *Pessimistic Locking* hingegen sollen Deadlocks grundsätzlich vermieden werden. Dazu muss ein Thread alle benötigten Sperren auf exklusiv nutzbare Ressourcen gleich zu Beginn der auszuführenden Transaktion anfordern. Entweder wird das Anfordern aller Sperren als atomare Aktion in einem kritischen Abschnitt zusammengefasst, wodurch die dritte der obigen Bedingungen hinfällig wird – oder die einzeln angeforderten Sperren werden in einer definierten Reihenfolge erlangt, wodurch keine Zyklen mehr entstehen können und die vierte der obigen Bedingungen gelöst wird.

Wer in Lehrbüchern oder im Internet zum Thema *Deadlock* recherchiert, wird relativ schnell auf das berühmte Beispielsproblem der [Dining Philosophers](#) stoßen. Die Problemstellung wurde bereits [1965 von Edsger Dijkstra](#) für seine Studierenden formuliert und ist bis heute in der Informatik-Lehre etabliert. Wir stellen uns folgende Situation vor, die in der Abbildung unten veranschaulicht ist: 5 Philosophen sitzen an einem Tisch und denken. Vor jedem Philosophen liegt ein Teller, um gelegentlich davon zu essen. Zum Essen benötigt ein Philosoph zwei Gabeln, die links und rechts des Tellers liegen. Zu einem zufälligen Zeitpunkt nimmt jeder Philosoph zwei Gabeln und isst zufällig lange, bis er sich wieder dem Denken zuwendet. Es stehen auf dem Tisch aber insgesamt nur 5 Gabeln zur Verfügung, und jeder Philosoph benötigt die beiden Gabeln links und rechts seines Tellers zum Essen.



Veranschaulichung des Problems der Dining Philosophers

Die Philosophen sind in diesem Beispiel die Threads, die um den exklusiven Zugriff auf die Gabeln als Ressourcen konkurrieren. Wenn alle Philosophen zeitgleich zuerst die rechte und dann die linke Gabel nehmen, entsteht ein Deadlock, bei dem jeder Philosoph auf seinen linken Nachbarn warten muss, bis dieser die benötigte Gabel wieder freigibt. In diesem Zustand würden alle Philosophen verhungern. Ein Deadlock kann vermieden werden, wenn das Erlangen beider Gabeln als atomare Aktion in einem kritischen Abschnitt realisiert wird.

Ein Blick auf die obige Abbildung zeigt, dass immer 2 Philosophen parallel essen können, z.B. Philosoph 1 und Philosoph 3 und anschließend Philosoph 2 und Philosoph 4. Wenn diese 4 Philosophen sich jeweils paarweise abwechseln, wird der fünfte Philosoph verhungern. Dieses Verhungern (engl. *Starvation*) eines Threads auf Kosten der anderen, weil es keine faire Warteschlange gibt, ist ebenfalls zu vermeiden. Für das Philosophen-Problem gibt es unterschiedliche Ansätze um Starvation zu vermeiden:

- FIFO-Warteschlange – ineffizient, da Wartezeiten nicht zu optimaler Auslastung führen
- Timeouts beim Warten, d.h. Vorrang bei der Belegung von Ressourcen für Threads, die bereits lange warten
- Asymmetrische Lösung durch einen zusätzlichen Butler-Thread, der die Belegung der Ressourcen möglichst fair steuert

Es wird an dieser Stelle kein Java-Code als Lösung für das Philosophen-Problem dargestellt. Stattdessen soll die Lösung mit Hilfe der folgenden Klassen eigenständig entwickelt werden!

Philosopher

```
1 class Philosopher extends Thread {
2     int number;
3
4     @Override
5     public void run() {
6         while (true) {
7             // TODO: think, takeFork, eat, putFork
8         }
9     }
10 }
```

Table

```
1 class Table {
2     boolean[] forkUsed;
3
4     synchronized void takeFork(int number) {
5         // TODO ...
6     }
7
8     synchronized void putFork(int number) {
9         // TODO ...
10    }
11 }
```

Java Concurrency API

Die [Java Concurrency API](#) im Package `java.util.concurrent` stellt seit Java 5 diverse Interfaces und Klassen bereit, um nebenläufige Threads zu steuern und zu koordinieren. Dazu zählen:

- **Executors:** Die Klassen [ThreadPoolExecutor](#) und [ScheduledThreadPoolExecutor](#) stellen anpassbare, flexible Thread-Pools bereit.
- **Queues:** Die Klasse [ConcurrentLinkedQueue](#) stellt eine effiziente, skalierbare, Thread-sichere und nicht blockierende FIFO-Warteschlange bereit. Fünf weitere Klassen implementieren das Interface [BlockingQueue](#), das die blockierenden Methoden `put` and `take` definiert: [LinkedBlockingQueue](#), [ArrayBlockingQueue](#), [SynchronousQueue](#), [PriorityBlockingQueue](#) und [DelayQueue](#). Diese Klassen decken die häufigsten Verwendungen für Erzeuger-Verbraucher-Probleme, Nachrichtenaustausch (engl. *Messaging*) und die Verarbeitung nebenläufiger Aufgaben ab.
- **Concurrent Collections:** Neben Warteschlangen werden `Collection`-Implementierungen angeboten, die für die Verwendung im Multithreading-Kontext konzipiert sind, z.B. [ConcurrentHashMap](#) und [CopyOnWriteArrayList](#). Wenn zu erwarten ist, dass viele Threads auf eine bestimmte Collection zugreifen, ist eine `ConcurrentHashMap` i.d.R. einer synchronisierten `HashMap` vorzuziehen. Eine `CopyOnWriteArrayList` ist einer synchronisierten `ArrayList` vorzuziehen, wenn die erwartete Anzahl an Lesezugriffen und Iterationen die Anzahl an Schreibzugriffen auf einer Liste übersteigt.
- **Locks:** Die Implementierungen der Interfaces [Lock](#) und [Condition](#) ermöglichen eine größere Flexibilität bei der Verwendung von Sperren und Bedingungen, jedoch auf Kosten einer unhandlicheren Syntax im Vergleich zum Schlüsselwort `synchronized`.
- **Timing:** Die Klasse [TimeUnit](#) bietet unterschiedliche Granularitäten (inkl. Nanosekunden) zum Steuern von Timeout-basierten Vorgängen.
- **Atomics:** Eine Sammlung von Klassen, die eine Thread-sichere Programmierung ohne Sperren für einzelne Variablen unterstützen, findet sich im Package `java.util.concurrent.atomic` – insbesondere für Variablen, die von einem Basisdatentyp

sind. Instanzen der Klasse [AtomicLong](#) o.ä. bieten jeweils Zugriff und Aktualisierungen für eine einzelne Variable des entsprechenden Typs. Mittels [AtomicLong](#) kann im Multithreading-Kontext z.B. die Vergabe einer eindeutigen Sequenznummer realisiert werden, wie das folgende Code-Beispiel demonstriert.

```
1 class Sequencer {
2     private final AtomicLong sequenceNumber = new AtomicLong(0);
3     public long next() {
4         return sequenceNumber.getAndIncrement(); // atomic and thread-safe variant of sequenceNumber++
5     }
6 }
```

- **Synchronizers:** Es werden weitere Klassen für spezielle Synchronisationsidiome angeboten, wie z.B.:
 - Ein [CountDownLatch](#) ist eine sehr einfache, verbreitete Klasse zum Blockieren eines Threads, bis eine bestimmte Anzahl von Signalen, Ereignissen oder Bedingungen erreicht wird.
 - Ein [CyclicBarrier](#) ist eine Klasse zur Synchronisation, mit der eine Gruppe von Threads aufeinander warten kann, bis jeder von ihnen eine gemeinsame Barriere als Synchronisationspunkt erreicht hat. Die Barriere ist zyklisch, da sie nach dem Freigeben der wartenden Threads bei Bedarf erneut verwendet werden kann.
 - Ein [Exchanger](#) ermöglicht, dass zwei Threads Objekte an einem Rendezvous-Punkt austauschen. Dies ist häufig bei Pipelines zur Datenverarbeitung nützlich.

Futures und parallele Streams

Futures

Ein [Runnable](#) hat die Einschränkung, dass seine Methode `run` kein Ergebnis zurückgibt. Falls in einem nebenläufigen Thread ein Ergebnis berechnet werden soll, bietet sich dafür das funktionale Interface [Callable<V>](#) an, das über die Methode `call` ein Ergebnis vom Typ `V` zurückgibt. `Callables` werden häufig im Zusammenhang mit [Futures](#) verwendet. Sowohl `Callables` als auch `Futures` wurden in Java 5 (2004) eingeführt. Ein `Future<V>` ist ein Platzhalter für ein Ergebnis vom Typ `V`, das noch nicht vollständig berechnet worden ist. Es verspricht, das Ergebnis einer nebenläufigen Berechnung zu speichern, sobald diese abgeschlossen ist (engl. *Promise*). Die Berechnung findet typischerweise in einem `Callable` statt, das durch einen [ExecutorService](#) ausgeführt wird. Der Aufrufer der nebenläufigen Berechnung kann auf dem `Future` mittels der Methode `isDone` prüfen, ob die Berechnung bereits abgeschlossen ist, und mittels der Methode `get` blockierend auf das fertige Ergebnis warten. Das folgende Code-Beispiel illustriert das Warten auf ein Future in Zeile 20.

```
1  import java.util.concurrent.Callable;
2  import java.util.concurrent.Future;
3  // ...
4
5  class MyFuture {
6
7      public static void main(String[] args) throws ExecutionException, InterruptedException {
8
9          // task being executed concurrently
10         Callable<Integer> callable = () -> {
11             Thread.sleep(1000);
12             return 1;
13         };
14
15         ExecutorService exec = Executors.newSingleThreadExecutor();
16         Future<Integer> f = exec.submit(callable); // start execution of concurrent task
17
18         System.out.println(f.isDone()); // prints 'false'
19
20         System.out.println(f.get()); // waits until the task is done, then prints '1'
21
22         exec.shutdown();
23     }
24 }
```

In Java 8 (2014) wurden ergänzend [CompletableFuture](#) eingeführt. Ein `CompletableFuture` ist eine Weiterentwicklung eines regulären `Future` – inspiriert durch [ListenableFutures](#) in Google's [Guava](#)-Bibliothek. `CompletableFuture` sollen es ermöglichen, mehrere voneinander abhängige, nebenläufige Berechnungen in einer Verarbeitungskette hintereinander zu schalten. Der Aufrufer kann z.B. formulieren, dass in einem nebenläufigen Thread zunächst Aufgabe A bearbeitet werden soll und anschließend Aufgabe B, die auf das Ergebnis von Aufgabe A als Input angewiesen ist. Es folgt ein einfaches Code-Beispiel zur Veranschaulichung:

```

1  import java.util.concurrent.CompletableFuture;
2  import java.util.function.Supplier;
3  import java.util.function.Function;
4  // ...
5
6  class MyCompletableFuture {
7
8      public static void main(String[] args) throws ExecutionException, InterruptedException {
9
10         // task A being executed concurrently
11         Supplier<Integer> supplier = () -> {
12             try {
13                 Thread.sleep(1000);
14             } catch (InterruptedException e) { }
15             return 1;
16         };
17
18         // task B being executed concurrently, dependent on the result of task A
19         Function<Integer, Integer> plusOne = (x) -> x + 1;
20
21         ExecutorService exec = Executors.newSingleThreadExecutor();
22
23         CompletableFuture<Integer> f = CompletableFuture.supplyAsync(supplier, exec); // start execution of task A
24         System.out.println(f.isDone()); // prints 'false'
25
26         CompletableFuture<Integer> f2 = f.thenApplyAsync(plusOne, exec); // start task B when task A is done >> chaining of tasks
27         System.out.println(f2.get()); // waits until both tasks are done, then prints '2'
28
29         exec.shutdown();
30     }
31 }

```

Parallele Streams

Ein [Stream](#) wird in Java per Default sequentiell in einem Thread ausgeführt. Streams stehen seit Java 8 (2014) zur Verfügung. Im folgenden Code-Beispiel wird eine Liste von 64 Zahlen in einem Stream verarbeitet, um die größte der Zahlen zu ermitteln (Zeile 3).

```

1  List<Integer> numbers = IntStream.rangeClosed(1, 64).boxed().collect(Collectors.toList());
2  Collections.shuffle(numbers);
3
4  int max = numbers.stream().reduce(Integer.MIN_VALUE, Math::max);

```

Durch den Aufruf der Methode `parallel` kann der Stream parallelisiert werden, so dass seine Verarbeitung auf mehrere nebenläufige Threads aufgeteilt wird. Per Default werden so viele Threads genutzt, wie Prozessorkerne gemäß `Runtime.getRuntime().availableProcessors()` zur Verfügung stehen. Für den Entwickler ist die Parallelisierung von Streams damit sehr einfach gestaltet.

```

1  int max = numbers.stream().parallel().reduce(Integer.MIN_VALUE, Math::max);

```

Die grundsätzliche Erwartung ist, dass durch die Nutzung mehrerer parallel arbeitender Threads der parallelisierte Stream schneller fertig wird als der sequentielle Stream. Das lässt sich mit einem ausreichend großen Berechnungsaufwand für den Stream und entsprechender Multicore-Hardware auch demonstrieren ([siehe hier](#)).

Grundsätzlich ist die Annahme, dass ein sequentieller und ein paralleler Stream stets zum gleichen Ergebnis führen. Es gibt aber Ausnahmefälle, die entsprechend dokumentiert sind, z.B. die häufig genutzte Methode [forEach](#), mit der über die Elemente eines Streams iteriert werden kann. Die Methode `forEach` ist wie folgt dokumentiert: *"The behavior of this operation is explicitly nondeterministic. For parallel stream pipelines, this operation does not guarantee to respect the encounter order of the stream, as doing so would sacrifice the benefit of parallelism. For any given element, the action may be performed at whatever time and in whatever thread the library chooses. If the action accesses shared state, it is responsible for providing the required synchronization."* Die folgende Methode `log` gibt die Elemente eines parallelen Streams daher nicht unbedingt in der richtigen Reihenfolge aus.

```

1  void log(Stream<String> stream) { stream.forEach(System.out::println); }

```

Über die Methode `isParallel` kann für einen Stream ermittelt werden, ob es sich um einen sequentiellen oder parallelen Stream handelt. Eine Fallunterscheidung in der Methode `log` sorgt dafür, dass sie sich deterministisch verhält.


```

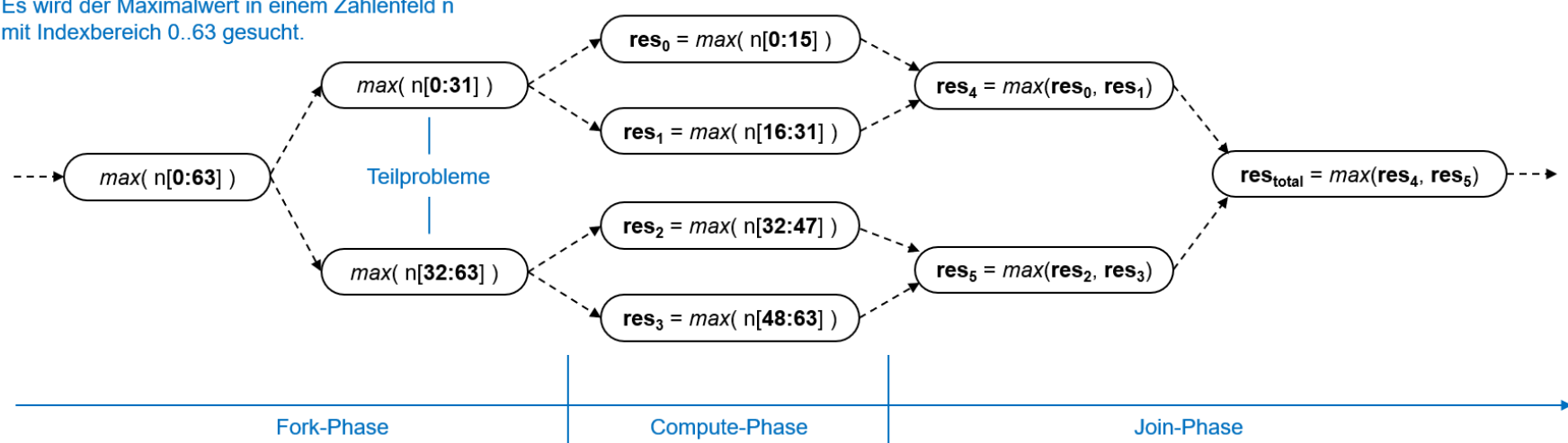
1 void log(Stream<String> stream) {
2     if (!stream.isParallel()) stream.forEach(System.out::println);
3     else stream.forEachOrdered(System.out::println);
4 }

```

Tatsächlich könnte auch in beiden Fällen die Methode `forEachOrdered` genutzt werden, was die obige Fallunterscheidung wieder überflüssig macht.

Die interne Implementierung paralleler Streams basiert auf dem Fork-Join-Framework, das in Java 7 (2011) eingeführt worden ist. Das Fork-Join-Framework agiert nach dem Prinzip des Divide & Conquer-Algorithmus. Die grundlegende Idee besteht dabei darin, dass ein Ausgangsproblem so in Teilprobleme zerlegt wird, dass die später zusammengeführten Ergebnisse der Teilprobleme das Ausgangsproblem lösen. Ein *Fork* ist dabei das Zerlegen in Teilprobleme und ein *Join* das Zusammenfassen der Teilergebnisse zum Gesamtergebnis des Ausgangsproblems. Der Algorithmus lässt sich rekursiv über mehrere Hierarchiestufen anwenden, d.h. dass die Teilprobleme der ersten Stufen sukzessive in noch kleinere, eigene Teilprobleme zerlegt werden können. Die folgende Abbildung visualisiert das Prinzip des Divide & Conquer-Algorithmus anhand des Ausgangsproblems die größte Zahl im Indexbereich 0..63 eines Arrays oder einer Liste zu finden.

Es wird der Maximalwert in einem Zahlenfeld n mit Indexbereich 0..63 gesucht.



Divide & Conquer-Prinzip des Fork-Join-Framework

Beim ersten Fork wird der Indexbereich in die beiden Teilbereiche 0..31 und 32..63 aufgeteilt. Damit sind 2 Teilaufgaben entstanden: (a) Finden der größten Zahl im Indexbereich 0..31 sowie (b) finden der größten Zahl im Indexbereich 32..63. Diese Teilaufgaben können in der Fork-Phase rekursiv in noch kleinere Teilaufgaben zerlegt werden: Finden der größten Zahl in den Indexbereichen 0..15, 16..31, 32..47 und 48..63. Entscheidend ist, dass jede Teilaufgabe unabhängig von den anderen in einem nebenläufigen Thread gelöst werden kann. Die Methode `reduce` (s. oben) führt dazu die Elemente eines Streams paarweise zusammen. In diesem Fall wird der Methode `reduce` dazu die Methode `Math.max` als `BinaryOperator` übergeben, um jeweils die größere von zwei Zahlen zurückzugeben. Angenommen es liegt eine Quadcore-Hardware zugrunde, würden in der Fork-Phase 4 Threads genutzt, die unabhängig voneinander die Teilaufgaben bearbeiten. In der anschließenden Join-Phase werden die Ergebnisse der Teilaufgaben aggregiert, indem sie paarweise als Argumente dem `BinaryOperator` (hier `Math.max`) übergeben werden, der in der Methode `reduce` ausgeführt wird.

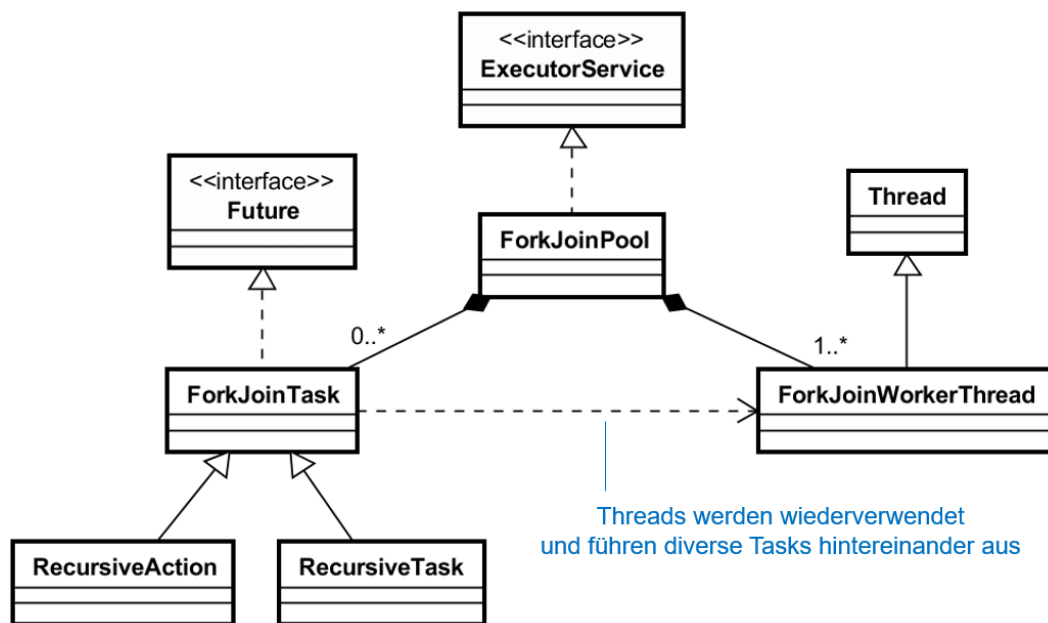
Wenn N die Anzahl der verfügbaren Prozessorkerne ist, wird das Ausgangsproblem tatsächlich nicht nur in N Teilaufgaben zerlegt sondern in etwa $4 \cdot N$. Die Anzahl der erzeugten Teilaufgaben entspricht der Zweierpotenz, die größer als $4 \cdot (N-1)$ ist. Auf einer Quadcore-Hardware werden also 16 Teilaufgaben erzeugt, die nacheinander von 4 Threads bearbeitet werden. Dieses Verhalten lässt sich auch beobachten ([siehe hier](#)). Der per Default genutzte Thread-Pool namens `ForkJoinPool.common` lässt sich bei Bedarf verändern oder austauschen. Über die folgende Anweisung wird z.B. die Anzahl der Threads in diesem Thread-Pool auf 8 erhöht.

```

1 System.setProperty("java.util.concurrent.ForkJoinPool.common.parallelism", "8");

```

Das Fork-Join-Framework unterscheidet also zwischen der Anzahl an Teilaufgaben (= *Tasks*), die abgearbeitet werden müssen, und der Anzahl an Threads, die dazu zur Verfügung stehen. Das folgende UML-Klassendiagramm visualisiert die Beziehungen zwischen den beteiligten Interfaces und Klassen.



Grundlegende Klassen des Fork-Join-Framework

- **Tasks:** [ForkJoinTasks](#) sind abgeschlossene Arbeitseinheiten/Teilprobleme, die nicht auf andere Tasks warten müssen. Sie sind nicht an einen bestimmten Thread zur Ausführung gebunden und implementieren das [Future](#)-Interface, versprechen also i.d.R. ein Ergebnis bestimmten Typs (= [RecursiveTask](#)). Tasks ohne Ergebnis heißen [RecursiveAction](#).
- **Threads:** In einem [ForkJoinPool](#) werden spezielle Threads, nämlich [ForkJoinWorkerThreads](#) verwaltet, die [ForkJoinTasks](#) ausführen. Ein [ForkJoinPool](#) unterscheidet sich von anderen [ExecutorServices](#) durch sogenanntes *Work Stealing*, d.h. alle Threads im Pool versuchen aktiv, in den Pool gesendete Tasks zu ergreifen und auszuführen. Dies ermöglicht eine effiziente Verarbeitung, insbesondere wenn die Tasks ihrerseits neue Tasks erzeugen – dadurch dass ein [ForkJoinTask](#) typischerweise eine Teilaufgabe rekursiv in kleinere Teilaufgaben zerlegt und diese wieder an den Pool sendet.

Das Fork-Join-Framework kann auch manuell ohne parallele Streams eingesetzt werden. Das folgende Code-Beispiel demonstriert wie die Berechnung einer bestimmten Fibonacci-Zahl über eine [RecursiveTask](#) parallelisiert werden kann. Das Beispiel ist aus der Dokumentation der [RecursiveTask](#)-Klasse übernommen. Die Implementierung ist nicht effizient, da die gebildeten Teilaufgaben zu klein sind. Stattdessen ist – wie bei allen Fork-Join-Anwendungen – eine sinnvolle, minimale Granularität zu wählen, ab der eine Aufgabe sequentiell gelöst wird, anstatt sie zu unterteilen. Die Methode [fork](#) in Zeile 9 erzeugt eine Teilaufgabe, die nebenläufig über den [ForkJoinPool](#) abgearbeitet wird. Über die Methode [join](#) in Zeile 11 wird im aufrufenden Thread auf das Ergebnis dieser Teilaufgabe gewartet.

```

1  class FibonacciTask extends RecursiveTask<Integer> {
2      int n;
3      FibonacciTask(int n) { this.n = n; }
4
5      @Override
6      protected Integer compute() {
7          if (n <= 1) return n;
8          FibonacciTask f1 = new FibonacciTask(n - 1);
9          f1.fork(); // fork: execute this task concurrently in the same pool the current task is running in
10         FibonacciTask f2 = new FibonacciTask(n - 2);
11         return f2.compute() + f1.join(); // join: wait for the concurrent task to be done
12     }
13
14     public static void main(String[] args) {
15         System.out.println(new FibonacciTask(7).compute()); // 7-th Fibonacci number is 13
16     }
17 }

```

Eine beliebte Übungsaufgabe ist es, die grundlegenden Sortieralgorithmen [QuickSort](#) und [MergeSort](#), die ebenfalls auf dem Divide & Conquer-Prinzip basieren, über das Fork-Join-Framework zu parallelisieren. Probiere es aus! Vergleiche die Laufzeiten einer sequentiellen und einer parallelisierten Implementierung der Sortieralgorithmen! Hilfreich ist hierbei die Methode [invokeAll](#), die auf einem [ForkJoinTask](#) ausgeführt werden kann, um mit einer Anweisung eine Menge von Teilaufgaben zu erzeugen ([fork](#)) und direkt auf alle Teilergebnisse zu warten ([join](#)).

In 2004 prägte eine wissenschaftliche Veröffentlichung von Google Research maßgeblich den Begriff [MapReduce](#), dem wir insbesondere im Umfeld der Massendatenverarbeitung in verteilten Systemen (z.B. mittels [Hadoop](#) und [Spark](#)) und allgemein in vielen APIs zur funktionalen Programmierung begegnen. MapReduce beschreibt ein Programmiermodell, das ebenfalls auf dem Divide & Conquer-Prinzip aufbaut, und daher dem Fork-Join-Framework sehr ähnlich ist, wobei die *Map*-Phase der Fork-Phase entspricht und die *Reduce*-Phase der Join-Phase. Die Ausführung der parallelen Threads ist bei MapReduce-Anwendungen aber i.d.R. nicht auf eine Maschine beschränkt, sondern verteilt sich auf ein ganzes Cluster von Maschinen.

Reaktive Streams

In Java sind reaktive Streams seit Java 9 (2017) durch die sogenannte [Flow API](#) standardisiert. Es werden hier insbesondere die Interfaces [Publisher](#) und [Subscriber](#) bereitgestellt, über die eine asynchrone Stream-Verarbeitung mit nicht-blockierenden Gegendruck (engl. *Backpressure*) realisiert werden kann. Eine Datenquelle (*Publisher*) informiert seine Beobachter (*Subscriber*) über neue Nachrichten bzw. Zustandsänderungen, wobei sämtliche Publisher und Subscriber in nebenläufigen Threads ausgeführt werden. Der Gegendruck entsteht für einen Subscriber dadurch, dass er auch bei hoher Frequenz neu eingehender Nachrichten vom Publisher, diese verarbeiten können muss. Der max. mögliche Durchsatz zur Nachrichtenverarbeitung auf der Seite des Subscriber muss bestimmt werden und schränkt ggf. ein, wie viele eingehende Nachrichten aus den beobachteten Datenquellen akzeptiert werden können. Ein Code-Beispiel für die Verwendung des [Publisher](#)- und [Subscriber](#)-Interface ist bereits am Ende des Kapitels zum [Entwurfsmuster Beobachter](#) dargestellt worden. Bemerkenswert ist, dass die Flow API einen Konsens darstellt, der im Rahmen der Initiative [Reactive Streams for the JVM](#) entstanden ist. Die Interfaces der Flow API ([Publisher](#), [Subscriber](#), [Subscription](#) und [Processor](#)) werden von verschiedenen beliebten Stream-Bibliotheken wie [RxJava](#), [Akka](#) und [Vert.x](#) implementiert. Dadurch können Datenströme, die auf Basis dieser Bibliotheken entwickelt worden sind, unter Aufrechterhaltung des Gegendrucks miteinander verbunden werden.

Desktop-Anwendungen mit JavaFX

[JavaFX](#) ist ein auf Java basierendes UI-Framework insbesondere für Desktop-Anwendungen. Es kann auch zur Entwicklung mobiler Anwendungen für spezifische eingebettete Systeme und für die sehr verbreiteten Zielplattformen Android und iOS eingesetzt werden. Als Laufzeitumgebung bietet sich für diesen Fall die [Gluon VM](#) an.

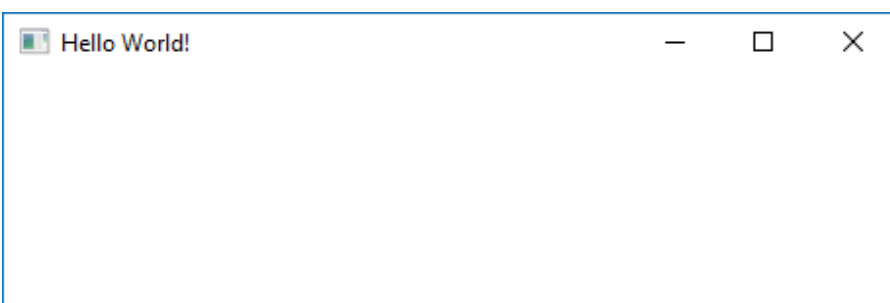
JavaFX war seit Version 7 (2012) bis inkl. Version 10 (2018) Teil der [Java SE](#) und damit das standardmäßig empfohlene UI-Framework. Im Zuge der Modularisierung und Verschlinkung der Java SE ist auch JavaFX seit Java 11 nicht mehr Teil des JDK und muss als separate Bibliothek eingebunden werden. Oracle erläutert diese Entscheidung in einem [Blog-Artikel](#). Die Weiterentwicklung von JavaFX erfolgt heute über die Open Source-Community [OpenJFX](#). Da OpenJFX unter der [GPL](#) veröffentlicht ist, lassen sich mittels OpenJFX auf Basis des [OpenJDK](#) nun freie, nicht durch Lizenzen eingeschränkte Desktop-Anwendungen mit Java konstruieren, was nicht nur für freie Linux-Distributionen erfreulich ist. In der Dokumentation zu OpenJFX finden sich ausführliche und bebilderte [Installationsanleitungen](#) für verschiedene Entwicklungsumgebungen und Build-Management-Tools.

JavaFX stellt eine Vielzahl von grundlegenden Steuerungselementen (sogenannte [UI Controls](#)) zur Interaktion mit dem Anwender bereit. Dazu zählen *Button*, *Text Field*, *Radio Button*, *Checkbox*, *List View*, *Table View*, *Slider*, *Progress Bar* usw. Die Darstellung der Steuerungselemente kann individuell über ein *Look and Feel-Theme* angepasst werden – z.B. mittels [JFoenix](#) an das [Material Design](#) von Google. Zum Einstieg bietet sich der grafische Editor [SceneBuilder](#) an, der es erlaubt das UI per Drag & Drop zusammenzustellen. SceneBuilder lässt sich in geläufige Java-Entwicklungsumgebungen wie IntelliJ oder Eclipse integrieren, um schnell zwischen der grafischen Editoransicht und dem generierten deklarativen Code wechseln zu können. Für JavaFX-Anfänger bietet der SceneBuilder-Editor die Möglichkeit die vorhandenen Steuerungselemente und ihre anpassbaren Eigenschaften explorativ zu erkunden und zu beobachten, welcher Code durch die Zusammenstellung per Drag & Drop entsteht. Weiter unten folgt ein Video, das eine Einführung in den SceneBuilder-Editor bietet.

Trennung von Präsentation und Steuerung einer View

Bevor wir die Motivation hinter der Trennung von Präsentation und Steuerungslogik in einem UI erläutern, klären wir kurz den allgemeinen Begriff "View": Als eine View bezeichnen wir hier den Ausschnitt des gesamten UI, der dem Anwender aktuell angezeigt wird. Das gesamte UI besteht also aus mehreren Views, zwischen deren Anzeige der Anwender i.d.R. durch eigene Interaktion, wie das Anklicken eines Buttons, navigiert. Die möglichen Transitionen zwischen den Views einer Anwendung können als Zustandsautomat modelliert werden. Je nach UI-Framework werden die Views unterschiedlich bezeichnet: In der Web-Entwicklung wird klassischerweise von Seiten (engl. *Pages*) oder in neueren Web-Frameworks wie [Angular](#), [Vue.js](#) oder [React](#) generisch von Komponenten (engl. *Components*) gesprochen. Im Android SDK navigiert der Anwender zwischen unterschiedlichen [Activities](#) (oder ihren Fragmenten). In JavaFX wurde zur Bezeichnung der Views eine Analogie zur Theaterbühne (engl. *Stage*) gewählt, auf der mehrere Szenen (engl. *Scenes*) mit unterschiedlichem Bühnenbild ablaufen. Das Fenster, in dem eine Anwendung dargestellt wird, ist dabei die [Stage](#). Der Anwender kann durch Interaktion zwischen unterschiedlichen [Scenes](#) wechseln.

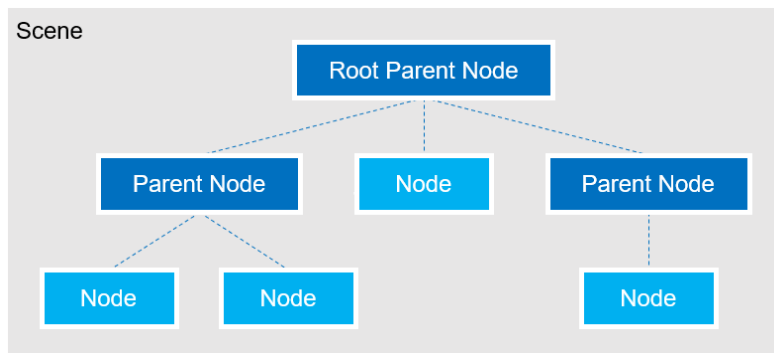
```
1  import javafx.application.Application;
2  import javafx.stage.Stage;
3
4  public class HelloWorldApp extends Application {
5
6      public static void main(String[] args) {
7          launch(args);
8      }
9
10     public void start(Stage stage) {
11         stage.setTitle("Hello World!");
12         stage.show();
13     }
14 }
```



Minimale JavaFX-Anwendung mit leerer Stage

Der Inhalt einer *Scene* besteht in JavaFX aus mehreren Knoten (die entsprechende Klasse heißt [Node](#)), die in einer Baumstruktur ineinander verschachtelt sind. Diese Baumstruktur wird in JavaFX als *Scene Graph* bezeichnet und entspricht konzeptionell dem *Document Object Model (DOM)* in der Web-Entwicklung. Container-Knoten, die mehrere andere Knoten als Kindelemente aufnehmen können, werden als [Parent](#) bezeichnet. Jede *Scene* hat einen Wurzelknoten, den *Root Parent Node*. Eine beliebig tief geschachtelte Baumstruktur entsteht dadurch, dass ein Container-Knoten auch andere Container-Knoten als Kindelemente

aufnehmen kann. Grafische Animationen, wie z.B. Verschiebungen, Rotationen und Skalierungen oder andere Effekte, die auf einen **Parent** angewendet werden, gelten auch für ihre untergeordneten Kindelemente. Konkrete Knoten, d.h. die Unterklassen der allgemeinen Klasse **Node** sind sämtliche Klassen für Steuerungselemente, Bilder oder Texte, die im UI angezeigt werden. Die folgende Abbildung und das folgende Code-Beispiel verdeutlichen den Aufbau eines *Scene Graph* in JavaFX.



Scene Graph in JavaFX

```
1  HBox rootNode = new HBox(); // HBox is a concrete Parent subclass
2  Scene scene = new Scene(rootNode);
3
4  Node node = new Button("Button 1"); // Button is a concrete Node subclass
5  rootNode.getChildren().add(node);
6
7  stage.setScene(scene);
8  stage.show();
```

Wie bei den anderen oben genannten UI-Frameworks aus dem Web- und Mobile-Bereich hat es sich auch in JavaFX bewährt, die Präsentation einer View von ihrer Steuerungslogik zu trennen. In nahezu alle modernen UI-Frameworks wird aktuell der Ansatz verfolgt, die grundlegende Präsentation einer View als Baumstruktur deklarativ in XML oder HTML zu formulieren und die zugehörige Steuerungslogik in einer imperativen Programmiersprache wie Java. Vor- und Nachteile dieses Ansatzes sollen kurz zusammengefasst werden:

- **Deklarative Programmierung der Präsentation:** In einer separaten Ressourcen-Datei wird die grundlegende Struktur einer View in einer deklarativen Auszeichnungssprache wie XML oder HTML gespeichert. Bei JavaFX ist dies [FXML](#). Derartige Auszeichnungssprachen sind sehr gut geeignet, um Baumstrukturen übersichtlich abzubilden. Das Framework ist in der Lage eine valide Ressourcen-Datei zu verarbeiten und für alle Knoten der Baumstruktur ein entsprechendes **Node**-Objekt zu erzeugen. Die einzelnen Einträge in Listen und Tabellen können zur Entwurfszeit natürlich nicht deklarativ angelegt werden, da sie erst dynamisch zur Laufzeit der Anwendung über einen Webservice o.ä. geladen werden.
- **Imperative Programmierung der Steuerungslogik:** Die Steuerung der View wird in einer imperativen Programmiersprache wie Java implementiert und beinhaltet insbesondere die Verarbeitung von sämtlichen Ereignissen, die zur Laufzeit auftreten können – wie z.B. die Navigation zur nächsten View nach einem Klickereignis und das damit verbundene dynamische Nachladen von Daten. Würde zusätzlich die grundlegende Struktur beim initialen Laden der Views imperativ zusammengestellt werden, entsteht i.d.R. sehr viel unleserlicher Code zum Aufbau der UI-Elemente, zum Setzen ihrer Eigenschaften und zum Positionieren im jeweiligen Container-Element (= Vaterknoten in XML). Änderungen an diesem Code sind aufwändig, da auch für kleine Verschiebungen von UI-Elementen viel Code bewegt wird. In der Versionsverwaltung sind die Effekte der Änderungen schwer nachzuvollziehen. Wer einen Blick auf diesen Code wirft, wird schnell erkennen, dass er sich sehr gut zur automatisierten Generierung durch ein Framework anbietet.

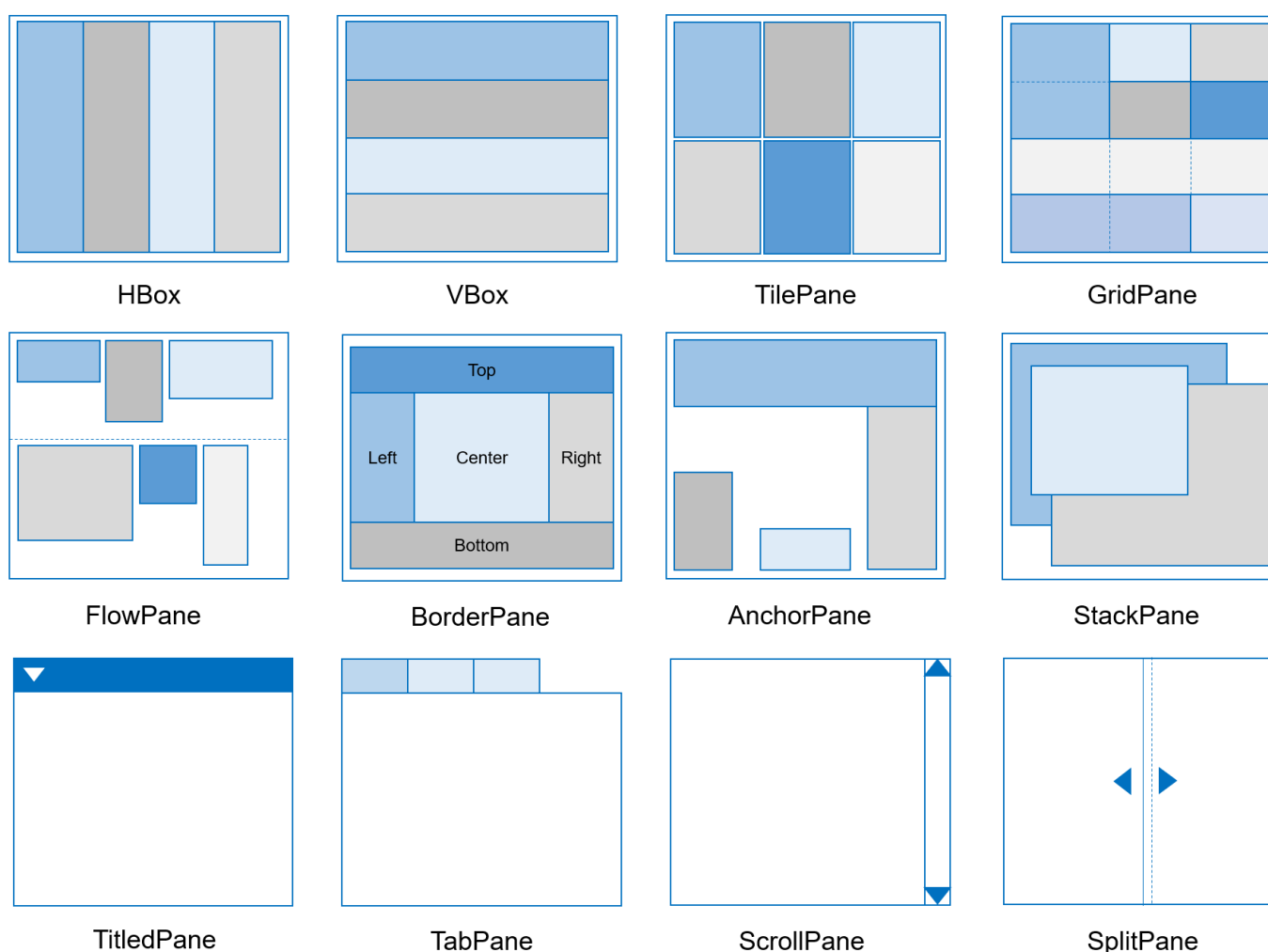
Ein weiterer Vorteil der klaren Trennung von Präsentation und Steuerung einer View in der Anwendungsentwicklung ist, dass die Rollenaufteilung zwischen spezialisierten Teammitgliedern vereinfacht wird: UI-Designer sind i.d.R. den Umgang mit deklarativen Auszeichnungssprachen wie HTML gewohnt und müssen sich nicht mehr mit der imperativen Programmiersprache auseinandersetzen. Im [MVC-Kapitel](#) ist bereits vorgestellt worden, wie das MVC-Muster zum MVVM-Muster ausgebaut werden kann, um die Steuerungslogik der View (= *ViewModel* in MVVM) unabhängig von einer konkreten Präsentation (= *View* in MVVM) zu gestalten. Die Präsentation kann dann relativ einfach ausgetauscht werden, ohne die Steuerungslogik der View anpassen zu müssen – z.B. von FXML zu HTML oder umgekehrt. Im Kapitel [MVC und MVVM in JavaFX](#) ist bereits dargestellt worden, wie eine einfache View inkl. Controller bzw. ViewModel in JavaFX aufgebaut werden kann.

Layout Panes

Ein [Pane](#) ist in JavaFX ein bestimmter Container-Knoten, in dem mehrere andere Knoten vereint und nach einem bestimmten Layout-Prinzip im UI angeordnet werden. Die wichtigsten der vorgefertigten [Layout Panes](#) sollen im Folgenden kurz erläutert werden.

- **HBox:** Die Kindelemente werden horizontal nebeneinander dargestellt. Es kann ein Abstand zwischen den Kindelementen angegeben werden.
- **VBox:** Die Kindelemente werden entsprechend vertikal untereinander dargestellt.

- **TilePane:** Die Kindelemente werden auf gleich großen Kacheln dargestellt. Die Anzahl der gewünschten Zeilen und Spalten kann angegeben werden.
- **GridPane:** Die Kindelemente werden auf einem Gitter dargestellt, das aus einer definierten Anzahl an Zeilen und Spalten besteht. Kindelemente können sich über mehrere Zeilen und/oder Spalten des Gitters erstrecken.
- **FlowPane:** Die Kindelemente werden horizontal oder vertikal hintereinander dargestellt und können unterschiedlich breit und hoch sein. Wenn die Breite des Containers (bei horizontaler Flussrichtung) bzw. die Höhe des Containers (bei vertikaler Flussrichtung) ausgeschöpft ist, wird der Fluss der Kindelemente in der nächsten Zeile bzw. Spalte fortgesetzt.
- **BorderPane:** Die Kindelemente können in 5 definierten Regionen (*Center, Top, Left, Bottom, Right*) des Containers positioniert werden. Die Regionen dürfen auch leer bleiben.
- **AnchorPane:** Die Kindelemente können an den Kanten und den Ecken des Containers mit fixen Koordinaten ausgerichtet werden.
- **StackPane:** Die Kindelemente können gezielt übereinander gestapelt werden, wenn sie sich überlappen sollen, um Tiefe als dritte Dimension anzudeuten.
- **TitledPane:** Der Container besteht aus einem Titel- und einem Inhaltsbereich. Der Inhaltsbereich kann mit einem Klick auf den Titel zugeklappt werden.
- **TabPane:** Der Container besteht aus mehreren Tabs, von denen jeweils nur eines sichtbar ist.
- **ScrollPane:** Der Container ist horizontal oder vertikal scrollbar und kapselt i.d.R. einen weiteren Container.
- **SplitPane:** Der Container ist horizontal oder vertikal in mehrere Bereiche unterteilt, deren Breite bzw. Höhe der Anwender ggf. interaktiv verändern kann.



Layout Panes in JavaFX

Navigation zwischen Views

Die verschiedenen Views einer Anwendung teilen sich häufig bestimmte inhaltliche Bereiche wie einen Header, einen Footer oder eine Sidebar. Das Navigationsmenu selbst ist i.d.R. in einem dieser Bereiche untergebracht. Es ergeben sich zwei alternative Ansätze, um die Navigation zwischen den Views einer Anwendung zu realisieren, ohne den Code für die geteilten Bereiche zu duplizieren:

- **Multiple scenes:** Alle Views sind strukturell gleich aufgebaut und inkludieren die geteilten Bereiche. Im folgenden Code-Beispiel werden der Header und die Sidebar aus separaten Dateien eingebunden. Weitere Views der Anwendung sind gleich aufgebaut wie die folgend dargestellte FXML-Datei `myFirstView.fxml` und unterscheiden sich nur im inhaltlichen Bereich. Nachteil: Wenn die allgemeine Struktur angepasst werden muss (z.B. Verschieben der Sidebar von links nach rechts), müssen alle Views einzeln editiert werden.


```

1 <BorderPane fx:id="myFirstView">
2   <top>
3     <fx:include source="header.fxml"></fx:include>
4   </top>
5   <left>
6     <fx:include source="sidebar.fxml"></fx:include>
7   </left>
8   <center>
9     <!-- specific content of this view -->
10  </center>
11 </BorderPane>

```

Vorteil in JavaFX: Jede View kann über einen eindeutigen Dateinamen in einer neuen Szene geladen werden.

```

1 Scene scene = new Scene(FXMLLoader.load(getClass().getResource("myFirstView.fxml")));
2 stage.setScene(scene);

```

- **Single scene, dynamic content:** Es wird eine FXML-Datei (z.B. namens `main.fxml`) angelegt, die den strukturellen Rahmen inkl. Header und Sidebar für alle Views definiert.

```

1 <BorderPane fx:id="main" fx:controller="MainController">
2   <top>
3     <fx:include source="header.fxml"></fx:include>
4   </top>
5   <left>
6     <fx:include source="sidebar.fxml"></fx:include>
7   </left>
8   <center>
9     <StackPane fx:id="viewHolder">
10      <!-- specific content of the active view is loaded here -->
11    </StackPane>
12  </center>
13 </BorderPane>

```

Diese Datei wird initial in einer Szene geladen. Anstatt zur Laufzeit zu neuen Szenen zu wechseln, wird dynamisch der spezifische Inhalt einer View in ein dafür vorgesehenes *Layout Pane* (hier `StackPane viewHolder`) geladen.

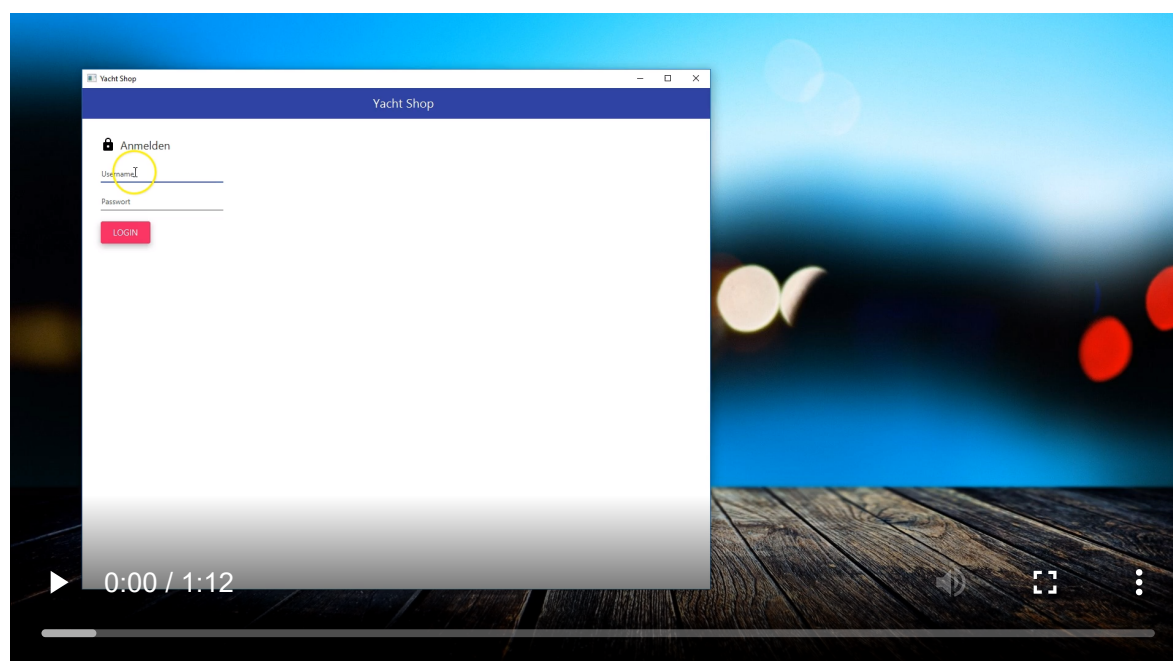
```

1 public class MainController {
2
3   @FXML
4   private StackPane viewHolder;
5
6   public void changeView(String fxmlFilename) {
7     Node view = FXMLLoader.load(getClass().getResource(fxmlFilename));
8     viewHolder.getChildren().setAll(view); // clears the list of child elements and adds the view as a new child element
9   }
10 }

```

Beispielanwendung "Yacht Shop"

In folgendem Video wird eine JavaFX-Beispielanwendung vorgestellt, die mit der im [Spring-Kapitel](#) erstellten REST-API kommuniziert. Spezifische JavaFX-UI-Elemente werden im Video hervorgehoben. Der vollständige Code der Beispielanwendung findet sich im Verzeichnis `/ui/javafx-shop` des Modul-Repository.



JavaFX-Beispielanwendung "Yacht Shop"

Die Anwendung folgt dem Ansatz "Single scene, dynamic content" und lädt dynamisch 3 unterschiedliche Views in Form von FXML-Dateien in den Inhaltsbereich eines *ViewHolder-Pane*. Die 3 Views bilden folgende Anwendungsfälle ab:

- `login.fxml`
 - Anmelden als registrierter User
- `catalog.fxml`
 - Einsehen des Produktkatalogs
 - Buchen eines Produkts
- `bookings.fxml`
 - Einsehen aller Buchungen des angemeldeten Users in einer Tabelle
 - Löschen einer Buchung des angemeldeten Users

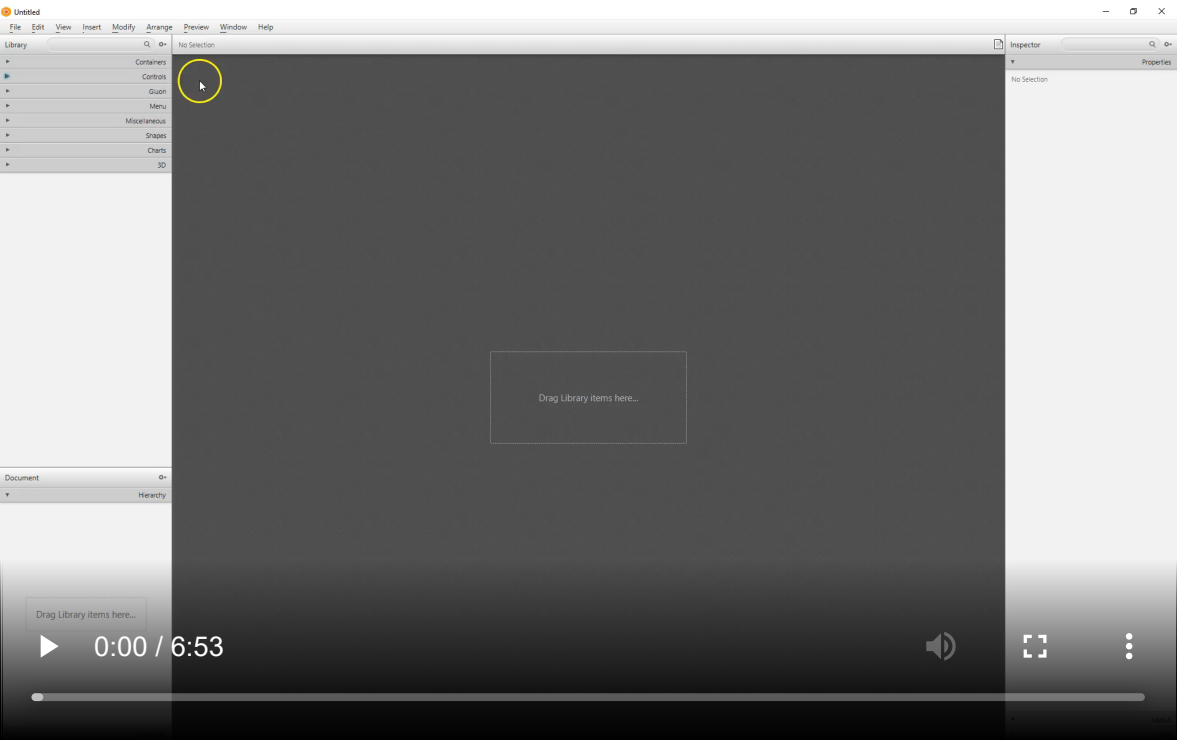
Zusätzlich besteht die Anwendung noch aus 2 weiteren FXML-Dateien:

- `main.fxml`: Enthält die gemeinsame Rahmenstruktur aller inhaltlichen Views, insbesondere Header und Navigationsmenu.
- `menu.fxml`: Enthält das Navigationsmenu, das in einem *MenuDrawer* dargestellt wird. Über das Menu kann zwischen den Views gewechselt werden.

Im Folgenden wird der Code auszugsweise und teilweise vereinfacht anhand ausgewählter Aspekte erläutert.

FXML, Controller und CSS in JavaFX

Das folgende Video zeigt, wie das Anmeldeformular einer Anwendung mittels des SceneBuilder-Editors ohne Programmierkenntnisse entworfen werden kann. Als Container werden im Video exemplarisch nur eine `VBox` und eine `HBox` verwendet, die verschiedene UI-Steuerungselemente aufnehmen und ineinander geschachtelt werden.



FXML-Entwurf im SceneBuilder-Editor

Die Ausgabe des Entwurfsvorgangs im SceneBuilder ist eine FXML-Datei, hier namens `login.fxml`, die noch nicht an einen Controller gebunden ist. Der Login-Button hat dadurch noch keine Funktionalität. In dem folgenden Code-Beispiel wird dem Anmeldeformular durch die Referenz auf die Klasse `LoginController` in Zeile 1 ein Controller zugeordnet, der automatisch durch das JavaFX-Framework instantiiert wird. Die Methode `initialize` versehen mit der Annotation `@FXML` (Zeilen 12-15 in `LoginController`) wird beim Erzeugen des Controllers durch das Framework ausgeführt. In Zeile 1 der FXML-Datei wird zusätzlich die CSS-Datei `app.css` eingebunden, so dass alle UI-Elemente die dort definierten Stylesheet-Klassen über das Attribut `styleClass` adressieren können.

FXML (login.fxml)

```
1  <VBox xmlns="http://javafx.com/javafx/10.0.1" xmlns:fx="http://javafx.com/fxml/1" fx:controller="shop.controller.LoginController"
2  stylesheets="@app.css">
3      <HBox>
4          <MaterialIconView glyphName="LOCK"></MaterialIconView>
5          <Label>Anmelden</Label>
6      </HBox>
7      <JFXTextField fx:id="username" promptText="Username" styleClass="input"></JFXTextField>
8      <JFXPasswordField fx:id="password" promptText="Passwort" styleClass="input"></JFXPasswordField>
9      <JFXButton onAction="#loginAction" text="LOGIN" styleClass="button-raised"></JFXButton>
10     <Label fx:id="loginFailure" visible="false">Login fehlgeschlagen!</Label>
    </VBox>
```

Controller (LoginController.java)

```
1 public class LoginController {
2
3     @FXML
4     private TextField username;
5
6     @FXML
7     private PasswordField password;
8
9     @FXML
10    private Label loginFailure;
11
12    @FXML
13    public void initialize() {
14        // executed when the controller object is created by the framework
15    }
16
17    public void loginAction(ActionEvent e1) {
18        // execute login request in async task
19        PostLoginTask loginTask = new PostLoginTask(username.getText(), password.getText());
20        new Thread(loginTask).start();
21
22        // login task response handler
23        loginTask.setOnSucceeded((WorkerStateEvent e2) -> {
24            // login is successfull if the return value is not null
25            if (loginTask.getValue() == null) {
26                loginFailure.setVisible(true);
27                return;
28            }
29            // navigate to catalog view
30            MainController.getInstance().changeView("catalog");
31        });
32    }
33 }
```

CSS (app.css)

```
1 .button-raised {
2     -jfx-button-type: RAISED;
3     -fx-background-color: deeppink;
4     -fx-text-fill: white;
5     -fx-font-size: 14;
6     -fx-padding: 8 20;
7 }
8
9 .input {
10     -fx-max-width: 200;
11 }
12
13 #loginFailure {
14     -fx-text-fill: red;
15     -fx-font-size: 16;
16 }
```

Die UI-Elemente aus der FXML-Datei können an Attribute in der Controller-Klasse gebunden werden (*Two-Way-Binding*). Auch diese Attribute werden per Dependency Injection durch das Framework instantiiert. Dazu muss ...

- der Bezeichner des Attributs in der Java-Klasse gleich dem Wert des Attributs `fx:id` in der FXML-Datei sein,
- der Datentyp des Attributs in der Java-Klasse gleich dem Datentyp in der FXML-Datei sein
- und das Attribut in der Java-Klasse mit der Annotation `@FXML` versehen sein.

Die Methode `loginAction` (Zeilen 17-32 in `LoginController`) ist durch das Attribut `onAction="#loginAction"` in der FXML-Deklaration des Login-Buttons als *EventHandler* an diesen Button gebunden. Die Implementierung dieser Methode, insbesondere die Klasse `PostLoginTask`, wird weiter unten genauer erläutert.

Das Styling der UI-Elemente erfolgt in JavaFX wie aus der Web-Entwicklung bekannt über [Cascading Style Sheets \(CSS\)](#). Es ist demzufolge zu empfehlen, dass eine FXML-Datei nur die strukturelle Anordnung der UI-Elemente definiert und deren Styling über wiederverwendbare Stylesheet-Klassen in eine separate CSS-Datei ausgelagert wird. Eine Styling-Klasse kann über das Attribut `styleClass` adressiert werden. Über das Attribut `style` können die CSS-Properties eines UI-Elements genau wie in HTML direkt verändert werden. Eine Referenz über alle verfügbaren [CSS-Properties in JavaFX](#) findet sich in der Dokumentation von Oracle.

Das folgende Code-Beispiel zeigt exemplarisch, wie ein Button alternativ aus dem imperativen Java-Code heraus einer View hinzugefügt werden kann. Das ist insbesondere für die Kindelemente von Listen ([ListView](#)) und Tabellen ([TableView](#)) sinnvoll, die zur Laufzeit dynamisch angelegt werden müssen, weil ihre Ausprägungen zur Entwurfszeit noch nicht bekannt sind.

```

1  class LoginController {
2
3      @FXML
4      private Pane container;
5
6      @FXML
7      protected void initialize() {
8          Button loginButton = new Button("LOGIN");
9          container.getChildren().add(loginButton); // add button to a parent container
10
11         // register button event handler
12         loginButton.setOnAction((e) -> {
13             // process login action here ...
14         });
15
16         // button styling
17         loginButton.getStyleClass().add("button-raised");
18         loginButton.setStyle("-fx-background-color: deeppink");
19     }
20 }

```

- In Zeile 9 wird der neue Button dem in der zugehörigen FXML-Datei deklarierten *UI-Pane* `container` hinzugefügt.
- In den Zeilen 12-14 wird ein anonymer EventHandler für den Button registriert.
- In den Zeilen 17-18 wird das Styling des Buttons mittels CSS angepasst.

Asynchrone Tasks in einer JavaFX-Anwendung

In einer JavaFX-Anwendung wird das UI aus dem sogenannten *JavaFX Application Thread* heraus gesteuert. Nur aus diesem Thread heraus darf das UI zur Laufzeit manipuliert werden, z.B. durch das Hinzufügen von Knoten zum *Scene Graph*. Es können in diesem Thread also nur nicht-blockierende EventHandler ausgeführt werden, damit sichergestellt ist, dass das UI stets auf Eingaben des Anwenders reagieren kann. Blockierende Aufrufe (wie z.B. das Senden eines HTTP-Request an eine REST-API) hingegen sind in nebenläufige Threads zu verschieben. Aus Sicht des *JavaFX Application Thread* wird in diesem Fall eine asynchrone Aufgabe (engl. *Task*) gestartet, deren Ergebnis – sobald es verfügbar ist – über eine Callback-Funktion verarbeitet wird, um das UI ggf. zu aktualisieren. Es bietet sich für derartige asynchrone Aufgaben in Java an, die Klasse [Task](#) zu erweitern. Diese Klasse implementiert die Interfaces `Future` und `Runnable` (s. Kapitel [Futures und parallele Streams](#)). Das folgende Code-Beispiel zeigt, wie das Senden eines HTTP-Request an eine REST-API und die Verarbeitung der zugehörigen HTTP-Response in eine spezielle, nebenläufig ausgeführte Task ausgelagert wird. In diesem Fall wird der HTTP-Request durch das Klicken auf den Login-Button im *JavaFX Application Thread* ausgelöst. Serverseitig soll geprüft werden, ob der Anwender eine gültige Kombination aus Username und Passwort eingegeben hat, und der Loginversuch damit erfolgreich war oder eben nicht.

JavaFX Application Thread

```

1  // the following code is executed in the event handler of the login button
2
3  PostLoginTask loginTask = new PostLoginTask(username, password);
4  new Thread(loginTask).start();
5
6  // login task response handler
7  loginTask.setOnSucceeded((WorkerStateEvent e) -> {
8      User user = loginTask.getValue();
9      if (user != null) {
10         // login is successfull
11     }
12     else {
13         // login is not successfull
14     }
15 });

```

Nebenläufiger Task

```
1 import javafx.concurrent.Task;
2 import com.mashape.unirest.http.Unirest;
3 import com.mashape.unirest.http.HttpResponse;
4 // ...
5
6 public class PostLoginTask extends Task<User> {
7
8     private String username;
9     private String password;
10
11     public PostLoginTask(String username, String password) { this.username = username; this.password = password; }
12
13     @Override
14     protected User call() {
15         String url = "https://example.com/login";
16         String json = "{\"name\": \"" + username + "\", \"passwordHash\": \"" + password + "\"}";
17
18         try {
19             HttpResponse<String> res = Unirest.post(url).header("Content-Type", "application/json").body(json).asString();
20             if (res.getStatus() != 403) {
21                 String jsonWebToken = res.getHeaders().getFirst("Authorization");
22                 return new User(username, jsonWebToken);
23             }
24         } catch (UnirestException e) {
25             e.printStackTrace();
26         }
27         return null;
28     }
29 }
```

- In Zeile 4 des *JavaFX Application Thread* wird der Task nebenläufig gestartet. In Zeile 7 wird mittels der Methode `setOnSucceeded` ein EventHandler registriert, der ausgeführt wird, sobald der Task abgeschlossen ist. In diesem EventHandler kann über die Methode `getValue` auf das Ergebnis des Task zugegriffen werden.
- Die Klasse `Task<V>` ist eine generische Klasse, für die ein bestimmter Rückgabety `V` für die Methode `call` spezifiziert werden kann. Für die Klasse `PostLoginTask` ist dies im vorliegenden Beispiel die Datenstruktur-Klasse `User` (vgl. Zeilen 6 und 14).
- In Zeile 19 des Task wird ein blockierender HTTP-Request gesendet, der ggf. eine Exception wirft, wenn z.B. der Server nicht zu erreichen ist. Insbesondere wegen dieser Zeile wird überhaupt ein asynchroner Task angelegt. In den Zeilen 20-23 wird die HTTP-Response verarbeitet.
- Zum Senden des HTTP-Request wird hier exemplarisch die Bibliothek [Unirest](#) verwendet. Diese würde alternativ auch das Senden eines nicht-blockierenden HTTP-Request ermöglichen, wenn am Ende von Zeile 19 anstatt der Methode `asString` die Methode `asStringAsync` aufgerufen wird. Die Rückgabe wäre dann vom Typ `Future<HttpResponse<String>>`. Dadurch würde sich das Auslagern des blockierenden Aufrufs aus dem eigenen Code in den internen Code der Bibliothek verlagern. Ein Task wäre nicht mehr unbedingt erforderlich, falls die Vorbereitung des HTTP-Request und die Verarbeitung der HTTP-Response als nicht sonderlich aufwändig eingestuft werden und es tolerierbar ist, diese im *JavaFX Application Thread* auszuführen.

Animationen in JavaFX

In einem Spiel bewegen sich häufig unterschiedliche Objekte über den Bildschirm, die miteinander im Rahmen der Spielregeln interagieren. In JavaFX wird zur Animation von Objekten die abstrakte Klasse [AnimationTimer](#) bereitgestellt. Mit dieser Klasse kann ein Timer erstellt werden, der in jedem Frame aufgerufen wird, während er aktiv ist. Die Frame-Rate ist variabel und hängt von der zugrundeliegenden Ausführungsumgebung und Hardware ab. Grundsätzlich ist es zu empfehlen, die Frame-Rate, in der das UI aktualisiert wird, und die Spielphysik voneinander zu entkoppeln (siehe Diskussion [hier](#)). Zur Verwendung eines konkreten `AnimationTimer` muss die Methode `handle` sinnvoll überschrieben werden. Die Methoden `start` und `stop` erlauben das Starten bzw. Stoppen des Timers. Das folgende Code-Beispiel erzeugt eine Anwendung, in der ein animierter Ball mit fixer Geschwindigkeit an den Kanten des Anwendungsfensters abprallt (siehe [SingleBouncingBall.java](#)).

SingleBouncingBallApp

```
1  import javafx.application.Application;
2  import javafx.animation.AnimationTimer;
3  // ...
4
5  public class SingleBouncingBall extends Application {
6
7      @Override
8      public void start(Stage stage) {
9
10         final Pane ballContainer = new Pane();
11         final Scene scene = new Scene(ballContainer, 600, 400);
12         final Ball ball = new Ball(300, 200, 25, Color.DARKGRAY, 300, Math.toRadians(45));
13         ballContainer.getChildren().add(ball);
14
15         stage.setScene(scene);
16         stage.show();
17
18         final LongProperty lastUpdateTime = new SimpleLongProperty(0);
19         final AnimationTimer timer = new AnimationTimer() {
20             @Override
21             public void handle(long now) {
22                 if (lastUpdateTime.get() > 0) {
23
24                     // change the ball position
25                     double elapsedSeconds = (now - lastUpdateTime.get()) / 1_000_000_000.;
26                     ball.setCenterX(ball.getCenterX() + elapsedSeconds * ball.velocity * Math.cos(ball.angle));
27                     ball.setCenterY(ball.getCenterY() + elapsedSeconds * ball.velocity * Math.sin(ball.angle));
28
29                     // bounce against a vertical wall
30                     if (ball.getCenterX() <= ball.getRadius()
31                         || ball.getCenterX() >= (ballContainer.getWidth() - ball.getRadius())) {
32                         ball.angle = Math.PI - ball.angle;
33                     }
34                     // bounce against a horizontal wall
35                     if (ball.getCenterY() <= ball.getRadius()
36                         || ball.getCenterY() >= (ballContainer.getHeight() - ball.getRadius())) {
37                         ball.angle = 2 * Math.PI - ball.angle;
38                     }
39                 }
40                 lastUpdateTime.set(now);
41             }
42         };
43         timer.start();
44     }
45
46     public static void main(String[] args) { launch(); }
47 }
```

Ball

```
1  import javafx.scene.shape.Circle;
2
3  class Ball extends Circle {
4      double velocity;
5      double angle;
6
7      Ball(double centerX, double centerY, double radius, Paint fill, double velocity, double angle) {
8          super(centerX, centerY, radius, fill);
9          this.velocity = velocity;
10         this.angle = angle;
11     }
12 }
```

Die Klasse `Ball` erweitert die Klasse `javafx.scene.shape.Circle`. Ein `Circle` ist ein spezieller `Shape`, der als `Node` dem *Scene Graph* hinzugefügt werden kann. Weitere Shapes sind `Rectangle`, `Polygon`, `Text`, `SVGPath`, uvm. Alternativ zur Verwendung von Shapes können animierte Objekte in JavaFX auch auf einer `Canvas` über deren `GraphicsContext` gezeichnet werden. Die beiden folgenden Code-Beispiele visualisieren jeweils den Text "Hello World!" und einen roten Kreis – im ersten Code-Beispiel werden dazu Shapes angelegt, im zweiten Code-Beispiel wird auf eine Canvas gezeichnet.


```

1 // display text and circle as shapes
2 Pane shapesContainer = new Pane();
3
4 Shape text = new Text(20, 20, "Hello World!"); // x = 20, y = 20
5 shapesContainer.getChildren().add(text);
6
7 Shape circle = new Circle(100, 100, 30, Color.RED); // centerX = 100, centerY = 100, radius = 30
8 shapesContainer.getChildren().add(circle);

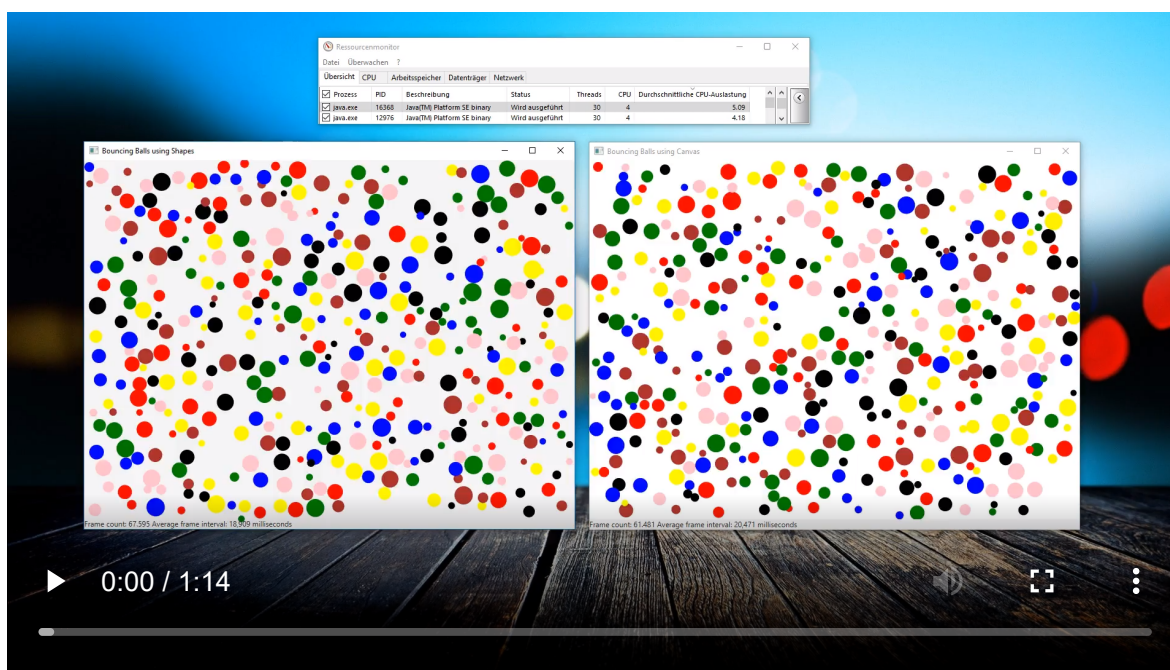
```

```

1 // draw text and circle on canvas
2 Canvas canvas = new Canvas(300, 200); // canvas is 300 x 200 pixels
3 GraphicsContext gc = canvas.getGraphicsContext2D();
4
5 gc.fillText("Hello World!", 20, 20); // x = 20, y = 20
6
7 gc.setFill(Color.RED);
8 gc.fillOval(100, 100, 60, 60); // x = 100, y = 100, width = 60, height = 60 => radius = 30

```

Die alternativen Implementierungen in den Klassen [BouncingBallsShapes.java](#) und [BouncingBallsCanvas.java](#) lassen erkennen, dass im Vergleich von Shapes- und Canvas-Animationen sich keine signifikanten Unterschiede hinsichtlich der Frame-Rate und des Hauptspeicherbedarfs ergeben, aber die Shapes-Animationen eine etwas höhere CPU-Auslastung verursachen. Das folgende Video demonstriert diese Beobachtungen.



Animationen in JavaFX mittels Shapes und Canvas

Simulation der Spielphysik

Die Spielphysik kann – wie in den obigen Beispielen – durch stark vereinfachte Annahmen bei der [Kollision von Objekten](#) eigenständig implementiert werden. Wenn in der Spielphysik allerdings auch spezielle Effekte und Kräfte wie Reibung, Dichte oder Rotationsgeschwindigkeit berücksichtigt werden sollen, bietet sich die Verwendung einer Physik-Engine wie z.B. [dyn4j](#) an. Die Physik-Engine ist unabhängig von JavaFX und kann gleichermaßen mit jedem anderen UI-Framework verwendet werden.

In dyn4j wird eine 2D-Welt simuliert. Die Welt (Klasse [World](#)) kann diverse Körper (Klasse [Body](#)) enthalten, die ihrerseits aus mehreren miteinander verbundenen Körperteilen (Klasse [BodyFixture](#)) bestehen (s. [Getting Started zu dyn4j](#)). Auf die Körper können Kräfte und Impulse wirken. Das folgende Code-Beispiel zeigt wie einer dyn4j-Welt ein rechteckiger und ein runder Körper hinzugefügt werden.

```

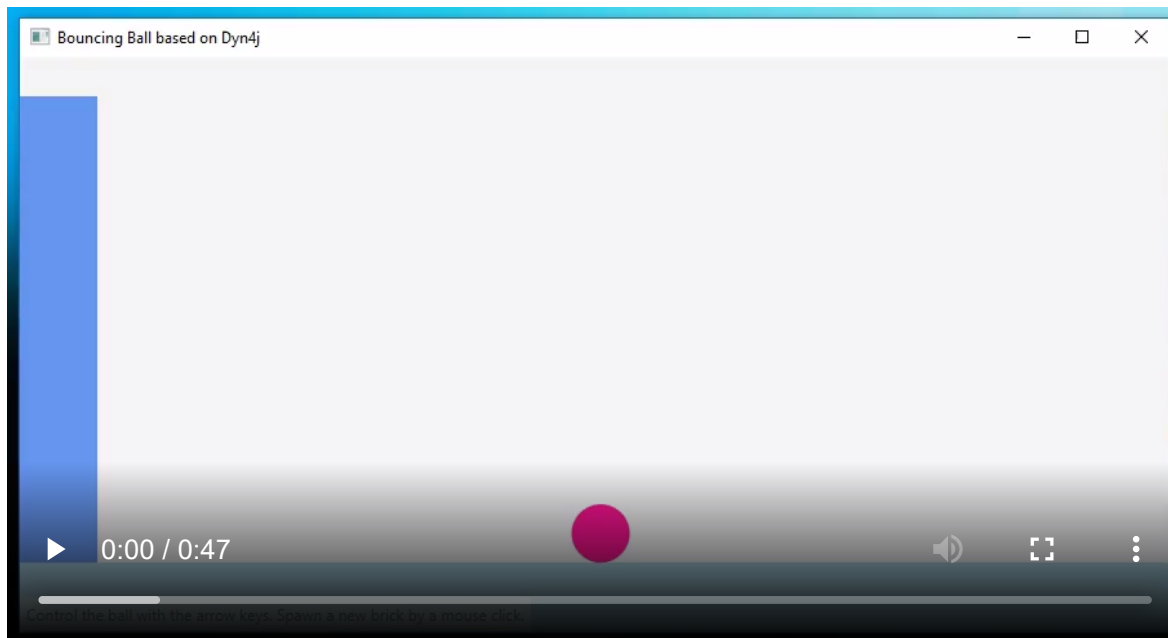
1 World world = new World();
2
3 Body rect = new Body();
4 rect.addFixture(Geometry.createRectangle(width, height)); // 1st body fixture is a rectangle
5 rect.setMass(MassType.INFINITY);
6 body.translate(0, 0); // sets the rectangle's center (x, y)
7 world.addBody(rectangle);
8
9 Body circle = new Body();
10 circle.addFixture(Geometry.createCircle(radius), density, friction, restitution);
11 circle.setMass(MassType.NORMAL);
12 circle.translate(0, 10); // sets the circle's center (x, y)
13 world.addBody(circle);
14
15 world.setGravity(World.EARTH_GRAVITY); // the circle will fall down, the rectangle is fixed due to an infinite mass
16
17 circle.applyImpulse(new Vector2(x, y)); // applies an impulse on the circle in the direction of (x, y)

```

Mittels der Methode `update` auf der Klasse `World` kann die Simulation der Welt in flexiblen zeitlichen Intervallen fortgeführt werden.

```
1 ScheduledExecutorService scheduler = new ScheduledThreadPoolExecutor(1);
2 scheduler.scheduleAtFixedRate(() -> {
3     world.update(interval); // simulation of the world model is updated here
4 }, 0, interval, TimeUnit.MILLISECONDS);
```

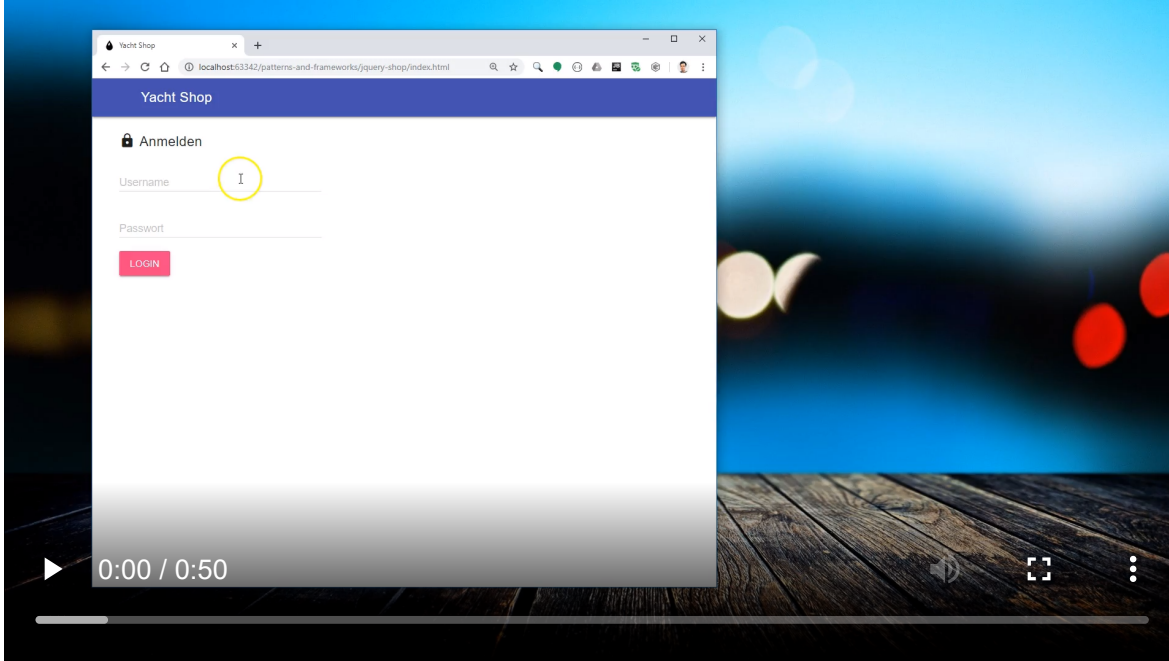
Das folgende Video zeigt eine Beispielanwendung, in der dyn4j zur Kollisionserkennung der animierten Objekte eingesetzt wird. Der Ball kann über die Pfeiltasten gesteuert werden. Durch Mausklicks können herabfallende Rechtecke erzeugt werden. Der vollständige Code der Beispielanwendung findet sich in der Klasse `BouncingBallDyn4j` im Verzeichnis `/ui/javafx-animation` des Modul-Repository.



Spielphysik in JavaFX mittels dyn4j

Web-Anwendungen mit jQuery und Angular

Dieses Kapitel soll keine Einführung in die grundlegenden Web-Technologien wie HTML, JavaScript und CSS geben, die bereits in anderen Modulen des Studiengangs ausführlich behandelt worden sind. Es wird lediglich die Struktur einer Web-Anwendung dargestellt, die ähnlich zu der [Desktop-Anwendung in JavaFX](#) aus dem vorherigen Kapitel mit der im [Spring-Kapitel](#) erstellten REST-API kommuniziert. Dadurch soll praktisch demonstriert werden, wie mehrere Clients, die in unterschiedlichen Programmiersprachen erstellt worden sind, sich eine gemeinsame API teilen können. Die Web-Anwendung weist die gleiche Funktionalität wie die JavaFX-Anwendung auf, was in folgendem Video zusammengefasst wird. Die Web-Anwendung nutzt die UI-Elemente von [Material Design Lite](#) zur Gestaltung des UI.



Web-Beispielanwendung "Yacht Shop"

Im Fokus dieses Kapitels steht nun, wie die serverseitige API aus der clientseitigen Web-Anwendung heraus aufgerufen wird. Die Web-Anwendung ist exemplarisch in zwei unterschiedlichen Ansätzen implementiert worden:

- **jQuery (Programmiersprache JavaScript)**: Im Verzeichnis /ui/jquery-shop des Modul-Repository findet sich eine Implementierung als [Single Page Application \(SPA\)](#) auf Basis der JavaScript-Bibliothek [jQuery](#).
- **Angular (Programmiersprache TypeScript)**: Im Verzeichnis /ui/angular-shop des Modul-Repository findet sich eine alternative Implementierung auf Basis des Frameworks [Angular](#).

Sowohl jQuery als auch Angular sind *open source* unter der MIT-Lizenz verfügbar. Ein entscheidender Unterschied ist, dass jQuery nur den Charakter einer Bibliothek aufweist und dementsprechend dem eigenen Projekt keine grundlegende Struktur vorgibt, während Angular als Framework die Verzeichnisstruktur des Projekts und das Zusammenspiel verschiedener Komponenten der Architektur klar vorgibt. Angular entspricht diesbezüglich anderen aktuellen Web-Frameworks wie [Vue.js](#) oder [React](#).

API-Aufrufe in jQuery

jQuery ist heute weiterhin die mit Abstand am stärksten verbreitete JavaScript-Bibliothek ([siehe hier](#)). jQuery bietet neben einfachen Funktionen zur Selektion und zur Manipulation von DOM-Elementen, zum Event-Handling und zur Erzeugung von Animationen auch Funktionen zum Senden und Verarbeiten von [AJAX-Anfragen](#). Die folgenden Code-Beispiele demonstrieren beispielhaft, wie die erstellte REST-API über die jQuery-Funktion [\\$.ajax](#) angesprochen werden kann.

Produktkatalog laden

```
1 $(document).ready(function () { // jQuery event handler which is executed when the DOM is fully loaded
2
3     $.ajax({ // send XHR request
4         url: "https://example.com/products",
5         method: "GET",
6         success: function (data) { // async success handler
7
8             $(data._embedded.products).each(function () { // iterate over the products in the returned JSON data
9                 var product = this;
10                // create and append a new DOM element for each product ...
11            });
12        }
13    });
14 });
```

Login

```
1  $("#login-button").on("click", function () { // login button event handler
2
3      var credentials = {
4          name: $("#username").val(),
5          passwordHash: $("#password").val()
6      };
7
8      $.ajax({ // send XHR request
9          url: "https://example.com/login",
10         method: "POST",
11         data: JSON.stringify(credentials),
12         contentType: "application/json",
13         success: function (data, status, xhr) { // async success handler
14             var jsonWebToken = xhr.getResponseHeader("Authorization"); // extract JWT from the HTTP response header
15             // ...
16         },
17         error: function () { // async error handler
18             $(".login-failure").html("Login fehlgeschlagen!");
19         }
20     });
21 });
```

Buchung löschen

```
1  $(".delete-booking").on("click", function () { // delete booking event handler
2
3      var bookingId = Number($(this).attr("id"));
4      $.ajax({ // send XHR request
5          url: "https://example.com/bookings/" + bookingId,
6          method: "DELETE",
7          headers: {"Authorization": user.jsonWebToken}, // send JWT as part of the HTTP request header
8          success: function () {
9              // ...
10         }
11     });
12 });
```

Der Vorteil von jQuery liegt in seiner Einfachheit. Zur Verwendung der Bibliothek ist nur eine einzige JavaScript-Datei einzubinden, die wie üblich als minimierte Version für die Produktivumgebung und als nicht-minimierte Version für die Entwicklungsumgebung zum Download oder über ein *Content Delivery Network (CDN)* angeboten wird. Durch die Einbindung der Bibliothek steht in JavaScript die globale Funktion `jQuery` zur Verfügung, die einen sehr kurzen Alias namens `$` hat. Durch verschiedene [Plugins](#) kann diese Funktion erweitert werden.

API-Aufrufe in Angular

Das Web-Framework Angular wird maßgeblich von Google entwickelt. Die Programmiersprache in Angular ist [TypeScript](#). TypeScript ist im Vergleich zu JavaScript von Beginn an auf objektorientierte Programmierung ausgelegt und eine [statisch typisierte Sprache](#), d.h. dass der Typ einer Variable bereits zur Kompilierungszeit bestimmt wird und nicht erst zur Laufzeit wie bei dynamisch typisierten Sprachen. TypeScript wird zu JavaScript [transkompiliert](#) (oder kurz: transpiliert), damit es von der JavaScript-Engine eines Browsers ausgeführt werden kann. In der Dokumentation von Angular findet sich ein sehr anschaulich aufbereitetes [Tutorial](#), indem durchgängig eine kleine Beispielanwendung entwickelt wird, um den Einstieg in das Framework zu erleichtern. An dieser Stelle sei auf dieses Tutorial verwiesen, um die im Folgenden dargestellten API-Aufrufe aus den Angular-Komponenten besser nachvollziehen zu können.

In Angular wird jede View durch eine [Komponente](#) mit der Annotation `@Component` gesteuert. Je Komponente wird eine TypeScript-Klasse angelegt. Das folgende Code-Beispiel zeigt exemplarisch eine Login-Komponente `login.component.ts`, die ein einfaches Anmeldeformular steuert, und eine Produktkatalog-Komponente `catalog.component.ts`, die mehrere Produkte in einer Liste ausgibt. Beide Komponenten teilen sich einen Shop-Service `shop.service.ts`. Der Produktkatalog soll nach erfolgreichem Login über die API geladen und angezeigt werden. Das HTML-basierte View-Template einer Komponente in Angular kann entweder direkt in die Komponentenklassse integriert werden (s. Zeilen 3-7 in `login.component.ts`) oder in eine separate Datei ausgegliedert und referenziert werden (s. Zeile 3 in `catalog.component.ts`).

Login-Komponente (login.component.ts)

```
1  import {Component} from '@angular/core';
2  import {Router} from '@angular/router';
3  import {ShopService} from './shop.service';
4  import {User} from './datamodel';
5
6  @Component({
7    selector: 'app-login',
8    template: `
9      <input [(ngModel)]="user.name" type="text">
10     <input [(ngModel)]="user.passwordHash" type="password">
11     <button (click)="loginAction()" type="submit">Login</button>
12     <div class="login-failure">{{failureMessage}}</div>`
13  })
14  export class LoginComponent {
15
16    // component attributes are bound to the view template by the framework
17    user = new User();
18    failureMessage: String;
19
20    // router and service are injected by the framework
21    constructor(private router: Router, private shopService: ShopService) { }
22
23    // login button event handler
24    loginAction() {
25      this.shopService.login(this.user).subscribe(res => {
26        this.router.navigateByUrl('/catalog');
27      }, error => {
28        this.failureMessage = 'Login fehlgeschlagen!';
29      });
30    }
31  }
```

Produktkatalog-Komponente (catalog.component.ts)

```
1  import {Component, OnInit} from '@angular/core';
2  import {ShopService} from './shop.service';
3  import {Booking, Product, User} from './datamodel';
4
5  @Component({
6    selector: 'app-catalog',
7    templateUrl: './catalog.component.html'
8  })
9  export class CatalogComponent implements OnInit {
10
11    products: Product[]; // bound to the view template by the framework
12
13    constructor(private shopService: ShopService) { }
14
15    ngOnInit(): void {
16      // API request is executed each time the component is loaded
17      this.shopService.loadProductCatalog().subscribe(res => {
18        this.products = res;
19      });
20    }
21  }
```


Shop-Service (shop.service.ts)

```
1 import {Injectable} from '@angular/core';
2 import {HttpClient} from '@angular/common/http';
3 import {Observable} from 'rxjs';
4 import {map} from 'rxjs/operators';
5 import {Product, User} from './datamodel';
6
7 @Injectable()
8 export class ShopService {
9
10     // HTTP client is injected by the framework
11     constructor(private http: HttpClient) {}
12
13     login(user: User) {
14         return this.http.post('https://example.com/login', user);
15     }
16
17     loadProductCatalog(): Observable<Product[]> {
18         return this.http.get('https://example.com/products').pipe(
19             map(res => {
20                 const products = (res as any)._embedded.products as any[];
21                 products.forEach(product => this.loadProductImage(product));
22                 return products as Product[];
23             })
24         );
25     }
26
27     loadProductImage(product: any) { /* ... */ }
28 }
```

• Login-Komponente

- Die Attribute `user` und `failureMessage` der Komponente (Zeilen 17-18) werden durch das Framework an das View-Template gebunden und können dort mittels verschiedener Angular-Direktiven – wie z.B. `ngModel` für das *Two-Way-Binding* von Eingabefeldern (vgl. Zeilen 9-10) – adressiert werden.
- Da der Shop-Service mit der Annotation `@Injectable` versehen ist, kann er mittels [Dependency Injection](#) im Konstruktor der Login-Komponente impliziert durch das Framework eingebunden werden (Zeile 22).
- Die Methode `login` des Shop-Service wird bei einem Klick auf den Login-Button durch den registrierten EventHandler `loginAction` aufgerufen (Zeile 25). Diese Methode gibt ein [Observable](#) zurück, das den asynchronen HTTP-Request an die serverseitige API kapselt (s. Zeilen 13-15 in `shop.service.ts`). Tatsächlich wird der HTTP-Request erst durch Aufruf der Methode `subscribe` auf dem `Observable` ausgelöst. In der Methode `subscribe` kann die HTTP-Response in einem EventHandler für den Erfolgsfall (d.h. Status-Code < 400) und einem EventHandler für den Fehlerfall (d.h. Status-Code ≥ 400) verarbeitet werden (vgl. Zeilen 25-29). Bei erfolgreichem Login wird über den Router zur Katalog-Komponente navigiert (Zeile 26).

• Produktkatalog-Komponente

- In der Produktkatalog-Komponente wird ebenfalls der Shop-Service injiziert (Zeile 13), um die Methode `loadProductCatalog` aufzurufen (Zeile 17). Dieser Aufruf ist in die Methode `ngOnInit` eingebettet, die das Framework per Konvention jedesmal ausführt, wenn die Komponente geladen wird. In der vorliegenden Implementierung wird also der HTTP-Request, über den die Produkte geladen werden, jedesmal ausgeführt, wenn erneut zur View dieser Komponente navigiert wird. Daher kann es ggf. je nach Anwendungsfall sinnvoll sein, die Ergebnisse eines HTTP-Request zwischenspeichern, wenn der abgefragte, serverseitige Zustand sich nicht kontinuierlich ändert oder nicht unbedingt der aktuelle Zustand angezeigt werden muss. Bei kontinuierlichen Zustandsaktualisierungen ist das WebSocket- oder SSE-Protokoll zu bevorzugen, um Polling zu vermeiden.
- Die Methode `loadProductCatalog` gibt ein `Observable<Product[]>` zurück (s. Zeile 17 in `shop.service.ts`). Da der Body der HTTP-Response noch nicht dem passenden Rückgabotyp `Product[]` entspricht (bzw. das JSON-Dokument, das die API zurückgibt, nicht direkt in diesen Typ überführt werden kann), wird der Body der HTTP-Response über die Methode [map](#) noch entsprechend transformiert (Zeilen 19-23 in `shop.service.ts`).